

Welcome

Thank you for choosing Freenove products!

How to Start

When reading this, you should have downloaded the ZIP file for this product.
Unzip it and you will get a folder containing tutorials and related files. Please start with this PDF tutorial.

- ! Unzip the ZIP file instead of opening the file in the ZIP file directly.
- ! Do not move, delete or rename files in the folder just unzipped.

Get Support

Encounter problems? Don't worry! Refer to "TroubleShooting.pdf" or contact us.

When there are packaging damage, quality problems, questions encountering in use, etc., just send us an email. We will reply to you within one working day and provide a solution.

support@freenove.com

Attention

Pay attention to safety when using and storing this product:

- This product is not suitable for children under 12 years of age because of small parts and sharp parts.
- Minors should use this product under the supervision and guidance of adults.
- This product contains small and sharp parts. Do not swallow, prick and scratch to avoid injury.
- This product contains conductive parts. Do not hold them to touch power supply and other circuits.
- To avoid personal injury, do not touch parts rotating or moving while working.
- The wrong operation may cause overheat. Do not touch and disconnect the power supply immediately.
- Operate in accordance with the requirements of the tutorial. Fail to do so may damage the parts.
- Store this product in a dry and dark environment. Keep away from children.
- Turn off the power of the circuit before leaving.

About

Freenove provides open source electronic products and services.

Freenove is committed to helping customers learn programming and electronic knowledge, quickly implement product prototypes, realize their creativity and launch innovative products. Our services include:

- Kits for learning programming and electronics
- Kits compatible with Arduino®, Raspberry Pi®, micro:bit®, ESP32®, etc.
- Kits for robots, smart cars, drones, etc.
- Components, modules and tools
- Design and customization

To learn more about us or get our latest information, please visit our website:

<http://www.freenove.com>

Copyright

All the files provided in the ZIP file are released under [Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License](https://creativecommons.org/licenses/by-nc-sa/3.0/). You can find a copy of the license in the ZIP file.



It means you can use these files on your own derived works, in part or completely. But not for commercial use.

Freenove® brand and logo are trademarks of Freenove Creative Technology Co., Ltd. Must not be used without permission.



Other registered trademarks and their owners appearing in this document:

Arduino® is a trademark of Arduino LLC (<https://www.arduino.cc/>).

Raspberry Pi® is a trademark of Raspberry Pi Foundation (<https://www.raspberrypi.org/>).

micro:bit® is a trademark of Micro:bit Educational Foundation (<https://www.microbit.org/>).

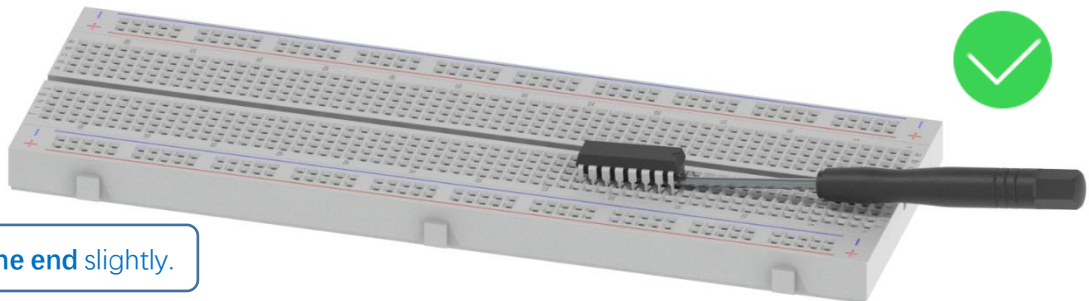
ESPRESSIF® and ESP32® are trademarks of ESPRESSIF Systems (Shanghai) Co., Ltd (<https://www.espressif.com/>).

Remove the Chips

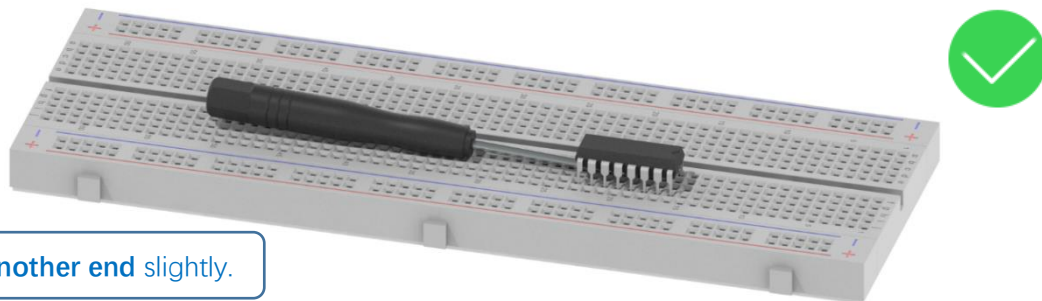
Some chips and modules are inserted into the breadboard to protect their pins.

You need to remove them from breadboard before use. (There is no need to remove GPIO Extension Board.)

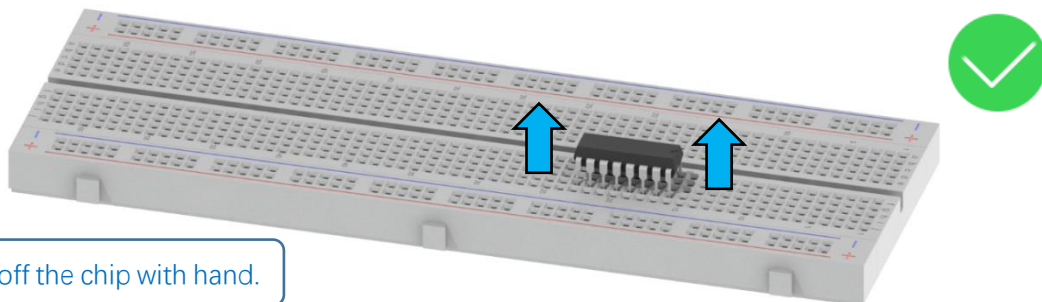
Please find a tool (like a little screw driver) to handle them like below:



Step 1, lift **one end** slightly.

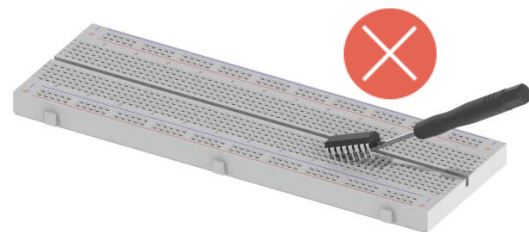
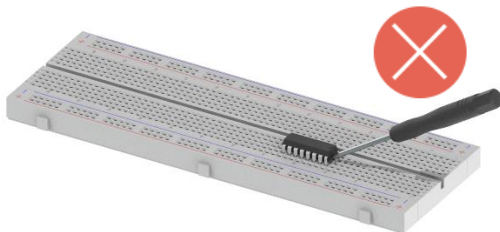


Step 2, lift **another end** slightly.



Step 3, take off the chip with hand.

Avoid lifting one end with big angle directly.



Contents

Welcome.....	1
Remove the Chips.....	3
Contents	4
Prepare	7
ESP32-WROVER	8
Extension board of the ESP32-WROVER.....	11
Chapter 0 Ready (Important)	12
0.1 Installing Thonny (Important)	12
0.2 Basic Configuration of Thonny	17
0.3 Installing CH340 (Important).....	19
0.4 Burning Micropython Firmware (Important).....	29
0.5 Testing codes (Important)	34
0.6 Thonny Common Operation.....	41
0.7 Note	46
Chapter 1 LED (Important).....	48
Project 1.1 Blink.....	48
Project 1.2 Blink.....	57
Chapter 2 Button & LED	65
Project 2.1 Button & LED	65
Project 2.2 MINI table lamp	72
Chapter 3 LED Bar	77
Project 3.1 Flowing Light	77
Chapter 4 Analog & PWM	83
Project 4.1 Breathing LED	83
Project 4.2 Meteor Flowing Light.....	90
Chapter 5 RGBLED	96
Project 5.1 Random Color Light.....	96
Project 5.2 Gradient Color Light	102
Chapter 6 Buzzer	104
Project 6.1 Doorbell	104

Project 6.2 Alertor.....	110
Chapter 7 Serial Communication	112
Project 7.1 Serial Print	112
Project 7.2 Serial Read and Write	117
Chapter 8 AD/DA Converter	118
Project 8.1 Read the Voltage of Potentiometer.....	118
Chapter 9 TouchSensor.....	126
Project 9.1 Read Touch Sensor	126
Project 9.2 TouchLamp	131
Chapter 10 Potentiometer & LED	136
Project 10.1 Soft Light.....	136
Chapter 11 Photoresistor & LED.....	139
Project 11.1 NightLamp	139
Chapter 12 Thermistor	143
Project 12.1 Thermometer	143
Chapter 13 Joystick.....	149
Project 13.1 Joystick.....	149
Chapter 14 74HC595 & LED Bar Graph	154
Project 14.1 Flowing Water Light.....	154
Chapter 15 74HC595 & 7-Segment Display.....	160
Project 15.1 7-Segment Display.	160
Chapter 16 Relay & Motor.....	166
Project 16.1 Control Motor with Potentiometer	166
Chapter 17 Servo.....	175
Project 17.1 Servo Sweep	175
Project 17.2 Servo Knop.....	181
Chapter 18 LCD1602.....	185
Project 18.1 LCD1602.....	185
Chapter 19 Ultrasonic Ranging	192
Project 19.1 Ultrasonic Ranging.....	192
Project 19.2 Ultrasonic Ranging.....	198

Chapter 20 Bluetooth	201
Project 20.1 Bluetooth Low Energy Data Passthrough	201
Project 20.2 Bluetooth Control LED	211
Chapter 21 WiFi Working Modes	217
Project 21.1 Station mode	217
Project 21.2 AP mode	221
Project 21.3 AP+Station mode	225
Chapter 22 TCP/IP	229
Project 22.1 As Client	229
Project 22.2 As Server	240
Chapter 23 Camera Web Server	245
Project 23.1 Camera Web Server	245
What's next?	256
End of the Tutorial	256

Prepare

ESP32 is a micro control unit with integrated Wi-Fi launched by Espressif, which features strong properties and integrates rich peripherals. It can be designed and studied as an ordinary Single Chip Microcontroller (SCM) chip, or connected to the Internet and used as an Internet of Things device.

ESP32 can be developed both either with C/C++ language or micropython language. In this tutorial, we use micropython. With Micropython is as easy to learn as Python with little code, making it ideal for beginners. Moreover, the code of ESP32 is completely open-source, so beginners can quickly learn how to develop and design IOT smart household products including smart curtains, fans, lamps and clocks.

We divide each project into four parts, namely Component List, Component Knowledge, Circuit and Code. Component List helps you to prepare material for the experiment more quickly. Component Knowledge allows you to quickly understand new electronic modules or components, while Circuit helps you understand the operating principle of the circuit. And Code allows you to easily master the use of ESP32 and its accessory kit. After finishing all the projects in this tutorial, you can also use these components and modules to make products such as smart household, smart cars and robots to transform your creative ideas into prototypes and new and innovative products.

In addition, if you have any difficulties or questions with this tutorial or toolkit, feel free to ask for our quick and free technical support through support@freenove.com

ESP32-WROVER

ESP32-WROVER has launched a total of two antenna packages, PCB on-board antenna and IPEX antenna respectively. The PCB on-board antenna is an integrated antenna in the chip module itself, so it is convenient to carry and design. The IPEX antenna is a metal antenna derived from the integrated antenna of the chip module itself, which is used to enhance the signal of the module.

PCB on-board antenna

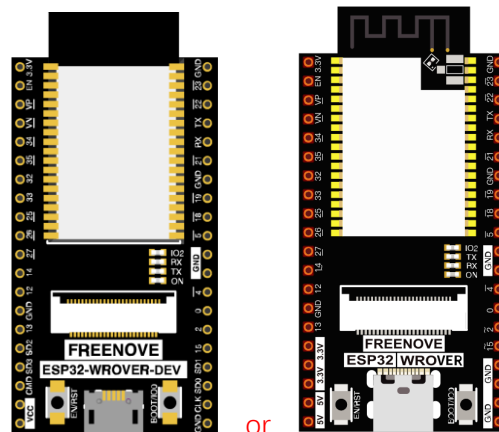


IPEX antenna



In this tutorial, the ESP32-WROVER is designed based on the PCB on-board antenna package.

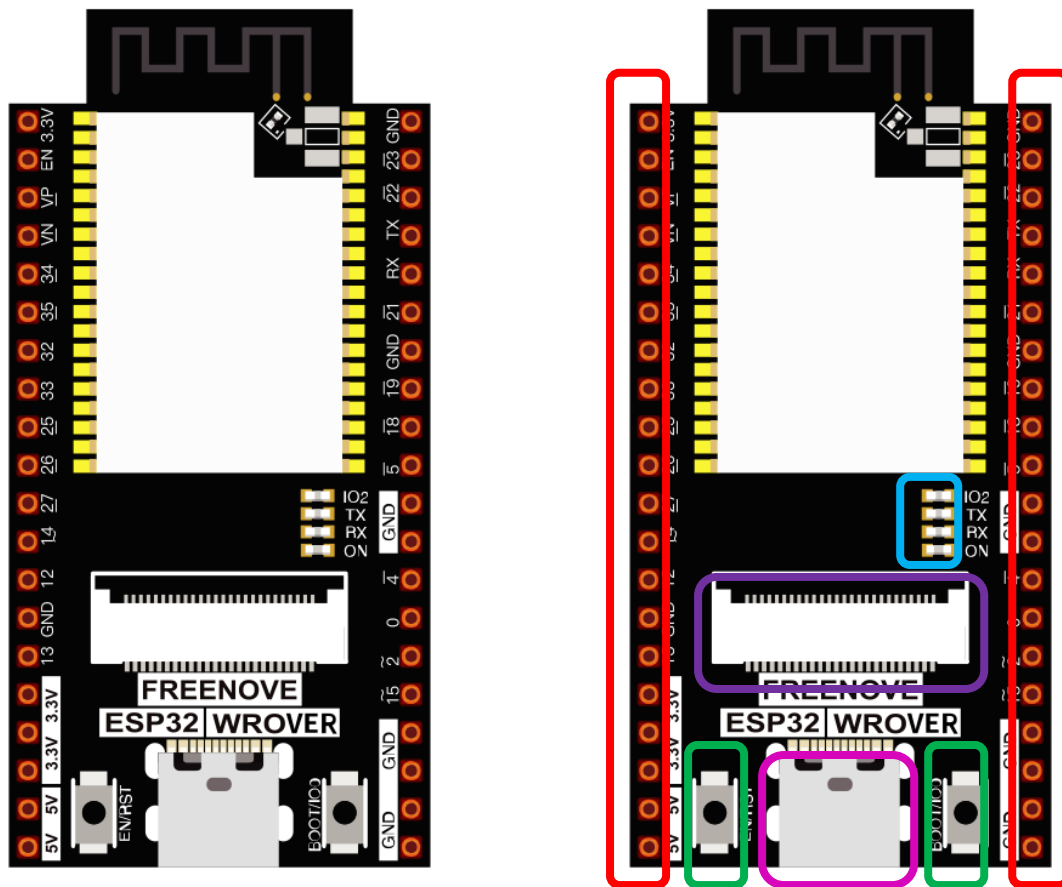
ESP32-WROVER



The version on the left is no longer mass-produced, and we mainly maintain the version on the right. Please note that there are many pirated versions of the ESP32 WROVER that look very similar to the version on the left. None of them will carry our logo and Freenove font.

We do not sell pirated ESP32 WROVER, nor do we provide after-sales service for pirated.

The hardware interfaces of ESP32-WROVER are distributed as follows:



Compare the left and right images. We've boxed off the resources on the ESP32-WROVER in different colors to facilitate your understanding of the ESP32-WROVER.

Box color	Corresponding resources introduction
	GPIO pin
	LED indicator
	Camera interface
	Reset button, Boot mode selection button
	USB port

Name	No.	Type	Function
GND	1	P	Ground
3V3	2	P	Power supply
EN	3	I	Module-enable signal. Active high.
SENSOR_VP	4	I	GPIO36, ADC1_CH0, RTC_GPIO0
SENSOR_VN	5	I	GPIO39, ADC1_CH3, RTC_GPIO3
IO34	6	I	GPIO34, ADC1_CH6, RTC_GPIO4
IO35	7	I	GPIO35, ADC1_CH7, RTC_GPIO5
IO32	8	I/O	GPIO32, XTAL_32K_P (32.768 kHz crystal oscillator input), ADC1_CH4, TOUCH9, RTC_GPIO9
IO33	9	I/O	GPIO33, XTAL_32K_N (32.768 kHz crystal oscillator output), ADC1_CH5, TOUCH8, RTC_GPIO8
IO25	10	I/O	GPIO25, DAC_1, ADC2_CH8, RTC_GPIO6, EMAC_RXD0
IO26	11	I/O	GPIO26, DAC_2, ADC2_CH9, RTC_GPIO7, EMAC_RXD1
IO27	12	I/O	GPIO27, ADC2_CH7, TOUCH7, RTC_GPIO17, EMAC_RX_DV
IO14	13	I/O	GPIO14, ADC2_CH6, TOUCH6, RTC_GPIO16, MTMS, HSPICLK, HS2_CLK, SD_CLK, EMAC_TXD2
IO12 ¹	14	I/O	GPIO12, ADC2_CH5, TOUCH5, RTC_GPIO15, MTDI, HSPIQ, HS2_DATA2, SD_DATA2, EMAC_TXD3
GND	15	P	Ground
IO13	16	I/O	GPIO13, ADC2_CH4, TOUCH4, RTC_GPIO14, MTCK, HSPID, HS2_DATA3, SD_DATA3, EMAC_RX_ER
SHD/SD2 ²	17	I/O	GPIO9, SD_DATA2, SPIHD, HS1_DATA2, U1RXD
SWP/SD3 ²	18	I/O	GPIO10, SD_DATA3, SPIWP, HS1_DATA3, U1TXD
SCS/CMD ²	19	I/O	GPIO11, SD_CMD, SPICSS0, HS1_CMD, U1RTS
SCK/CLK ²	20	I/O	GPIO6, SD_CLK, SPICLK, HS1_CLK, U1CTS
SDO/SD0 ²	21	I/O	GPIO7, SD_DATA0, SPIQ, HS1_DATA0, U2RTS
SDI/SD1 ²	22	I/O	GPIO8, SD_DATA1, SPID, HS1_DATA1, U2CTS
IO15	23	I/O	GPIO15, ADC2_CH3, TOUCH3, MTDO, HSPICSS0, RTC_GPIO13, HS2_CMD, SD_CMD, EMAC_RXD3
IO2	24	I/O	GPIO2, ADC2_CH2, TOUCH2, RTC_GPIO12, HSPWP, HS2_DATA0, SD_DATA0
IO0	25	I/O	GPIO0, ADC2_CH1, TOUCH1, RTC_GPIO11, CLK_OUT1, EMAC_TX_CLK
IO4	26	I/O	GPIO4, ADC2_CH0, TOUCH0, RTC_GPIO10, HSPHD, HS2_DATA1, SD_DATA1, EMAC_TX_ER
NC1	27	-	-
NC2	28	-	-
IO5	29	I/O	GPIO5, VSPICSS0, HS1_DATA6, EMAC_RX_CLK
IO18	30	I/O	GPIO18, VSPICLK, HS1_DATA7
IO19	31	I/O	GPIO19, VSPIQ, U0CTS, EMAC_TXD0
NC	32	-	-
IO21	33	I/O	GPIO21, VSPHD, EMAC_TX_EN
RXD0	34	I/O	GPIO3, U0RXD, CLK_OUT2
TXD0	35	I/O	GPIO1, U0TXD, CLK_OUT3, EMAC_RXD2
IO22	36	I/O	GPIO22, VSPWP, U0RTS, EMAC_TXD1
IO23	37	I/O	GPIO23, VSPID, HS1_STROBE
GND	38	P	Ground

Notice:

- GPIO12 is internally pulled high in the module and is not recommended for use as a touch pin.
- Pins SCK/CLK, SDO/SD0, SDI/SD1, SHD/SD2, SWP/SD3 and SCS/CMD, namely, GPIO6 to GPIO11 are connected to the SPI flash integrated on the module and are not recommended for other uses.

For more information, please visit: https://www.espressif.com/sites/default/files/documentation/esp32-wrover_datasheet_en.pdf

Chapter 0 Ready (Important)

Before starting building the projects, you need to make some preparation first, which is so crucial that you must not skip.

0.1 Installing Thonny (Important)

Thonny is a free, open-source software platform with compact size, simple interface, simple operation and rich functions, making it a Python IDE for beginners. In this tutorial, we use this IDE to develop ESP32 during the whole process.

Thonny supports various operating system, including Windows、Mac OS、Linux.

Downloading Thonny

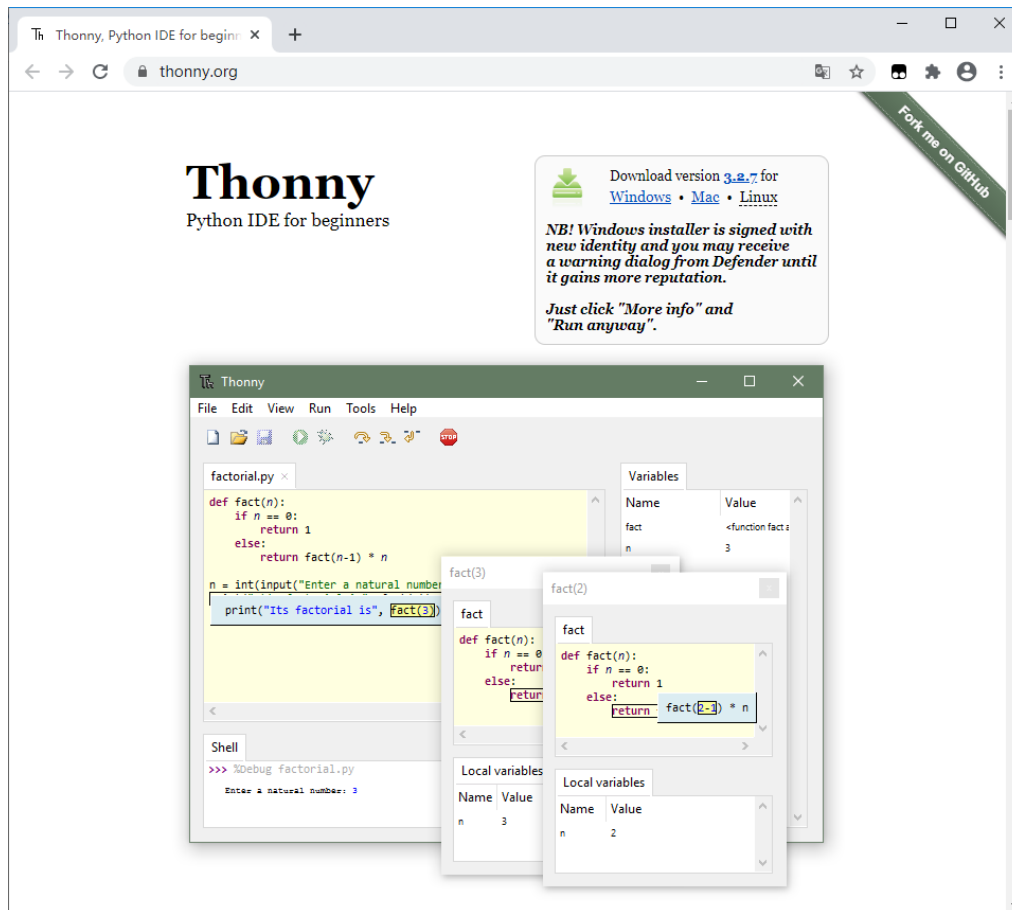
Official website of Thonny: <https://thonny.org>

Open-source code repositories of Thonny: <https://github.com/thonny/thonny>

Follow the instruction of official website to install Thonny or click the links below to download and install. (Select the appropriate one based on your operating system.)

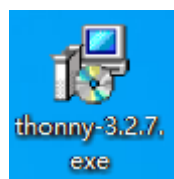
Operating System	Download links/methods
Windows	https://github.com/thonny/thonny/releases/download/v3.2.7/thonny-3.2.7.exe
Mac OS	https://github.com/thonny/thonny/releases/download/v3.2.7/thonny-3.2.7.pkg
Linux	The latest version: Binary bundle for PC (Thonny+Python): <code>bash <(wget -O - https://thonny.org/installer-for-linux)</code> With pip: <code>pip3 install thonny</code> Distro packages (may not be the latest version): Debian, Rasbian, Ubuntu, Mint and others: <code>sudo apt install thonny</code> Fedora: <code>sudo dnf install thonny</code>

You can also open “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Software**”, we have prepared it in advance.

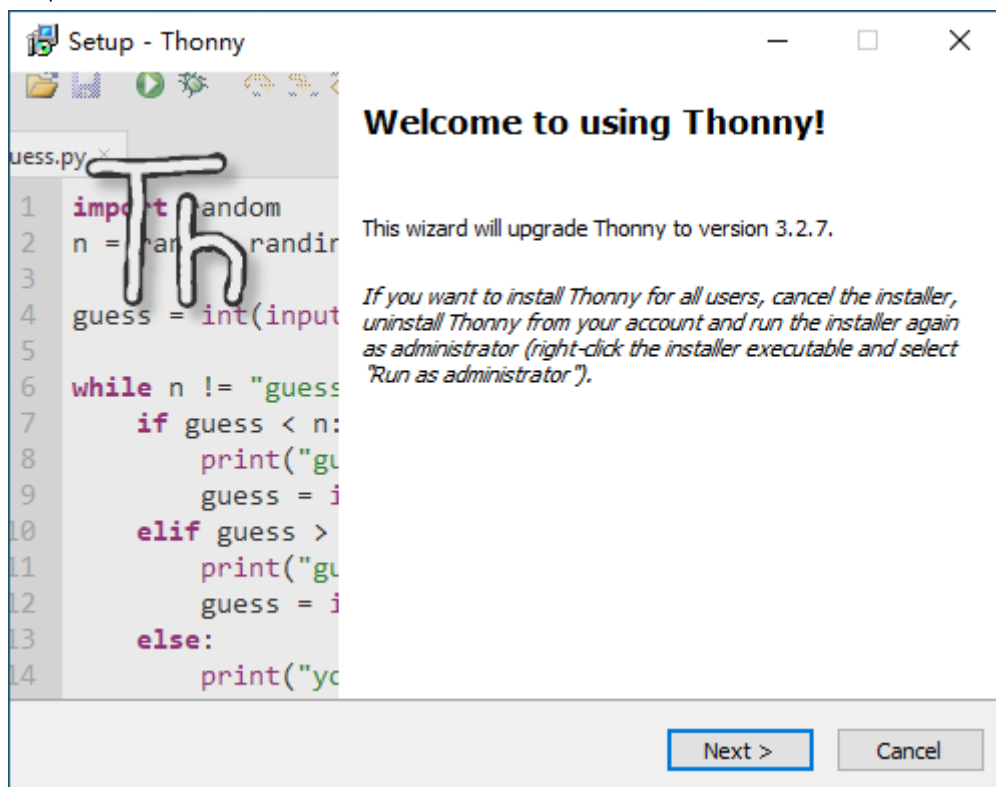


Installing on Windows

The icon of Thonny after downloading is as below. Double click "thonny-3.2.7.exe".

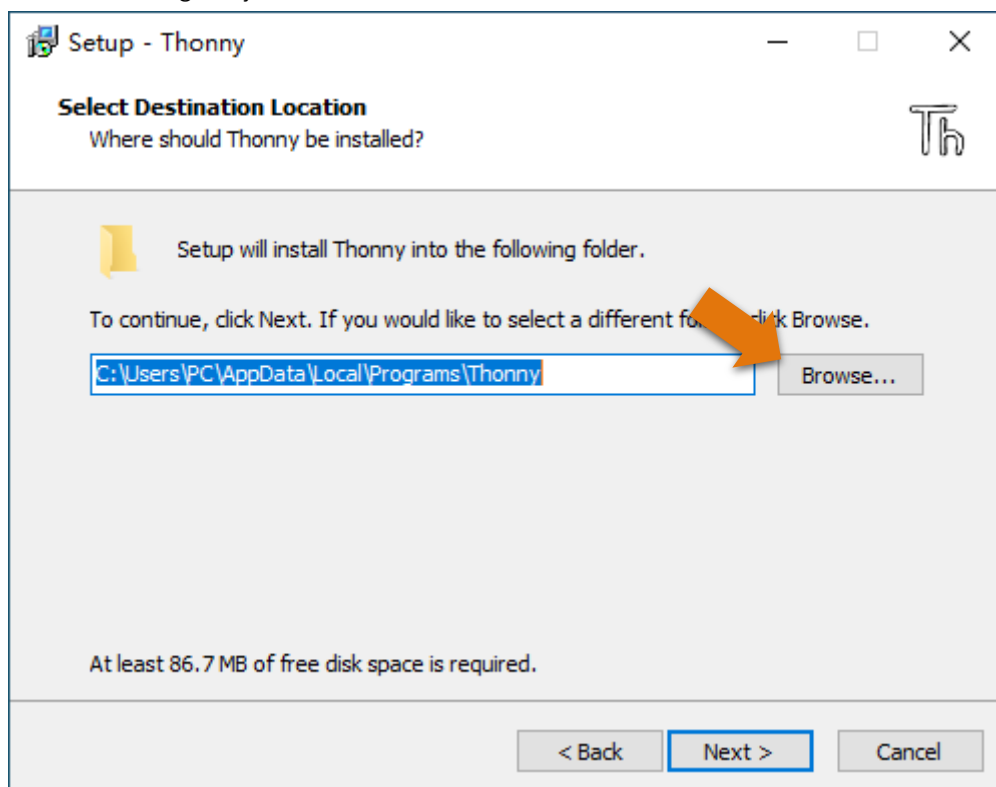


If you're not familiar with computer software installation, you can simply keep clicking "Next" until the installation completes.

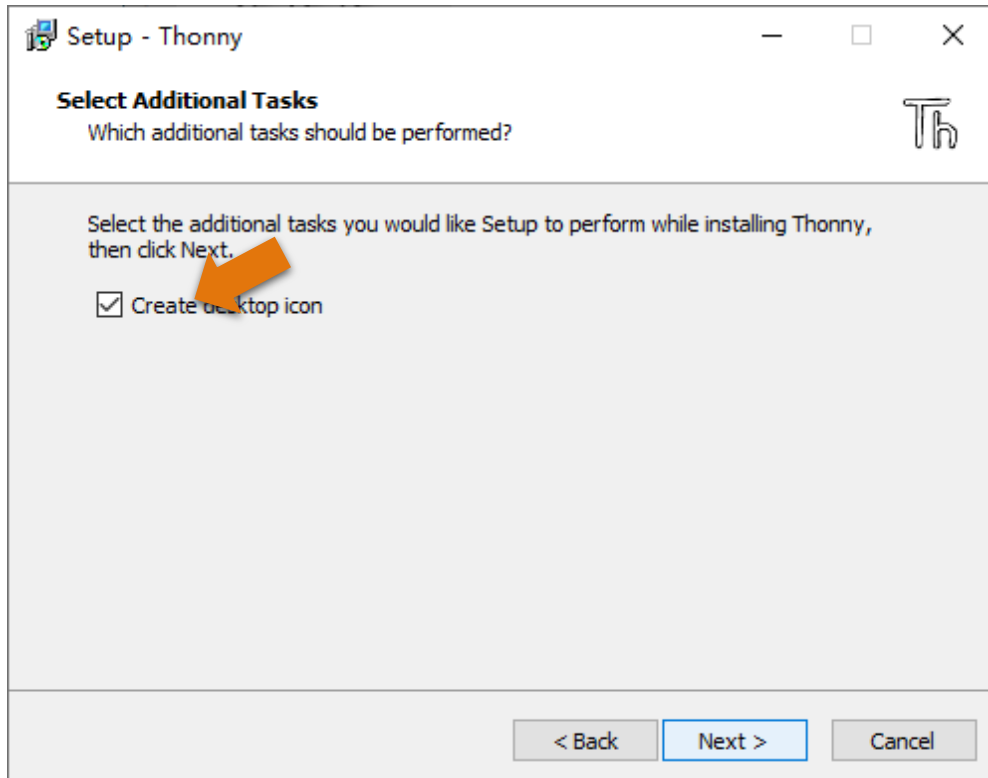


If you want to change Thonny's installation path, you can click "Browse" to modify it. After selecting installation path, click "OK".

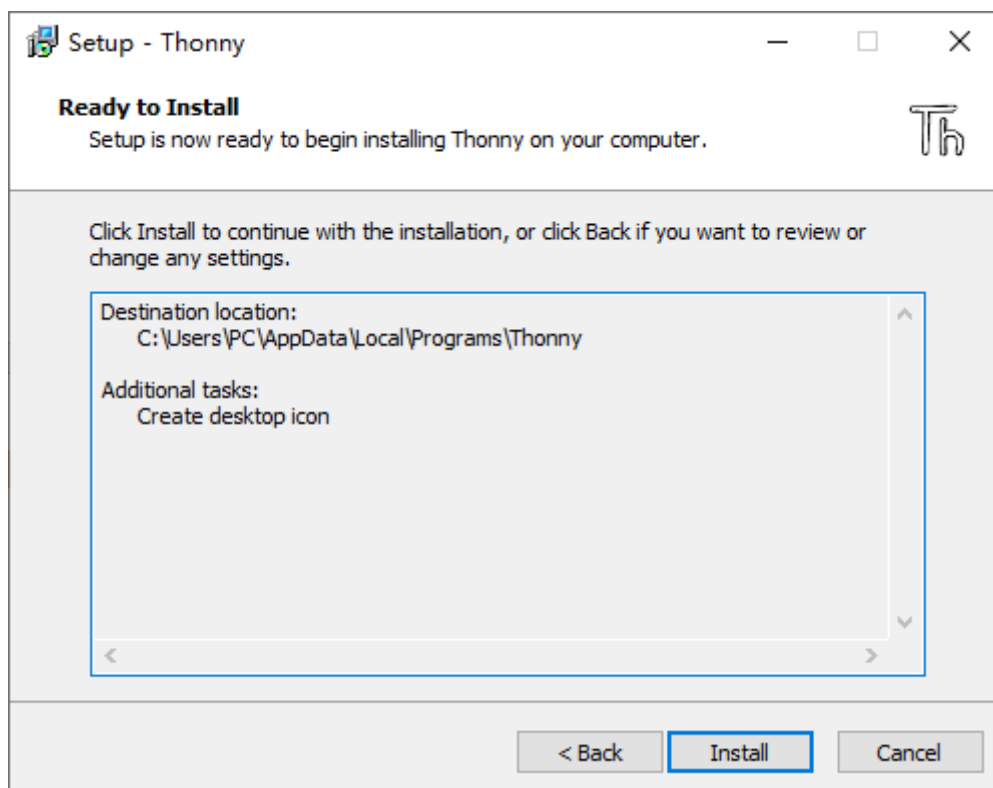
If you do not want to change it, just click "Next".



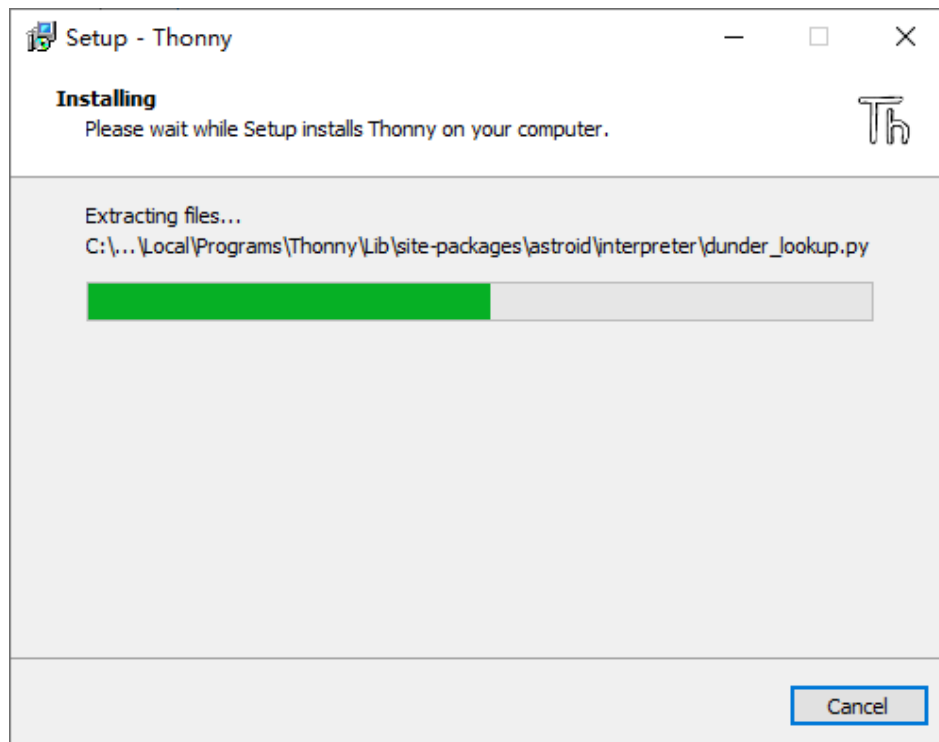
Check “Create desktop icon” and then it will generate a shortcut on your desktop to facilitate you to open Thonny later.



Click “install” to install the software.



During the installation process, you only need to wait for the installation to complete, and you must not click "Cancel", otherwise Thonny will fail to be installed.



Once you see the interface as below, Thonny has been installed successfully.

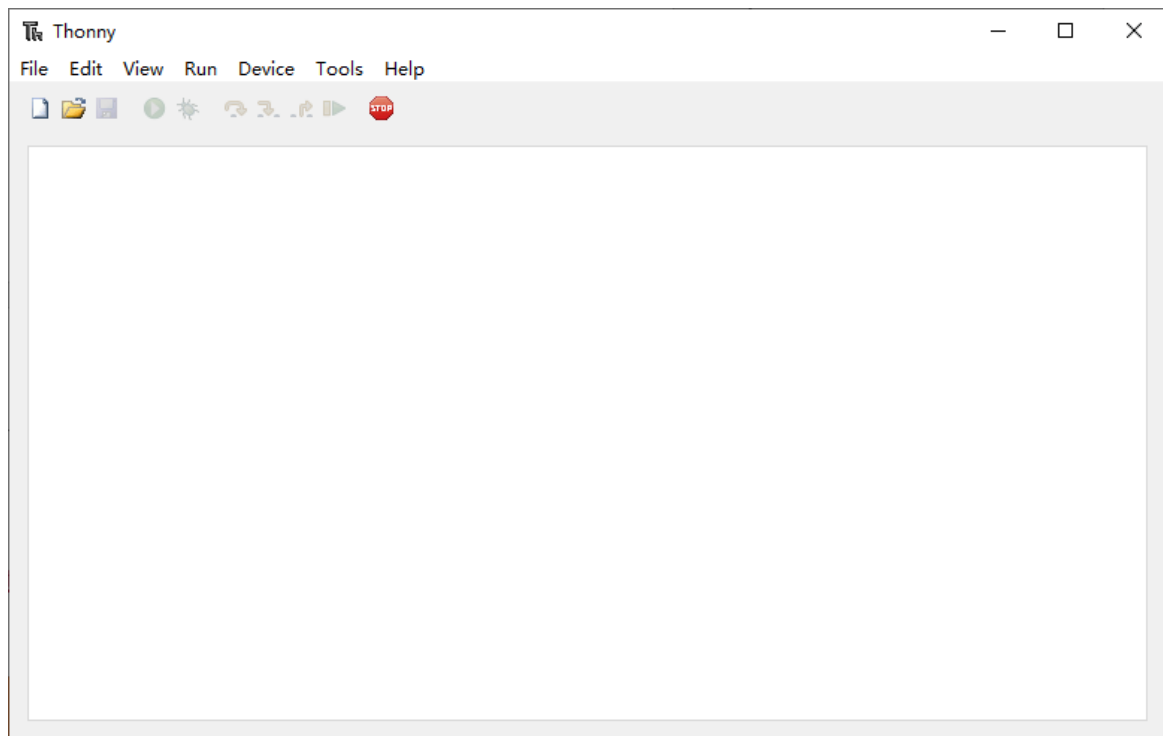


If you've checked "Create desktop icon" during the installation process, you can see the below icon on your desktop.

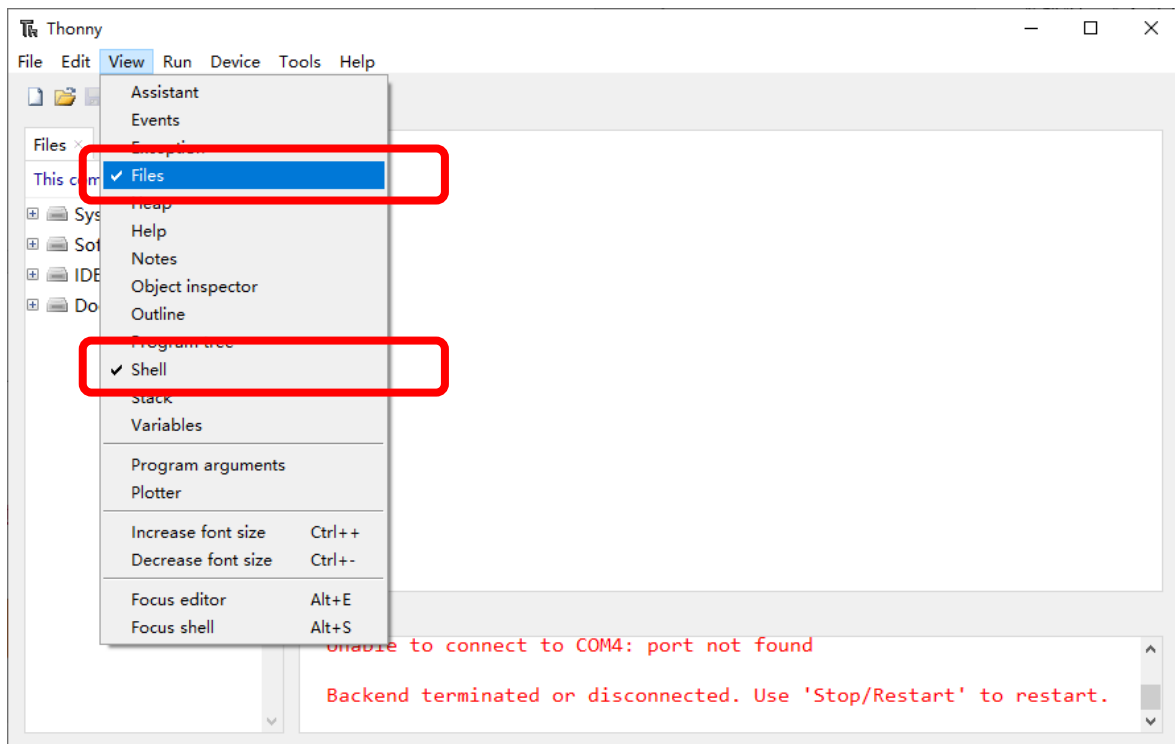


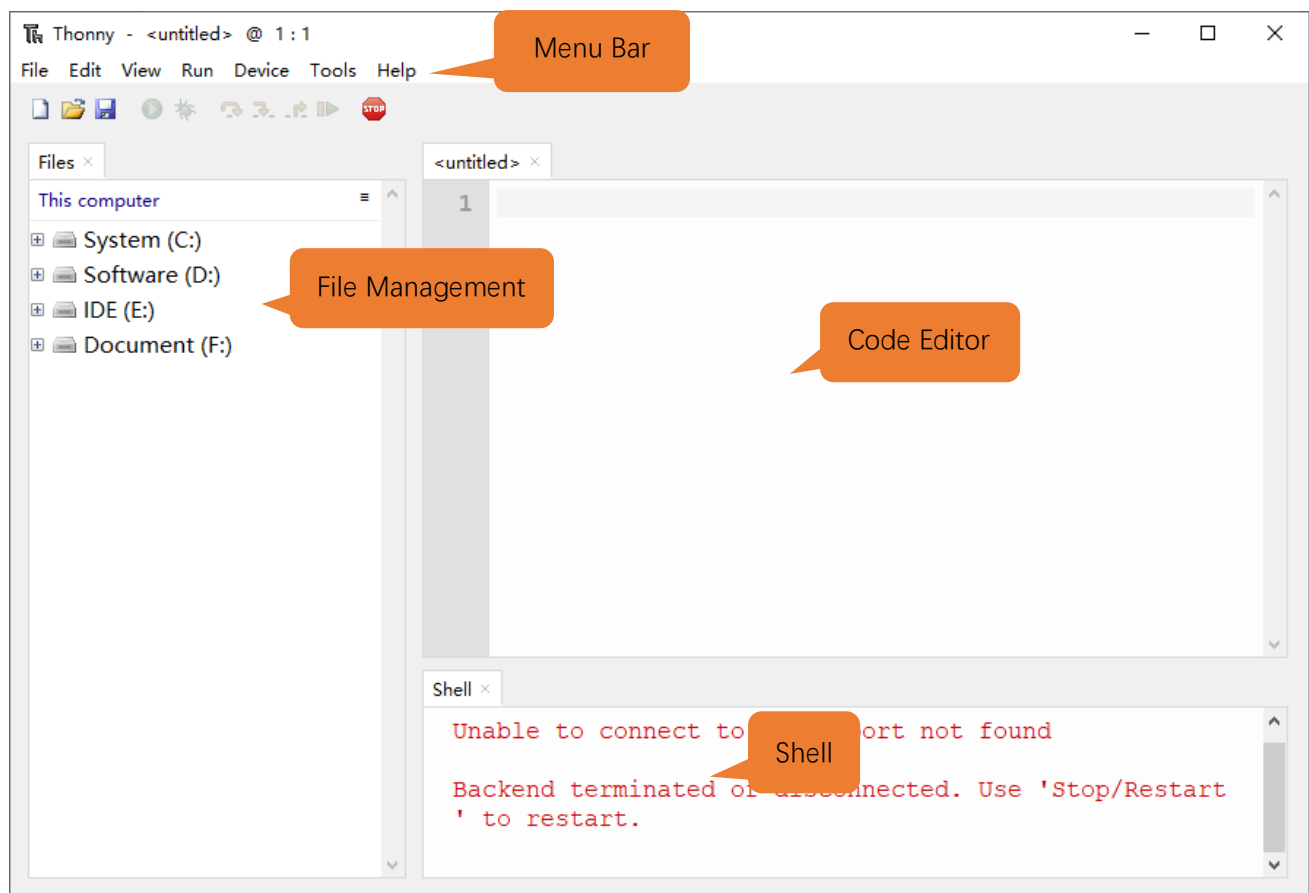
0.2 Basic Configuration of Thonny

Click the desktop icon of Thonny and you can see the interface of it as follows:



Select "View" → "Files" and "Shell".





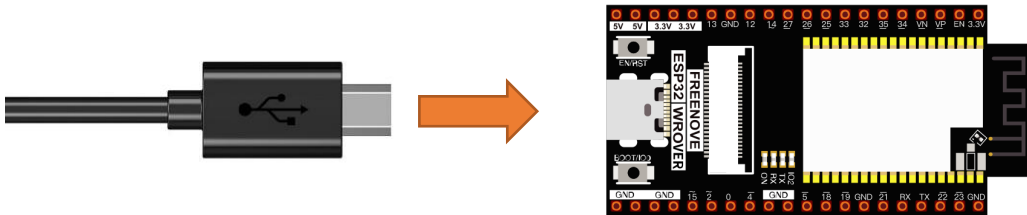
0.3 Installing CH340 (Important)

ESP32 uses CH340 to download codes. So before using it, we need to install CH340 driver in our computers.

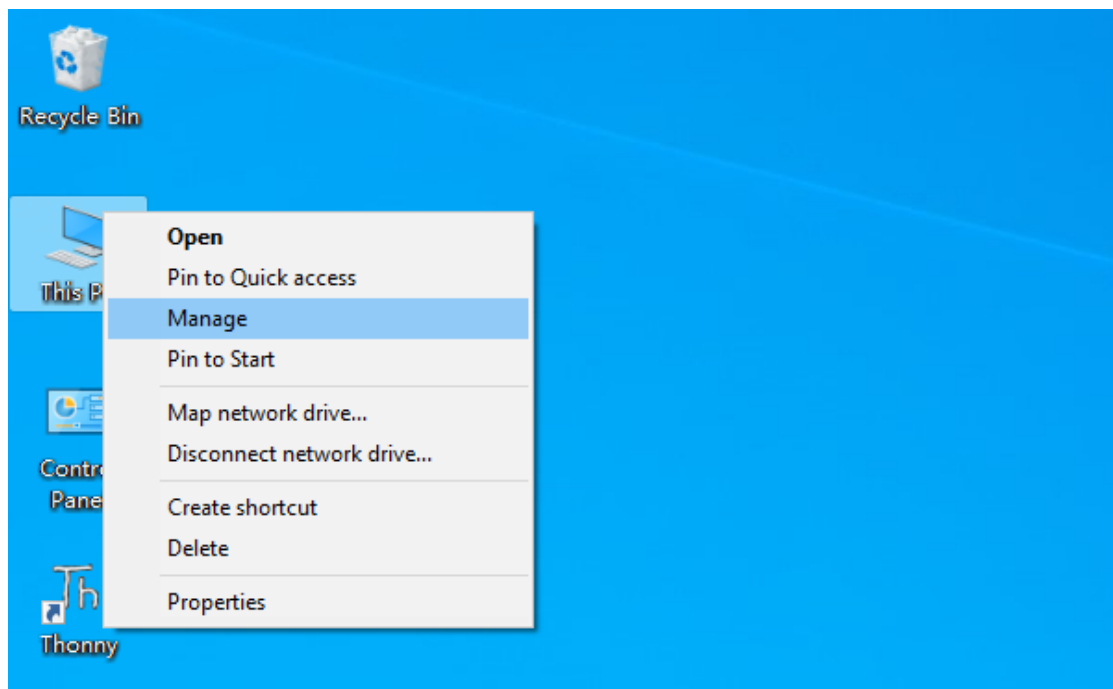
Windows

Check whether CH340 has been installed

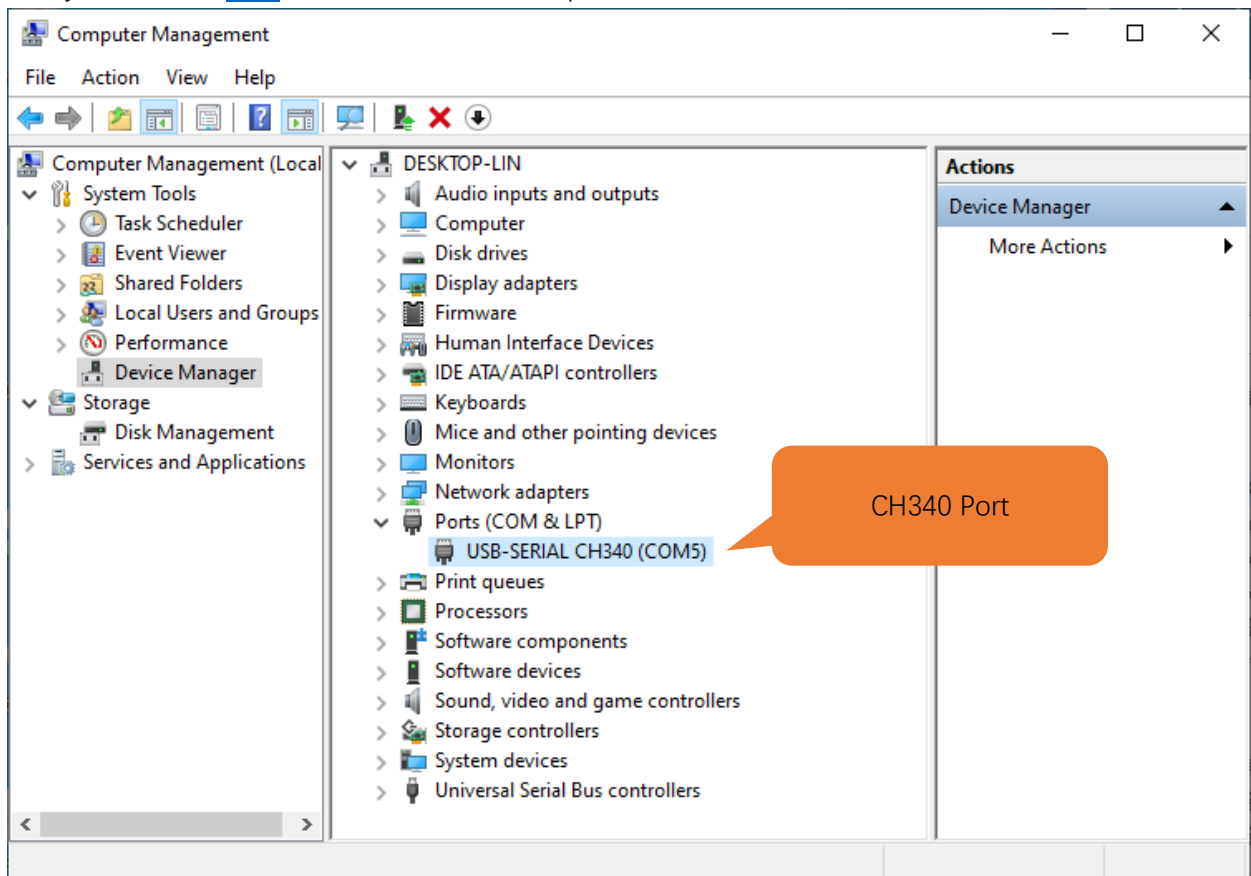
1. Connect your computer and ESP32 with a USB cable.



2. Turn to the main interface of your computer, select "This PC" and right-click to select "Manage".



3. Click “Device Manager”. If your computer has installed CH340, you can see “USB-SERIAL CH340 (COMx)”. And you can click [here](#) to move to the next step.



Installing CH340

- First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.

index / search / search CH340

All (14)

Downloads (7)

Products (4)

Application (2)

Video (1)

News (0)

keyword CH340

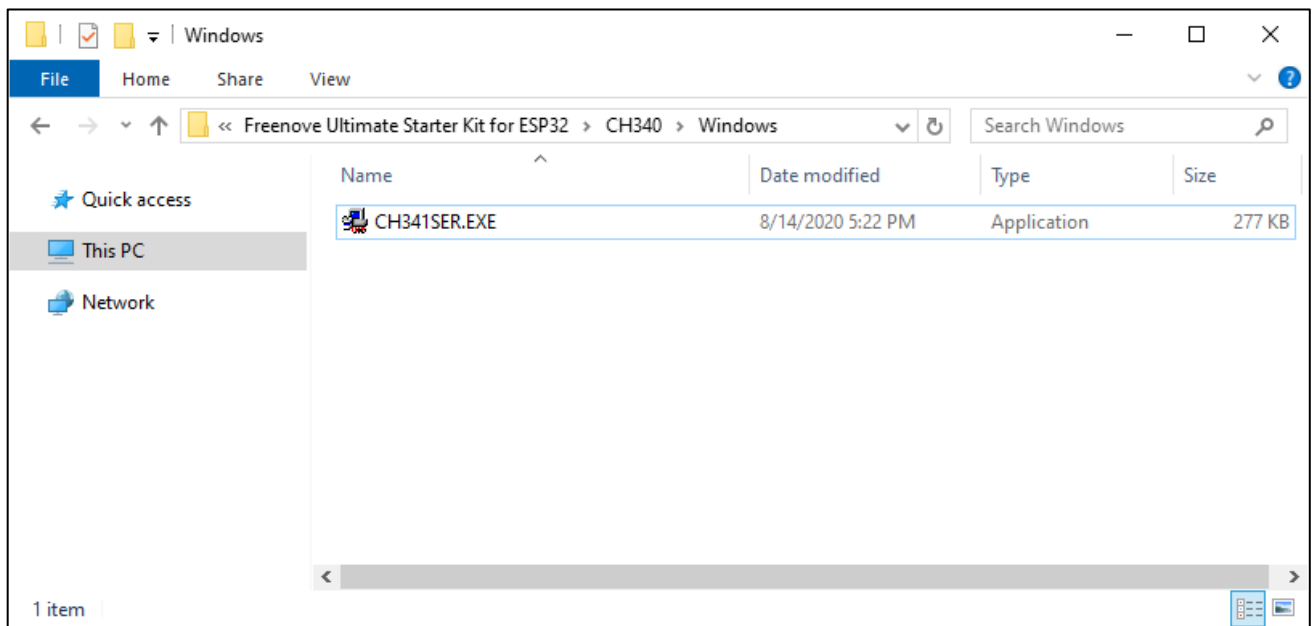
Downloads(7)

file category	file content	version	upload time
Driver&Tools			
CH341SER.EXE	CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-18
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-05
CH341SER_ANDROID...	CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib library, File (Host Driver), App Demo Example (USB to UART Den...	1.6	2019-04-19
CH341SER_LINUX...	CH340/CH341 USB to serial port LINUX driver	1.5	2018-03-18
CH341SER_MAC.ZIP	CH340/CH341 USB to serial port MAC OS driver	1.5	2018-07-05
Others			
PRODUCT_GUIDE.P...	Electronic selection of product selection manual, please refer to related product technical manual for more technical information.	1.4	2018-12-29
InstallNoteOn64...	Instructions for the driver after 18 years of August cannot be installed under some 64-bit WIN7 (English)	1.0	2019-01-10

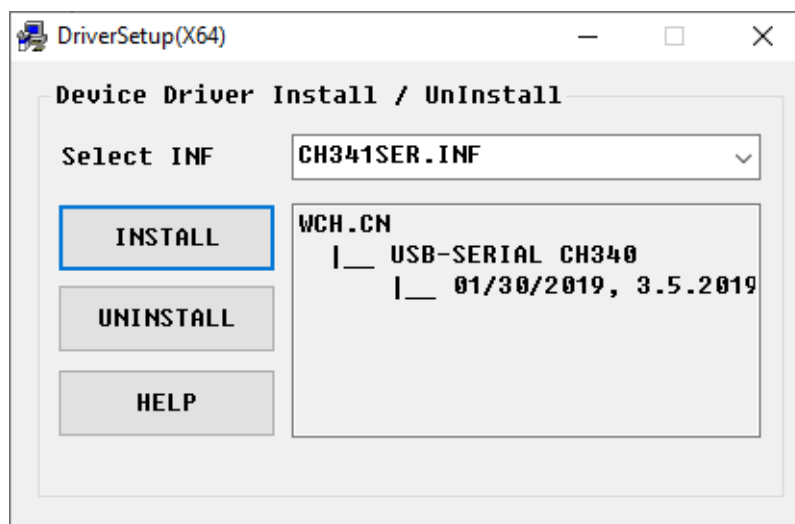
You can also open “Freenove_Super_Starter_Kit_for_ESP32/CH340”, we have prepared the installation package.

Name	Date modified	Type	Size
Linux	8/14/2020 5:24 PM	File folder	
MAC	8/14/2020 5:23 PM	File folder	
Windows	8/14/2020 5:23 PM	File folder	

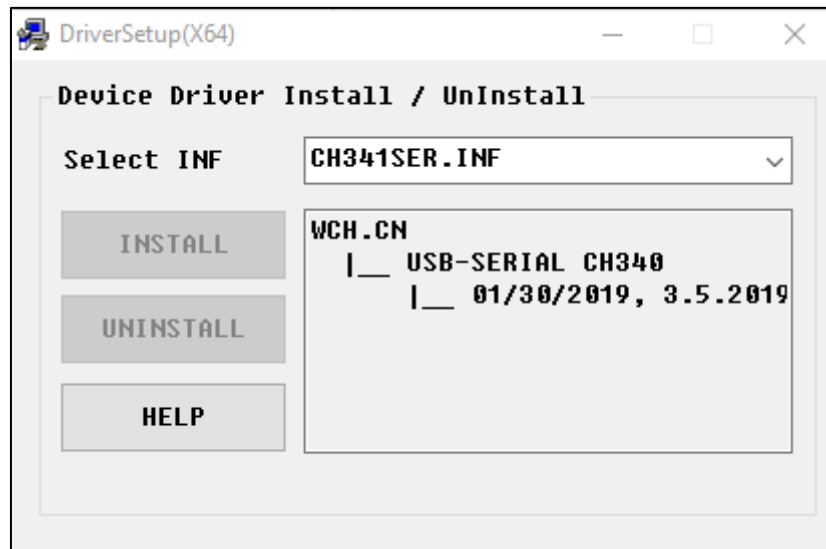
2. Open the folder “Freenove_Super_Starter_Kit_for_ESP32/CH340/Windows/ch341ser”



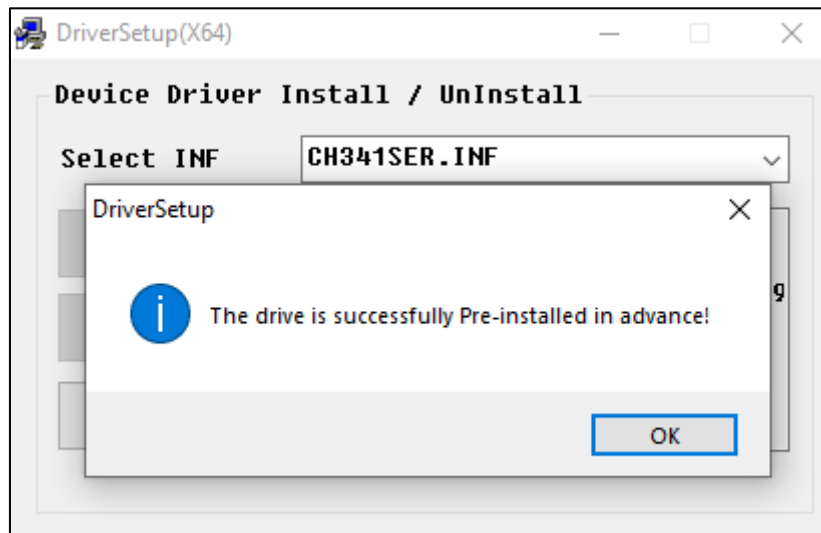
3. Double click “CH341SER.EXE”.



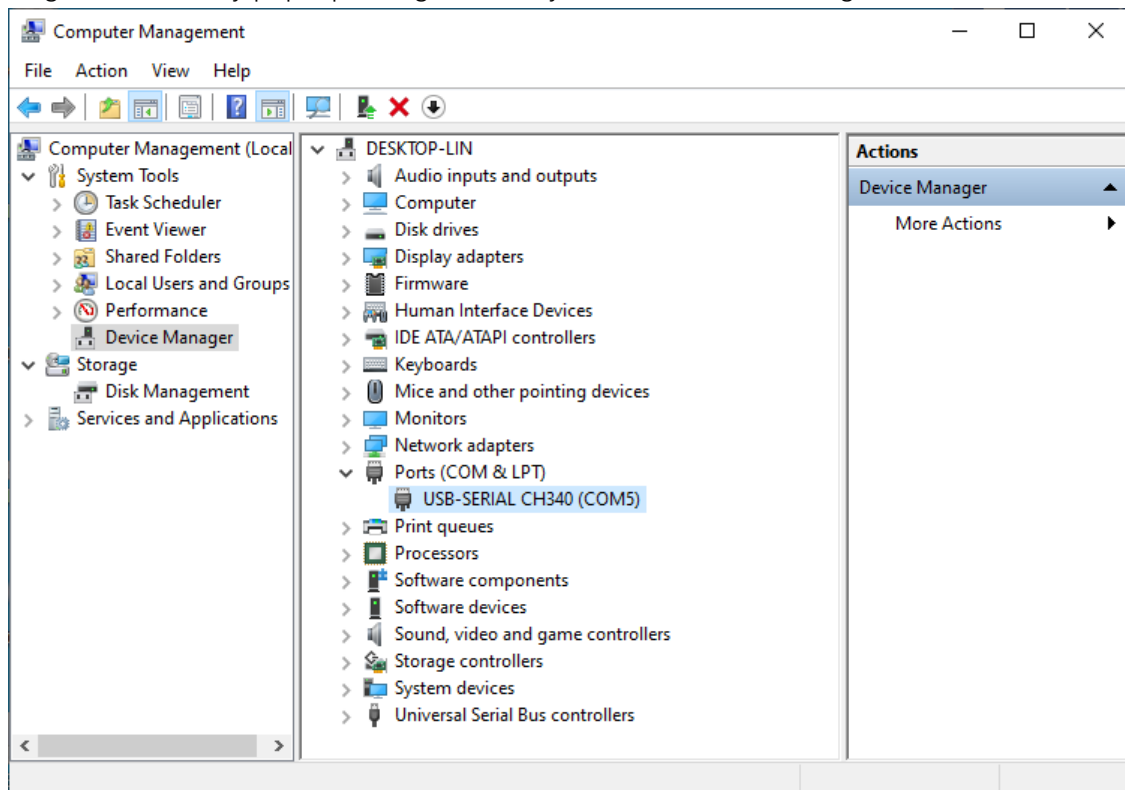
4. Click "INSTALL" and wait for the installation to complete.



5. Install successfully. Close all interfaces.



6. When ESP32 is connected to computer, select "This PC", right-click to select "Manage" and click "Device Manager" in the newly pop-up dialog box, and you can see the following interface.



7. So far, CH340 has been installed successfully. Close all dialog boxes.

MAC

First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.

keyword ch340

Downloads(7)

file category	file content	version	upload time
Driver&Tools			
CH341SER.EXE	CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-18
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-05
CH341SER_ANDROID...	CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib library file (Java Driver), App Demo Example, AT Demo SDK).	1.6	2019-04-19
CH341SER_LINUX...	CH340/CH341 USB to serial port LINUX driver	1.5	2018-03-18
CH341SER_MAC.ZI...	CH340/CH341 USB to serial port MAC OS driver	1.5	2018-07-05

Others

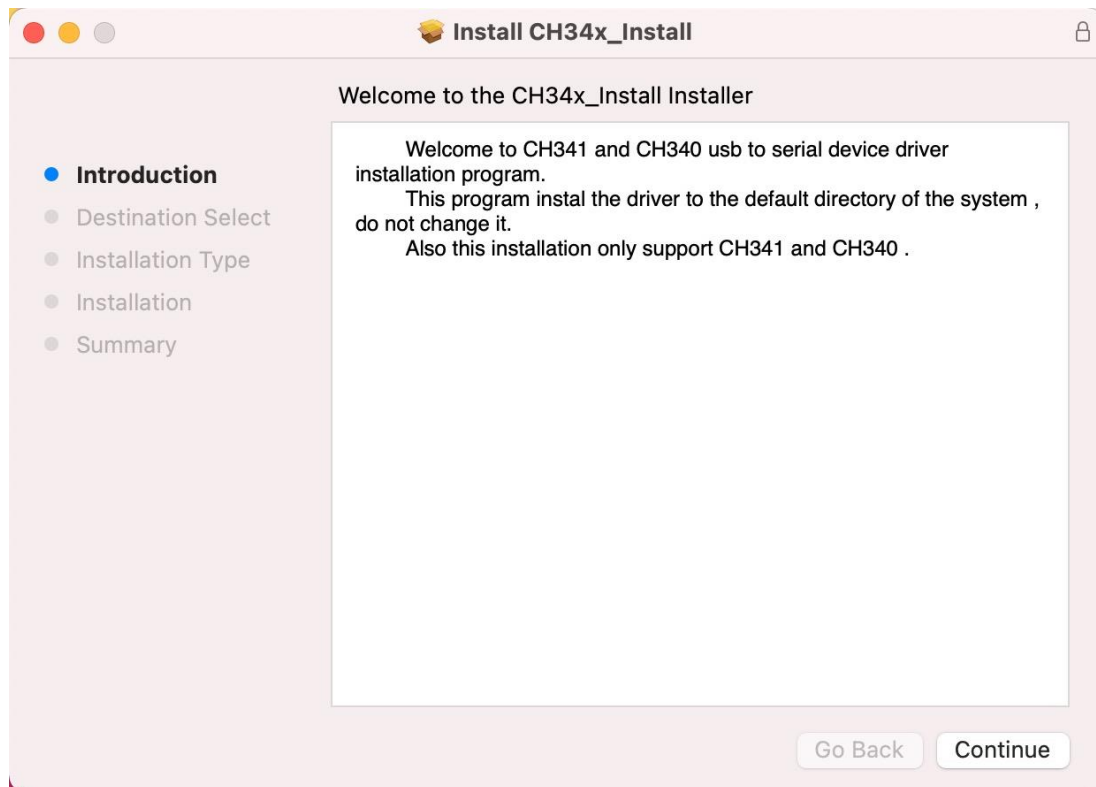
If you would not like to download the installation package, you can open “Freenove_Super_Starter_Kit_for_ESP32/CH340”, we have prepared the installation package.

Second, open the folder “Freenove_Super_Starter_Kit_for_ESP32/CH340/MAC/”

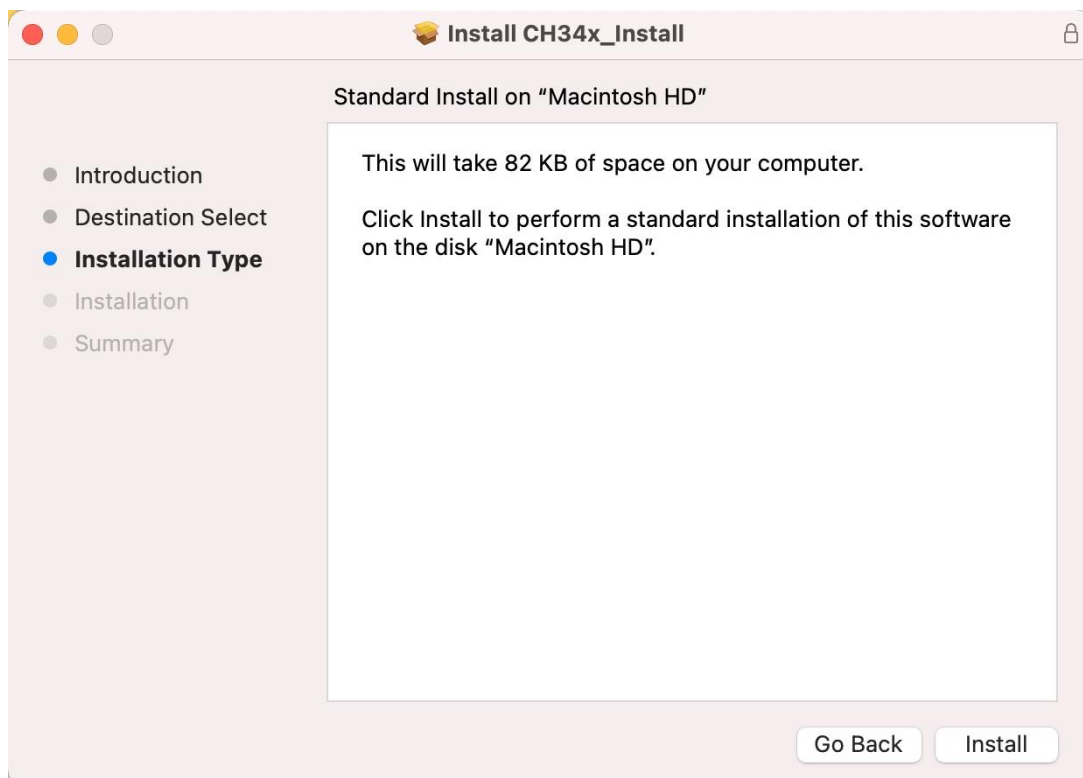
MAC

Name	Modified	Size	Kind
CH34x_Install_V1.5.pkg	Dec 8, 2018 at 2:00 PM	26 KB	Installer package
ReadMe.pdf	Dec 8, 2016 at 4:09 PM	146 KB	Adobe...ocument

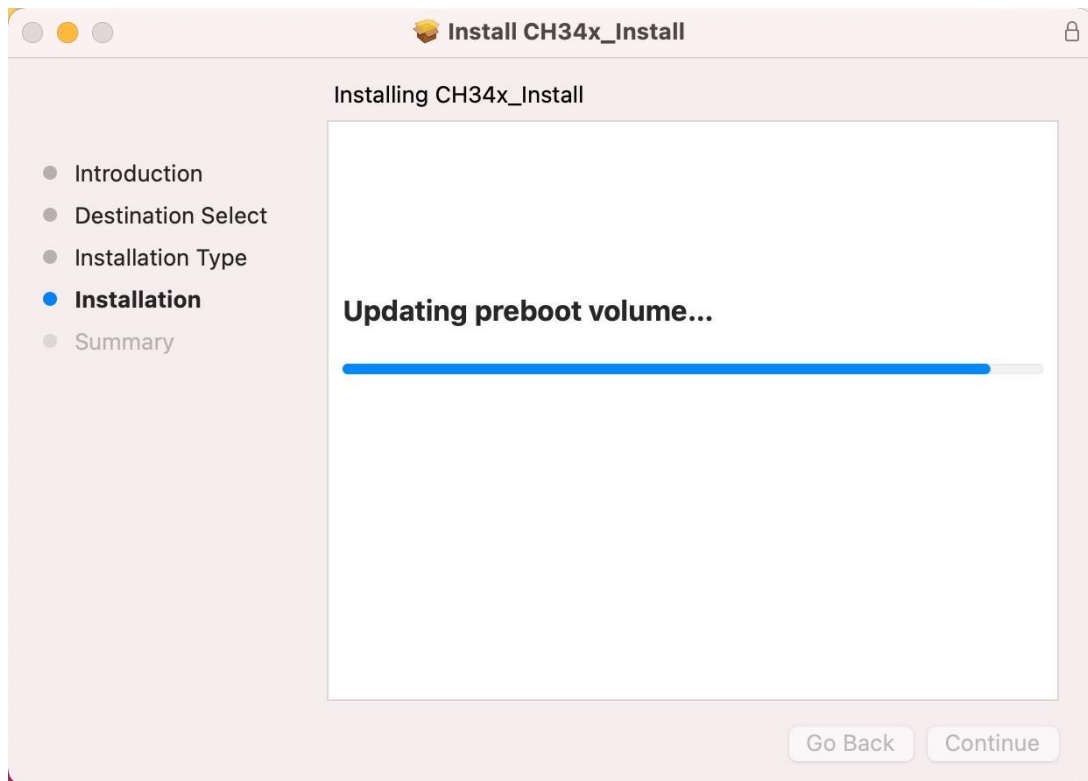
Third, click Continue.



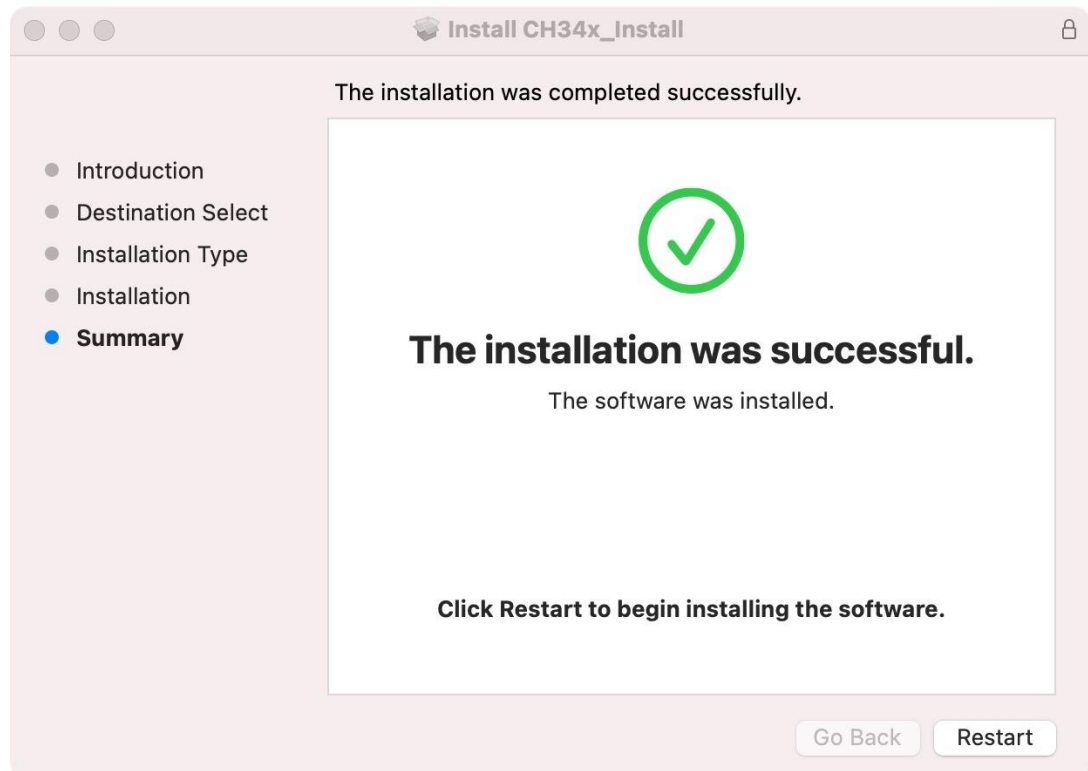
Fourth, click Install.



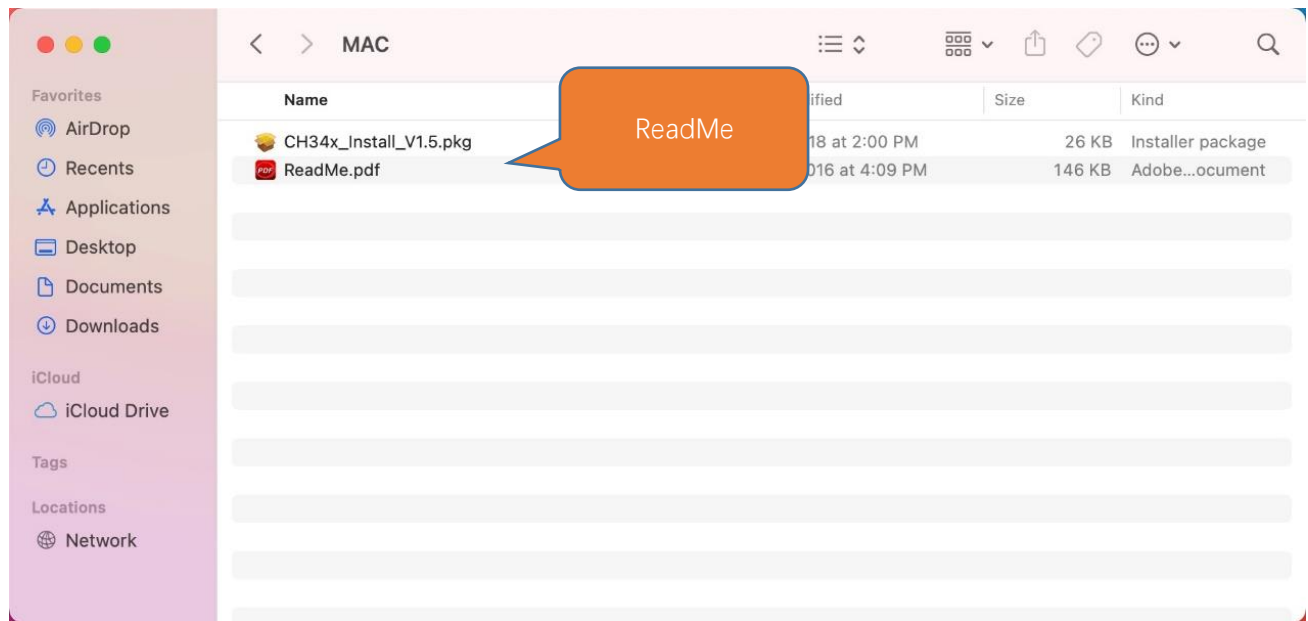
Then, waiting Finish.



Finally, restart your PC.



If you still haven't installed the CH340 by following the steps above, you can view readme.pdf to install it.



0.4 Burning Micropython Firmware (Important)

To run Python programs on ESP32, we need to burn a firmware to ESP32 first.

Downloading Micropython Firmware

Official website of microPython: <http://micropython.org/>

Webpage listing firmware of microPython for ESP32: <https://micropython.org/download/esp32spiram/>

Firmware

Releases

v1.18 (2022-01-17) .bin [.elf] [.map] [Release notes] (latest)
v1.17 (2021-09-02) .bin [.elf] [.map] [Release notes]
v1.16 (2021-06-23) .bin [.elf] [.map] [Release notes]
v1.15 (2021-04-18) .bin [.elf] [.map] [Release notes]
v1.14 (2021-02-02) .bin [.elf] [.map] [Release notes]
v1.13 (2020-09-02) .bin [.elf] [.map] [Release notes]
v1.12 (2019-12-20) .bin [.elf] [.map] [Release notes]

Nightly builds

v1.18-382-g014912daa (2022-04-27) .bin [.elf] [.map]
v1.18-377-geb9674822 (2022-04-26) .bin [.elf] [.map]
v1.18-370-g28e7e15c0 (2022-04-22) .bin [.elf] [.map]
v1.18-366-gef1c2cdab (2022-04-21) .bin [.elf] [.map]

Firmware (Compiled with IDF 3.x)

Releases

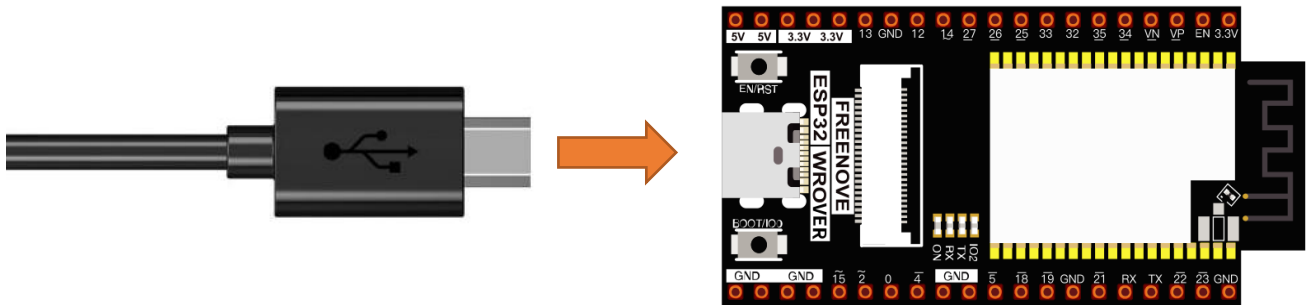
v1.14 (2021-02-02) .bin [.elf] [.map] [Release notes] (latest)
v1.13 (2020-09-02) .bin [.elf] [.map] [Release notes]
v1.12 (2019-12-20) .bin [.elf] [.map] [Release notes]
v1.11 (2019-05-29) .bin [.elf] [.map] [Release notes]
v1.10 (2019-01-25) .bin [.elf] [.map] [Release notes]
v1.9.4 (2018-05-11) .bin [.elf] [.map] [Release notes]

Firmware used in this tutorial is **esp32spiram-20220117-v1.17.bin**

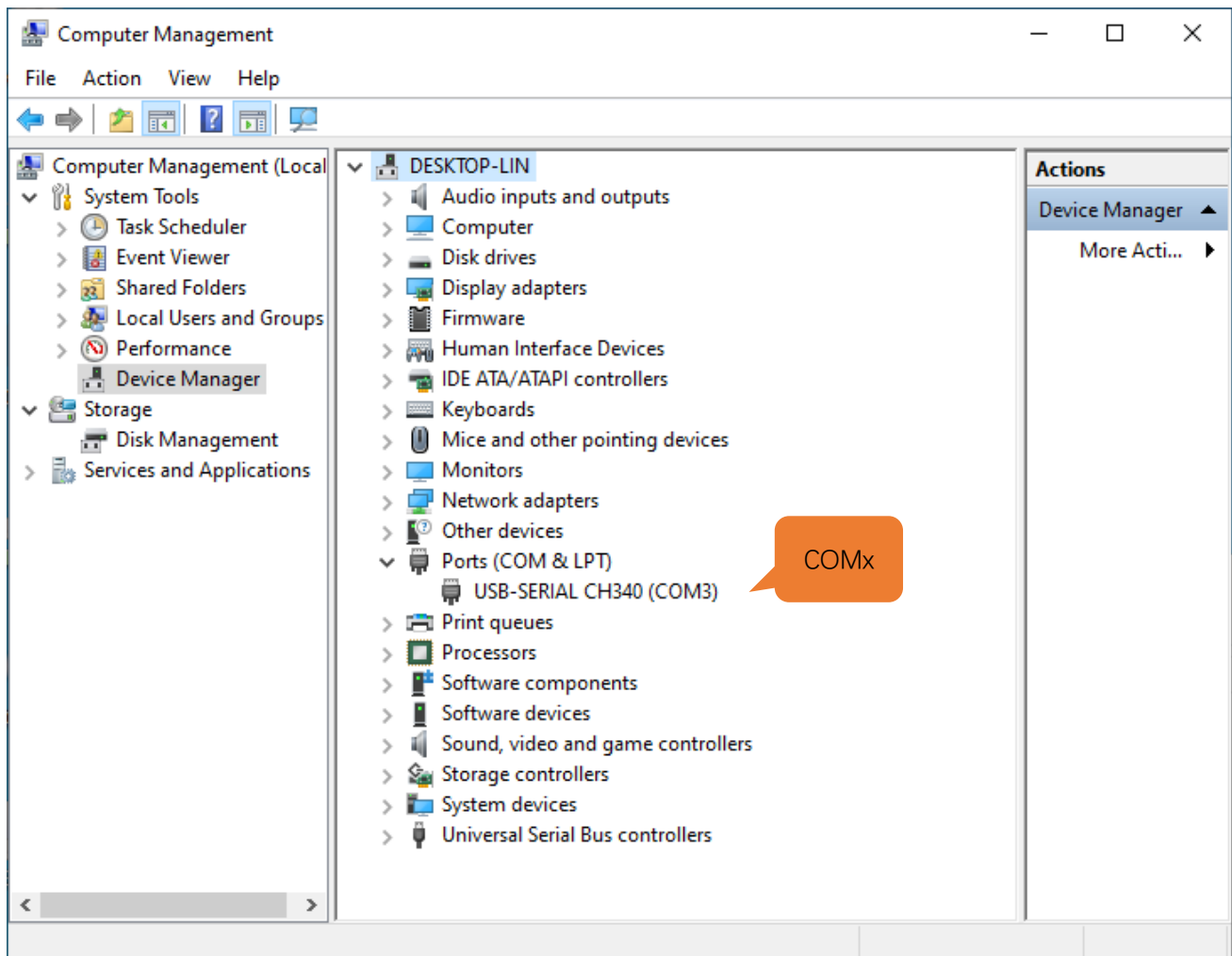
This file is also provided in our data folder "**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Firmware**".

Burning a Micropython Firmware

Connect your computer and ESP32 with a USB cable.

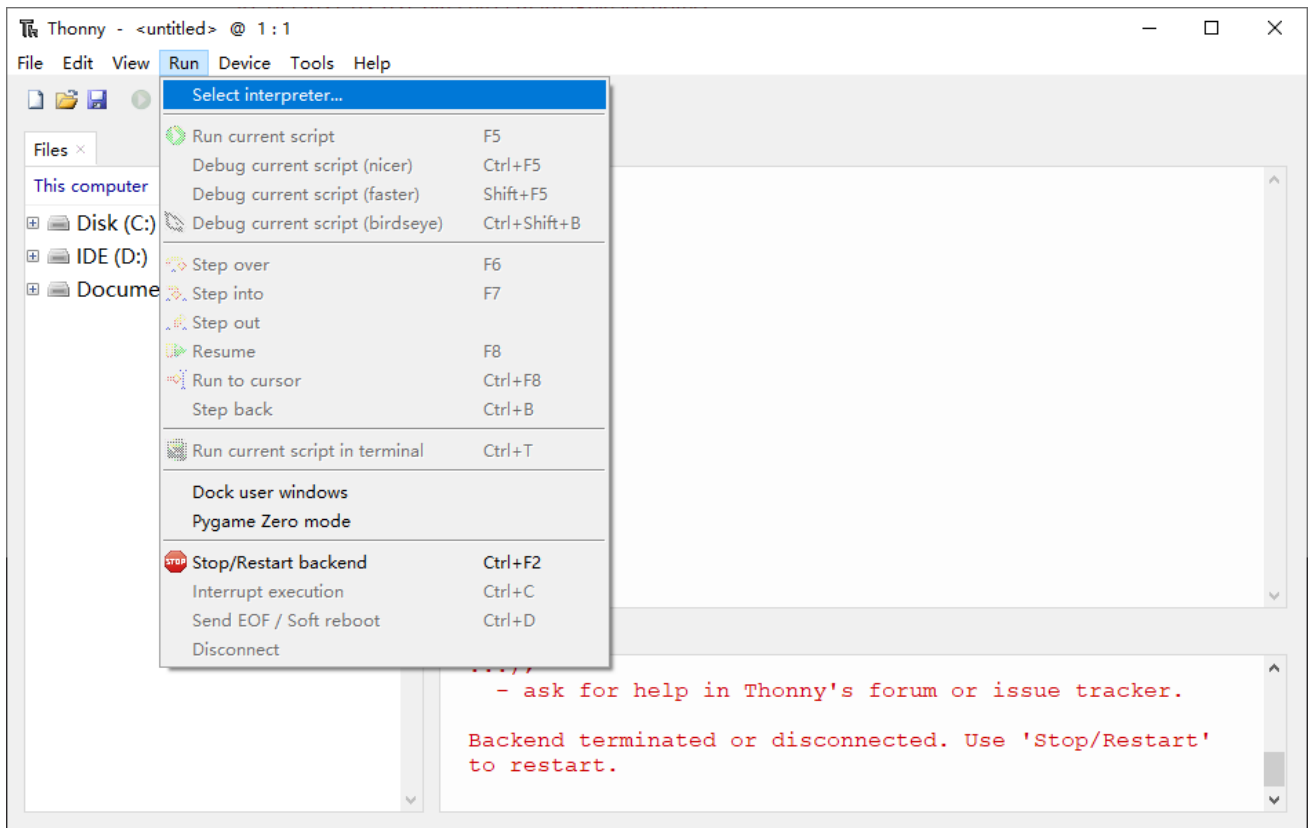


Make sure that the driver has been installed successfully and that it can recognize COM port correctly. Open device manager and expand "Ports".

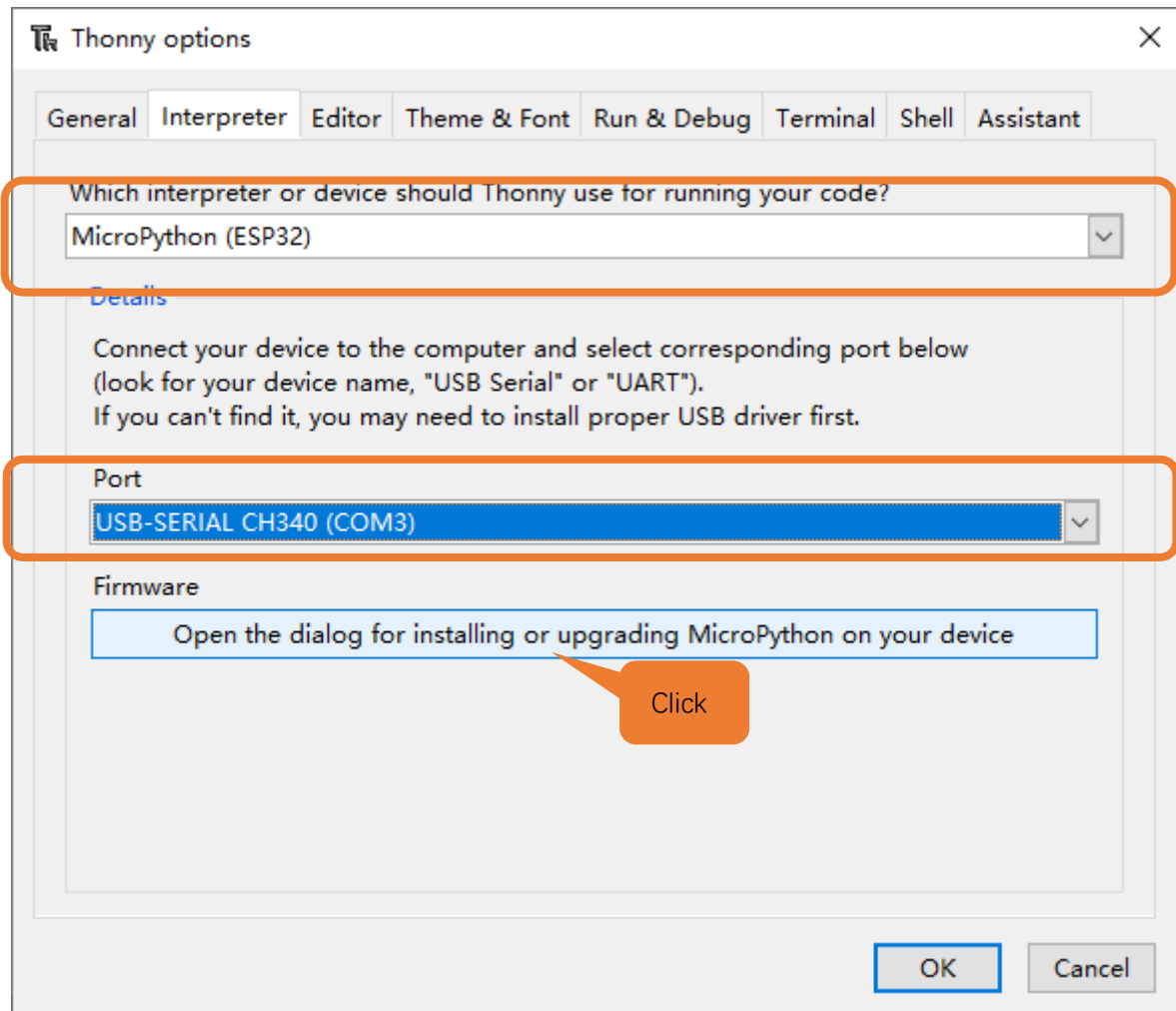


Note: the port of different people may be different, which is a normal situation.

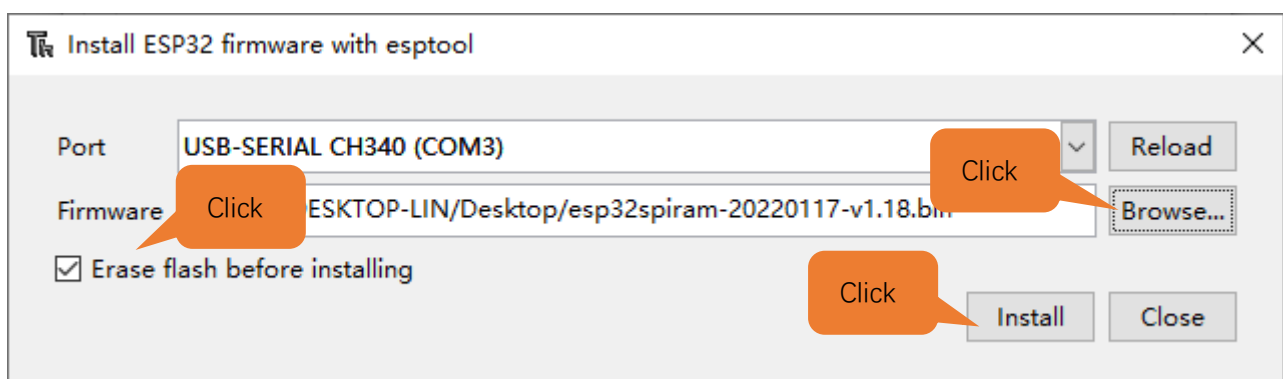
1. Open Thonny, click “run” and select “Select interpreter...”



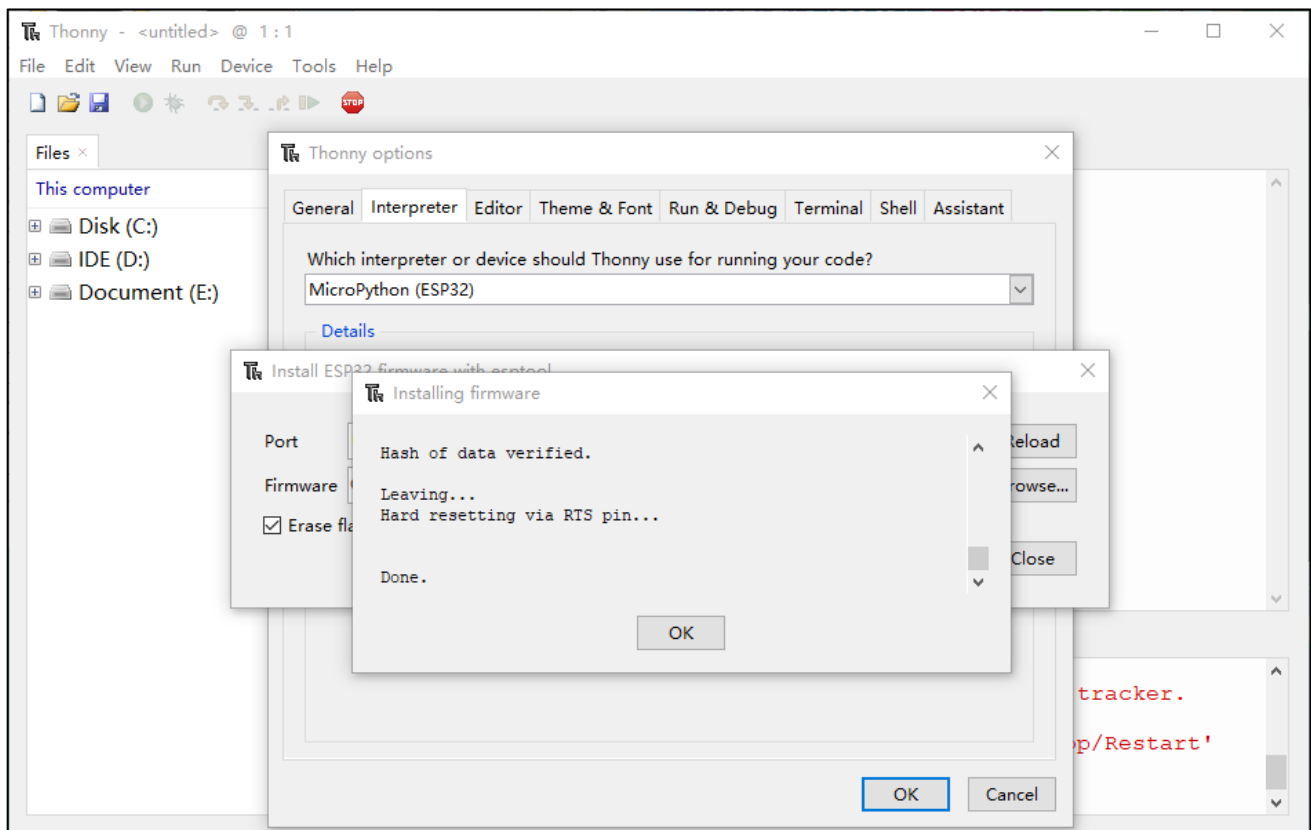
2. Select "Micropython (ESP32)", select "USB-SERIAL CH340 (COM4)", and then click the long button under "Firmware".



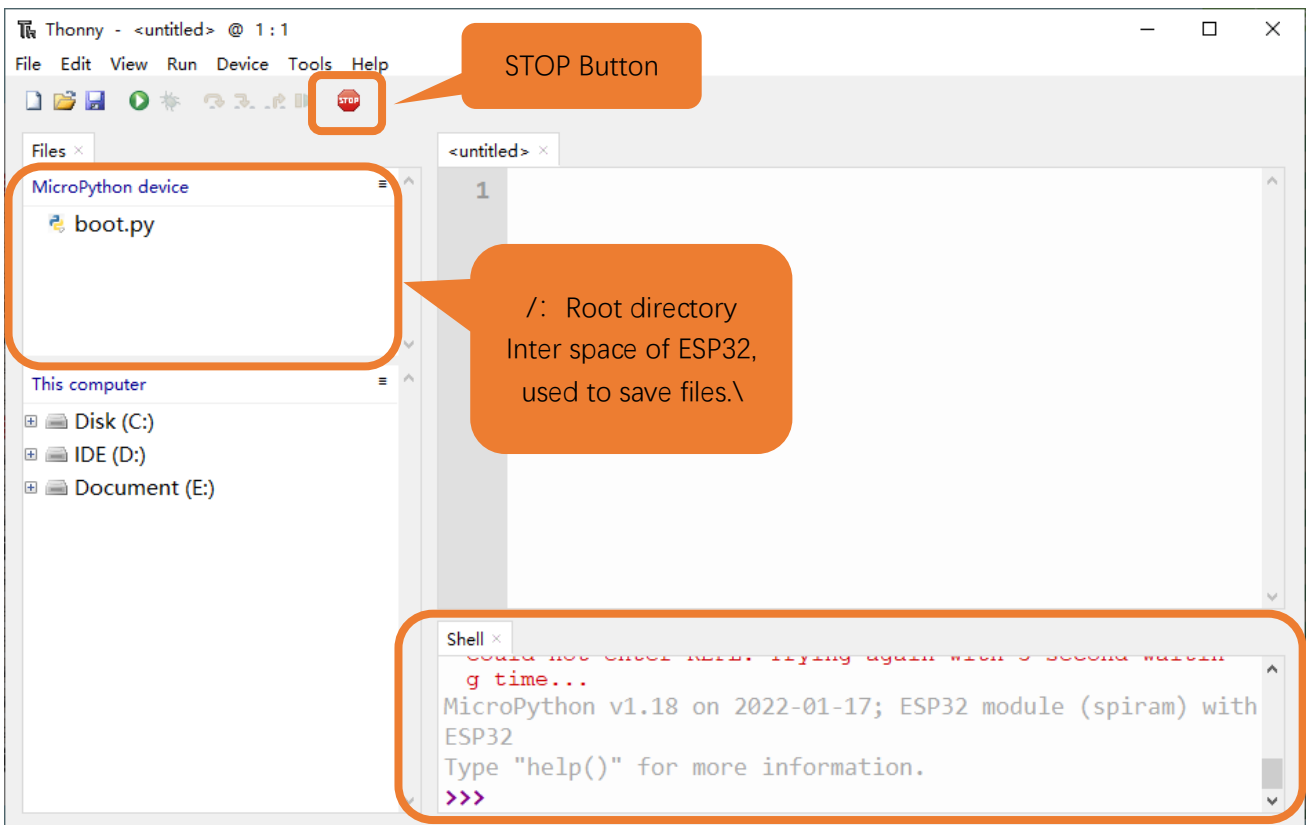
3. The following dialog box pops up. Select "USB-SERIAL CH340 (COM3)" for "Port" and then click "Browse...". Select the previous prepared microPython firmware "**esp32spiram-20220117-v1.17.bin**". Check "Erase flash before installing" and click "install" to wait for the prompt of finishing installation.



4. Wait for the installation to be done.



5. Close all dialog boxes, turn to main interface and click "STOP". As shown in the illustration below

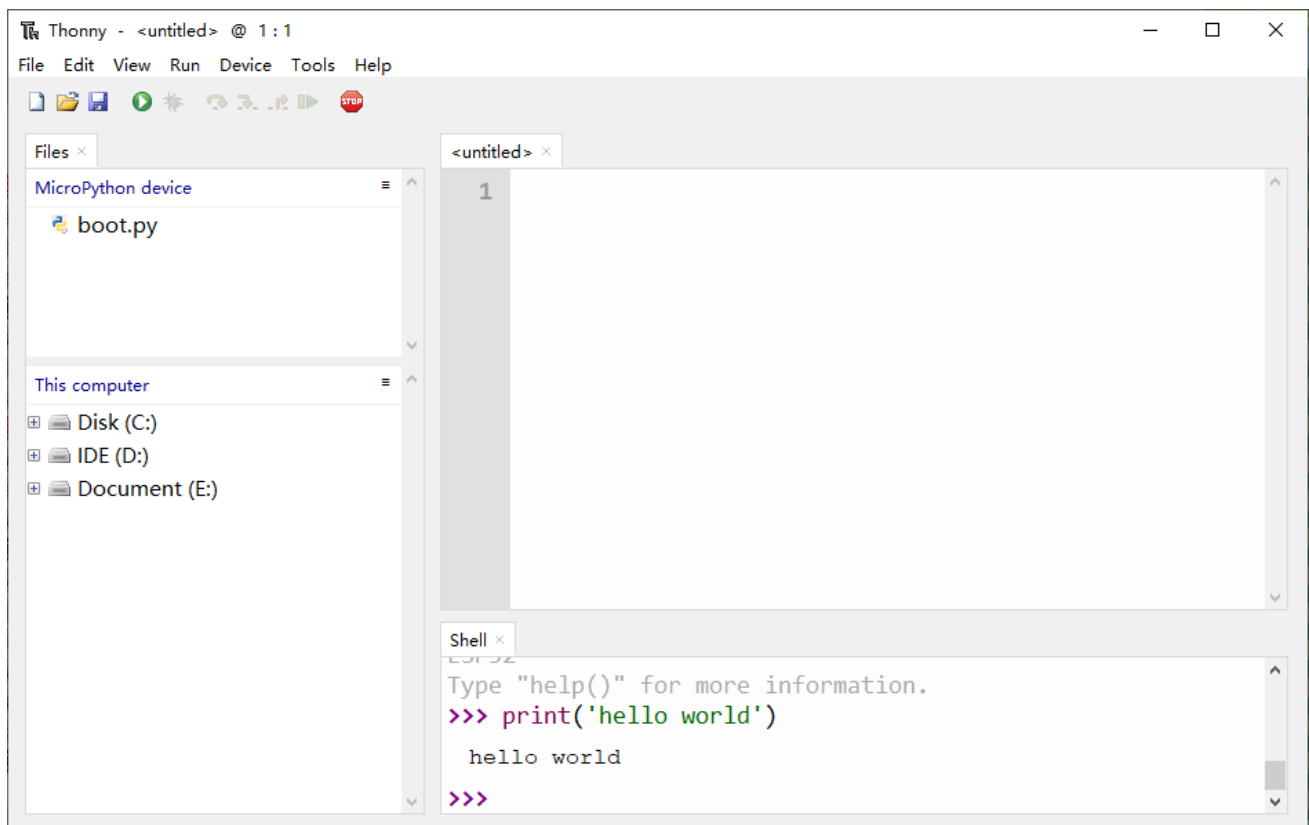


6. So far, all the preparations have been made.

0.5 Testing codes (Important)

Testing Shell Command

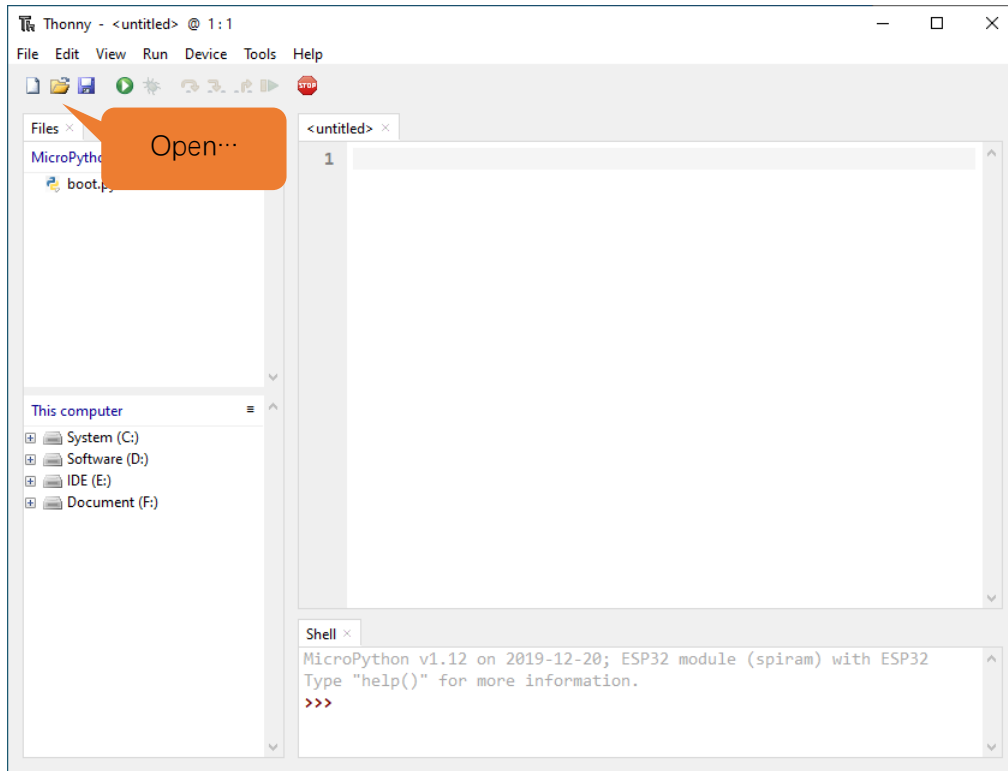
Enter `print('hello world')` in "Shell" and press Enter.



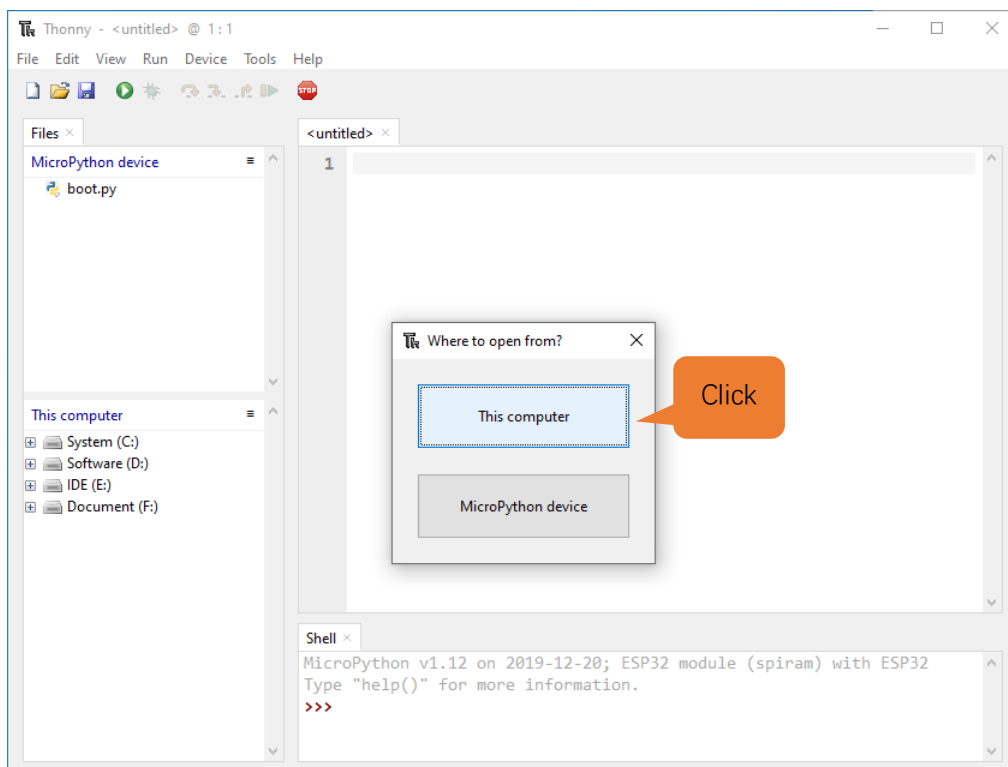
Running Online

ESP32 needs to be connected to a computer when it is run online. Users can use Thonny to write and debug programs.

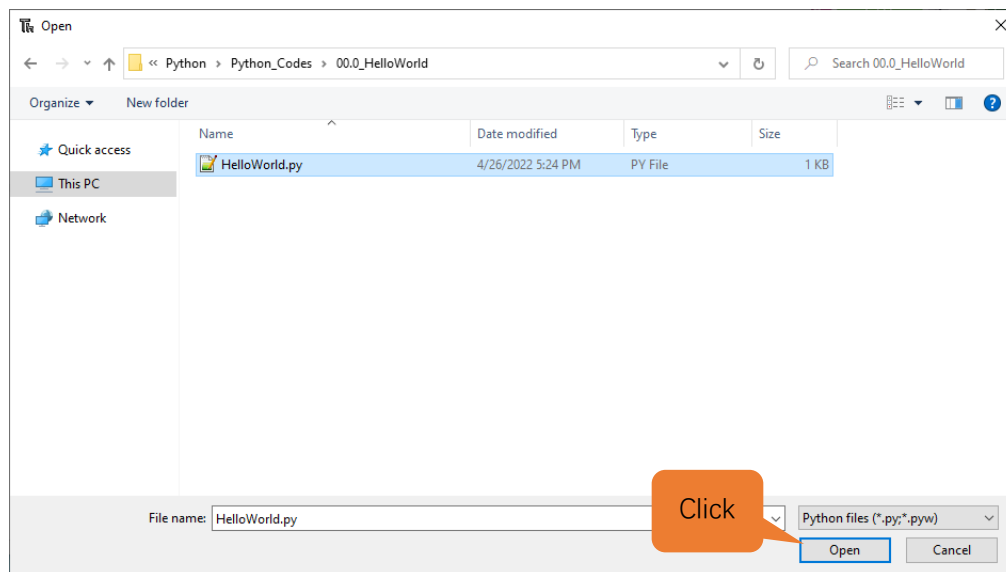
1. Open Thonny and click “Open…”.



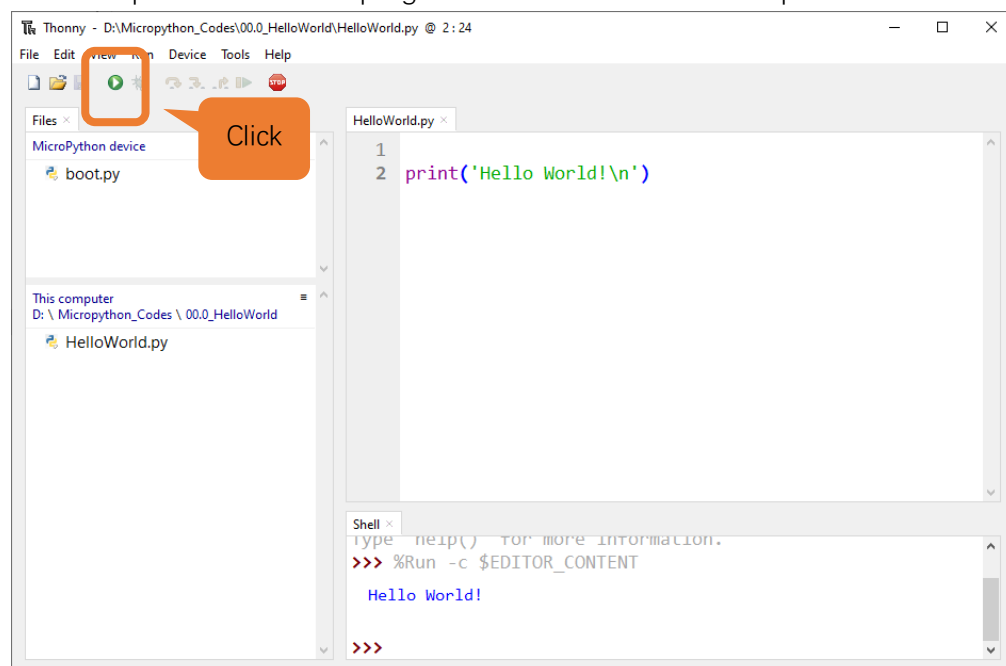
2. On the newly pop-up window, click “This computer”.



In the new dialog box, select “HelloWorld.py” in “Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes/00.0_HelloWorld” folder.



Click “Run current script” to execute the program and “Hello World” will be printed in “Shell”.

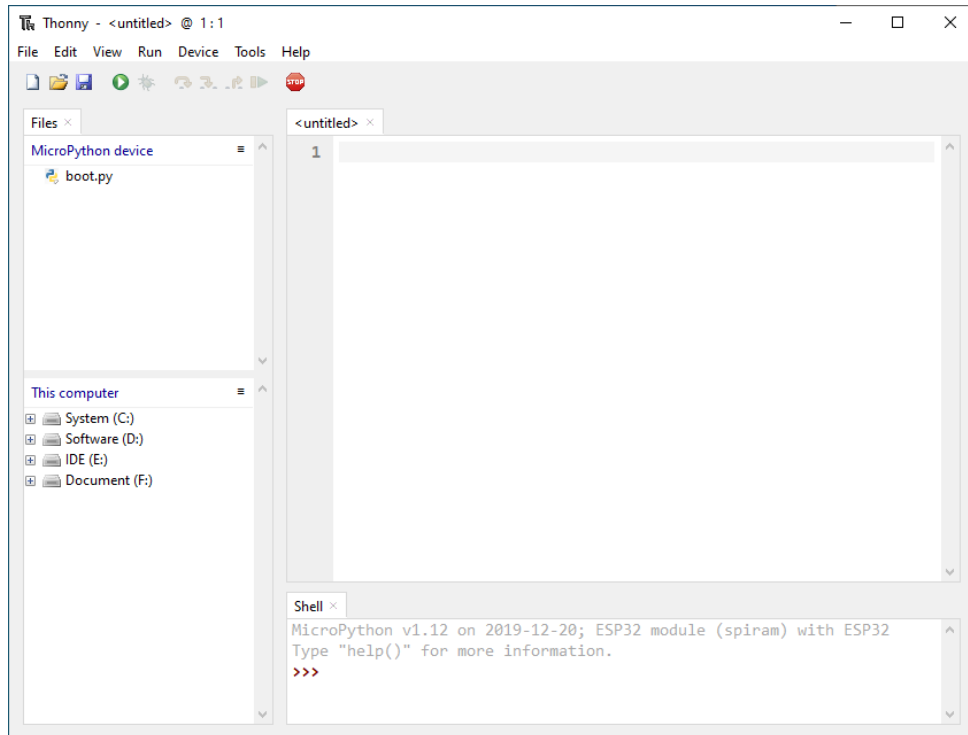


Note: When running online, if you press the reset key of ESP32, user’s code will not be executed again. If you wish to run the code automatically after resetting the code, please refer to the following [Running Offline](#).

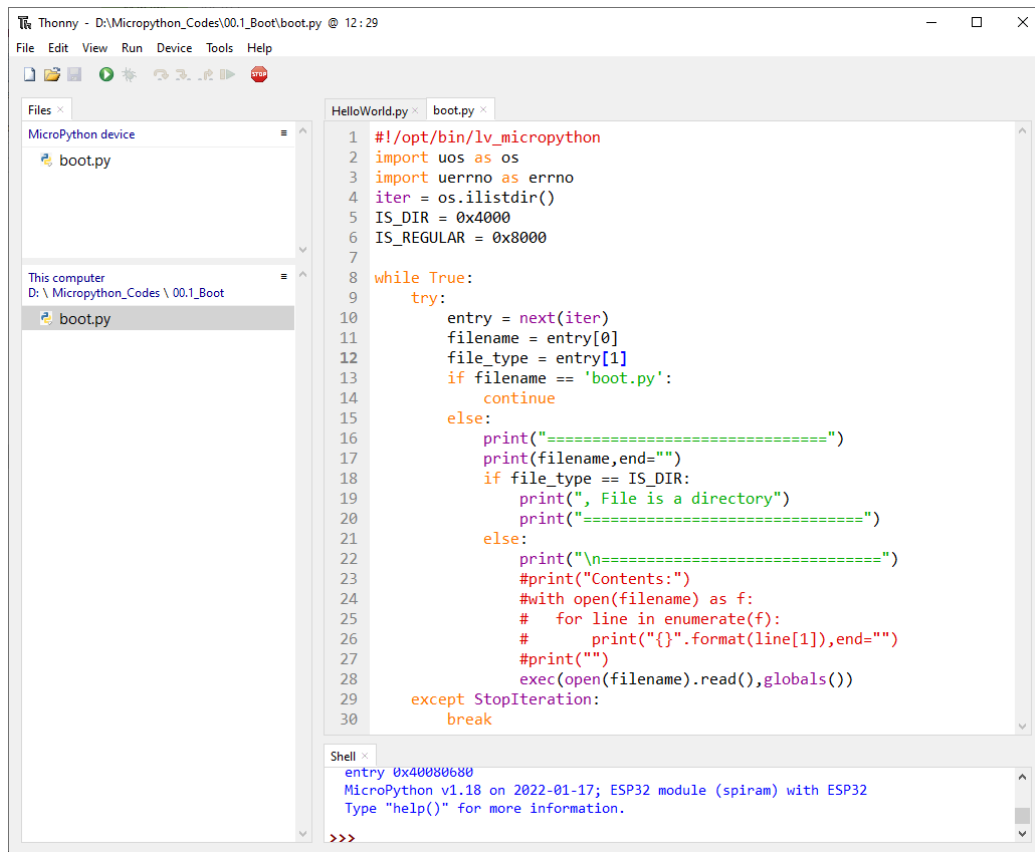
Running Offline (Importance)

After ESP32 is reset, it runs the file `boot.py` in root directory first and then runs file `main.py`, and finally, it enters “Shell”. Therefore, to make ESP32 execute user’s programs after resetting, we need to add a guiding program in `boot.py` to execute user’s code.

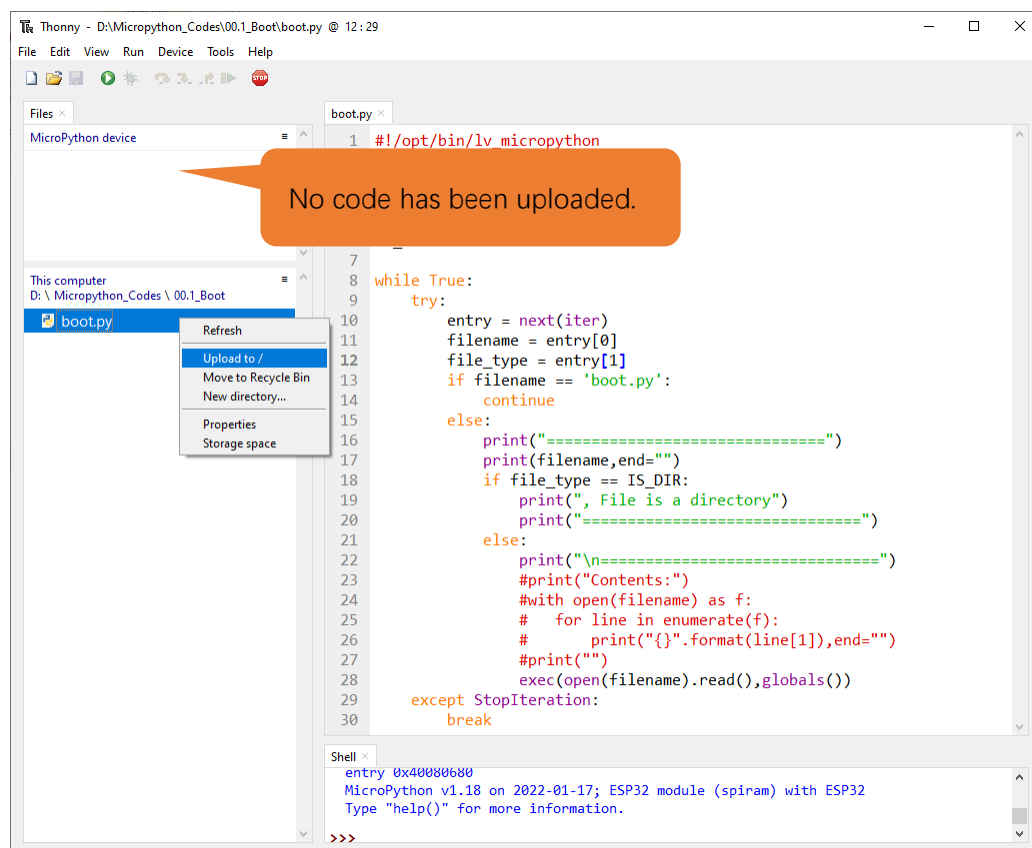
1. Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”. Open “Thonny”。



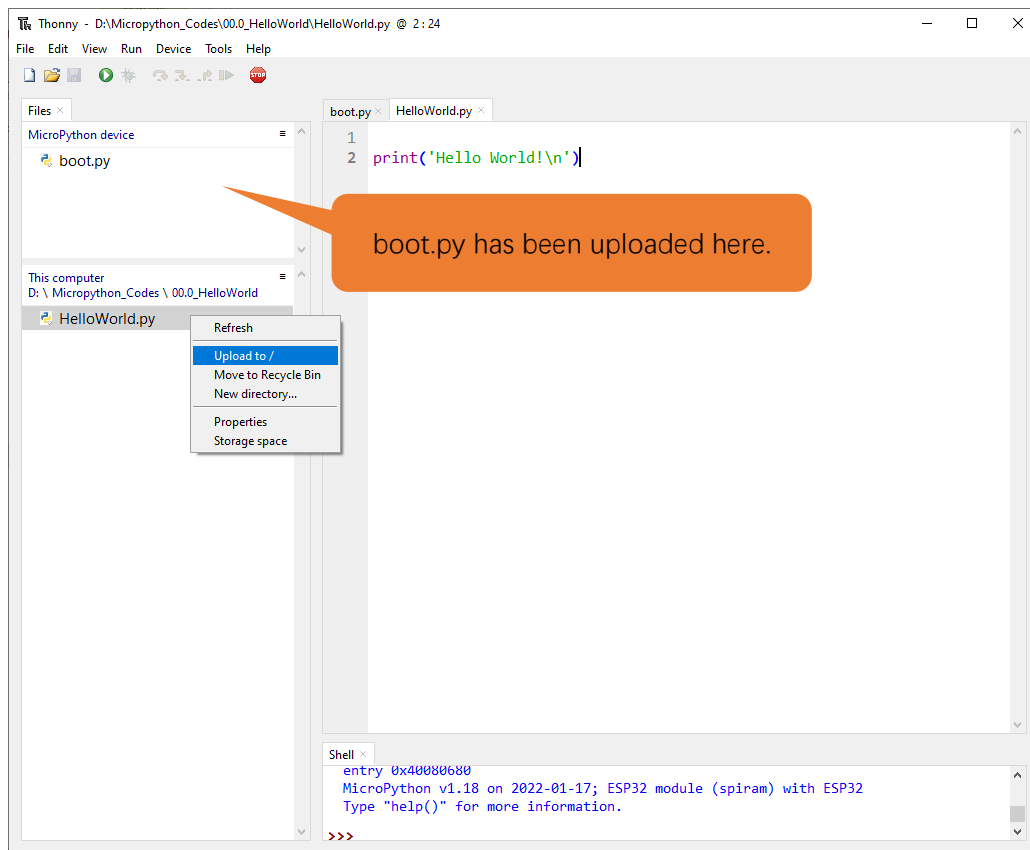
2. Expand “00.1_Boot” in the “Micropython_Codes” in the directory of disk(D), and double-click `boot.py`, which is provided by us to enable programs in “MicroPython device” to run offline.



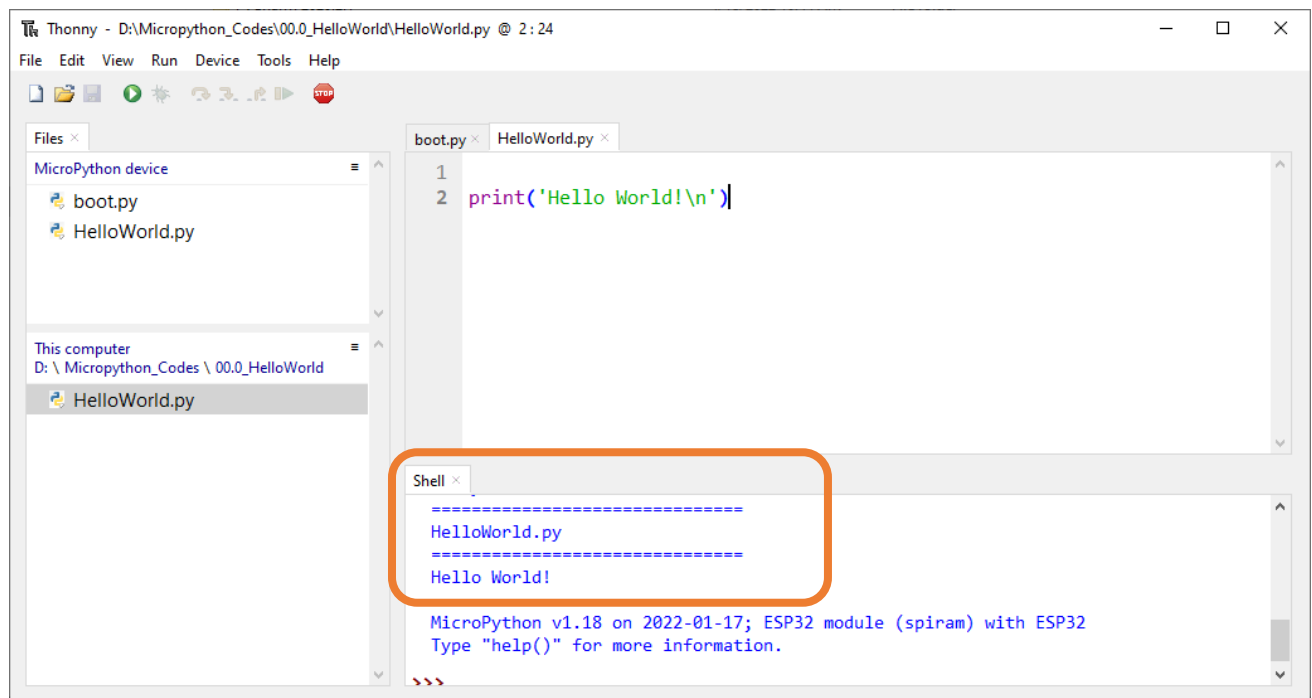
If you want your written programs to run offline, you need to upload `boot.py` we provided and all your codes to "MicroPython device" and press ESP32's reset key. Here we use programs 00.0 and 00.1 as examples. Select "boot.py", right-click to select "Upload to /".



Similarly, upload “HelloWorld.py” to “MicroPython device”.



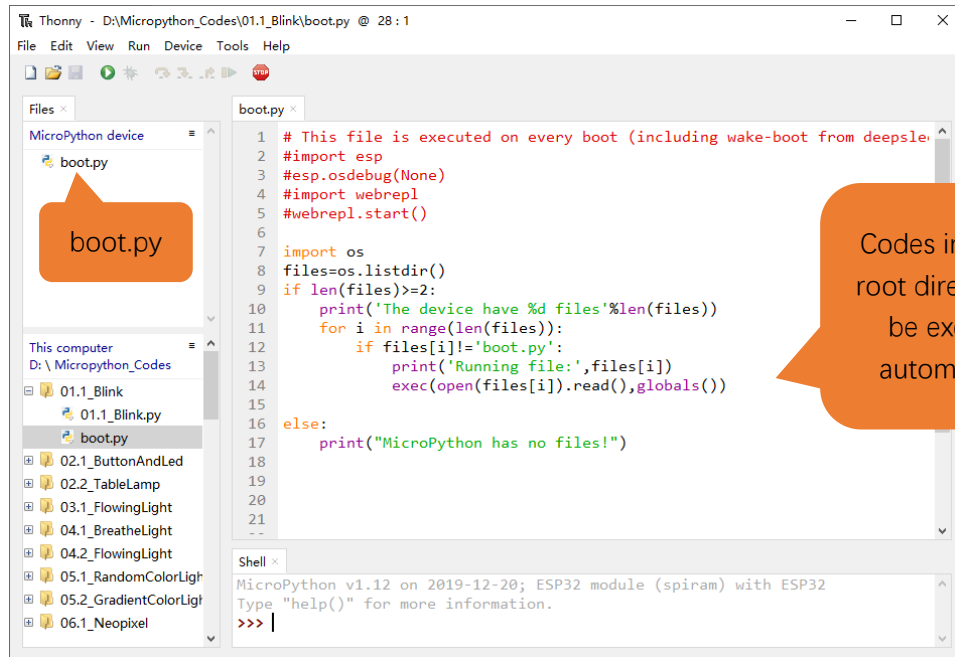
3. Press the reset key and in the box of the illustration below, you can see the code is executed.



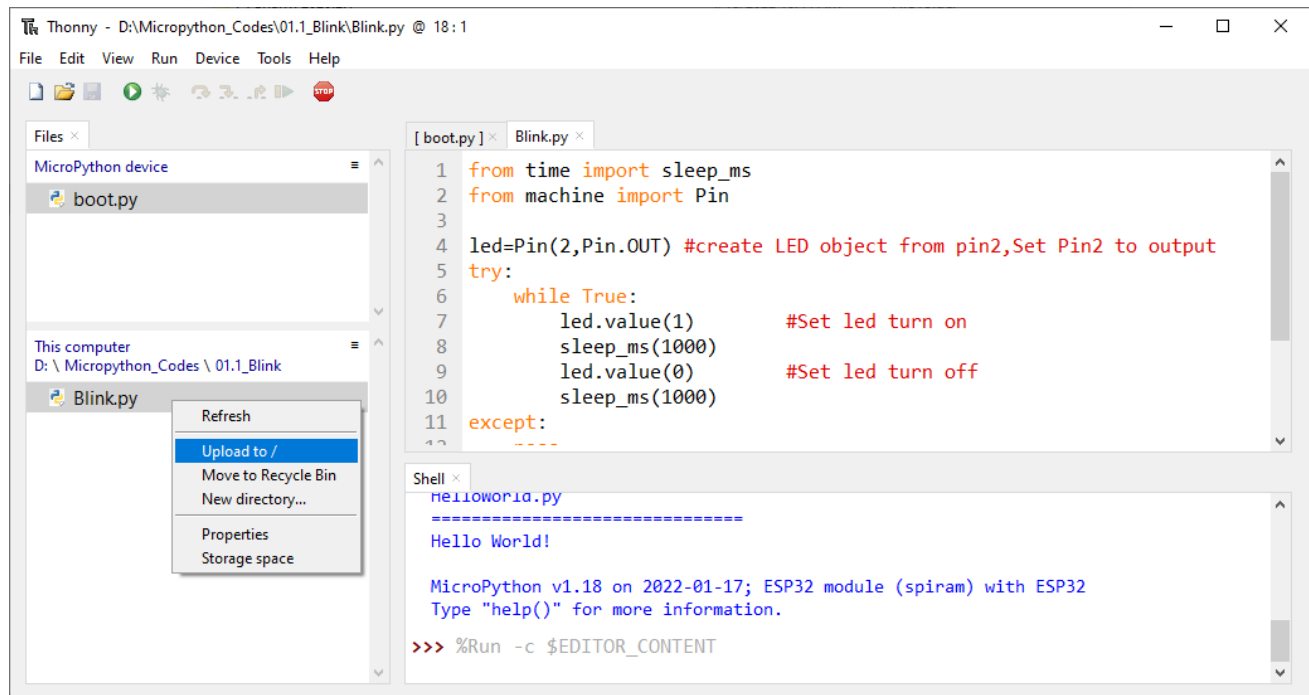
0.6 Thonny Common Operation

Uploading Code to ESP32

Each time when ESP32 restarts, if there is a “boot.py” in the root directory, it will execute this code first.

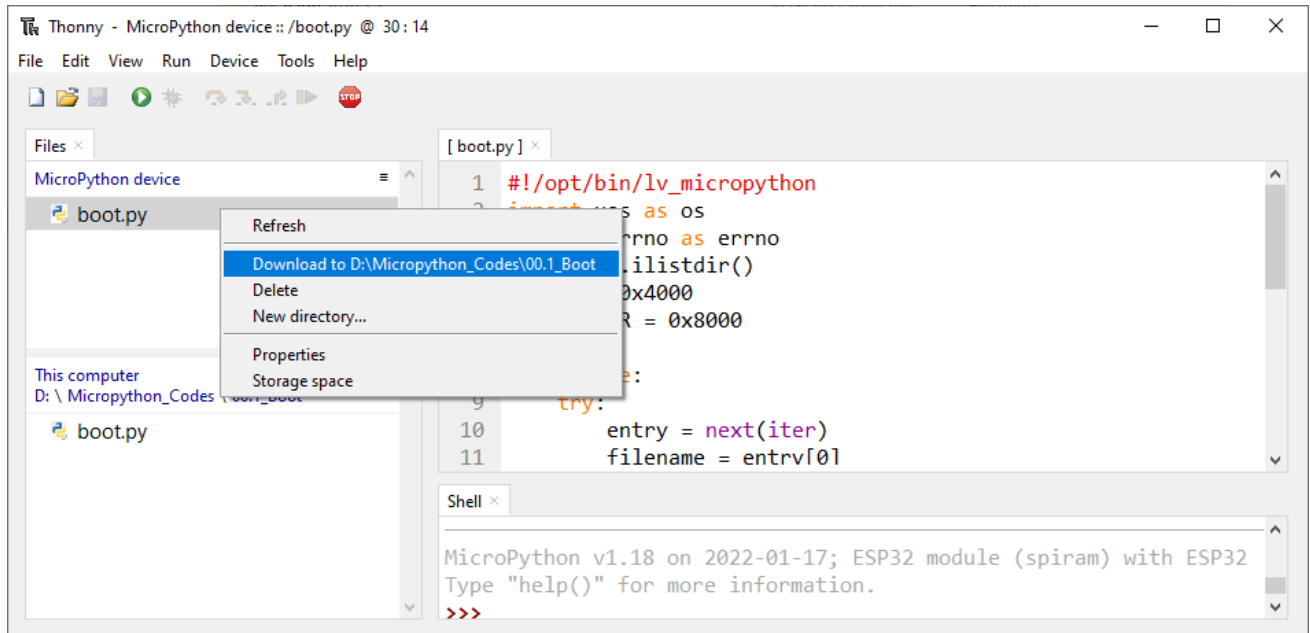


Select “Blink.py” in “01.1_Blink”, right-click your mouse and select “Upload to /” to upload code to ESP32's root directory.



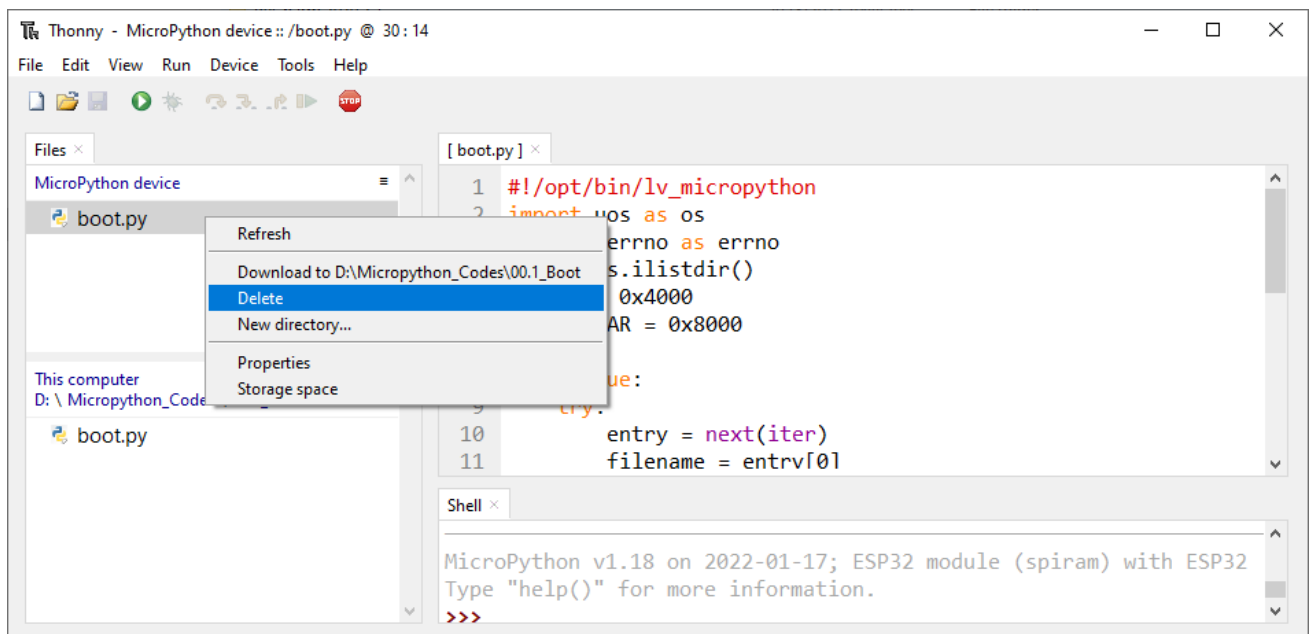
Downloading Code to Computer

Select “boot.py” in “MicroPython device”, right-click to select “Download to ...” to download the code to your computer.



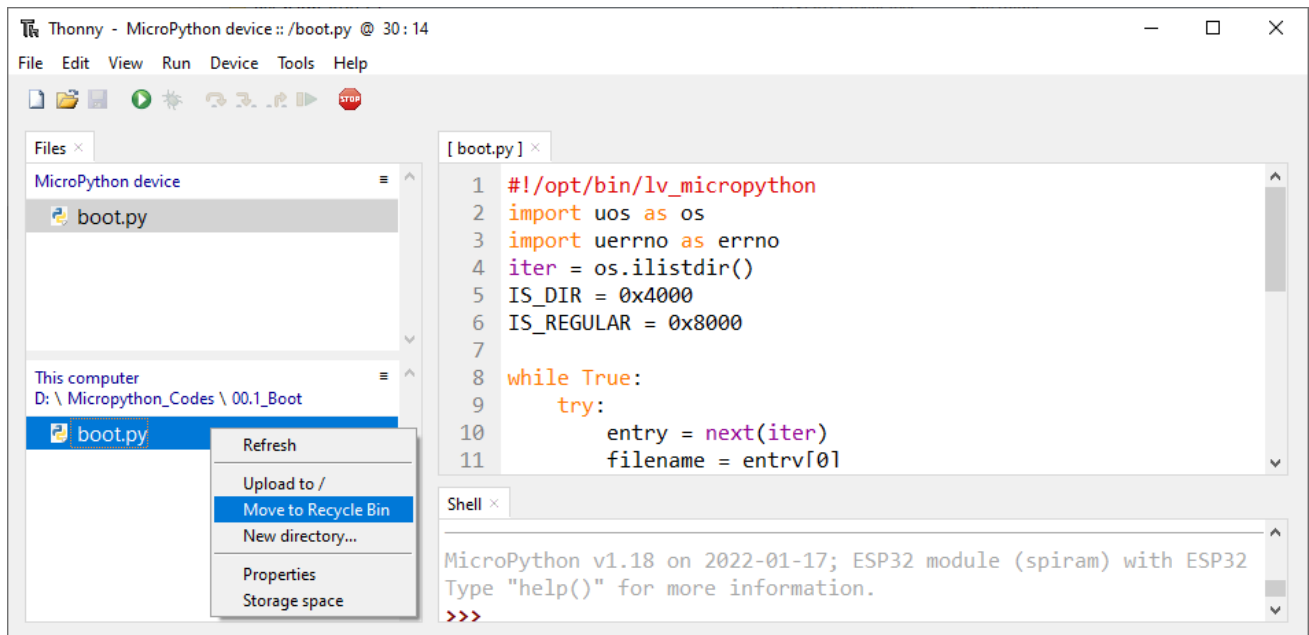
Deleting Files from ESP32's Root Directory

Select “boot.py” in “MicroPython device”, right-click it and select “Delete” to delete “boot.py” from ESP32's root directory.



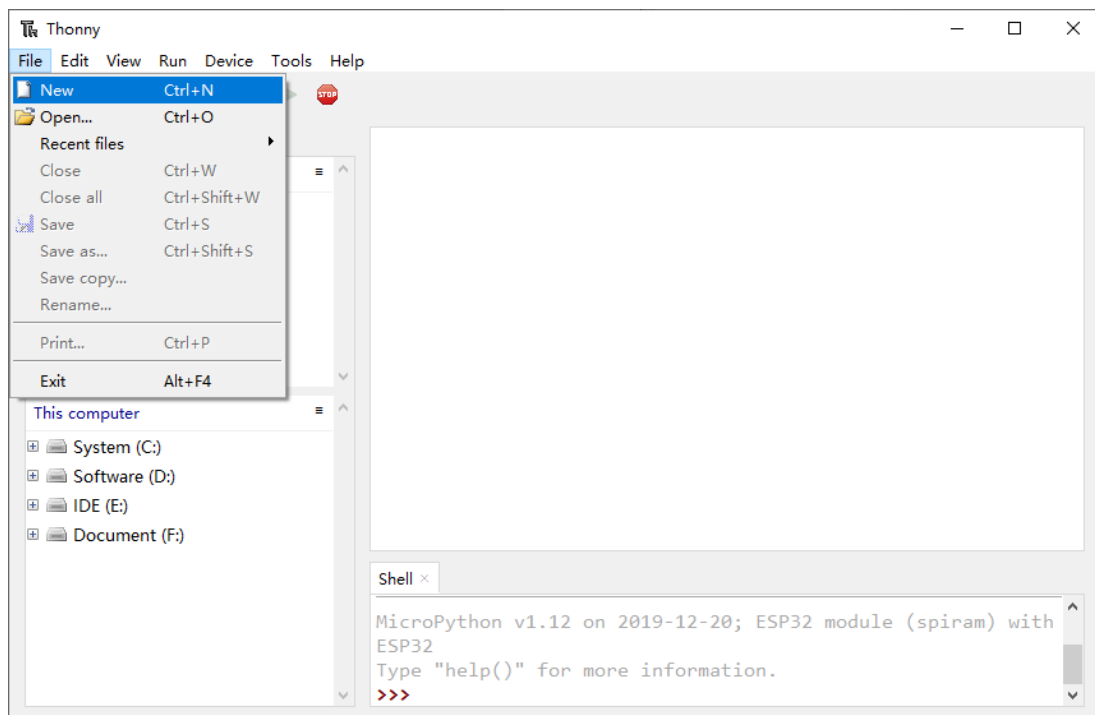
Deleting Files from your Computer Directory

Select “boot.py” in “00.1_Boot”, right-click it and select “Move to Recycle Bin” to delete it from “00.1_Boot”.

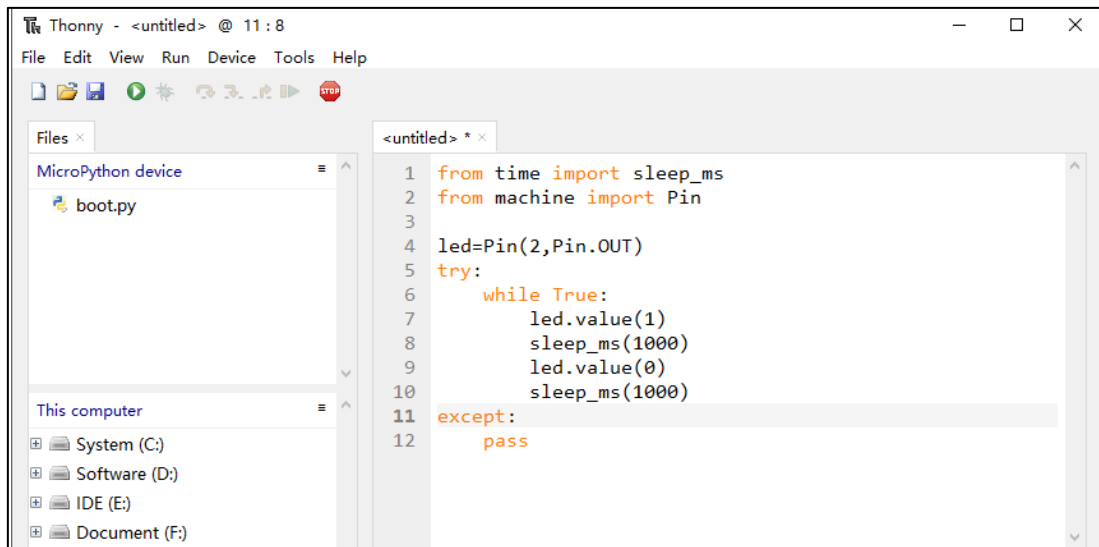


Creating and Saving the code

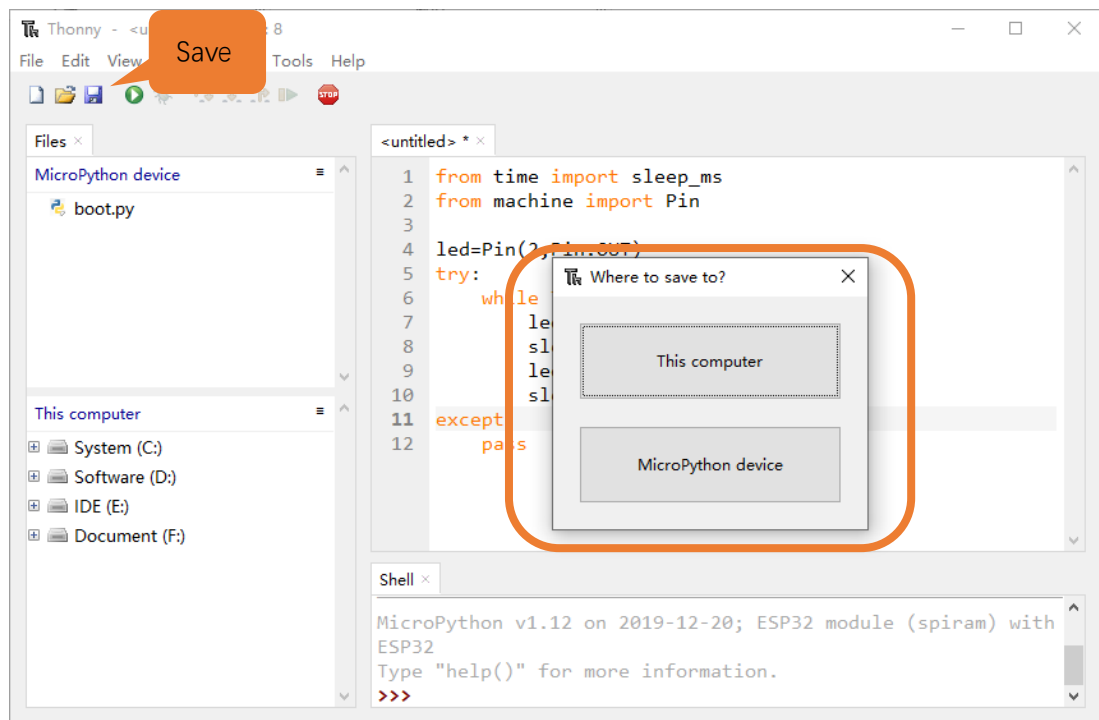
Click “File”→“New” to create and write codes.



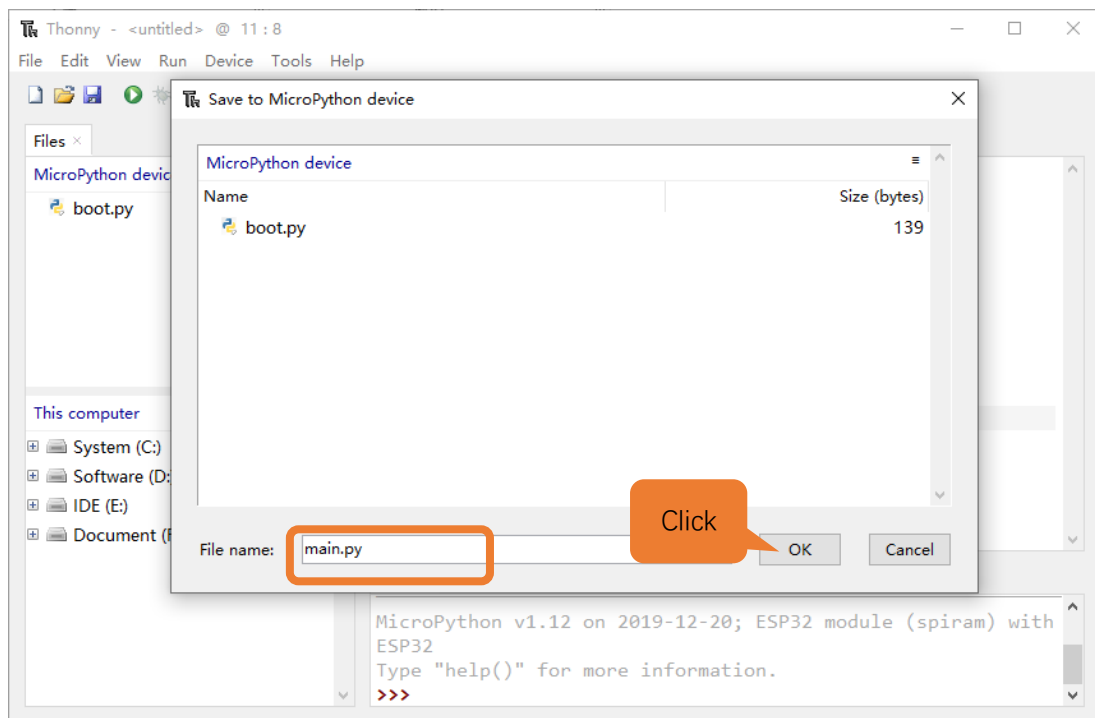
Enter codes in the newly opened file. Here we use codes of “01.1_Blink.py” as an example.



Click “Save” on the menu bar. You can save the codes either to your computer or to ESP32-WROVER.



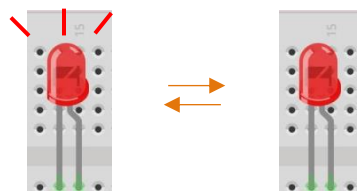
Select "MicroPython device", enter "main.py" in the newly pop-up window and click "OK".



You can see that codes have been uploaded to ESP32-WROVER.



Disconnect and reconnect USB cable, and you can see that LED is ON for one second and then OFF for one second, which repeats in an endless loop.



0.7 Note

Though there are many pins available on ESP32, some of them have been connected to peripheral equipment, so we should avoid using such pins to prevent pin conflicts. For example, when downloading programs, make sure that the pin state of Strapping Pin, when resetting, is consistent with the default level; do NOT use Flash Pin; Do NOT use Cam Pin when using Camera function.

Strapping Pin

The state of Strapping Pin can affect the functions of ESP32 after it is reset, as shown in the table below.

Voltage of Internal LDO (VDD_SDIO)					
Pin	Default	3.3 V		1.8 V	
MTDI	Pull-down	0		1	
Bootling Mode					
Pin	Default	SPI Boot		Download Boot	
GPIO0	Pull-up	1		0	
GPIO2	Pull-down	Don't-care		0	
Enabling/Disabling Debugging Log Print over U0TXD During Bootling					
Pin	Default	U0TXD Active		U0TXD Silent	
MTDO	Pull-up	1		0	
Timing of SDIO Slave					
Pin	Default	Falling-edge Sampling Falling-edge Output	Falling-edge Sampling Rising-edge Output	Rising-edge Sampling Falling-edge Output	Rising-edge Sampling Rising-edge Output
MTDO	Pull-up	0	0	1	1
GPIO5	Pull-up	0	1	0	1

If you have any difficulties or questions with this tutorial or toolkit, feel free to ask for our quick and free technical support through support@freenove.com at any time.

or check: https://www.espressif.com/sites/default/files/documentation/esp32-wrover_datasheet_en.pdf

Flash Pin

GPIO6-11 has been used to connect the integrated SPI flash on the module, and is used when GPIO 0 is power on and at high level. Flash is related to the operation of the whole chip, so the external pin GPIO6-11 cannot be used as an experimental pin for external circuits, otherwise it may cause errors in the operation of the program.

GPIO16-17 has been used to connect the integrated PSRAM on the module.

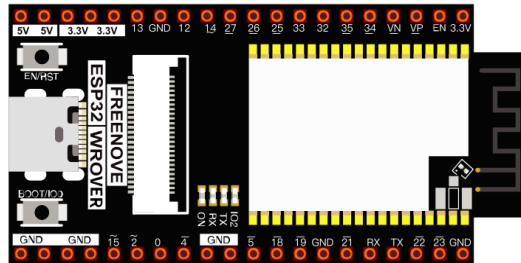
Because of external pull-up, MTDI pin is not suggested to be used as a touch sensor. For details, please refer to Peripheral Interface and Sensor chapter in "[ESP32 Data Sheet](#)".

For more relevant information, please click:

https://www.espressif.com/sites/default/files/documentation/esp32-wrover_datasheet_en.pdf.

Cam Pin

When using the cam camera of our ESP32-WROVER, please check the pins of it. Pins with underlined numbers are used by the cam camera function, if you want to use other functions besides it, please avoid using them.



CAM_Pin	GPIO_pin
I2C_SDA	GPIO26
I2C_SCL	GPIO27
CSI_VYSNC	GPIO25
CSI_HREF	GPIO23
CSI_Y9	GPIO35
XCLK	GPIO21
CSI_Y8	GPIO34
CSI_Y7	GPIO39
CSI_PCLK	GPIO22
CSI_Y6	GPIO36
CSI_Y2	GPIO4
CSI_Y5	GPIO19
CSI_Y3	GPIO5
CSI_Y4	GPIO18

If you have any questions about the information of GPIO, you can click [here](#) to go back to ESP32-WROVER to view specific information about GPIO.

Or check: https://www.espressif.com/sites/default/files/documentation/esp32-wrover_datasheet_en.pdf.

Chapter 1 LED (Important)

This chapter is the Start Point in the journey to build and explore ESP32 electronic projects. We will start with simple “Blink” project.

Project 1.1 Blink

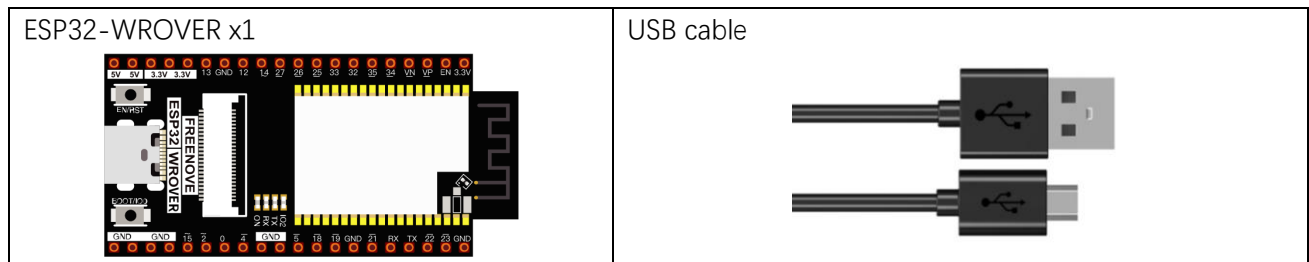
In this project, we will use ESP32 to control blinking a common LED.

If you have not yet installed Thonny, click [here](#).

If you have not yet downloaded Micropython Firmware, click [here](#).

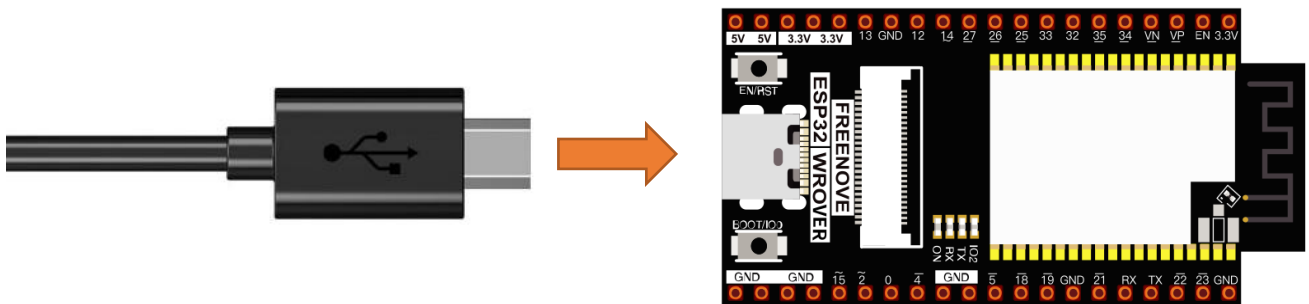
If you have not yet loaded Micropython Firmware, click [here](#).

Component List



Power

ESP32-WROVER needs 5v power supply. In this tutorial, we need connect ESP32-WROVER to computer via USB cable to power it and program it. We can also use other 5v power source to power it.



In the following projects, we only use USB cable to power ESP32-WROVER by default.

In the whole tutorial, we don't use T extension to power ESP32-WROVER. So 5V and 3.3V (including EXT 3.3V) on the extension board are provided by ESP32-WROVER.

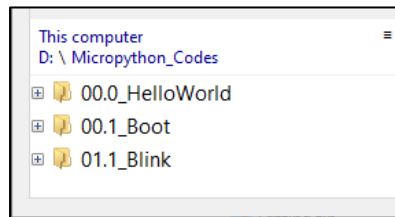
We can also use DC jack of extension board to power ESP32-WROVER. In this way, 5v and EXT 3.3v on extension board are provided by external power resource.

Code

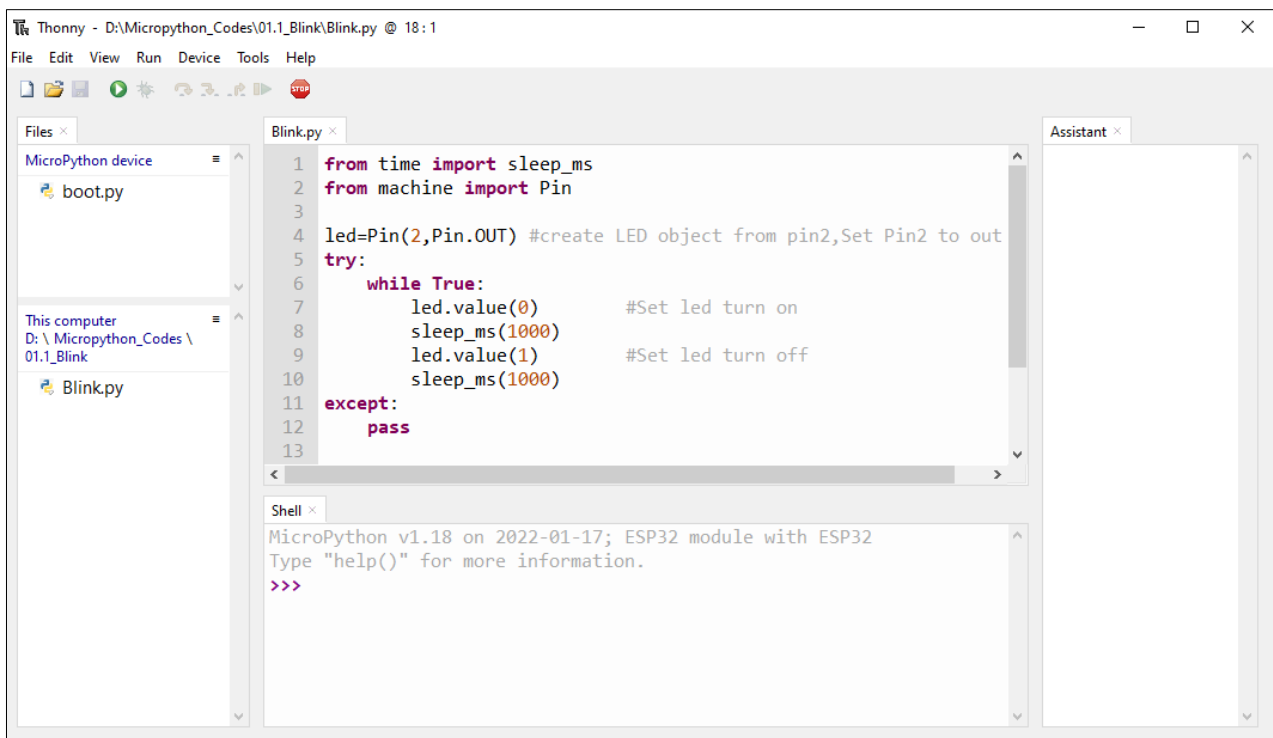
Codes used in this tutorial are saved in “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**”. You can move the codes to any location. For example, we save the codes in Disk(D) with the path of “**D:/Micropython_Codes**”.

01.1_Blink

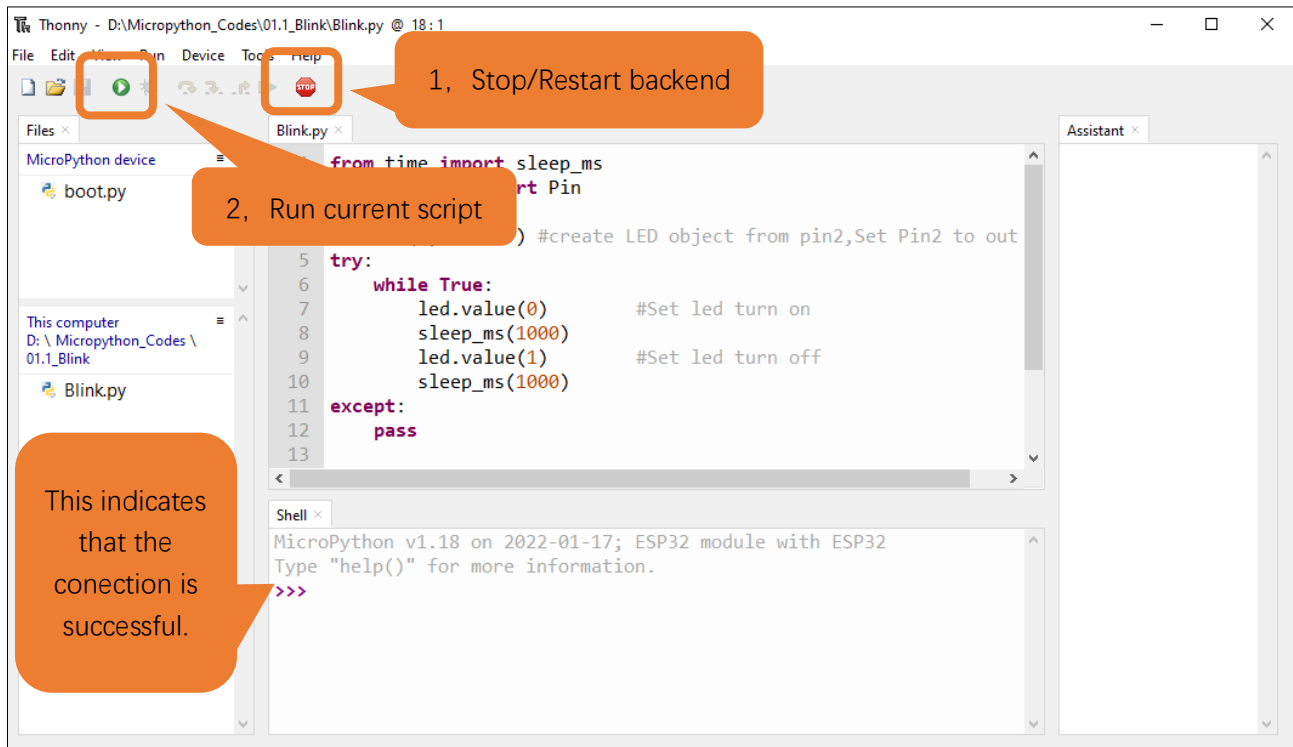
Open “Thonny”, click “This computer”→“D:”→“Micropython_Codes”.



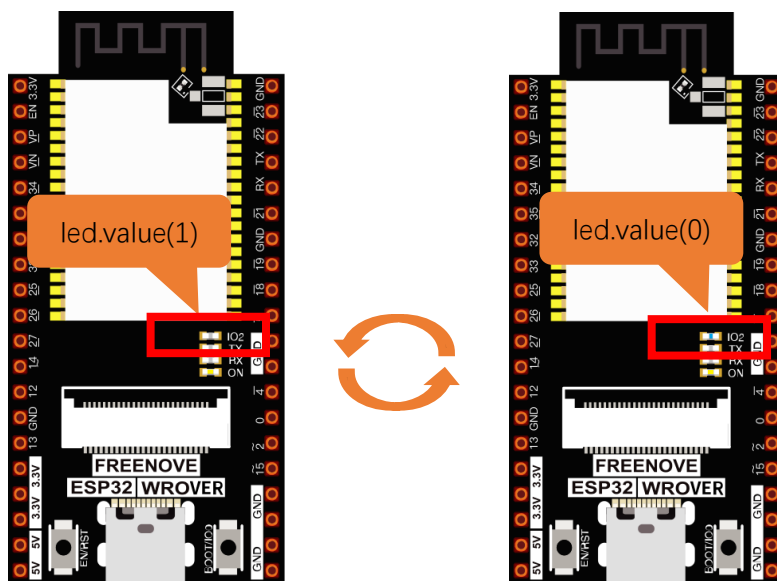
Expand folder “01.1_Blink” and double click “Blink.py” to open it. As shown in the illustration below.



Make sure ESP32 has been connected with the computer with ESP32 correctly. Click “Stop/Restart backend” or press the reset button, and then wait to see what interface will show up.



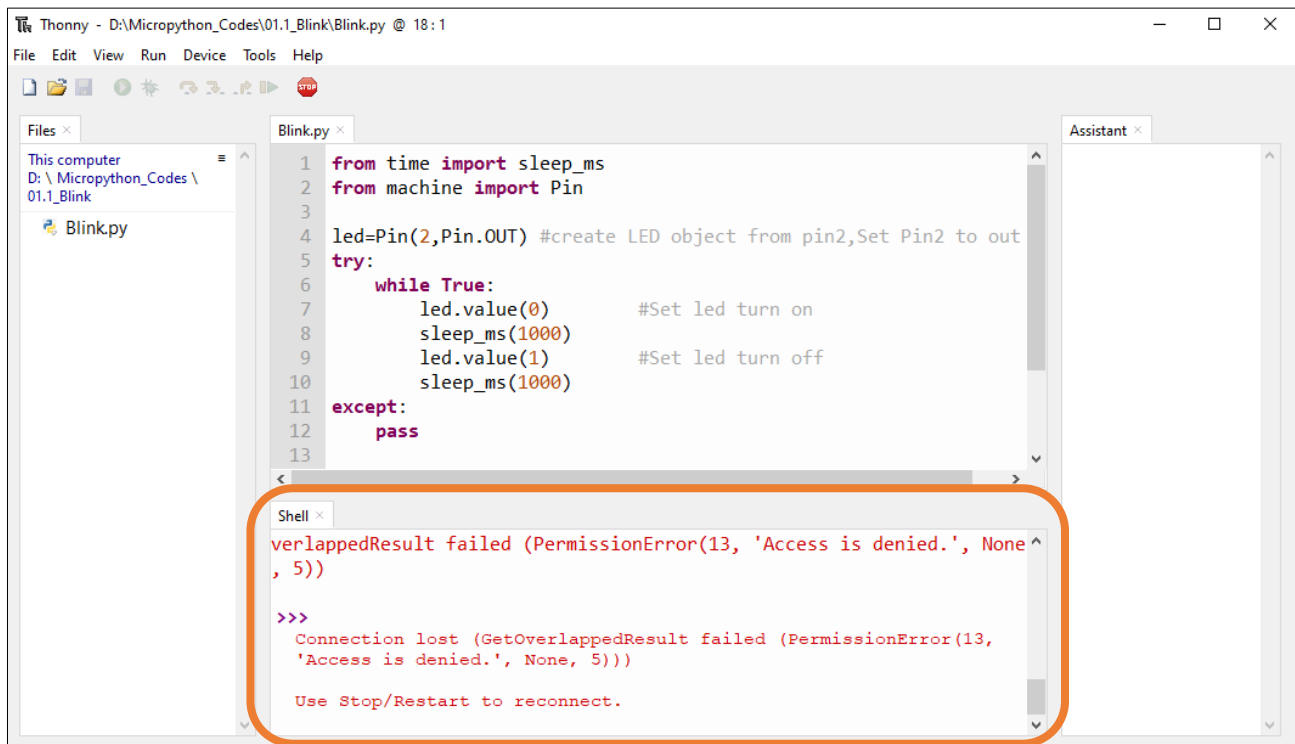
Click “Run current script” shown in the box above, the code starts to be executed and the LED in the circuit starts to blink.



Due to different versions, some board leds will show the opposite effect.

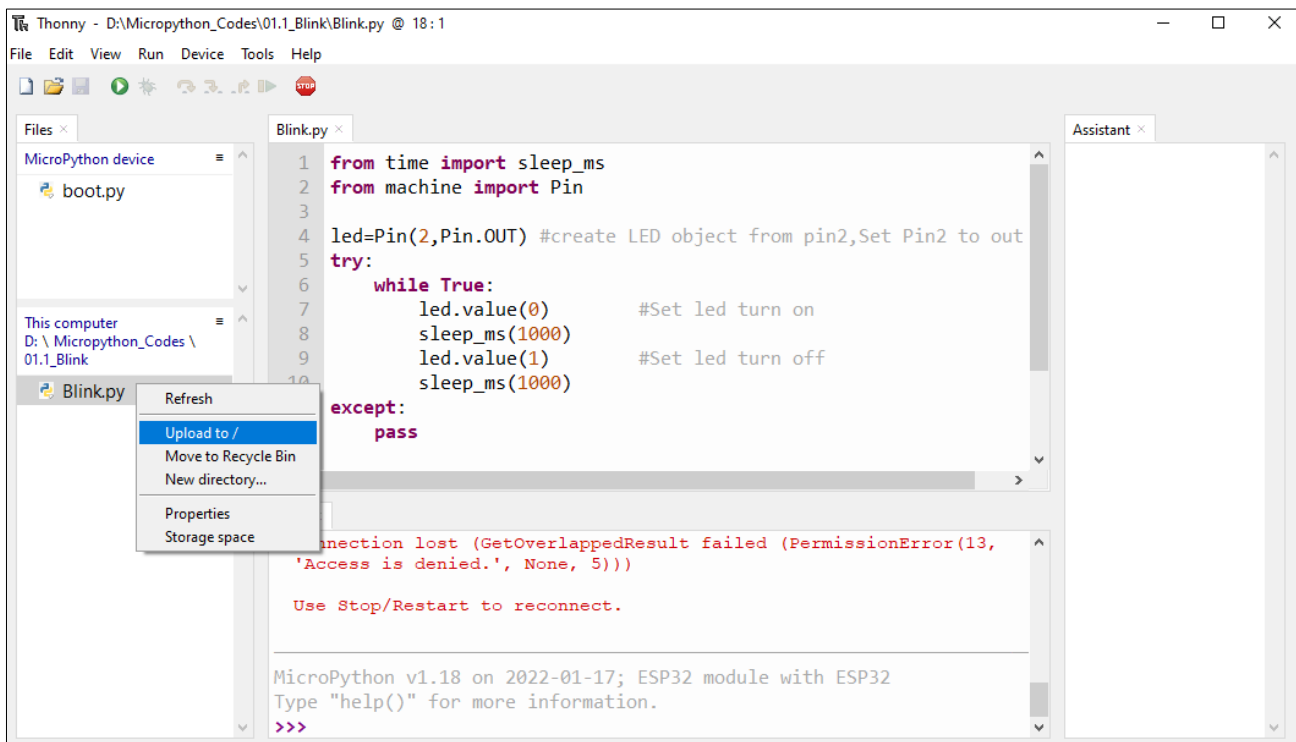
Note:

This is the code [running online](#). If you disconnect USB cable and repower ESP32 or press its reset key, LED stops blinking and the following messages will be displayed in Thonny.

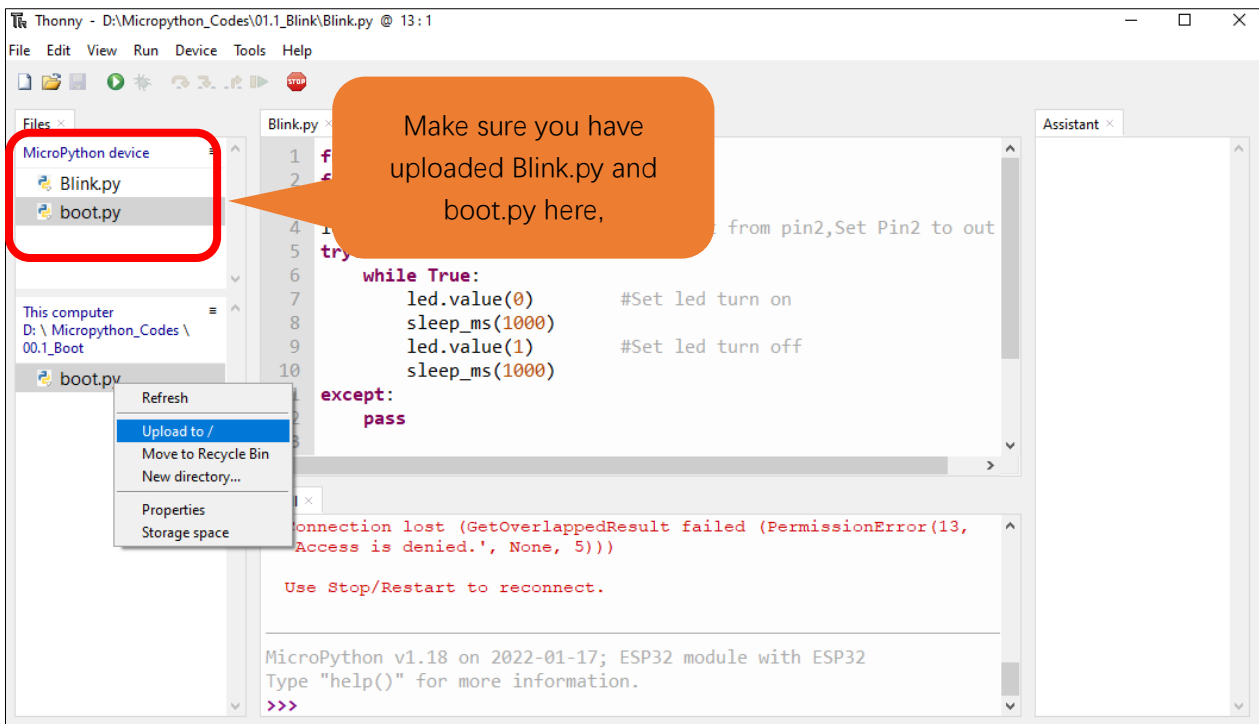


Uploading code to ESP32

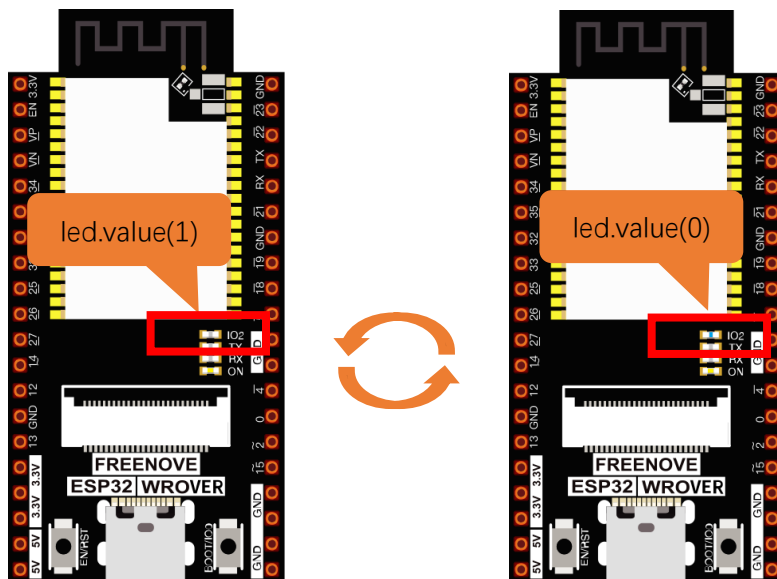
As shown in the following illustration, right-click the file Blink.py and select "Upload to /" to upload code to ESP32.



Upload boot.py in the same way.



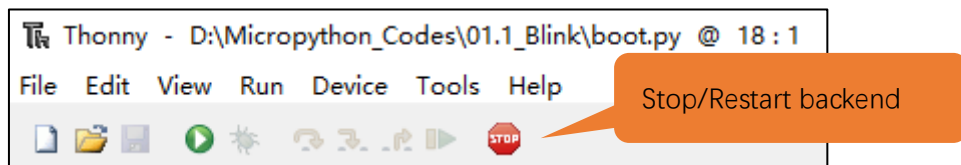
Press the reset key of ESP32 and you can see LED is ON for one second and then OFF for one second, which repeats in an endless loop.



Due to different versions, some board leds will show the opposite effect.

Note:

Codes here is run offline. If you want to stop running offline and enter Shell, just click "Stop" in Thonny.



If you have any concerns, please contact us via: support@freenove.com

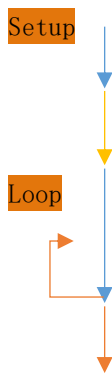
The following is the program code:

```

1  from time import sleep_ms
2  from machine import Pin
3
4  led=Pin(2,Pin. OUT) #create LED object from pin2,Set Pin2 to output
5  try:
6      while True:
7          led.value(0) #Set led turn on
8          sleep_ms(1000)
9          led.value(1) #Set led turn off
10         sleep_ms(1000)
11 except:
12     pass

```

Each time a new file is opened, the program will be executed from top to bottom. When encountering a loop construction, it will execute the loop statement according to the loop condition.



```

1  from time import sleep_ms
2  from machine import Pin
3
4  led=Pin(2,Pin. OUT) #create LED object from pin2,Set Pin2 to output
5  try:
6      while True:
7          ...
8          ...
11 except:
12     pass

```

Print() function is used to print data to Terminal. It can be executed in Terminal directly or be written in a Python file and executed by running the file.

```
print("Hello world!")
```

Each time when using the functions of ESP32, you need to import modules corresponding to those functions: Import sleep_ms module of time module and Pin module of machine module.

```

1  from time import sleep_ms
2  from machine import Pin

```

Configure GPIO2 of ESP32-WROVER to output mode and assign it to an object named "led".

```
4  led=Pin(2,Pin. OUT) #create LED object from pin2,Set Pin2 to output
```

It means that from now on, LED represents GPIO2 that is in output mode.

Set the value of LED to 1 and GPIO2 will output high level.

```
7  led.value(1) #Set led turn off
```

Set the value of LED to 0 and GPIO2 will output low level.

```
9  led.value(0) #Set led turn on
```

Execute codes in a while loop.

```

6      while True:
...          ...

```

Put statements that may cause an error in “try” block and the executing statements when an error occurs in “except” block. In general, when the program executes statements, it will execute those in “try” block. However, when an error occurs to ESP32 due to some interference or other reasons, it will execute statements in “except” block.

“Pass” is an empty statement. When it is executed, nothing happens. It is useful as a placeholder to make the structure of a program look better.

```
5  try:
...
11 except:
12     pass
```

The single-line comment of Micropython starts with a “#” and continues to the end of the line. Comments help us to understand code. When programs are running, Thonny will ignore comments.

```
9  #Set led turn on
```

MicroPython uses indentations to distinguish different blocks of code instead of braces. The number of indentations is changeable, but it must be consistent throughout one block. If the indentation of the same code block is inconsistent, it will cause errors when the program runs.

```
6  while True:
7      led.value(0) #Set led turn on
8      sleep_ms(1000)
9      led.value(1) #Set led turn off
10     sleep_ms(1000)
```

How to import python files

Whether to import the built-in python module or to import that written by users, the command “import” is needed.

If you import the module directly you should indicate the module to which the function or attribute belongs when using the function or attribute (constant, variable) in the module. The format should be: <module name>.<function or attribute>, otherwise an error will occur.

```
import random

num = random.randint(1, 100)
print(num)
```

If you only want to import a certain function or attribute in the module, use the from...import statement. The format is as follows

```
from random import randint
num = randint(1, 100)
print(num)
```

When using “from...import” statement to import function, to avoid conflicts and for easy understanding, you can use “as” statement to rename the imported function, as follows

```
from random import randint as rand
num = rand(1, 100)
print(num)
```

Reference

Class machine

Before each use of the **machine** module, please add the statement **"import machine"** to the top of python file.

machine.freq(freq_val): When freq_val is not specified, it is to return to the current CPU frequency; Otherwise, it is to set the current CPU frequency.

freq_val: 80000000(80MHz)、160000000(160MHz)、240000000(240MHz)

machine.reset(): A reset function. When it is called, the program will be reset.

machine.unique_id(): Obtains MAC address of the device.

machine.idle(): Turns off any temporarily unused functions on the chip and its clock, which is useful to reduce power consumption at any time during short or long periods.

machine.disable_irq(): Disables interrupt requests and return the previous IRQ state. The **disable_irq ()** function and **enable_irq ()** function need to be used together; Otherwise the machine will crash and restart.

machine.enable_irq(state): To re-enable interrupt requests. The parameter **state** should be the value that was returned from the most recent call to the **disable_irq()** function

machine.time_pulse_us(pin, pulse_level, timeout_us=1000000):

Tests the duration of the external pulse level on the given pin and returns the duration of the external pulse level in microseconds. When pulse level = 1, it tests the high level duration; When pulse level = 0, it tests the low level duration.

If the setting level is not consistent with the current pulse level, it will wait until they are consistent, and then start timing. If the set level is consistent with the current pulse level, it will start timing immediately.

When the pin level is opposite to the set level, it will wait for timeout and return "-2". When the pin level and the set level is the same, it will also wait timeout but return "-1". **timeout_us** is the duration of timeout.

Class Pin(id[, mode, pull, value])

Before each use of the **Pin** module, please add the statement “**from machine import Pin**” to the top of python file.

id: Arbitrary pin number

mode: Mode of pins

Pin.IN: Input Mode

Pin.OUT: Output Mode

Pin.OPEN_DRAIN: Open-drain Mode

Pull: Whether to enable the internal pull up and down mode

None: No pull up or pull down resistors

Pin.PULL_UP: Pull-up Mode, outputting high level by default

Pin.PULL_DOWN: Pull-down Mode, outputting low level by default

Value: State of the pin level, 0/1

Pin.init(mode, pull): Initialize pins

Pin.value([value]): Obtain or set state of the pin level, return 0 or 1 according to the logic level of pins. Without parameter, it reads input level. With parameter given, it is to set output level.

value: It can be either True/False or 1/0.

Pin.irq(trigger, handler): Configures an interrupt handler to be called when the pin level meets a condition.

trigger:

Pin.IRQ_FALLING: interrupt on falling edge

Pin.IRQ_RISING: interrupt on rising edge

3: interrupt on both edges

Handler: callback function

Class time

Before each use of the **time** module, please add the statement “**import time**” to the top of python file

time.sleep(sec): Sleeps for the given number of seconds

sec: This argument should be either an int or a float.

time.sleep_ms(ms): Sleeps for the given number of milliseconds, ms should be an int.

time.sleep_us(us): Sleeps for the given number of microseconds, us should be an int.

time.time(): Obtains the timestamp of CPU, with second as its unit.

time.ticks_ms(): Returns the incrementing millisecond counter value, which recounts after some values.

time.ticks_us(): Returns microsecond

time.ticks_cpu(): Similar to ticks_ms() and ticks_us(), but it is more accurate(return clock of CPU).

time.ticks_add(ticks, delta): Gets the timestamp after the offset.

ticks: ticks_ms()、ticks_us()、ticks_cpu()

delta: Delta can be an arbitrary integer number or numeric expression

time.ticks_diff(old_t, new_t): Calculates the interval between two timestamps, such as ticks_ms(), ticks_us() or ticks_cpu().

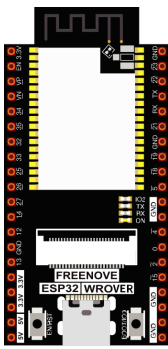
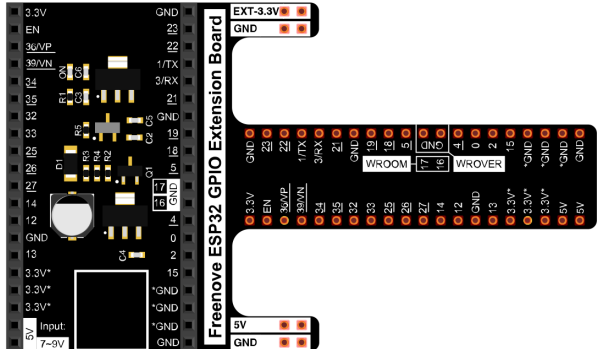
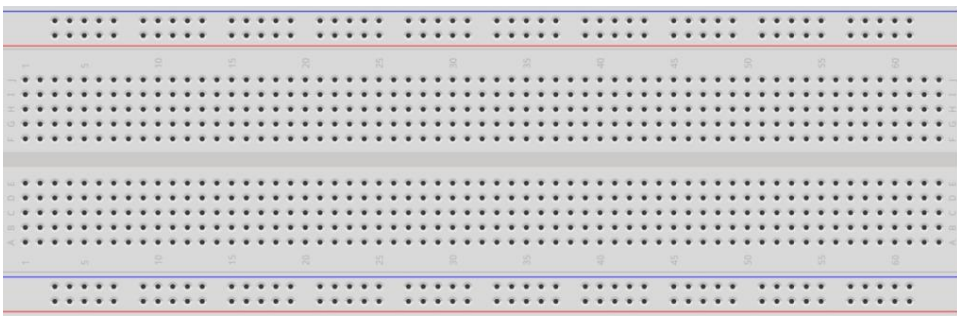
old_t: Starting time

new_t: Ending time

Project 1.2 Blink

In this project, we will use ESP32 to control blinking a common LED.

Component List

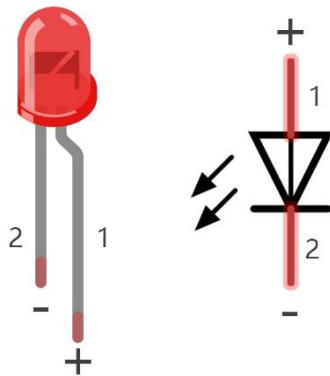
ESP32-WROVER x1 	GPIO Extension Board x1 	
Breadboard x1 		
LED x1 	Resistor 220Ω x1 	Jumper M/M x2 

Component knowledge

LED

An LED is a type of diode. All diodes only work if current is flowing in the correct direction and have two Poles. An LED will only work (light up) if the longer pin (+) of LED is connected to the positive output from a power source and the shorter pin is connected to the negative (-). Negative output is also referred to as Ground (GND). This type of component is known as "Polar" (think One-Way Street).

All common 2 lead diodes are the same in this respect. Diodes work only if the voltage of its positive electrode is higher than its negative electrode and there is a narrow range of operating voltage for most all common diodes of 1.9 and 3.4V. If you use much more than 3.3V the LED will be damaged and burn out.



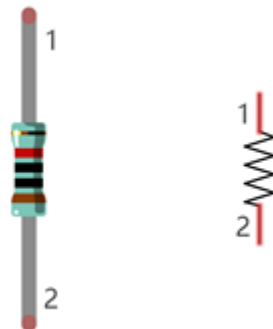
LED	Voltage	Maximum current	Recommended current
Red	1.9-2.2V	20mA	10mA
Green	2.9-3.4V	10mA	5mA
Blue	2.9-3.4V	10mA	5mA
Volt ampere characteristics conform to diode			

Note: LEDs cannot be directly connected to a power supply, which usually ends in a damaged component. A resistor with a specified resistance value must be connected in series to the LED you plan to use.

Resistor

Resistors use Ohms (Ω) as the unit of measurement of their resistance (R). $1M\Omega=1000k\Omega$, $1k\Omega=1000\Omega$.

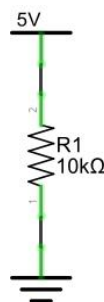
A resistor is a passive electrical component that limits or regulates the flow of current in an electronic circuit. On the left, we see a physical representation of a resistor, and the right is the symbol used to represent the presence of a resistor in a circuit diagram or schematic.



The bands of color on a resistor is a shorthand code used to identify its resistance value. For more details of resistor color codes, please refer to the appendix of this tutorial.

With a fixed voltage, there will be less current output with greater resistance added to the circuit. The relationship between Current, Voltage and Resistance can be expressed by this formula: $I=V/R$ known as Ohm's Law where I = Current, V = Voltage and R = Resistance. Knowing the values of any two of these allows you to solve the value of the third.

In the following diagram, the current through R1 is: $I=U/R=5V/10k\Omega=0.0005A=0.5mA$.

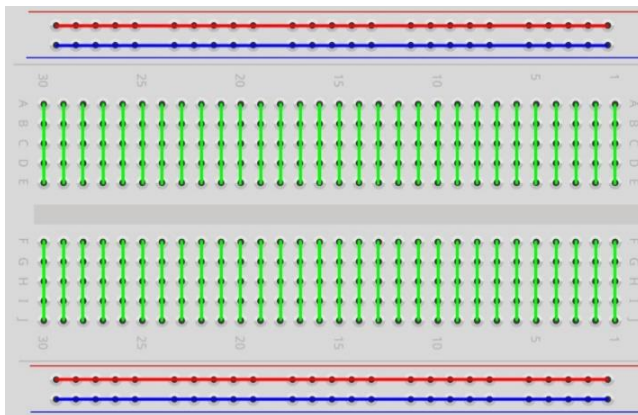


WARNING: Never connect the two poles of a power supply with anything of low resistance value (i.e. a metal object or bare wire) this is a Short and results in high current that may damage the power supply and electronic components.

Note: Unlike LEDs and Diodes, Resistors have no poles and are non-polar (it does not matter which direction you insert them into a circuit, it will work the same)

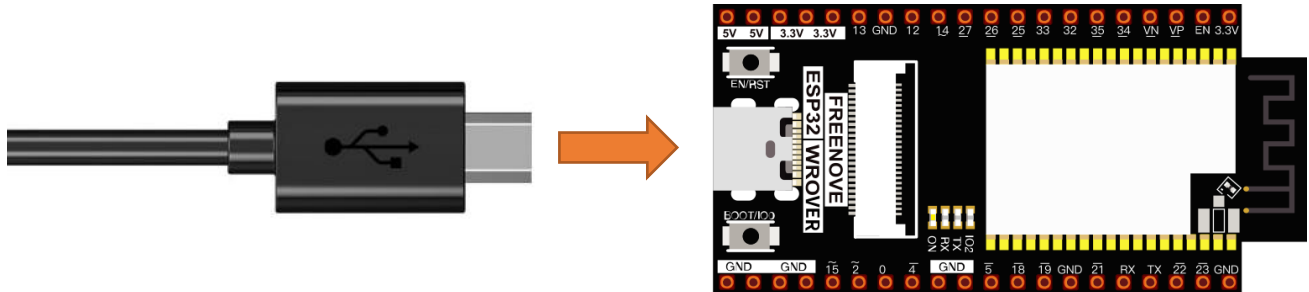
Breadboard

Here we have a small breadboard as an example of how the rows of holes (sockets) are electrically attached. The left picture shows the way to connect pins. The right picture shows the practical internal structure.



Power

ESP32-WROVER needs 5v power supply. In this tutorial, we need connect ESP32-WROVER to computer via USB cable to power it and program it. We can also use other 5v power source to power it.



Later, we only use USB cable to power ESP32-WROVER in default.

In the whole tutorial, we don't use T extension to power ESP32-WROVER. So 5V and 3.3V (include EXT 3.3V) on the extension board are from ESP32-WROVER.

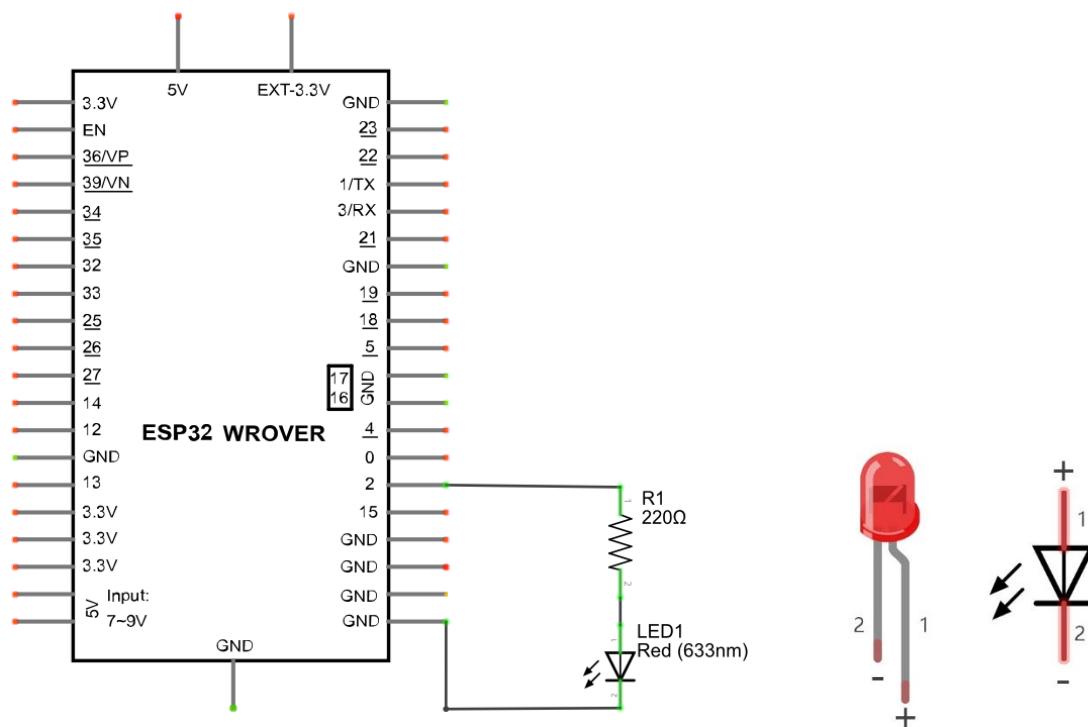
We can also use DC jack of extension board to power ESP32-WROVER. Then 5v and EXT 3.3v on extension board are from external power resource.

Circuit

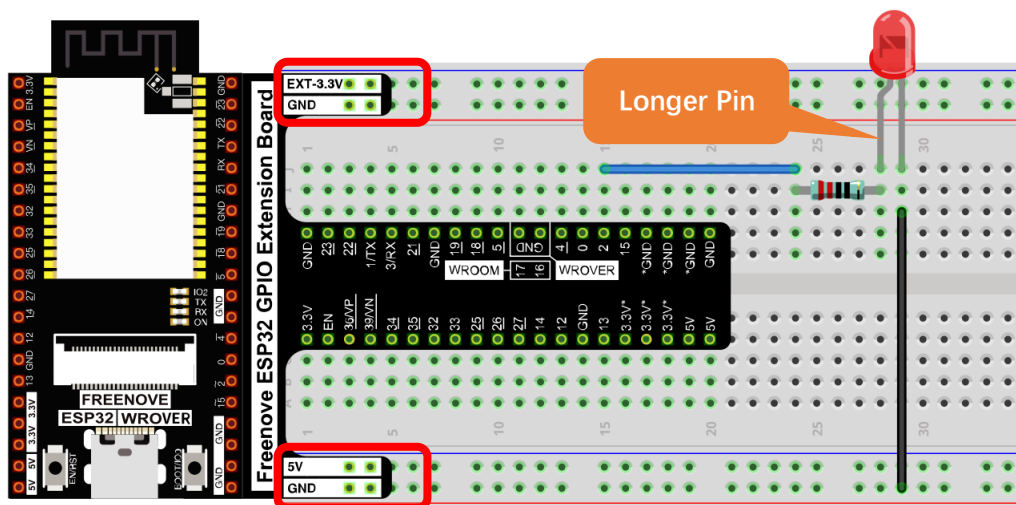
First, disconnect all power from the ESP32-WROVER. Then build the circuit according to the circuit and hardware diagrams. After the circuit is built and verified correct, connect the PC to ESP32-WROVER.

CAUTION: Avoid any possible short circuits (especially connecting 5V or GND, 3.3V and GND)! WARNING: A short circuit can cause high current in your circuit, create excessive component heat and cause permanent damage to your hardware!

Schematic diagram



Hardware connection. If you need any support, please contact us via: support@freenove.com



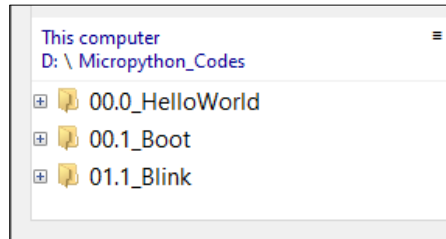
Don't rotate ESP32-WROVER 180° for connection.

Code

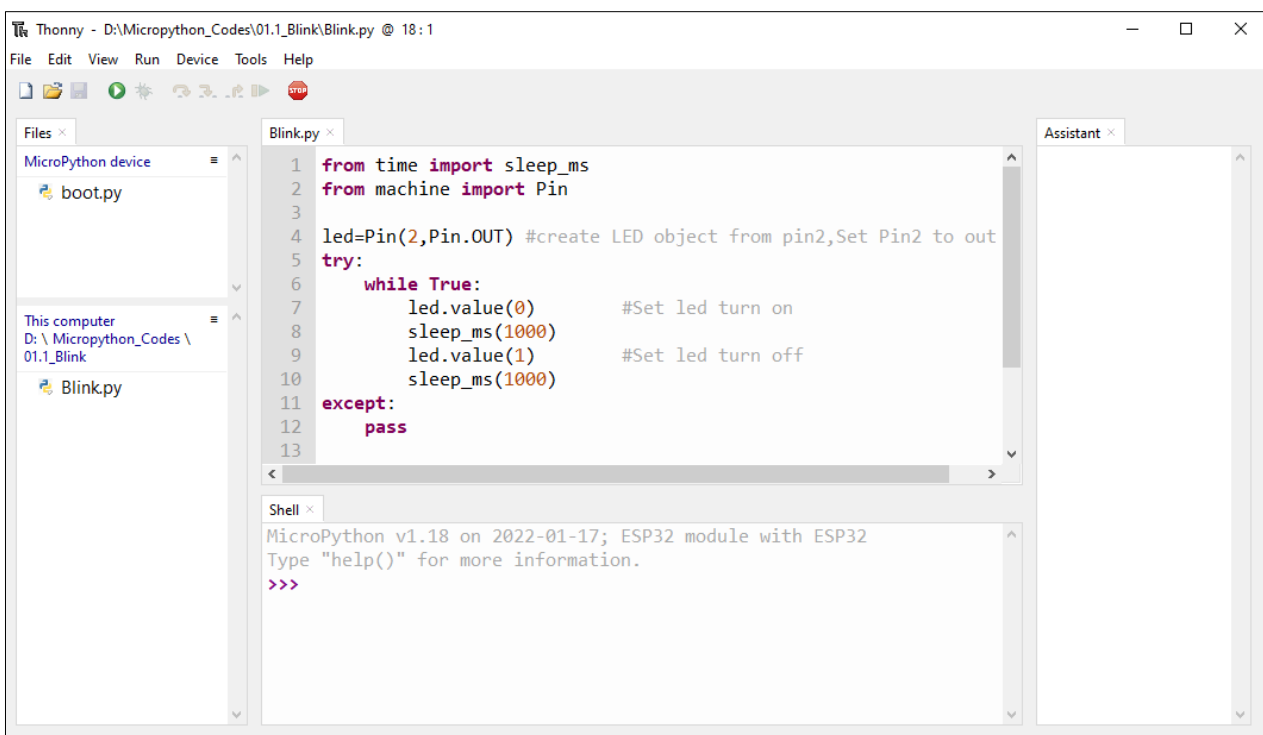
Codes used in this tutorial are saved in “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**”. You can move the codes to any location. For example, we save the codes in Disk(D) with the path of “**D:/Micropython_Codes**”.

01.1_Blink

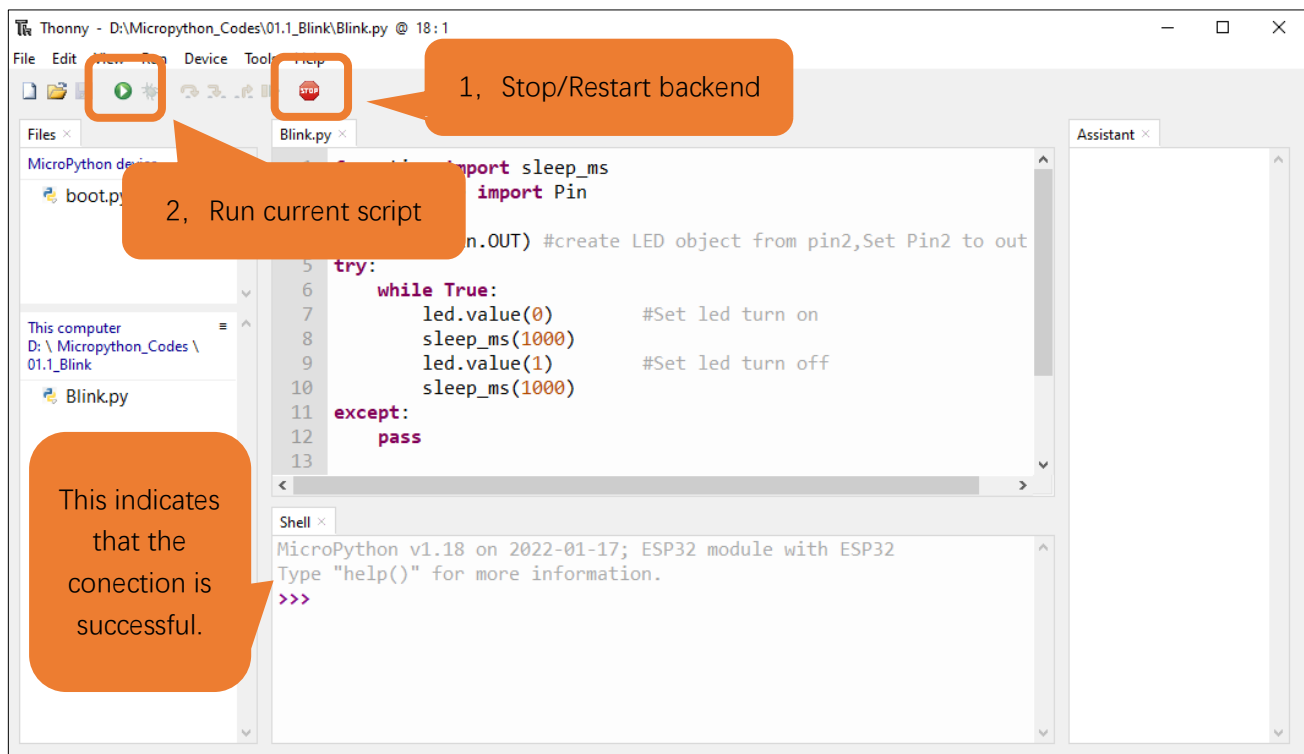
Open “Thonny”, click “This computer”→“D:”→“Micropython_Codes”.



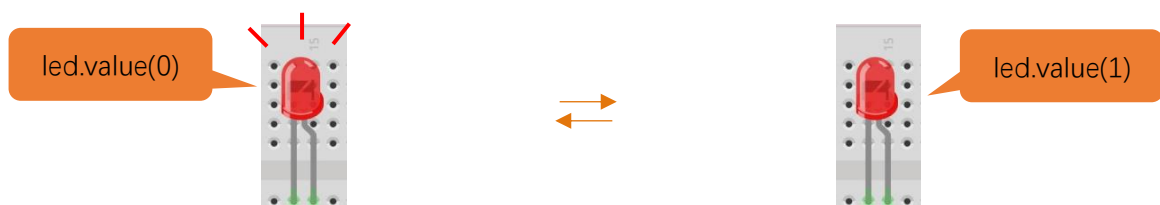
Expand folder “01.1_Blink” and double click “Blink.py” to open it. As shown in the illustration below.



Make sure ESP32 has been connected with the computer with ESP32 correctly. Click “Stop/Restart backend” or press the reset button, and then wait to see what interface will show up.

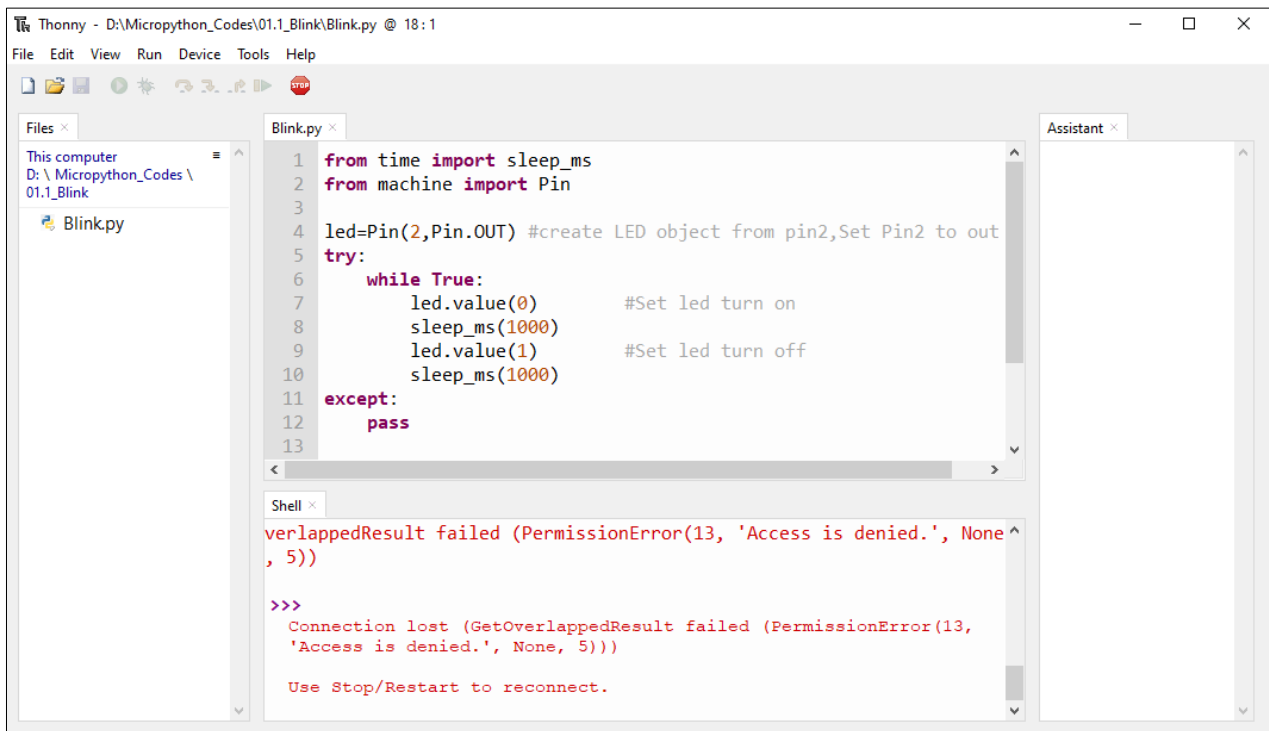


Click “Run current script” shown in the box above, the code starts to be executed and the LED in the circuit starts to blink.



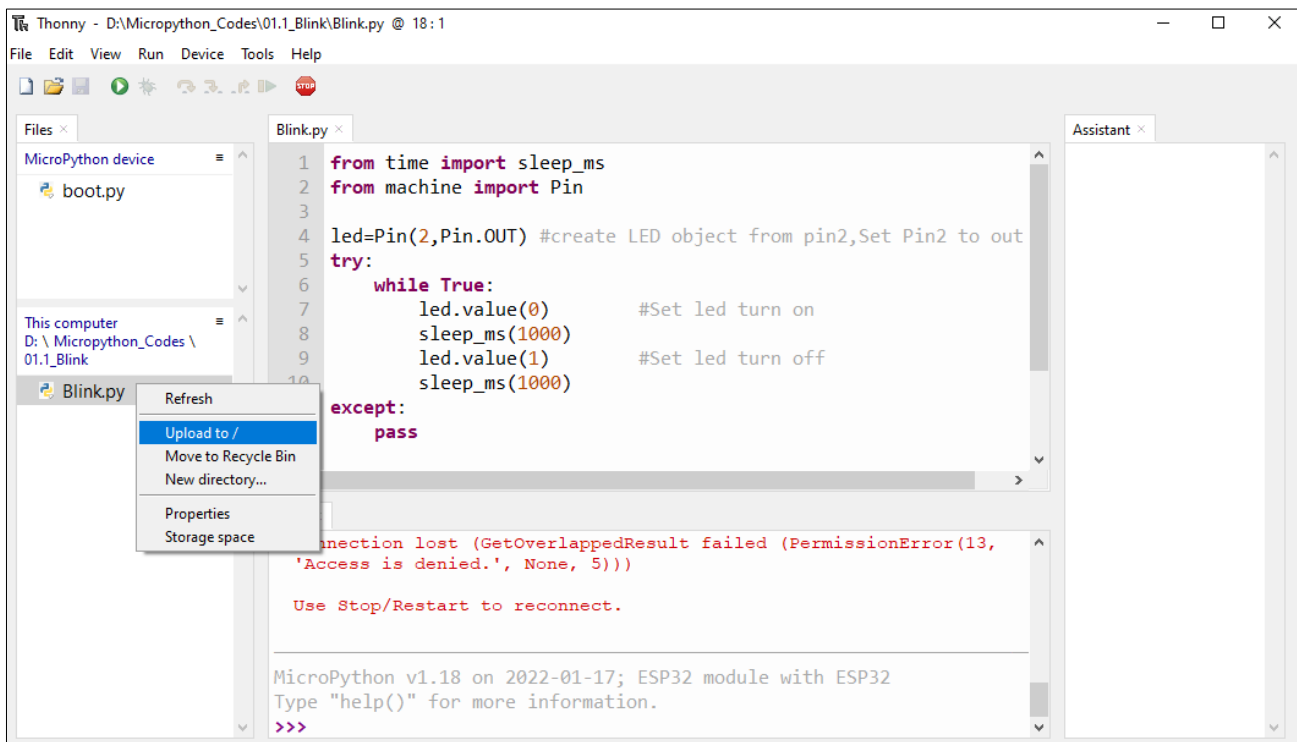
Note:

This is the code [running online](#). If you disconnect USB cable and repower ESP32 or press its reset key, LED stops blinking and the following messages will be displayed in Thonny.

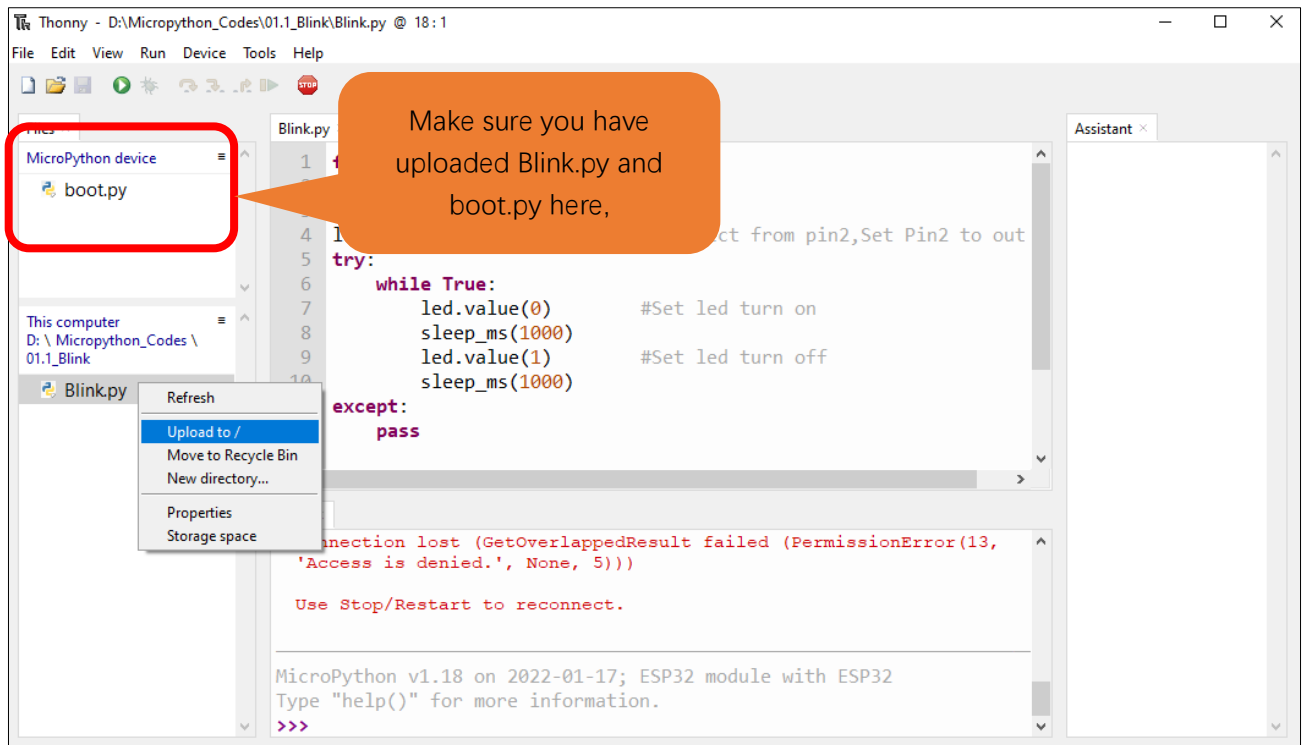


Uploading code to ESP32

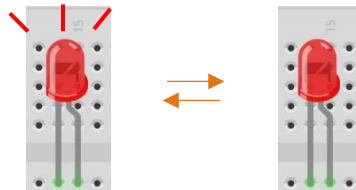
As shown in the following illustration, right-click the file Blink.py and select “Upload to /” to upload code to ESP32.



Upload boot.py in the same way.

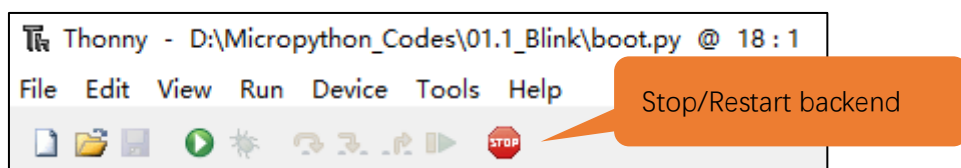


Press the reset key of ESP32 and you can see LED is ON for one second and then OFF for one second, which repeats in an endless loop.



Note:

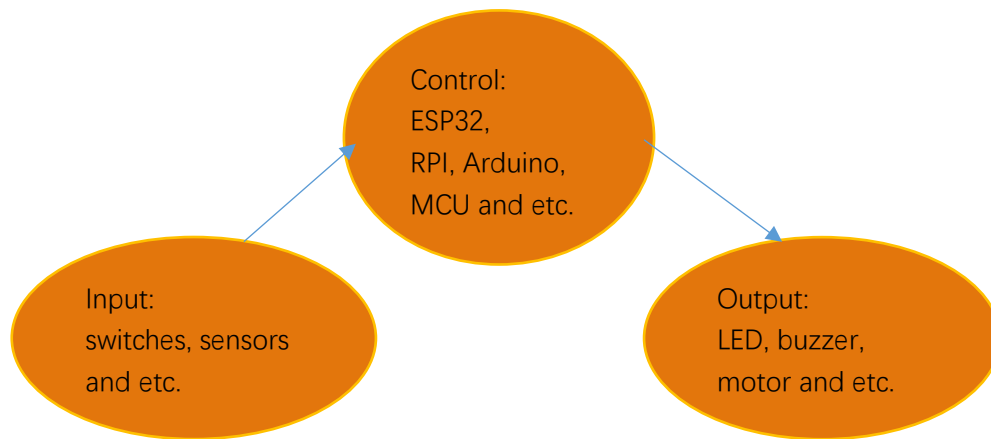
Codes here is run offline. If you want to stop running offline and enter Shell, just click "Stop" in Thonny.



If you have any concerns, please contact us via: support@freenove.com

Chapter 2 Button & LED

Usually, there are three essential parts in a complete automatic control device: INPUT, OUTPUT, and CONTROL. In last section, the LED module was the output part and ESP32 was the control part. In practical applications, we not only make LEDs flash, but also make a device sense the surrounding environment, receive instructions and then take the appropriate action such as LEDs light up, make a buzzer turn ON and so on.

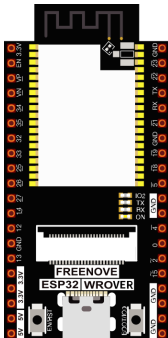
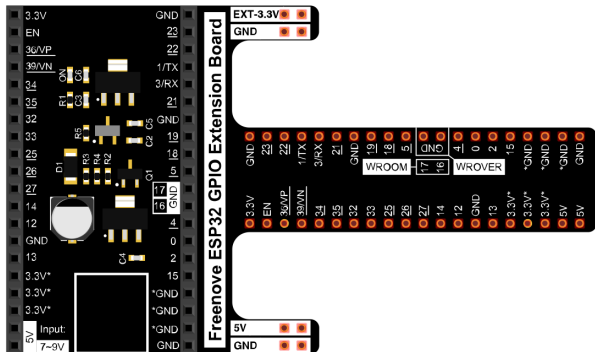
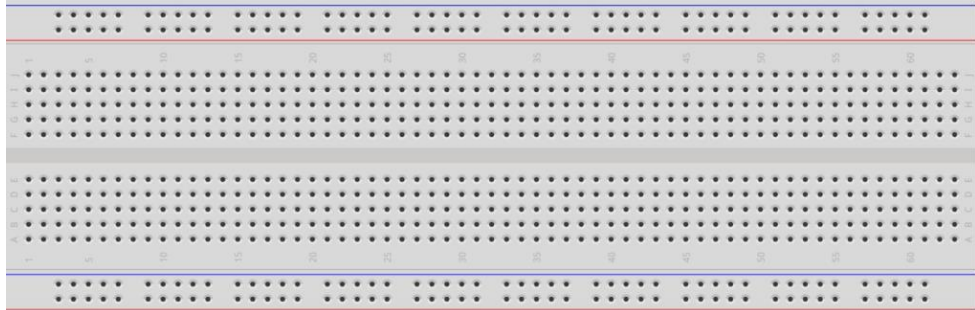


Next, we will build a simple control system to control an LED through a push button switch.

Project 2.1 Button & LED

In the project, we will control the LED state through a Push Button Switch. When the button is pressed, our LED will turn ON, and when it is released, the LED will turn OFF. This describes a Momentary Switch.

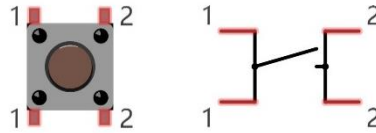
Component List

ESP32-WROVER x1 	GPIO Extension Board x1 			
Breadboard x1 				
Jumper M/M x4 	LED x1 	Resistor 220Ω x1 	Resistor 10kΩ x2 	Push button x1 

Component knowledge

Push button

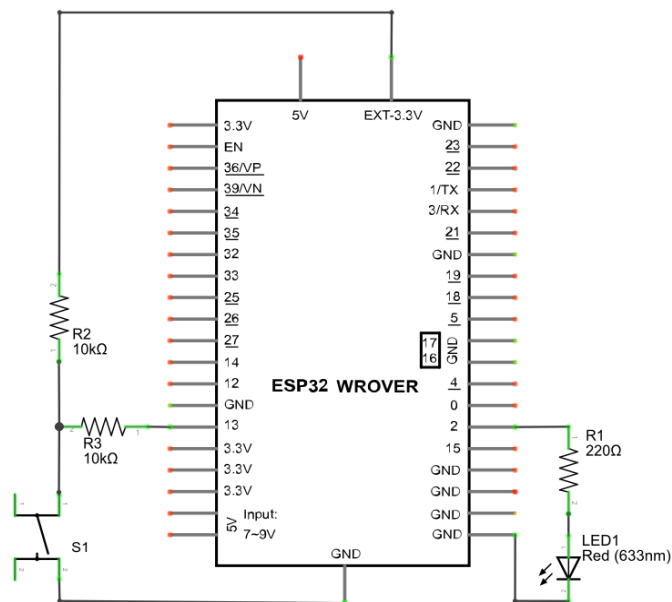
This type of Push Button Switch has 4 pins (2 Pole Switch). Two pins on the left are connected, and both left and right sides are the same per the illustration:



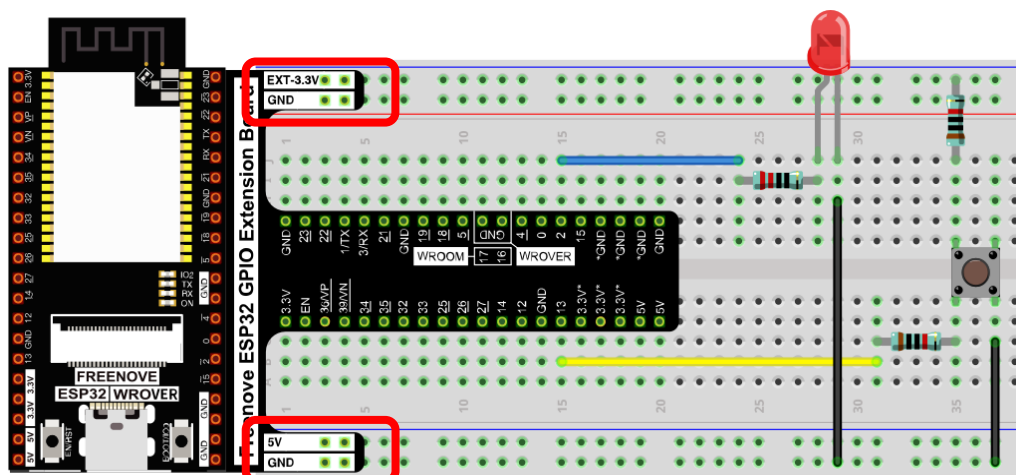
When the button on the switch is pressed, the circuit is completed (your project is Powered ON).

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



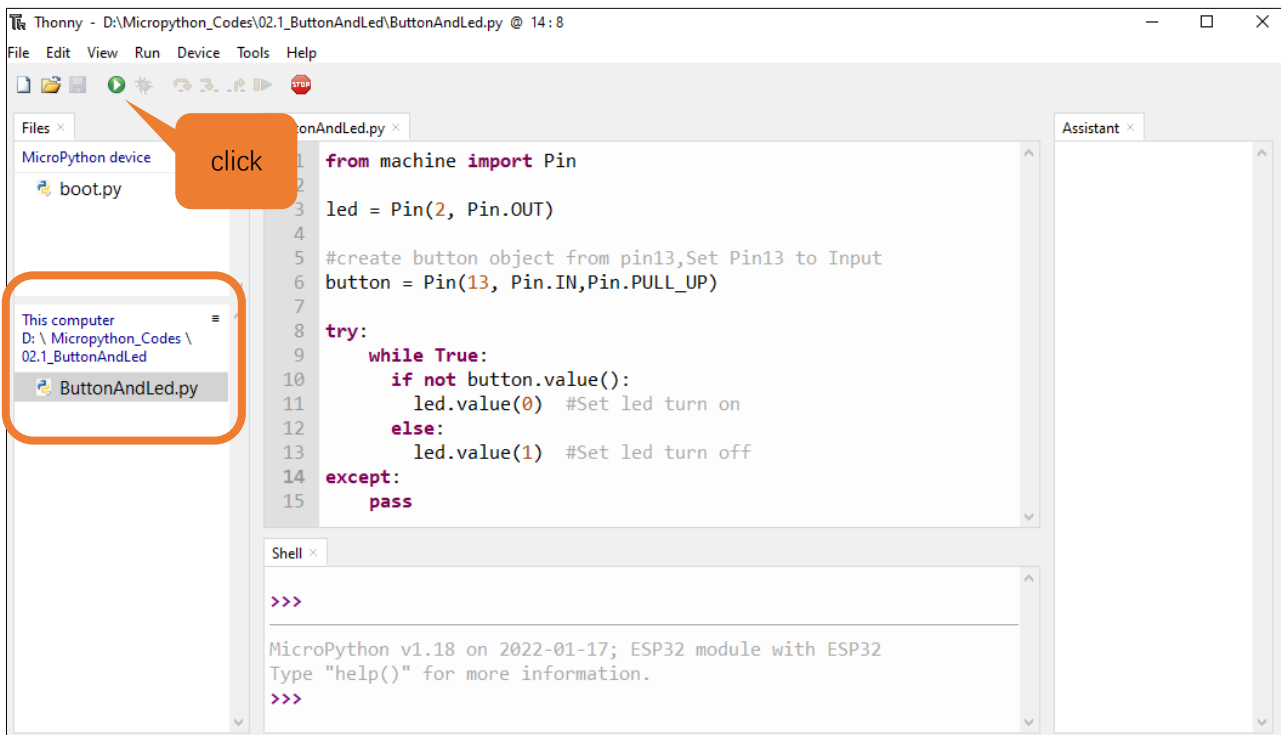
Code

This project is designed to learn to control an LED with a push button switch. First, we need to read the state of the switch and then decide whether the LED is turned on or not based on it.

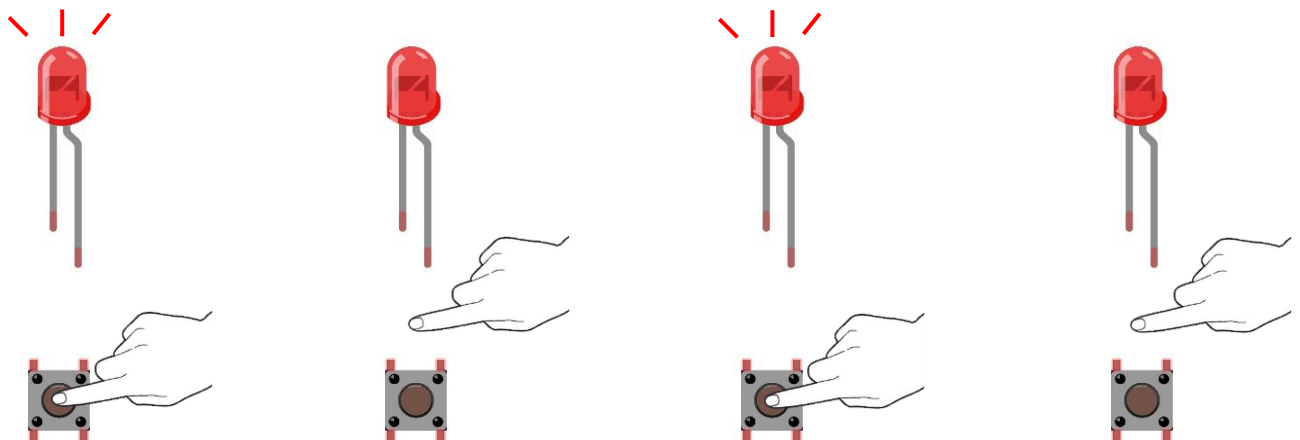
Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “02.1_ButtonAndLed” and double click “ButtonAndLed.py”.

02.1_ButtonAndLed

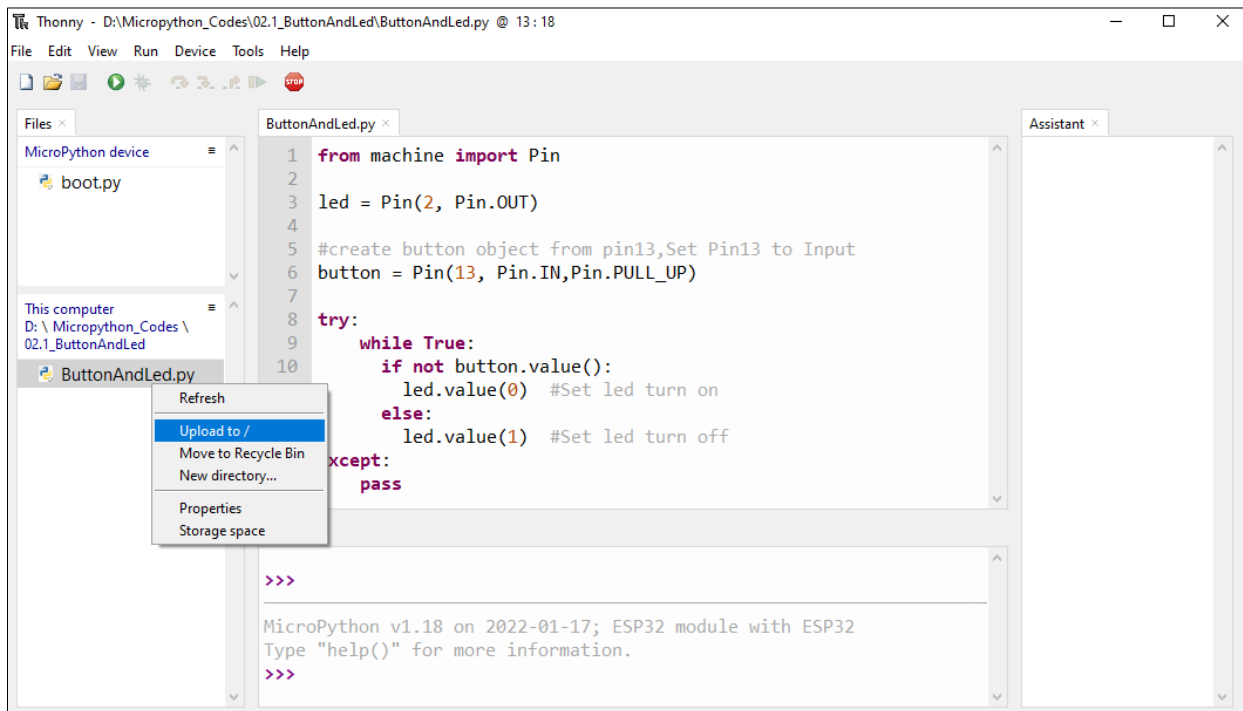


Click “Run current script” shown in the box of the above illustration, press the push button switch, LED turns ON; release the switch, LED turns OFF.

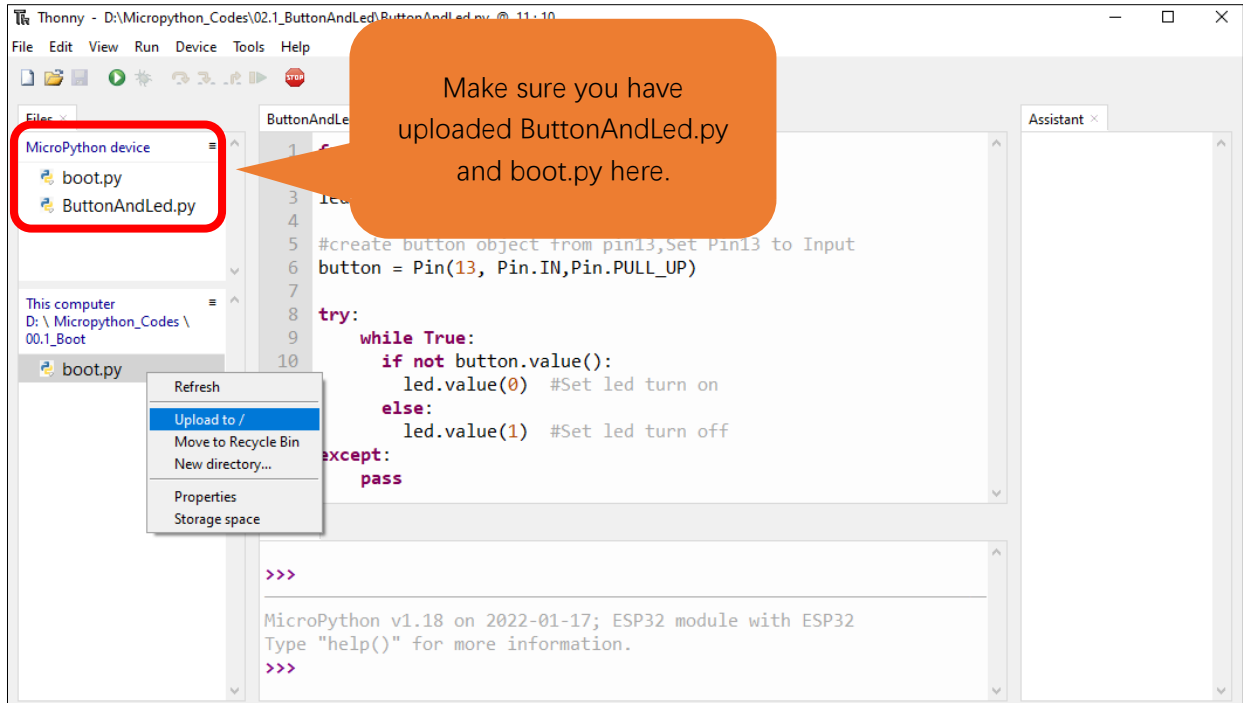


Upload Code to ESP32

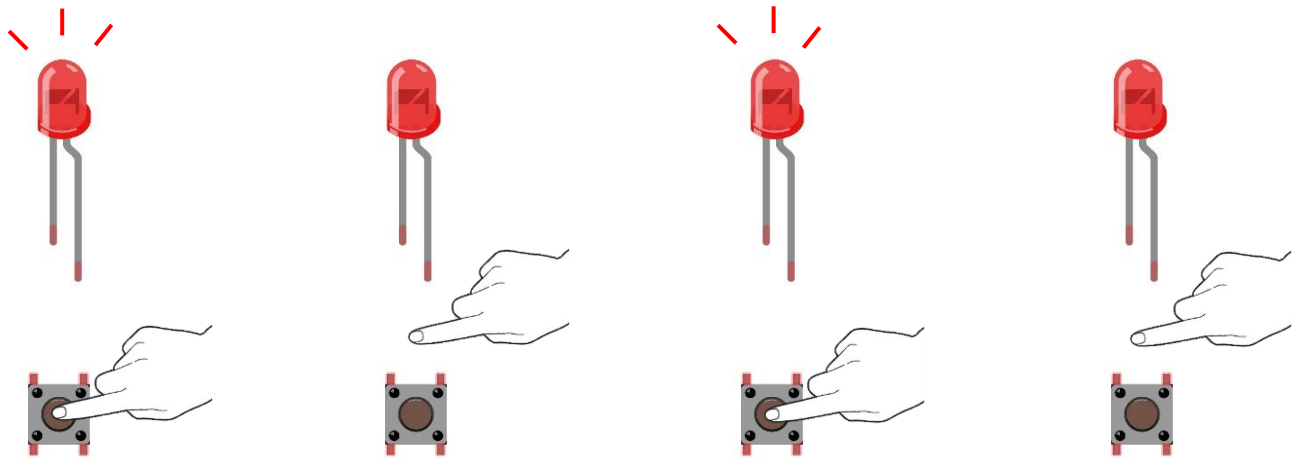
As shown in the following illustration, right-click file 02.1_ButtonAndLed and select "Upload to /" to upload code to ESP32.



Upload boot.py in the same way.



Press ESP32's reset key, and then push the button switch, LED turns ON; Push the button again, LED turns OFF.



The following is the program code:

```

1  from machine import Pin
2
3  led = Pin(2, Pin.OUT)
4
5  #create button object from pin13,Set Pin13 to Input
6  button = Pin(13, Pin.IN,Pin.PULL_UP)
7
8  try:
9      while True:
10         if not button.value():
11             led.value(0) #Set led turn on
12         else:
13             led.value(1) #Set led turn off
14 except:
15     pass

```

In this project, we use the Pin module of the machine, so before initializing the Pin, we need to import this module first.

```

1  from machine import Pin

```

In the circuit connection, LED and Button are connected with GPIO2 and GPIO13 respectively, so define led and button as 2 and 13 respectively.

```

3  led = Pin(2, Pin.OUT)
4
5  #create button object from pin13,Set Pin13 to Input
6  button = Pin(13, Pin.IN,Pin.PULL_UP)

```

Read the pin state of button with value() function. Press the button switch, the function returns low level and the result of "if" is true, and then LED will be turned ON; Otherwise, LED is turned OFF.

```
9      while True:
10         if not button.value():
11             led.value(0) #Set led turn on
12         else:
13             led.value(1) #Set led turn off
```

If statement is used to execute the next statement when a certain condition is proved to be true (or non0). It is often used together with "else" statement, which judges other statements except the if statement. If you need to judge if the result of a condition is 0, you can use if not statement.

```
10     if not button.value():
11         ...
12     else:
13         ...
```

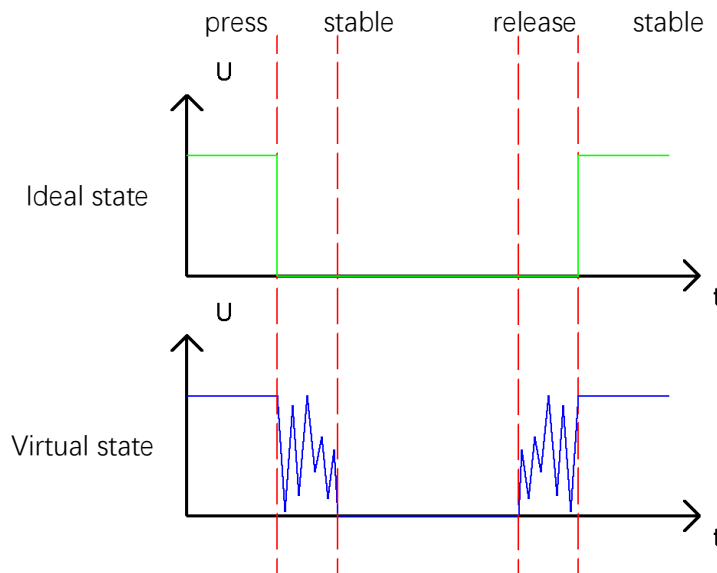
Project 2.2 MINI table lamp

We will also use a Push Button Switch, LED and ESP32 to make a MINI Table Lamp but this will function differently: Press the button, the LED will turn ON, and pressing the button again, the LED turns OFF. The ON switch action is no longer momentary (like a door bell) but remains ON without needing to continually press on the Button Switch.

First, let us learn something about the push button switch.

Debounce for Push Button

When a Momentary Push Button Switch is pressed, it will not change from one state to another state immediately. Due to tiny mechanical vibrations, there will be a short period of continuous buffeting before it completely reaches another state too fast for Humans to detect but not for computer microcontrollers. The same is true when the push button switch is released. This unwanted phenomenon is known as “bounce”.



Therefore, if we can directly detect the state of the Push Button Switch, there are multiple pressing and releasing actions in one pressing cycle. This buffeting will mislead the high-speed operation of the microcontroller to cause many false decisions. Therefore, we need to eliminate the impact of buffeting. Our solution: to judge the state of the button multiple times. Only when the button state is stable (consistent) over a period of time, can it indicate that the button is actually in the ON state (being pressed).

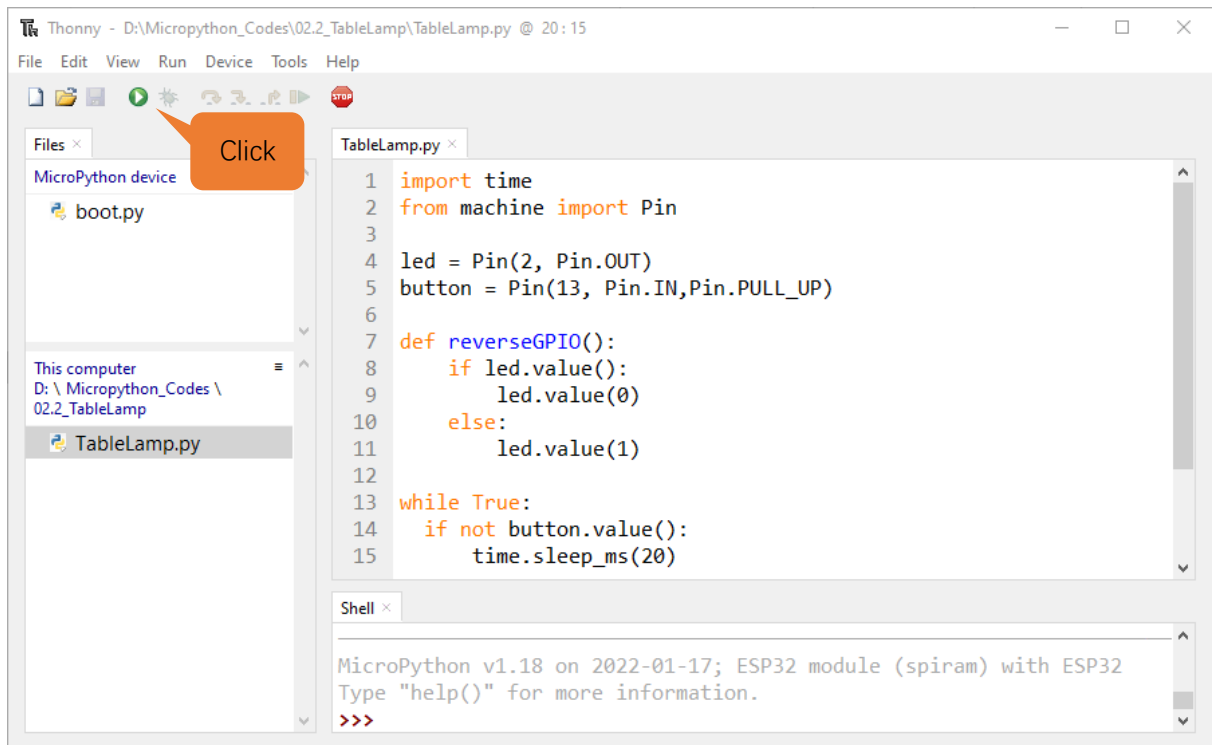
This project needs the same components and circuits as we used in the previous section.

Code

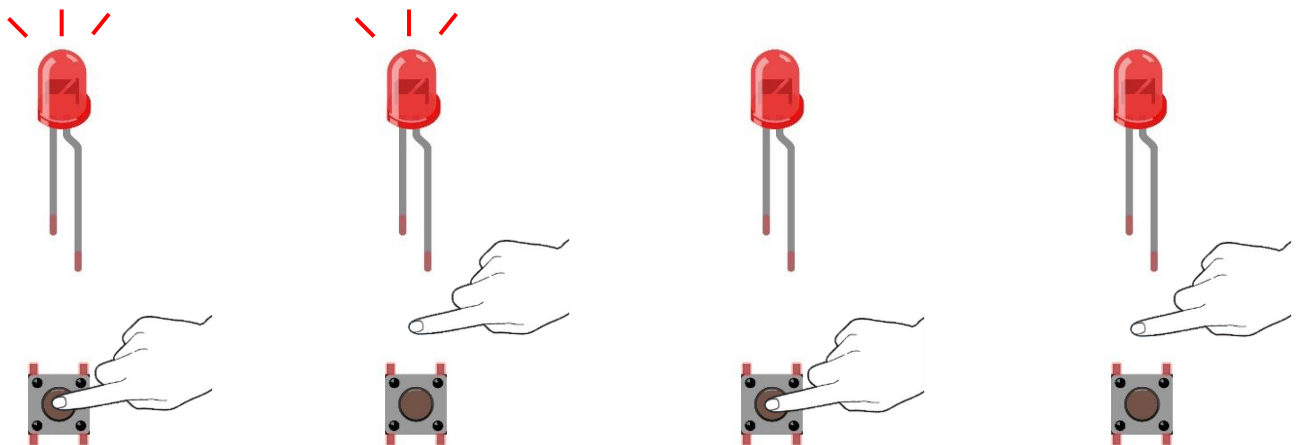
02.2_Tablelamp

Move the program folder “**Freenove Super Starter Kit for ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “02.2_TableLamp” and double click “TableLamp.py”.



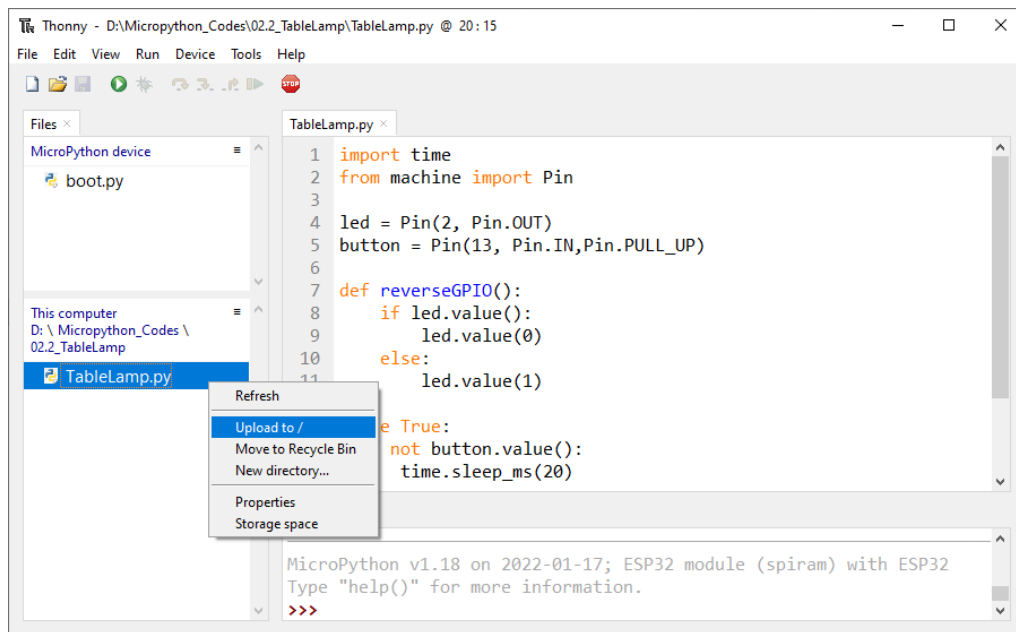
Click “Run current script” shown in the box of the above illustration, press the push button switch, LED turns ON; press it again, LED turns OFF.



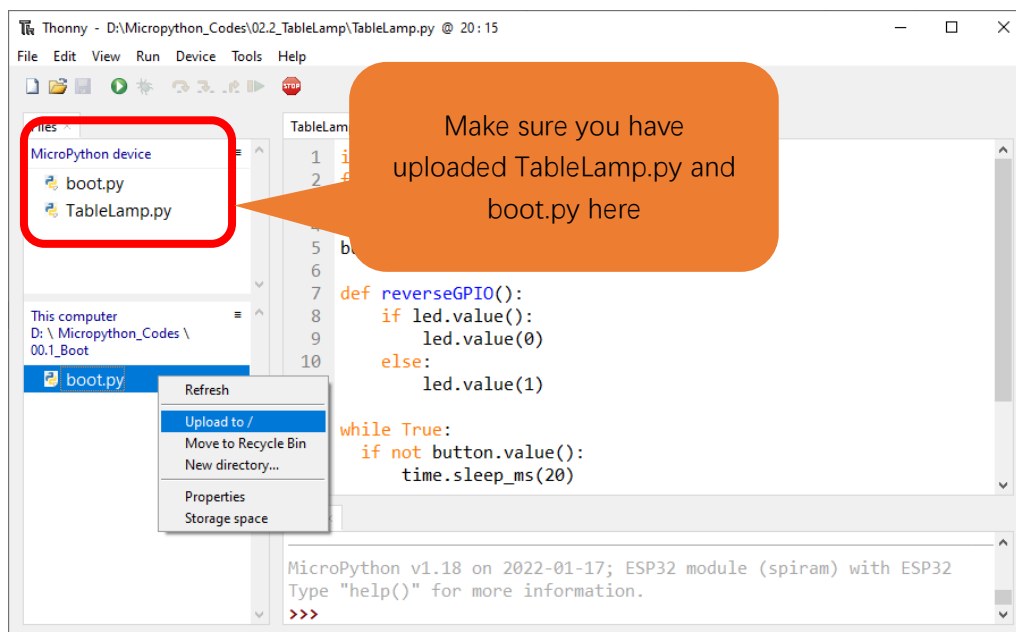
If you have any concerns, please contact us via: support@freenove.com

Upload code to ESP32

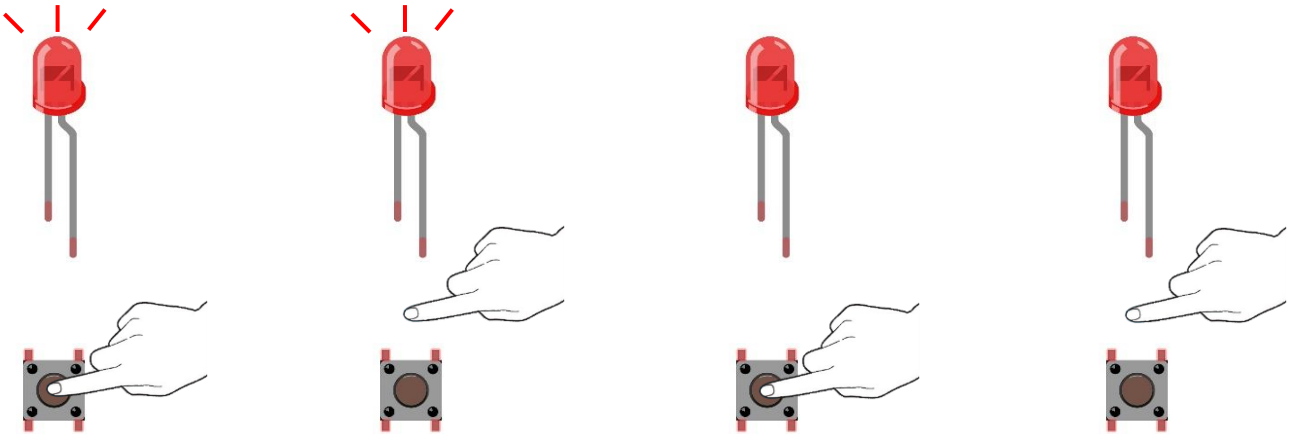
As shown in the following illustration, right-click file 02.2_TableLamp and select “Upload to /” to upload code to ESP32.



Upload boot.py in the same way.



Press ESP32's reset key, and then push the button switch, LED turns ON; Push the button again, LED turns OFF.



The following is the program code:

```

1  import time
2  from machine import Pin
3
4  led = Pin(2, Pin.OUT)
5  button = Pin(13, Pin.IN, Pin.PULL_UP)
6
7  def reverseGPIO():
8      if led.value():
9          led.value(0)
10     else:
11         led.value(1)
12
13 while True:
14     if not button.value():
15         time.sleep_ms(20)
16         if not button.value():
17             reverseGPIO()
18             while not button.value():
19                 time.sleep_ms(20)
20

```

When the button is detected to be pressed, delay 20ms to avoid the effect of bounce, and then check whether the button has been pressed again. If so, the conditional statement will be executed, otherwise it will not be executed.

```

13 while True:
14     if not button.value():
15         time.sleep_ms(20)
16         if not button.value():
17             reverseGPIO()
18             while not button.value():
19                 time.sleep_ms(20)

```

Customize a function and name it `reverseGPIO()`, which reverses the output level of the LED.

```
7 def reverseGPIO():
8     if led.value():
9         led.value(0)
10    else:
11        led.value(1)
```

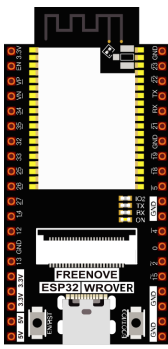
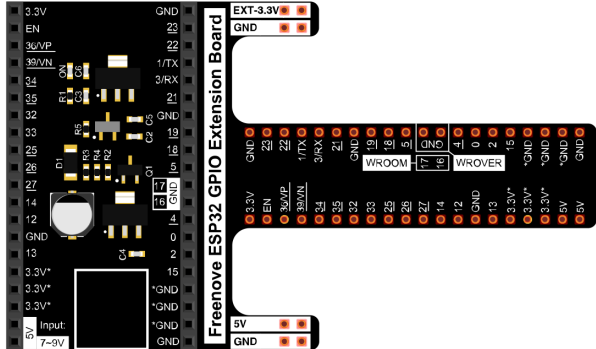
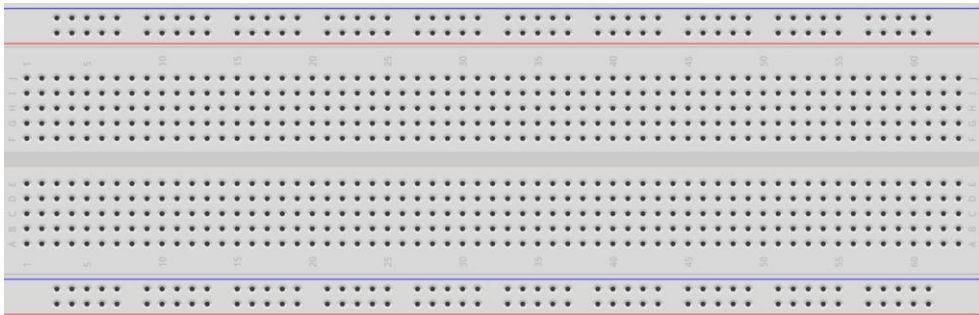

Chapter 3 LED Bar

We have learned how to control a LED blinking, next we will learn how to control a number of LEDs.

Project 3.1 Flowing Light

In this project, we use a number of LEDs to make a flowing light.

Component List

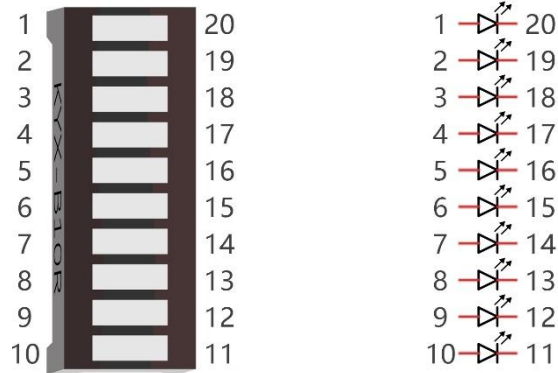
ESP32-WROVER x1 	GPIO Extension Board x1 	
Breadboard x1 		
Jumper M/M x10 	LED bar graph x1 	Resistor 220Ω x10 

Component knowledge

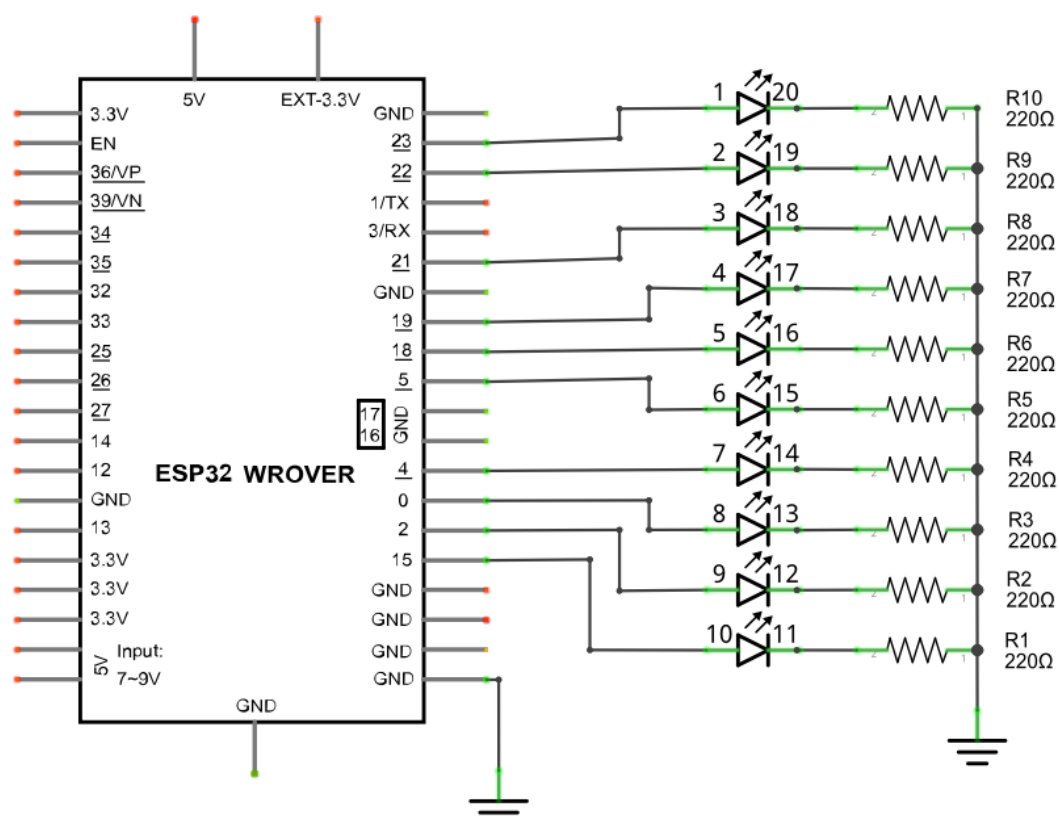
Let us learn about the basic features of these components to use and understand them better.

LED bar

A Bar Graph LED has 10 LEDs integrated into one compact component. The two rows of pins at its bottom are paired to identify each LED like the single LED used earlier.



Schematic diagram



The diagram illustrates the following connections:

- ESP32 GPIO Extension Board:** Its 5V pin is connected to the positive rail, and its GND pin is connected to the ground rail.
- WROOM Module:** Pin 1 (GND) connects to the ground rail. Pins 2 (EN), 3 (30VP), 4 (30VN), 5 (TX), 6 (RX), 7 (GND), 8 (GND), 9 (GND), 10 (GND), 11 (GND), 12 (GND), 13 (GND), 14 (GND), 15 (GND), 16 (GND), 17 (GND), 18 (GND), 19 (GND), 20 (GND), 21 (GND), 22 (GND), 23 (GND), 24 (GND), 25 (GND), 26 (GND), 27 (GND), 28 (GND), 29 (GND), 30 (GND), 31 (GND), 32 (GND), 33 (GND), 34 (GND), 35 (GND), 36 (GND), 37 (GND), 38 (GND), 39 (GND), 40 (GND), 41 (GND), 42 (GND), 43 (GND), 44 (GND), 45 (GND), 46 (GND), 47 (GND), 48 (GND), 49 (GND), 50 (GND), 51 (GND), 52 (GND), 53 (GND), 54 (GND), 55 (GND), 56 (GND), 57 (GND), 58 (GND), 59 (GND), 60 (GND), 61 (GND), 62 (GND), 63 (GND), 64 (GND), 65 (GND), 66 (GND), 67 (GND), 68 (GND), 69 (GND), 70 (GND), 71 (GND), 72 (GND), 73 (GND), 74 (GND), 75 (GND), 76 (GND), 77 (GND), 78 (GND), 79 (GND), 80 (GND), 81 (GND), 82 (GND), 83 (GND), 84 (GND), 85 (GND), 86 (GND), 87 (GND), 88 (GND), 89 (GND), 90 (GND), 91 (GND), 92 (GND), 93 (GND), 94 (GND), 95 (GND), 96 (GND), 97 (GND), 98 (GND), 99 (GND), 100 (GND).
- WROVER Module:** Pin 1 (GND) connects to the ground rail. Pins 2 (EN), 3 (30VP), 4 (30VN), 5 (TX), 6 (RX), 7 (GND), 8 (GND), 9 (GND), 10 (GND), 11 (GND), 12 (GND), 13 (GND), 14 (GND), 15 (GND), 16 (GND), 17 (GND), 18 (GND), 19 (GND), 20 (GND), 21 (GND), 22 (GND), 23 (GND), 24 (GND), 25 (GND), 26 (GND), 27 (GND), 28 (GND), 29 (GND), 30 (GND), 31 (GND), 32 (GND), 33 (GND), 34 (GND), 35 (GND), 36 (GND), 37 (GND), 38 (GND), 39 (GND), 40 (GND), 41 (GND), 42 (GND), 43 (GND), 44 (GND), 45 (GND), 46 (GND), 47 (GND), 48 (GND), 49 (GND), 50 (GND), 51 (GND), 52 (GND), 53 (GND), 54 (GND), 55 (GND), 56 (GND), 57 (GND), 58 (GND), 59 (GND), 60 (GND), 61 (GND), 62 (GND), 63 (GND), 64 (GND), 65 (GND), 66 (GND), 67 (GND), 68 (GND), 69 (GND), 70 (GND), 71 (GND), 72 (GND), 73 (GND), 74 (GND), 75 (GND), 76 (GND), 77 (GND), 78 (GND), 79 (GND), 80 (GND), 81 (GND), 82 (GND), 83 (GND), 84 (GND), 85 (GND), 86 (GND), 87 (GND), 88 (GND), 89 (GND), 90 (GND), 91 (GND), 92 (GND), 93 (GND), 94 (GND), 95 (GND), 96 (GND), 97 (GND), 98 (GND), 99 (GND), 100 (GND).

If LEDbar doesn't work, try to rotate LEDbar for 180°. The label is random.

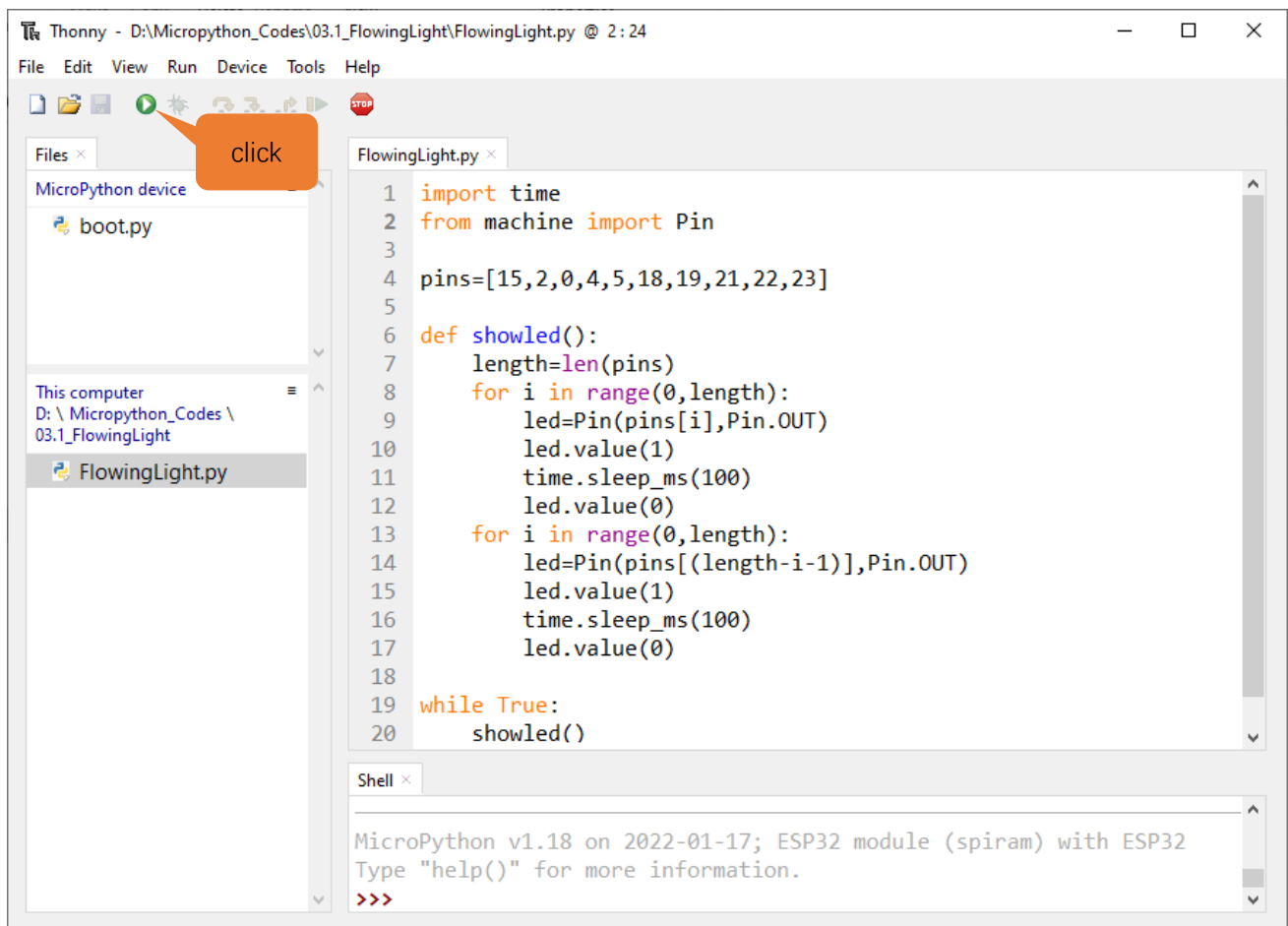
Code

This project is designed to make a flowing water lamp. Which are these actions: First turn LED #1 ON, then turn it OFF. Then turn LED #2 ON, and then turn it OFF... and repeat the same to all 10 LEDs until the last LED is turns OFF. This process is repeated to achieve the “movements” of flowing water.

03.1_FlowingLight

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “03.1_FlowingLight” and double click “FlowingLight.py”.



Click “Run current script” shown in the box above, LED Bar Graph will light up from left to right and then back from right to left.



If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
1  import time
2  from machine import Pin
3
4  pins=[15, 2, 0, 4, 5, 18, 19, 21, 22, 23]
5
6  def showled():
7      length=len(pins)
8      for i in range(0, length):
9          led=Pin(pins[i], Pin. OUT)
10         led.value(1)
11         time.sleep_ms(100)
12         led.value(0)
13     for i in range(0, length):
14         led=Pin(pins[(length-i-1)], Pin. OUT)
15         led.value(1)
16         time.sleep_ms(100)
17         led.value(0)
18
19 while True:
20     showled()
```

Use an array to define 10 GPIO ports connected to LED Bar Graph for easier operation.

```
4  pins=[15, 2, 0, 4, 5, 18, 19, 21, 22, 23]
```

Use len() function to obtain the amount of elements in the list and use a for loop to configure pins as output mode.

```
7      length=len(pins)
8      for i in range(0, length):
9          led=Pin(pins[i], Pin. OUT)
```

Use two for loops to turn on LEDs separately from left to right and then back from right to left.

```
8      for i in range(0, length):
9          led=Pin(pins[i], Pin. OUT)
10         led.value(1)
11         time.sleep_ms(100)
12         led.value(0)
13     for i in range(0, length):
14         led=Pin(pins[(length-i-1)], Pin. OUT)
15         led.value(1)
16         time.sleep_ms(100)
17         led.value(0)
```

Reference

for i in range(start,end,num: int=1)

For loop is used to execute a program endlessly and iterate in the order of items (a list or a string) in the sequence

start: The initial value, the for loop starts with it

end: The ending value, the for loop end with it

num: Num is automatically added each time to the data. The default value is 1

Chapter 4 Analog & PWM

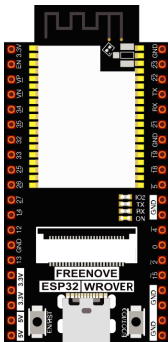
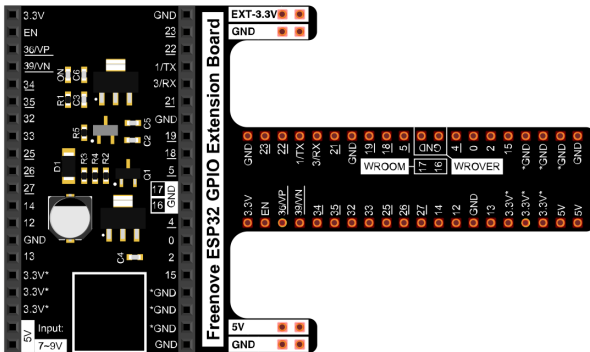
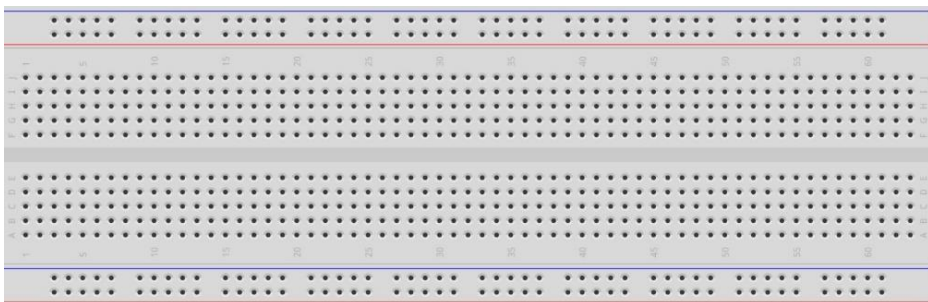
In previous study, we have known that one button has two states: pressed and released, and LED has light-on/off state, then how to enter a middle state? How to output an intermediate state to let LED "semi bright"? That's what we're going to learn.

First, let's learn how to control the brightness of a LED.

Project 4.1 Breathing LED

Breathing light, that is, LED is turned from off to on gradually, and gradually from on to off, just like "breathing". So, how to control the brightness of a LED? We will use PWM to achieve this target.

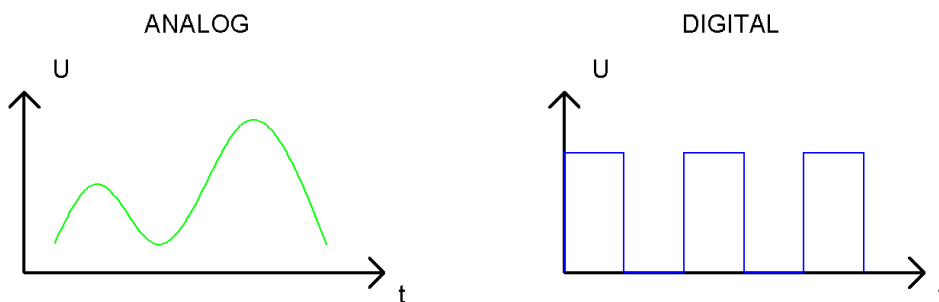
Component List

ESP32-WROVER x1 	GPIO Extension Board x1 	
Breadboard x1 		
LED x1 	Resistor 220Ω x1 	Jumper M/M x2 

Related knowledge

Analog & Digital

An Analog Signal is a continuous signal in both time and value. On the contrary, a Digital Signal or discrete-time signal is a time series consisting of a sequence of quantities. Most signals in life are analog signals. A familiar example of an Analog Signal would be how the temperature throughout the day is continuously changing and could not suddenly change instantaneously from 0°C to 10°C. However, Digital Signals can instantaneously change in value. This change is expressed in numbers as 1 and 0 (the basis of binary code). Their differences can more easily be seen when compared when graphed as below.



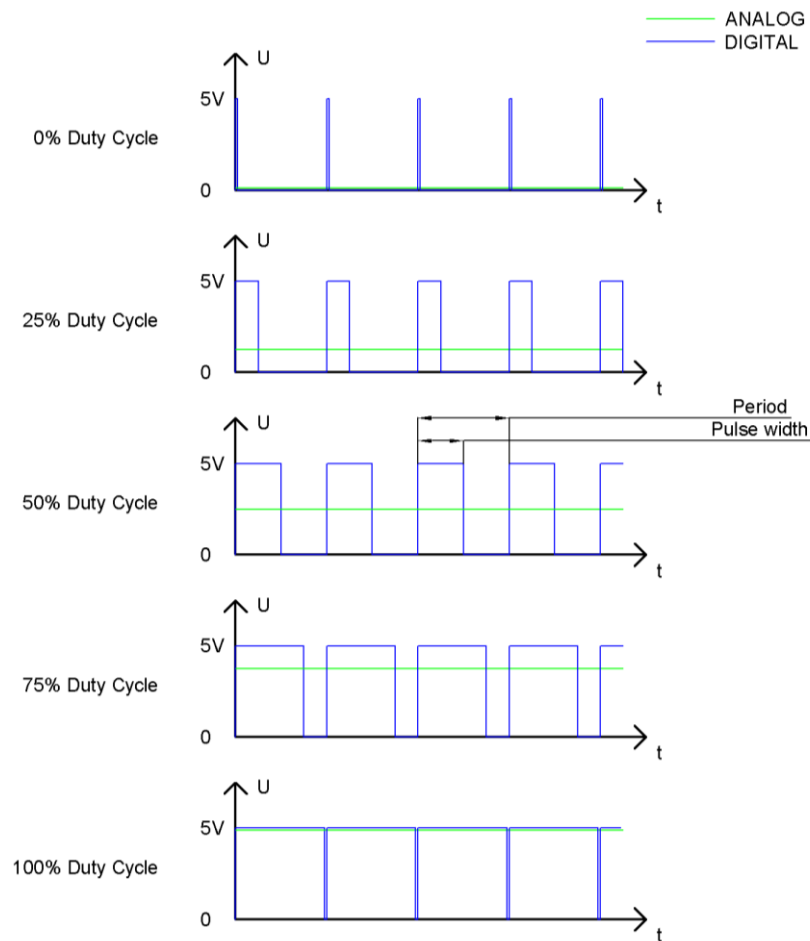
In practical application, we often use binary as the digital signal, that is a series of 0's and 1's. Since a binary signal only has two values (0 or 1), it has great stability and reliability. Lastly, both analog and digital signals can be converted into the other.

PWM

PWM, Pulse-Width Modulation, is a very effective method for using digital signals to control analog circuits. Common processors cannot directly output analog signals. PWM technology makes it very convenient to achieve this conversion (translation of digital to analog signals)

PWM technology uses digital pins to send certain frequencies of square waves, that is, the output of high levels and low levels, which alternately last for a while. The total time for each set of high levels and low levels is generally fixed, which is called the period (Note: the reciprocal of the period is frequency). The time of high level outputs are generally called "pulse width", and the duty cycle is the percentage of the ratio of pulse duration, or pulse width (PW) to the total period (T) of the waveform.

The longer the output of high levels last, the longer the duty cycle and the higher the corresponding voltage in the analog signal will be. The following figures show how the analog signal voltages vary between 0V-5V (high level is 5V) corresponding to the pulse width 0%-100%:



The longer the PWM duty cycle is, the higher the output power will be. Now that we understand this relationship, we can use PWM to control the brightness of an LED or the speed of DC motor and so on.

It is evident from the above that PWM is not real analog, and the effective value of the voltage is equivalent to the corresponding analog. so, we can control the output power of the LED and other output modules to achieve different effects.

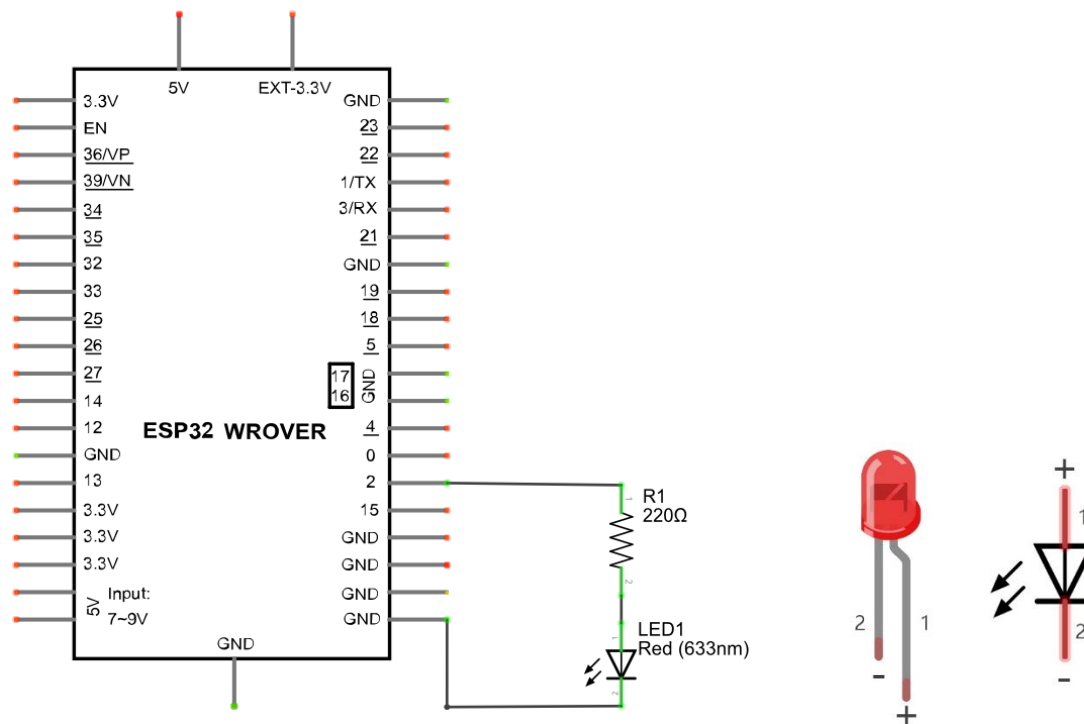
ESP32 and PWM

The ESP32 PWM controller has 8 independent channels, each of which can independently control frequency, duty cycle, and even accuracy. Unlike traditional PWM pins, the PWM output pins of ESP32 are configurable and they can be configured to PWM.

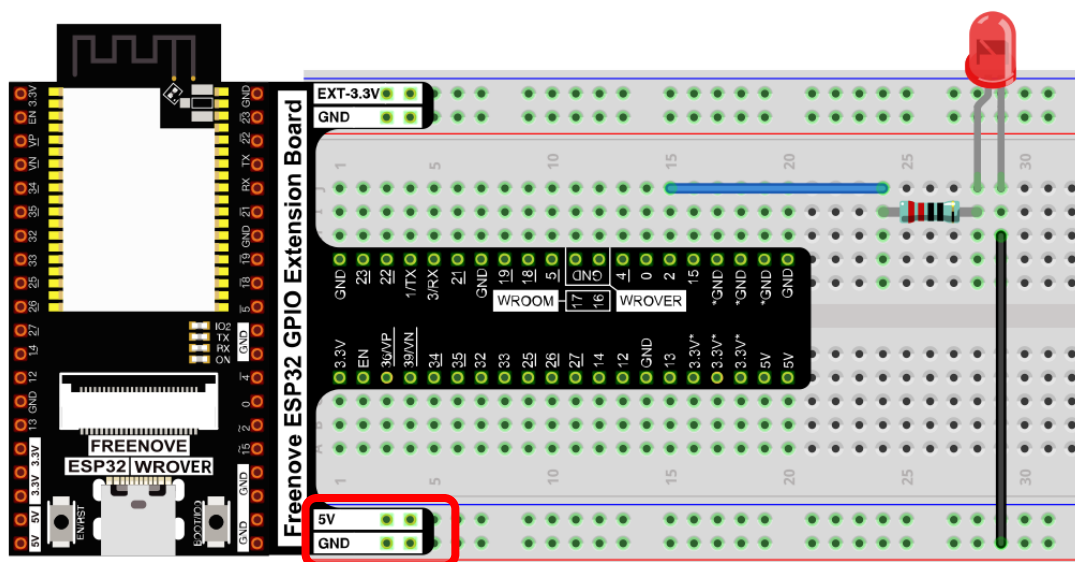
Circuit

This circuit is the same as the one in project Blink.

Schematic diagram



Hardware connection. **If you need any support, please contact us via: support@freenove.com**



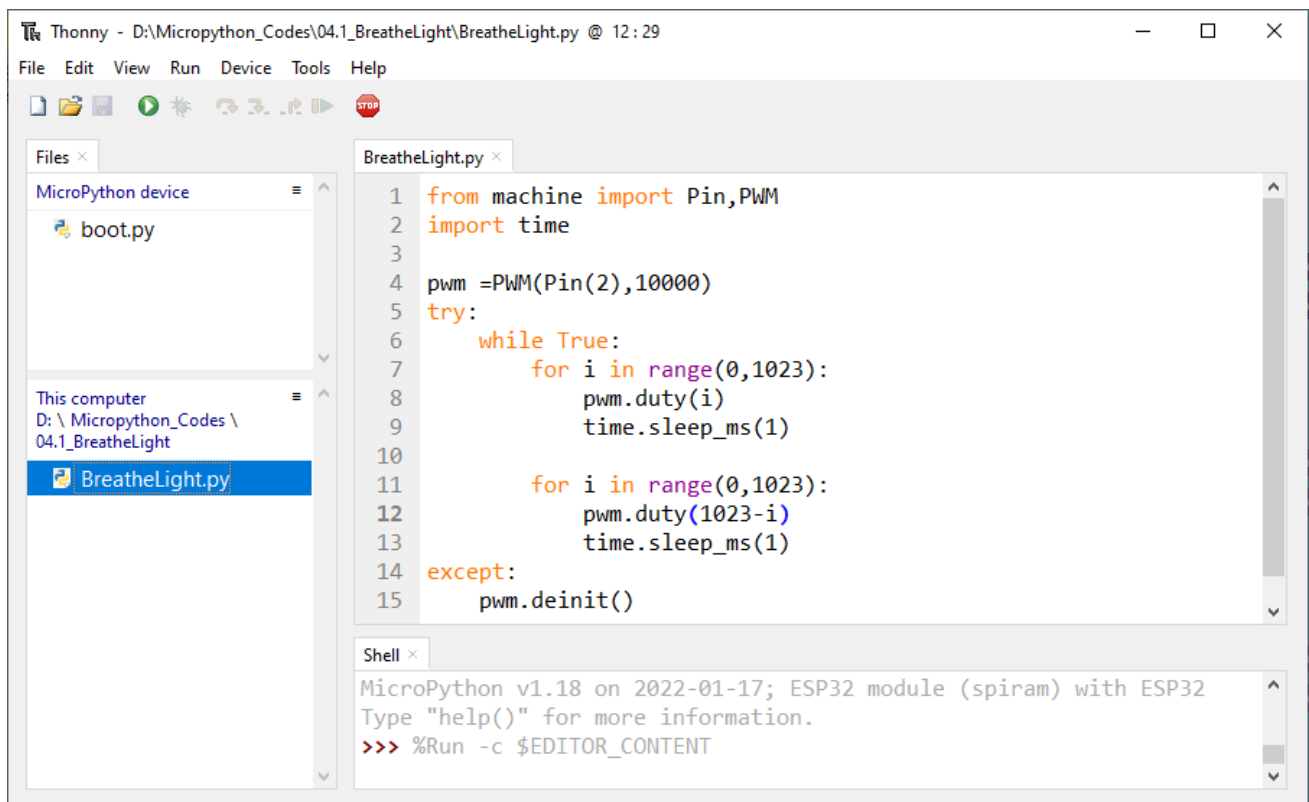
Code

This project is designed to make PWM output GPIO2 with pulse width increasing from 0% to 100%, and then reducing from 100% to 0% gradually.

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “04.1_BreatheLight” and double click “BreatheLight.py”.

04.1_BreatheLight



The screenshot shows the Thonny IDE interface. The main editor window displays the following Python code in `BreatheLight.py`:

```
1 from machine import Pin,PWM
2 import time
3
4 pwm =PWM(Pin(2),10000)
5 try:
6     while True:
7         for i in range(0,1023):
8             pwm.duty(i)
9             time.sleep_ms(1)
10
11         for i in range(0,1023):
12             pwm.duty(1023-i)
13             time.sleep_ms(1)
14 except:
15     pwm.deinit()
```

The left sidebar shows the file explorer with the following structure:

- MicroPython device
 - boot.py
- This computer
 - D: \ Micropython_Codes \ 04.1_BreatheLight
 - BreatheLight.py

The bottom shell window shows the following output:

```
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT
```

Click “Run current script”, and you'll see that LED is turned from ON to OFF and then back from OFF to ON gradually like breathing.



The following is the program code:

```

1  from machine import Pin, PWM
2  import time
3
4  pwm =PWM(Pin(2, Pin. OUT), 10000)
5  try:
6      while True:
7          for i in range(0, 1023):
8              pwm.duty(i)
9              time.sleep_ms(1)
10
11         for i in range(0, 1023):
12             pwm.duty(1023-i)
13             time.sleep_ms(1)
14 except:
15     pwm.deinit()
```

The way that the ESP32 PWM pins output is different from traditionally controllers. It can change frequency and duty cycle by configuring PWM's parameters at the initialization stage. Define GPIO2's output frequency as 10000Hz, and assign them to PWM.

```

4  pwm =PWM(Pin(2, Pin. OUT), 10000)
```

The range of duty cycle is 0-1023, so we use the first for loop to control PWM to change the duty cycle value, making PWM output 0% -100%; Use the second for loop to make PWM output 100%-0%.

```

7      for i in range(0, 1023):
8          pwm.duty(i)
9          time.sleep_ms(1)
10
11     for i in range(0, 1023):
12         pwm.duty(1023-i)
13         time.sleep_ms(1)
```

Each time PWM is used, the hardware Timer will be turned ON to cooperate it. Therefore, after each use of PWM, deinit() needs to be called to turned OFF the timer. Otherwise, the PWM may fail to work next time.

```

15  pwm.deinit()
```

Reference

Class PWM(pin, freq)

Before each use of PWM module, please add the statement **"from machine import PWM"** to the top of the python file.

pin: PWM pins are supported, such as Pin(0)、Pin(2)、Pin(4)、Pin(5)、Pin(10)、Pin(12~19)、Pin(21)、Pin(22)、Pin(23)、Pin(25~27).

freq: Output frequency, with the range of 0-78125 Hz

duty: Duty cycle, with the range of 0-1023.

PWM.init(freq, duty): Initialize PWM, parameters are the same as above.

PWM.freq([freq_val]): When there is no parameter, the function obtains and returns PWM frequency; When parameters are set, the function is used to set PWM frequency and returns nothing.

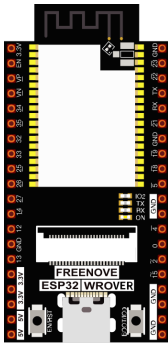
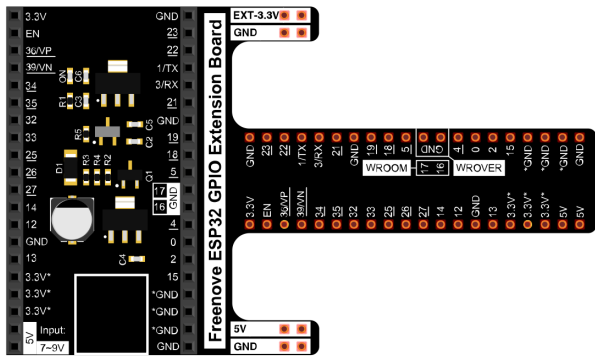
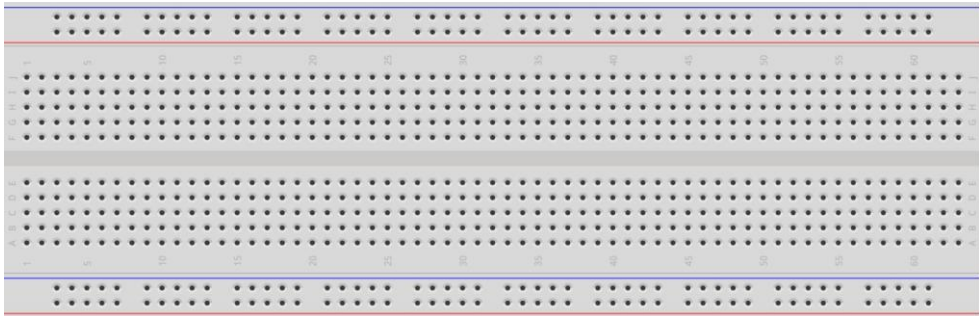
PWM.duty([duty_val]): When there is no parameter, the function obtains and returns PWM duty cycle; When parameters are set, the function is used to set PWM duty cycle.

PWM.deinit(): Turn OFF PWM.

Project 4.2 Meteor Flowing Light

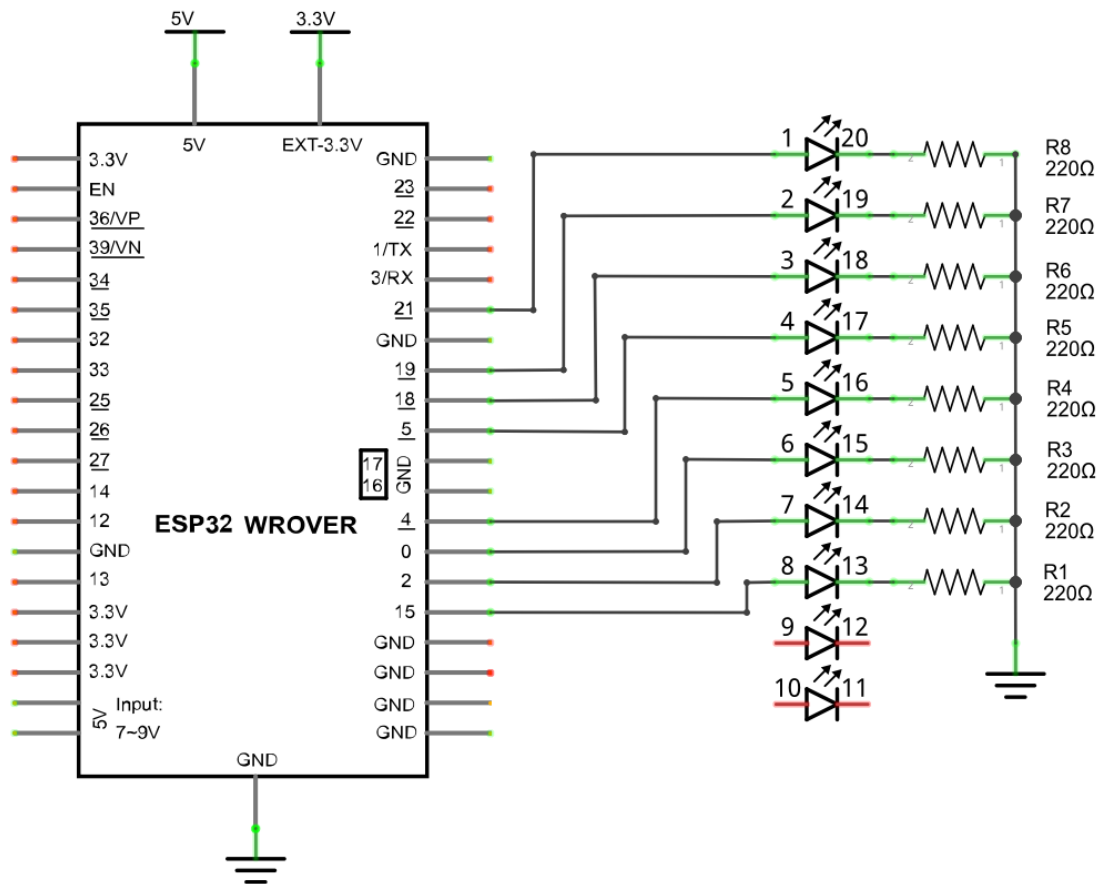
After learning about PWM, we can use it to control LED Bar Graph and realize a cooler Flowing Light.

Component List

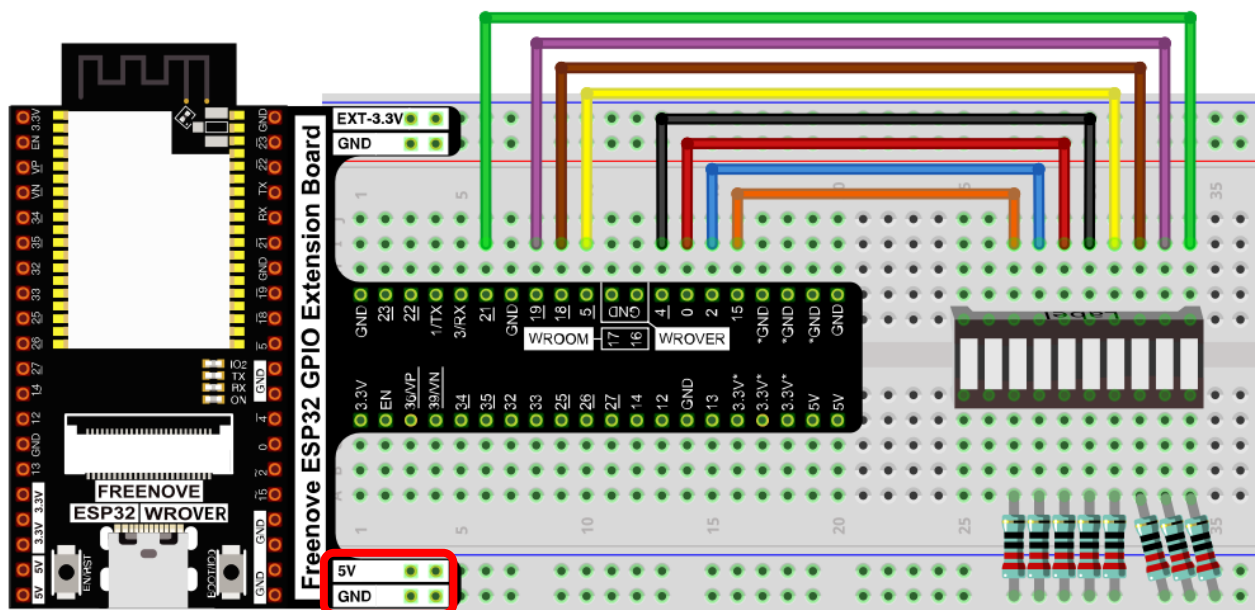
ESP32-WROVER x1 	GPIO Extension Board x1 	
Breadboard x1 		
Jumper M/M x10 	LED bar graph x1 	Resistor 220Ω x10 

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



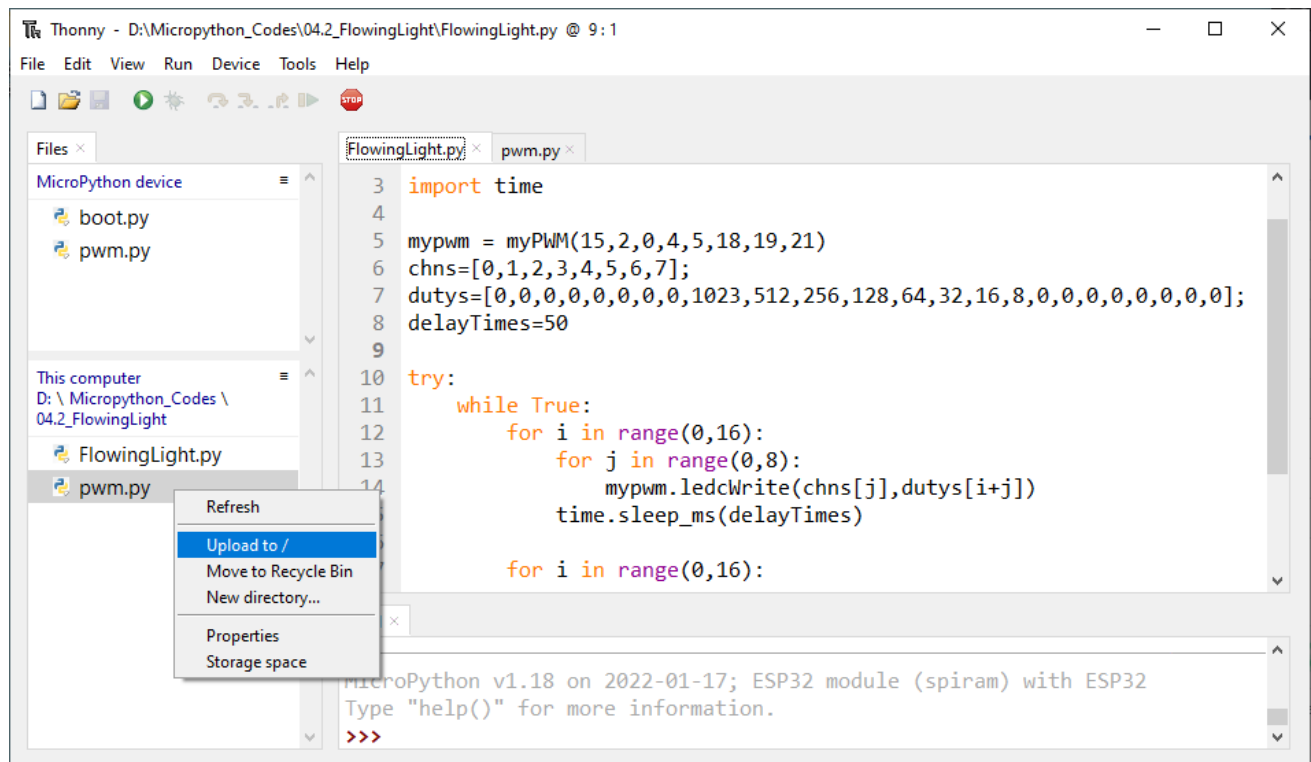
If LEDbar doesn't work, try to rotate LEDbar for 180°. The label is random.

Code

Flowing Light with tail was implemented with PWM.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “04.2_FlowingLight”. Select “pwm.py”, right click to select “Upload to /”, wait for “pwm.py” to be uploaded to ESP32-WROVER and then double click “FlowingLight.py”

04.2_FlowingLight



Click “Run current script”, and LED Bar Graph will gradually light up and out from left to right, then light up and out from right to left.

The following is the program code:

```

1  from machine import Pin, PWM
2  from pwm import myPWM
3  import time
4
5  mypwm = myPWM(15, 2, 0, 4, 5, 18, 19, 21)
6  chns=[0, 1, 2, 3, 4, 5, 6, 7];
7  dutys=[0, 0, 0, 0, 0, 0, 0, 1023, 512, 256, 128, 64, 32, 16, 8, 0, 0, 0, 0, 0, 0, 0];
8  delayTimes=50
9
10 try:
11     while True:
12         for i in range(0, 16):
13             for j in range(0, 8):
14                 mypwm.ledcWrite(chns[j], dutys[i+j])
15                 time.sleep_ms(delayTimes)
16
17         for i in range(0, 16):
18             for j in range(0, 8):
19                 mypwm.ledcWrite(chns[7-j], dutys[i+j])
20                 time.sleep_ms(delayTimes)
21 except:
22     mypwm.deinit()

```

Import the object myPWM from pwm.py and set corresponding pins for PWM channel.

```

2  from pwm import myPWM
...  ...
5  mypwm = myPWM(15, 2, 0, 4, 5, 18, 19, 21)

```

First we defined 8 GPIO, 8 PWM channels, and 24 pulse width values.

```

5  mypwm = myPWM(15, 2, 0, 4, 5, 18, 19, 21)
6  chns=[0, 1, 2, 3, 4, 5, 6, 7];
7  dutys=[0, 0, 0, 0, 0, 0, 0, 1023, 512, 256, 128, 64, 32, 16, 8, 0, 0, 0, 0, 0, 0, 0];

```

Call ledcWrite() to set duty cycle dutys[i+j] for the chns[j] channel of PWM.

```

14 mypwm.ledcWrite(chns[j], dutys[i+j])

```

Close the PWM of the object myPWM.

```

14 mypwm.deinit()

```

In the code, a nesting of two for loops are used to achieve this effect.

```

12     for i in range(0, 16):
13         for j in range(0, 8):
14             mypwm.ledcWrite(chns[j], dutys[i+j])
15             time.sleep_ms(delayTimes)
16
17     for i in range(0, 16):
18         for j in range(0, 8):
19             mypwm.ledcWrite(chns[7-j], dutys[i+j])
20             time.sleep_ms(delayTimes)

```

In the main function, a nested for loop is used to control the pulse width of the PWM. Every time *i* in the first for loop increases by 1, the LED Bar Graph will move one grid, and gradually change according to the value in the array *dutys*. As shown in the following table, the value in the second row is the value of the array *dutys*, and the 8 green grids in each row below represent the 8 LEDs on the LED Bar Graph. Each time *i* increases by 1, the value of the LED Bar Graph will move to the right by one grid, and when it reaches the end, it will move from the end to the starting point, achieving the desired effect.

0	1	2	3	4	5	6	7	8	9	1	1	1	1	1	1	1	1	1	2	2	2	2	2	
										0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
d	0	0	0	0	0	0	0		1	5	2	1	6	3	1	8	0	0	0	0	0	0	0	
i									0	1	5	2	4	2	6									
									2	2	6	8												
									3															
0																								
1																								
...																								
14																								
15																								
16																								

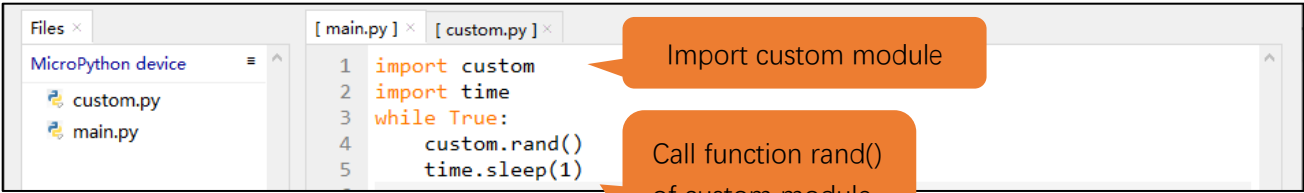
How to import a custom python module

Each Python file, as long as it's stored on the file system of ESP32, is a module. To import a custom module, the module file needs to be located in the MicroPython environment variable path or in the same path as the currently running program.

First, customize a python module "custom.py". Create a new py file and name it "custom.py". Write code to it and save it to ESP32.



Second, import custom module "custom" to main.py



The screenshot shows a MicroPython IDE interface. On the left, a file explorer lists 'custom.py' and 'main.py'. The main editor window shows the code in 'main.py':

```
1 import custom
2 import time
3 while True:
4     custom.rand()
5     time.sleep(1)
```

Two orange callout boxes provide annotations:

- The first callout points to line 1 and contains the text: "Import custom module".
- The second callout points to line 4 and contains the text: "Call function rand() of custom module".

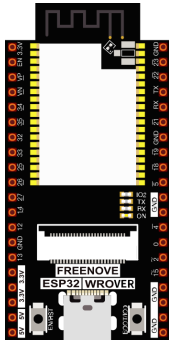
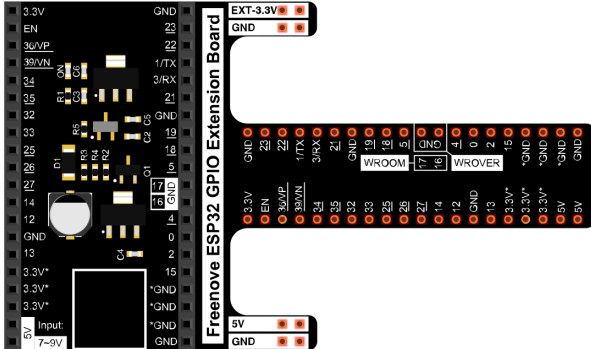
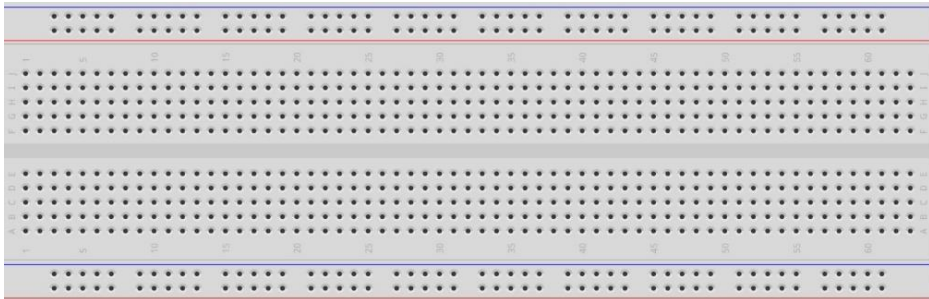
Chapter 5 RGBLED

In this chapter, we will learn how to control a RGBLED. It can emit different colors of light. Next, we will use RGBLED to make a multicolored light.

Project 5.1 Random Color Light

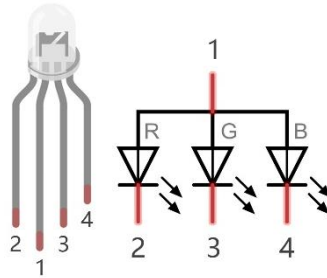
In this project, we will make a multicolored LED. And we can control RGBLED to switch different colors automatically.

Component List

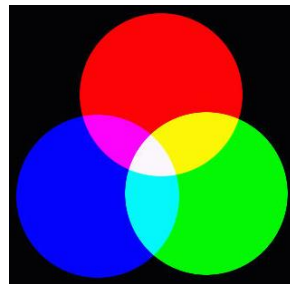
ESP32-WROVER x1 	GPIO Extension Board x1 	
Breadboard x1 		
RGBLED x1 	Resistor 220Ω x3 	Jumper M/M x4 

Related knowledge

RGB LED has integrated 3 LEDs that can respectively emit red, green and blue light. And it has 4 pins. The long pin (1) is the common port, that is, 3 LED 's positive or negative port. The RGB LED with common positive port and its symbol is shown below. We can make RGB LED emit various colors of light by controlling these 3 LEDs to emit light with different brightness,



Red, green, and blue light are known as three primary colors. When you combine these three primary-color lights with different brightness, it can produce almost all kinds of visible lights. Computer screens, single pixel of cell phone screen, neon, and etc. are working under this principle.

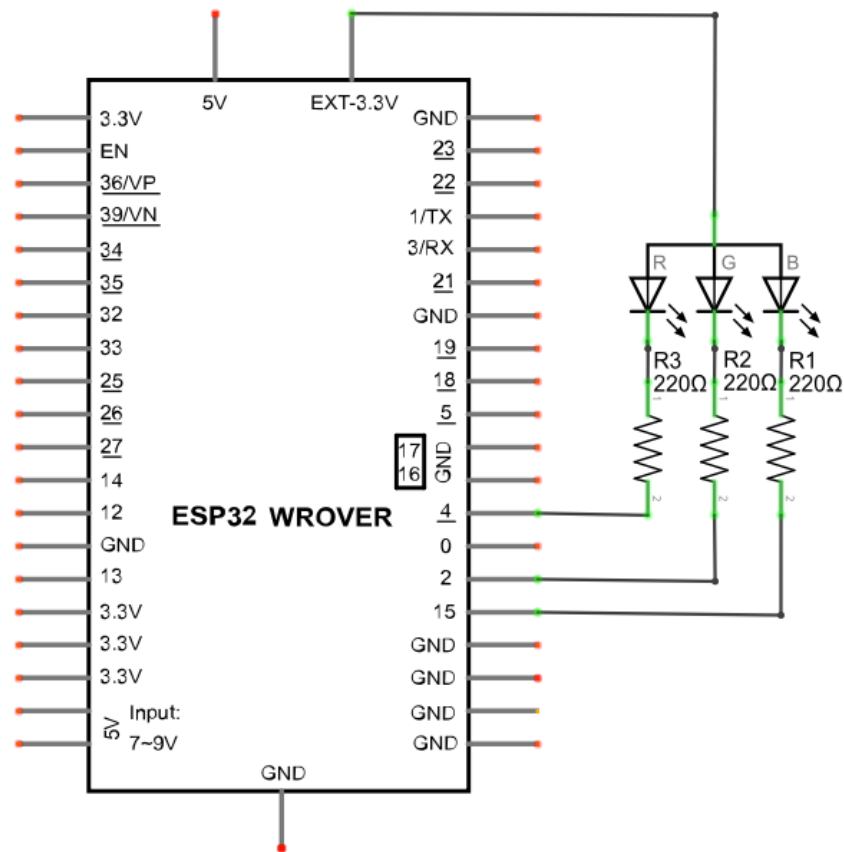


RGB

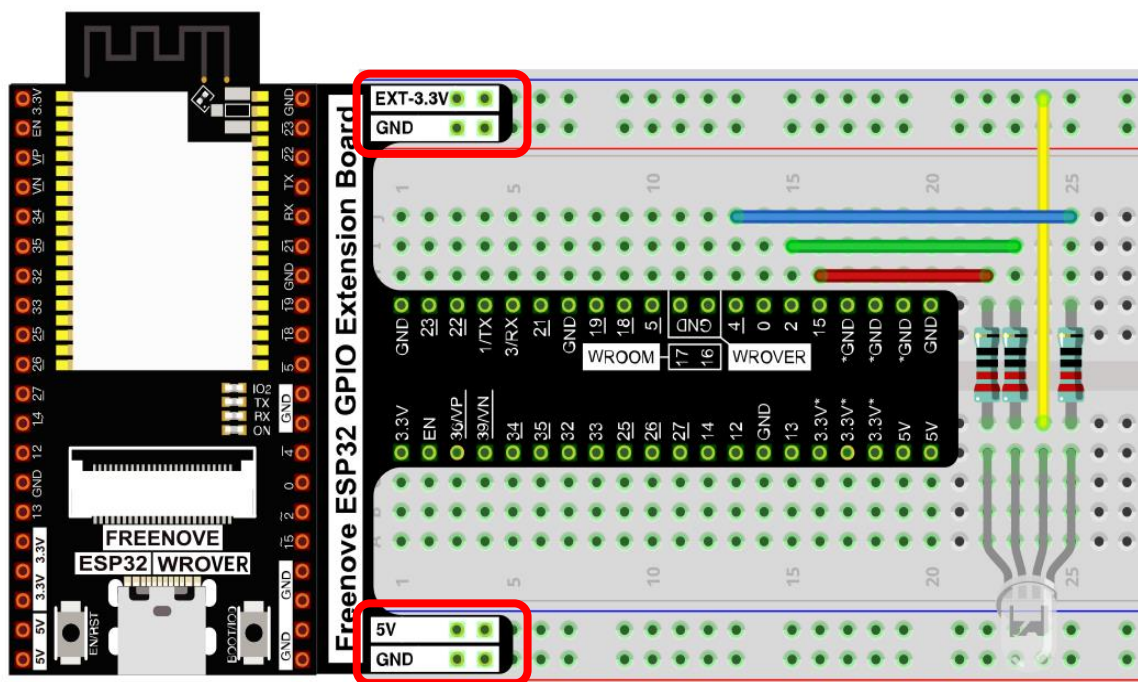
If we use three 10-bit PWM to control the RGBLED, in theory, we can create $2^{10} \times 2^{10} \times 2^{10} = 1,073,741,824$ (1 billion) colors through different combinations.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



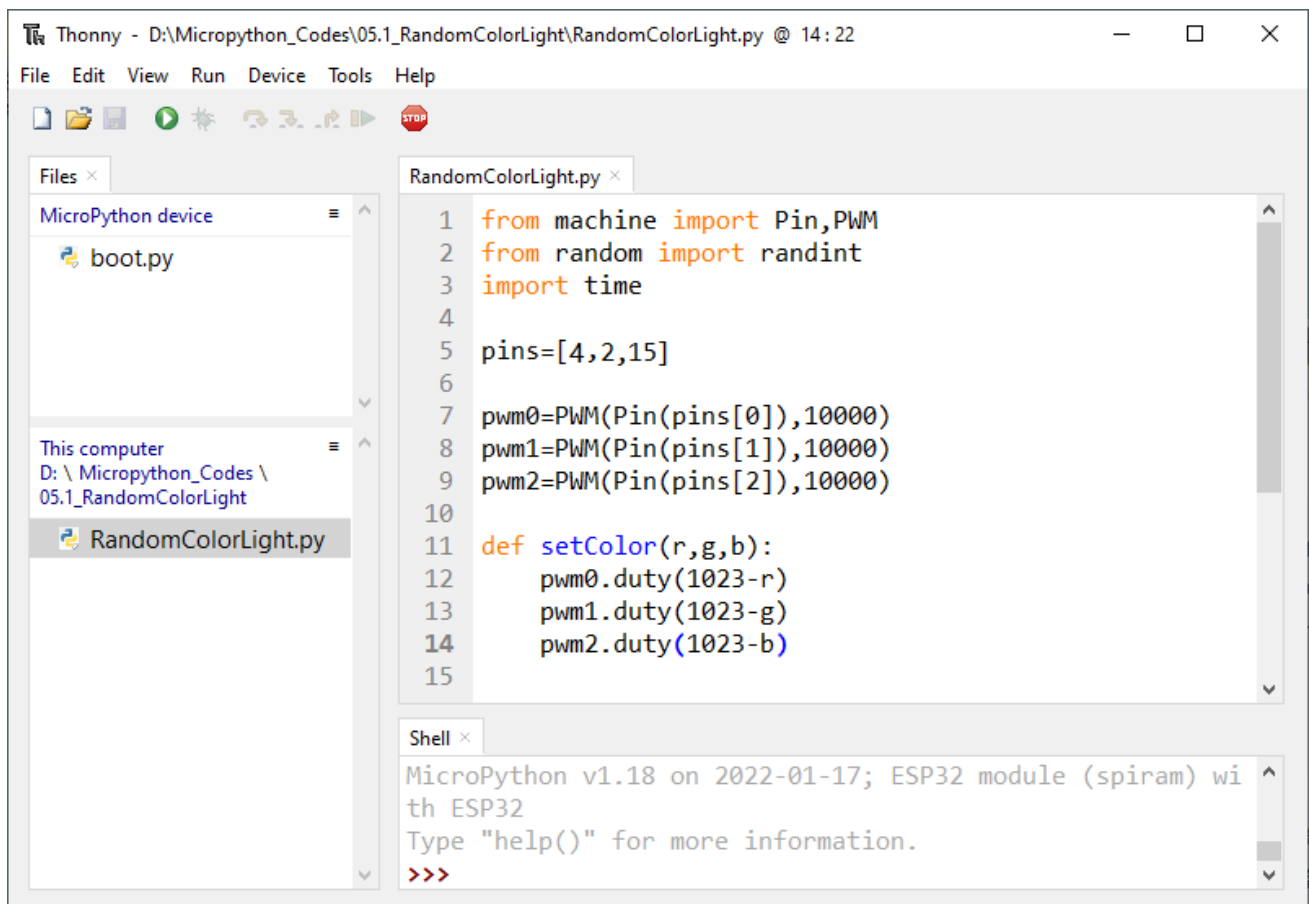
Code

We need to create three PWM channels and use random duty cycle to make random RGBLED color.

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “05.1_RandomColorLight” and double click “RandomColorLight.py”.

05.1_RandomColorLight



The screenshot shows the Thonny IDE interface. The main editor displays the `RandomColorLight.py` script. The left sidebar shows the file explorer with the project structure. The bottom panel shows the shell output.

```
Thonny - D:\Micropython_Codes\05.1_RandomColorLight\RandomColorLight.py @ 14:22
File Edit View Run Device Tools Help

Files
MicroPython device
  boot.py
This computer
  D: \ Micropython_Codes \
    05.1_RandomColorLight
      RandomColorLight.py

RandomColorLight.py
1 from machine import Pin,PWM
2 from random import randint
3 import time
4
5 pins=[4,2,15]
6
7 pwm0=PWM(Pin(pins[0]),10000)
8 pwm1=PWM(Pin(pins[1]),10000)
9 pwm2=PWM(Pin(pins[2]),10000)
10
11 def setColor(r,g,b):
12     pwm0.duty(1023-r)
13     pwm1.duty(1023-g)
14     pwm2.duty(1023-b)
15

Shell
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>>
```

Click “Run current script”, RGBLED begins to display random colors.

If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```

1  from machine import Pin, PWM
2  from random import randint
3  import time
4
5  pins=[4, 2, 15]
6
7  pwm0=PWM(Pin(pins[0]), 10000)
8  pwm1=PWM(Pin(pins[1]), 10000)
9  pwm2=PWM(Pin(pins[2]), 10000)
10
11 def setColor(r, g, b):
12     pwm0.duty(1023-r)
13     pwm1.duty(1023-g)
14     pwm2.duty(1023-b)
15
16 try:
17     while True:
18         red = randint(0, 1023)
19         green = randint(0, 1023)
20         blue = randint(0, 1023)
21         setColor(red, green, blue)
22         time.sleep_ms(200)
23 except:
24     pwm0.deinit()
25     pwm1.deinit()
26     pwm2.deinit()

```

Import Pin, PWM and Random Function modules.

```

12 from machine import Pin, PWM
13 from random import randint
14 import time

```

Configure output mode of GPIO15, GPIO2 and GPIO0 as PWM output and PWM frequency as 10000Hz

```

5 pins=[4, 2, 15]
6
7 pwm0=PWM(Pin(pins[0]), 10000)
8 pwm1=PWM(Pin(pins[1]), 10000)
9 pwm2=PWM(Pin(pins[2]), 10000)

```

Define a function to set the color of RGBLED.

```

11 def setColor(r, g, b):
12     pwm0.duty(1023-r)
13     pwm1.duty(1023-g)
14     pwm2.duty(1023-b)

```


Call random function randint() to generate a random number in the range of 0-1023 and assign the value to red.

```
18 red = randint(0, 1023)
```

Obtain 3 random number every 200 milliseconds and call function setColor to make RGBLED display dazzling colors.

```
17 while True:
18     red = randint(0, 1023)
19     green = randint(0, 1023)
20     blue = randint(0, 1023)
21     setColor(red, green, blue)
22     time.sleep_ms(200)
```

Reference

Class random

Before each use of the module **random**, please add the statement “**import random**” to the top of Python file.

randint(start, end): Randomly generates an integer between the value of start and end.

start: Starting value in the specified range, which would be included in the range.

end: Ending value in the specified range, which would be included in the range.

random(): Randomly generates a floating point number between 0 and 1.

random.uniform(start, end): Randomly generates a floating point number between the value of start and end

start: Starting value in the specified range, which would be included in the range.

end: Ending value in the specified range, which would be included in the range.

random.getrandbits(size): Generates an integer with **size** random bits

For example:

size = 4, it generates an integer in the range of 0 to 0b1111

size = 8, it generates an integer in the range of 0 to 0b11111111

random.randrange(start, end, step): Randomly generates a positive integer in the range from start to end and increment to step.

start: Starting value in the specified range, which would be included in the range

end: Ending value in the specified range, which would be included in the range.

step: An integer specifying the incrementation.

random.seed(sed): Specifies a random seed, usually being applied in conjunction with other random number generators

sed: Random seed, a starting point in generating random numbers.

random.choice(obj): Randomly generates an element from the object obj.

obj: list of elements

Project 5.2 Gradient Color Light

In the previous project, we have mastered the usage of RGBLED, but the random color display is rather stiff. This project will realize a fashionable Light with soft color changes.

Component list, the circuit is exactly the same as the project random color light.

Using a color model, the color changes from 0 to 255 as shown below.



In this code, the color model will be implemented and RGBLED will change colors along the model.

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "05.2_GradientColorLight" and double click "GradientColorLight.py".

05.2_GradientColorLight

The following is the program code:

```

1  from machine import Pin,PWM
2  import time
3
4  pins=[4, 2, 15];
5
6  pwm0=PWM(Pin(pins[0]), 1000)
7  pwm1=PWM(Pin(pins[1]), 1000)
8  pwm2=PWM(Pin(pins[2]), 1000)
9
10 red=0          #red
11 green=0        #green
12 blue=0         #blue
13
14 def setColor():
15     pwm0.duty(red)
16     pwm1.duty(green)
17     pwm2.duty(blue)
18
19 def wheel(pos):
20     global red, green, blue
21     WheelPos=pos%1023
22     print(WheelPos)
23     if WheelPos<341:
24         red=1023-WheelPos*3
25         green=WheelPos*3
26         blue=0

```

```
27
28     elif WheelPos>=341 and WheelPos<682:
29         WheelPos -= 341;
30         red=0
31         green=1023-WheelPos*3
32         blue=WheelPos*3
33     else :
34         WheelPos -= 682;
35         red=WheelPos*3
36         green=0
37         blue=1023-WheelPos*3
38
39 try:
40     while True:
41         for i in range(0, 1023):
42             wheel(i)
43             setColor()
44             time.sleep_ms(15)
45 except:
46     pwm0.deinit()
47     pwm1.deinit()
48     pwm2.deinit()
```

The function wheel() is a color selection method of the color model introduced earlier. The value range of the parameter pos is 0-1023. The function will return a data containing the duty cycle values of 3 pins.

```
19 def wheel(pos):
20     global red, green, blue
21     WheelPos=pos%1023
22     print(WheelPos)
23     if WheelPos<341:
24         red=1023-WheelPos*3
25         green=WheelPos*3
26         blue=0
27
28     elif WheelPos>=341 and WheelPos<682:
29         WheelPos -= 341;
30         red=0
31         green=1023-WheelPos*3
32         blue=WheelPos*3
33     else :
34         WheelPos -= 682;
35         red=WheelPos*3
36         green=0
37         blue=1023-WheelPos*3
```

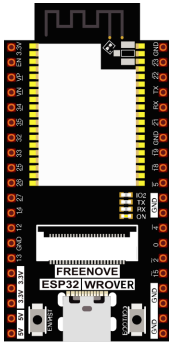
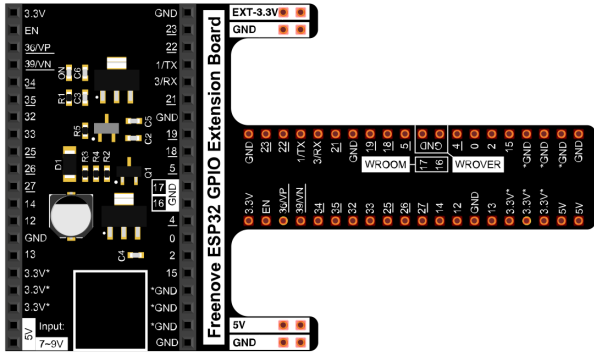
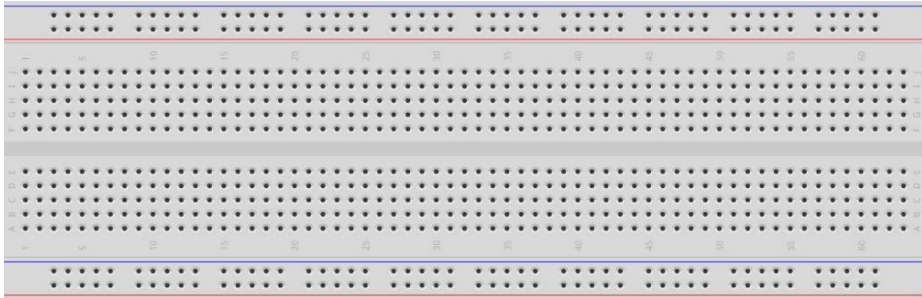
Chapter 6 Buzzer

In this chapter, we will learn about buzzers and the sounds they make.

Project 6.1 Doorbell

We will make this kind of doorbell: when the button is pressed, the buzzer sounds; and when the button is released, the buzzer stops sounding.

Component List

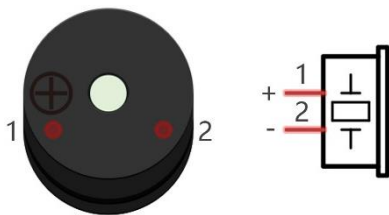
ESP32-WROVER x1 	GPIO Extension Board x1 			
Breadboard x1 				
Jumper M/M x6 				
NPN transistorx1 (S8050) 	Active buzzer x1 	Push button x1 	Resistor 1kΩ x1 	Resistor 10kΩ x2 

Component knowledge

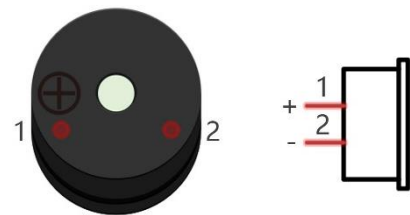
Buzzer

Buzzer is a sounding component, which is widely used in electronic devices such as calculator, electronic warning clock and alarm. Buzzer has two types: active and passive. Active buzzer has oscillator inside, which will sound as long as it is supplied with power. Passive buzzer requires external oscillator signal (generally use PWM with different frequency) to make a sound.

Active buzzer



Passive buzzer



Active buzzer is easy to use. Generally, it can only make a specific frequency of sound. Passive buzzer requires an external circuit to make a sound, but it can be controlled to make a sound with different frequency. The resonant frequency of the passive buzzer is 2kHz, which means the passive buzzer is loudest when its resonant frequency is 2kHz.

Next, we will use an active buzzer to make a doorbell and a passive buzzer to make an alarm.

How to identify active and passive buzzer?

1. Usually, there is a label on the surface of active buzzer covering the vocal hole, but this is not an absolute judgment method.
2. Active buzzers are more complex than passive buzzers in their manufacture. There are many circuits and crystal oscillator elements inside active buzzers; all of this is usually protected with a waterproof coating (and a housing) exposing only its pins from the underside. On the other hand, passive buzzers do not have protective coatings on their underside. From the pin holes viewing of a passive buzzer, you can see the circuit board, coils, and a permanent magnet (all or any combination of these components depending on the model).

Active buzzer



Passive buzzer

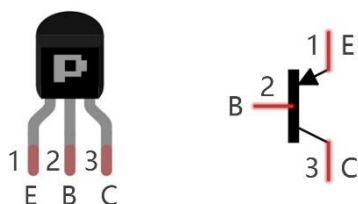


Transistor

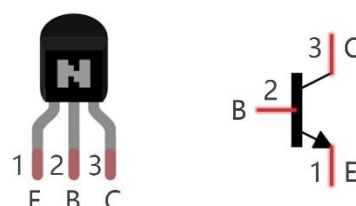
Because the buzzer requires such large current that GPIO of ESP32 output capability cannot meet the requirement, a transistor of NPN type is needed here to amplify the current.

Transistor, the full name: semiconductor transistor, is a semiconductor device that controls current. Transistor can be used to amplify weak signal, or works as a switch. It has three electrodes(PINs): base (b), collector (c) and emitter (e). When there is current passing between "be", "ce" will allow several-fold current (transistor magnification) pass, at this point, transistor works in the amplifying area. When current between "be" exceeds a certain value, "ce" will not allow current to increase any longer, at this point, transistor works in the saturation area. Transistor has two types as shown below: PNP and NPN,

PNP transistor



NPN transistor

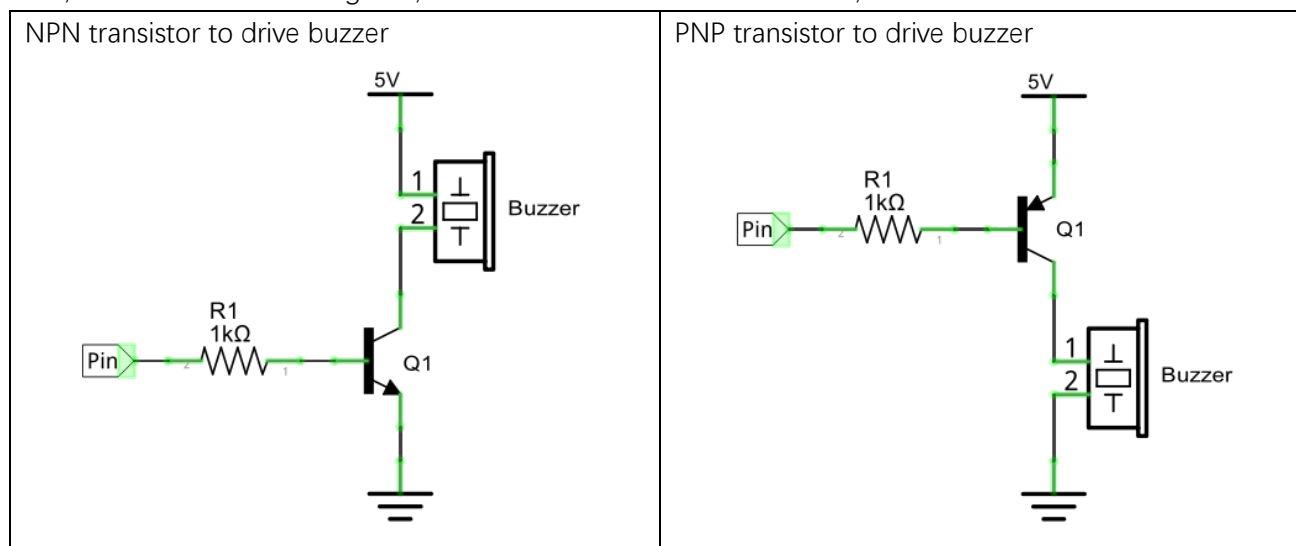


In our kit, the PNP transistor is marked with 8550, and the NPN transistor is marked with 8050.

Based on the transistor's characteristics, it is often used as a switch in digital circuits. As micro-controller's capacity to output current is very weak, we will use transistor to amplify current and drive large-current components.

When using NPN transistor to drive buzzer, we often adopt the following method. If GPIO outputs high level, current will flow through R1, the transistor will get conducted, and the buzzer will sound. If GPIO outputs low level, no current flows through R1, the transistor will not be conducted, and buzzer will not sound.

When using PNP transistor to drive buzzer, we often adopt the following method. If GPIO outputs low level, current will flow through R1, the transistor will get conducted, and the buzzer will sound. If GPIO outputs high level, no current flows through R1, the transistor will not be conducted, and buzzer will not sound.



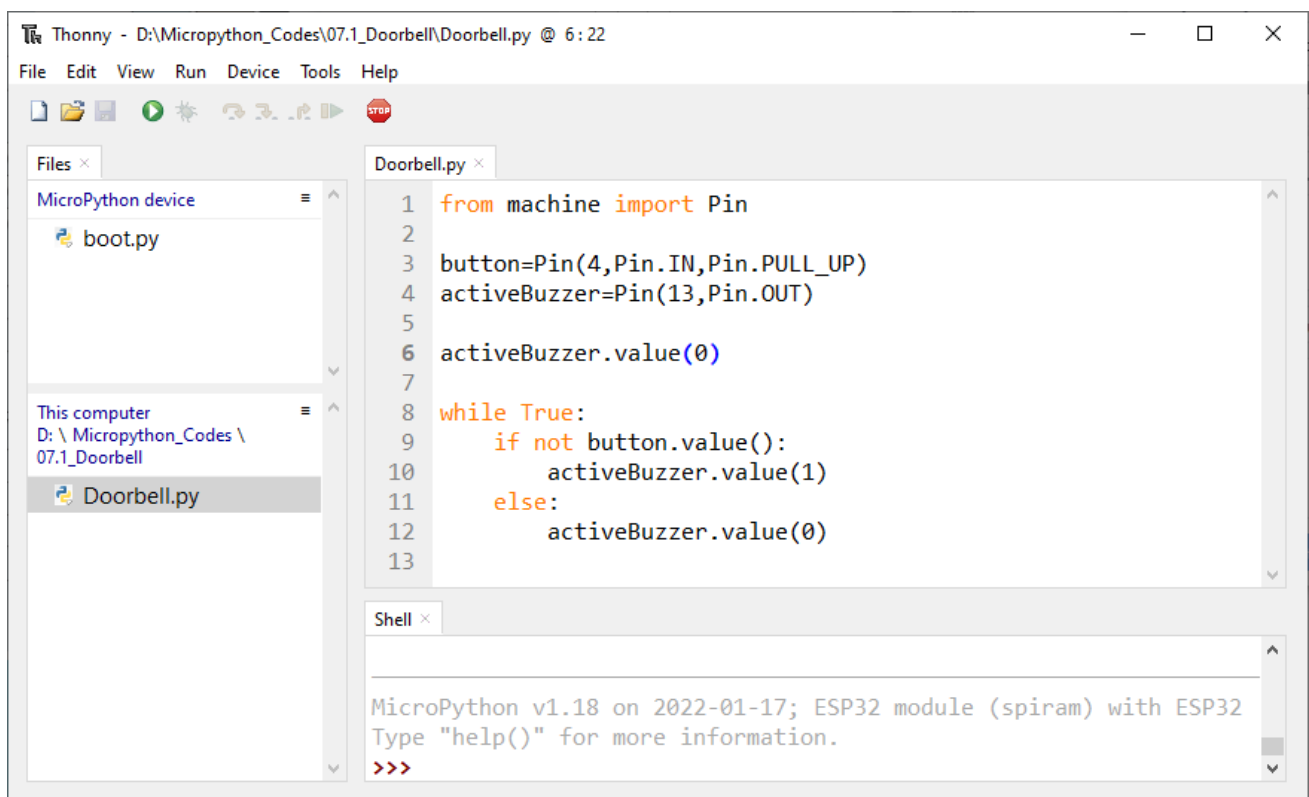
Code

In this project, a buzzer will be controlled by a push button switch. When the button switch is pressed, the buzzer sounds and when the button is released, the buzzer stops. It is analogous to our earlier project that controlled an LED ON and OFF.

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “06.1_Doorbell” and double click “Doorbell.py”.

06.1_Doorbell



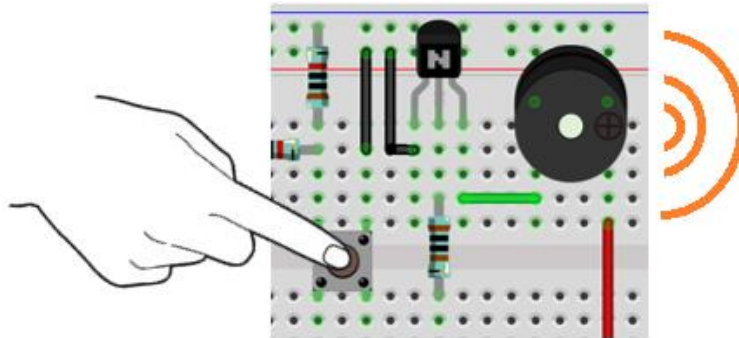
The screenshot shows the Thonny IDE interface. The top menu bar includes File, Edit, View, Run, Device, Tools, and Help. Below the menu is a toolbar with icons for file operations and execution. The left sidebar shows the file explorer with two sections: 'MicroPython device' containing 'boot.py' and 'This computer' containing 'D:\Micropython_Codes\07.1_Doorbell' and 'Doorbell.py'. The main editor window displays the code for 'Doorbell.py'.

```
1 from machine import Pin
2
3 button=Pin(4,Pin.IN,Pin.PULL_UP)
4 activeBuzzer=Pin(13,Pin.OUT)
5
6 activeBuzzer.value(0)
7
8 while True:
9     if not button.value():
10         activeBuzzer.value(1)
11     else:
12         activeBuzzer.value(0)
13
```

The bottom panel shows the shell output:

```
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>>
```


Click “Run current script”, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.



The following is the program code:

```
1  from machine import Pin
2
3  button=Pin(4,Pin.IN,Pin.PULL_UP)
4  activeBuzzer=Pin(13,Pin.OUT)
5
6  activeBuzzer.value(0)
7
8  while True:
9      if not button.value():
10         activeBuzzer.value(1)
11     else:
12         activeBuzzer.value(0)
```

The code is logically the same as using button to control LED.

Project 6.2 Alertor

Next, we will use a passive buzzer to make an alarm.

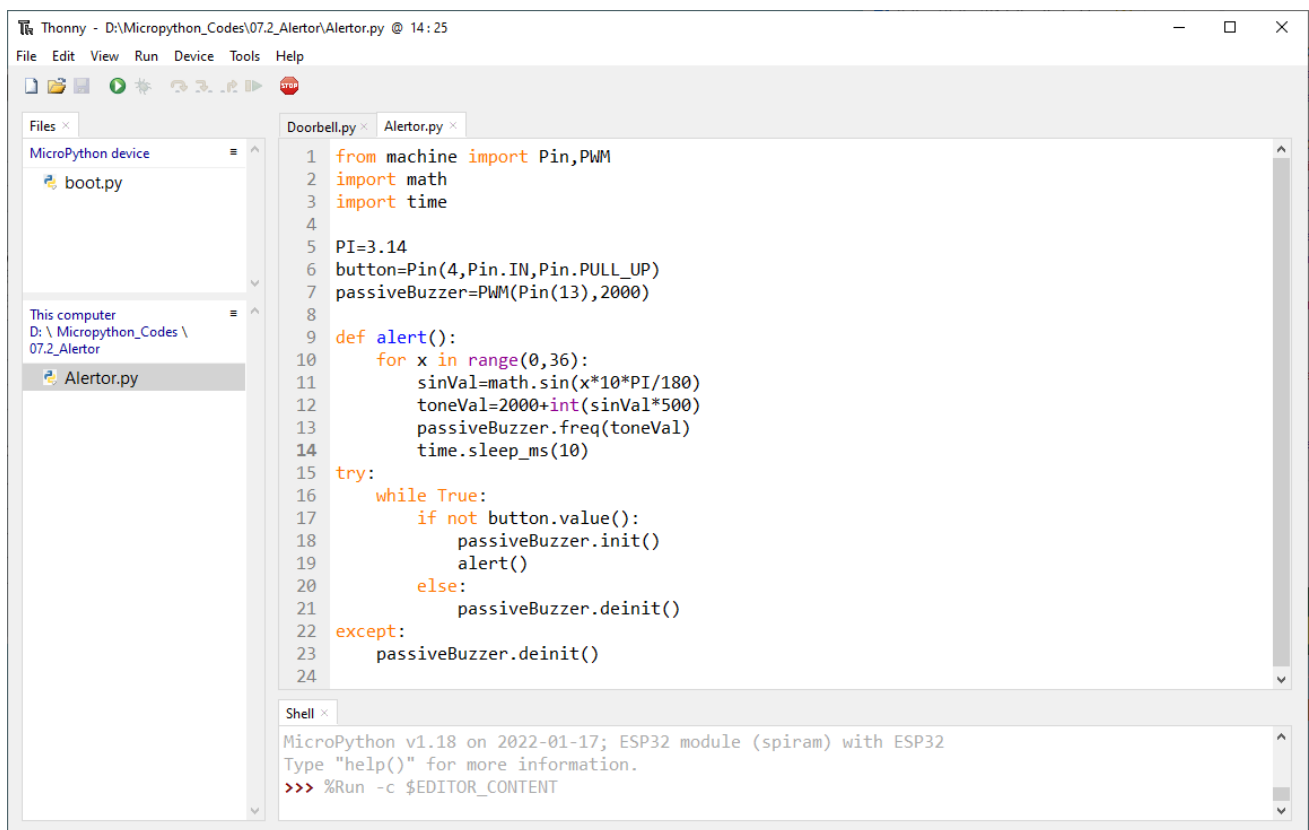
Component list and the circuit part is similar to last section. In the Doorbell circuit only the **active buzzer** needs to be **replaced** with a **passive buzzer**.

Code

In this project, the buzzer alarm is controlled by the button. Press the button, then buzzer sounds. If you release the button, the buzzer will stop sounding. In the logic, it is the same as using button to control LED. In the control method, passive buzzer requires PWM of certain frequency to sound.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “06.2_Alertor”, and double click “Alertor.py”.

06.2_Alertor



```

1 from machine import Pin,PWM
2 import math
3 import time
4
5 PI=3.14
6 button=Pin(4,Pin.IN,Pin.PULL_UP)
7 passiveBuzzer=PWM(Pin(13),2000)
8
9 def alert():
10     for x in range(0,36):
11         sinVal=math.sin(x*10*PI/180)
12         toneVal=2000+int(sinVal*500)
13         passiveBuzzer.freq(toneVal)
14         time.sleep_ms(10)
15
16 try:
17     while True:
18         if not button.value():
19             passiveBuzzer.init()
20             alert()
21         else:
22             passiveBuzzer.deinit()
23 except:
24     passiveBuzzer.deinit()

```

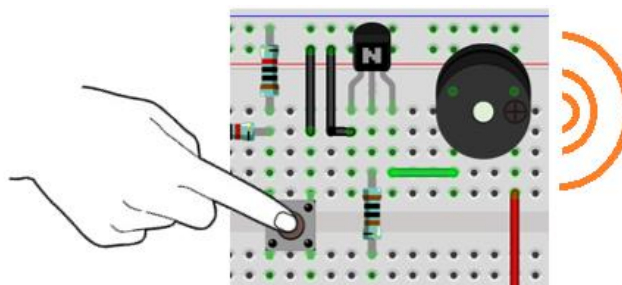
Shell

```

MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

```

Click “Run current script”, press the button, then alarm sounds. And when the button is release, the alarm will stop sounding.



The following is the program code:

```

1  from machine import Pin, PWM
2  import math
3  import time
4
5  PI=3.14
6  button=Pin(4, Pin.IN, Pin.PULL_UP)
7  passiveBuzzer=PWM(Pin(13), 2000)
8
9  def alert():
10     for x in range(0, 36):
11         sinVal=math.sin(x*10*PI/180)
12         toneVal=2000+int(sinVal*500)
13         passiveBuzzer.freq(toneVal)
14         time.sleep_ms(10)
15     try:
16         while True:
17             if not button.value():
18                 passiveBuzzer.init()
19                 alert()
20             else:
21                 passiveBuzzer.deinit()
22     except:
23         passiveBuzzer.deinit()

```

Import PWM, Pin, math and time modules.

```

1  from machine import Pin, PWM
2  import math
3  import time

```

Define the pins of the button and passive buzzer.

```

5  PI=3.14
6  button=Pin(4, Pin.IN, Pin.PULL_UP)
7  passiveBuzzer=PWM(Pin(13), 2000, 512)

```

Call sin function of math module to generate the frequency data of the passive buzzer.

```

9  def alert():
10     for x in range(0, 36):
11         sinVal=math.sin(x*10*PI/180)
12         toneVal=2000+int(sinVal*500)
13         passiveBuzzer.freq(toneVal)
14         time.sleep_ms(10)

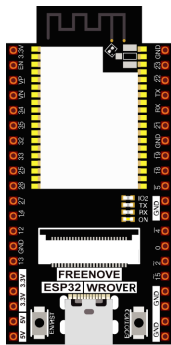
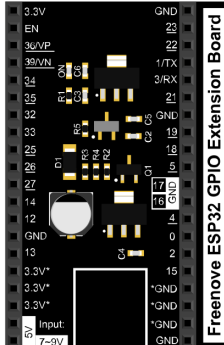
```

When not using PWM, please turn it OFF in time.

```

22  passiveBuzzer.deinit()

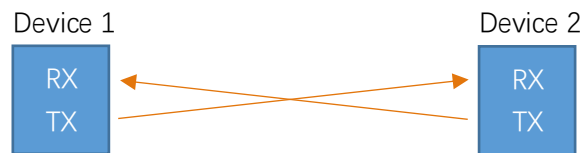
```

ESP32-WROVER x1  The image shows an ESP32-WROVER development module. It features a black PCB with a gold-plated micro-USB port on the left, a USB Type-C port on the right, and a 40-pin GPIO header at the bottom. The module is labeled "FREENOVA ESP32-WROVER".	GPIO Extension Board x1  The image shows a black GPIO Extension Board. It has a 40-pin header on the left, a 40-pin header on the right, and a 40-pin header at the bottom. The board is labeled "FREENOVA ESP32 GPIO Extension Board".	Micro USB Wire x1  The image shows a black Micro USB cable. It has a standard USB-A connector on one end and a Micro USB-B connector on the other. The cable is labeled "Micro USB Wire".
--	---	---

Related knowledge

Serial communication

Serial communication generally refers to the Universal Asynchronous Receiver/Transmitter (UART), which is commonly used in electronic circuit communication. It has two communication lines, one is responsible for sending data (TX line) and the other for receiving data (RX line). The serial communication connections two devices use is as follows:



Before serial communication starts, the baud rate of both sides must be the same. Communication between devices can work only if the same baud rate is used. The baud rates commonly used is 9600 and 115200.

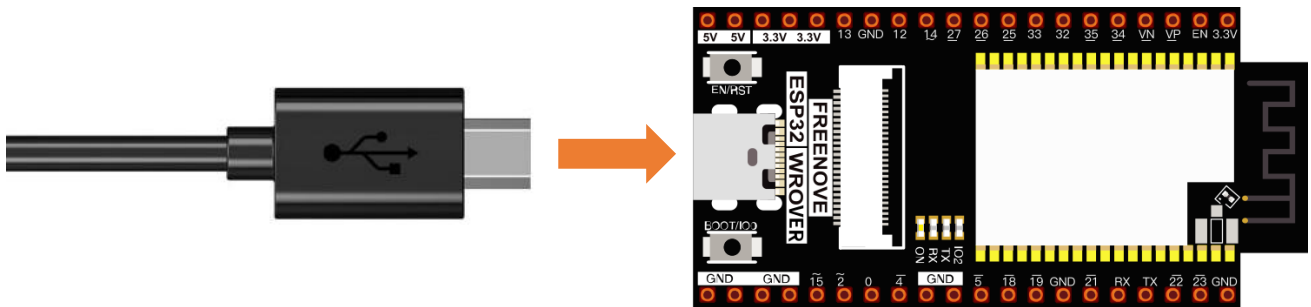
Serial port on ESP32

Freenove ESP32 has integrated USB to serial transfer, so it could communicate with computer connecting to USB cable.



Circuit

Connect Freenove ESP32 to the computer with USB cable

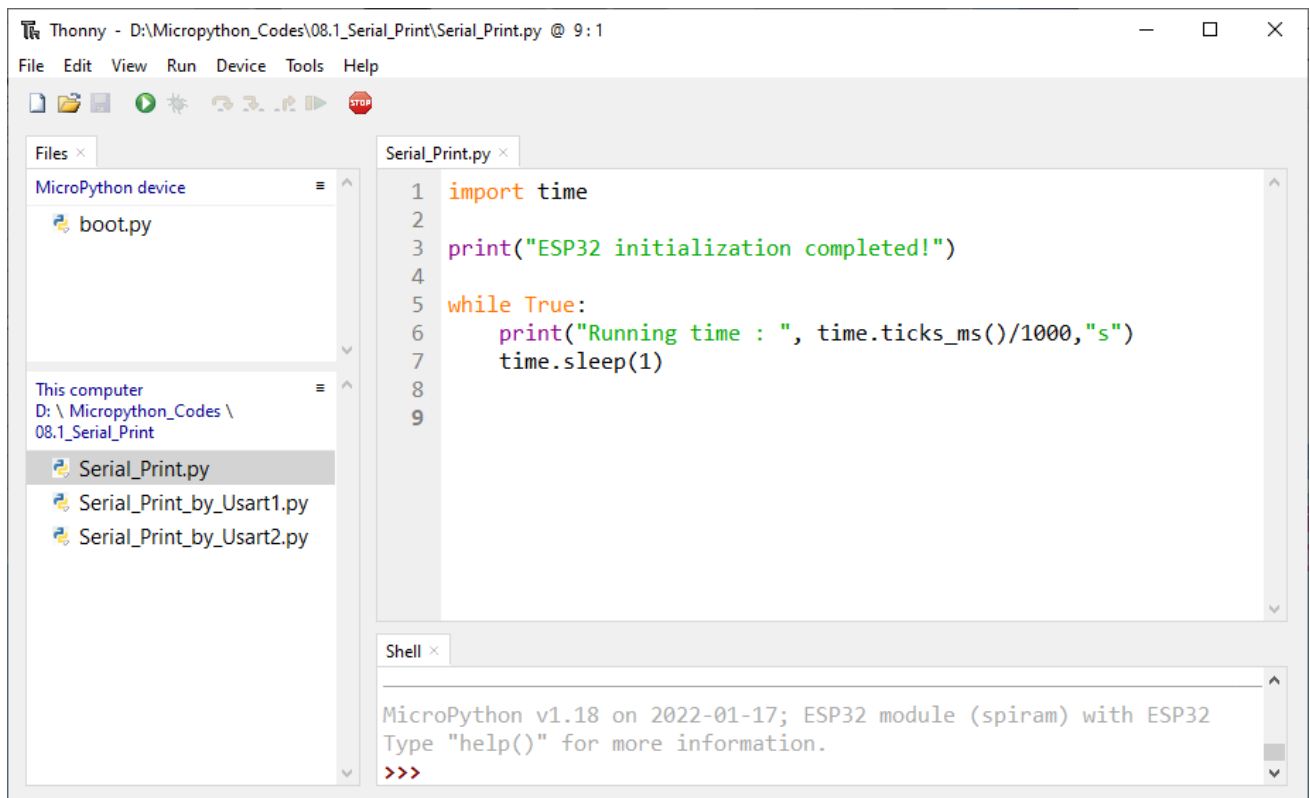


Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “07.1_Serial_Print” and double “Serial_Print.py”.

07.1_Serial_Print



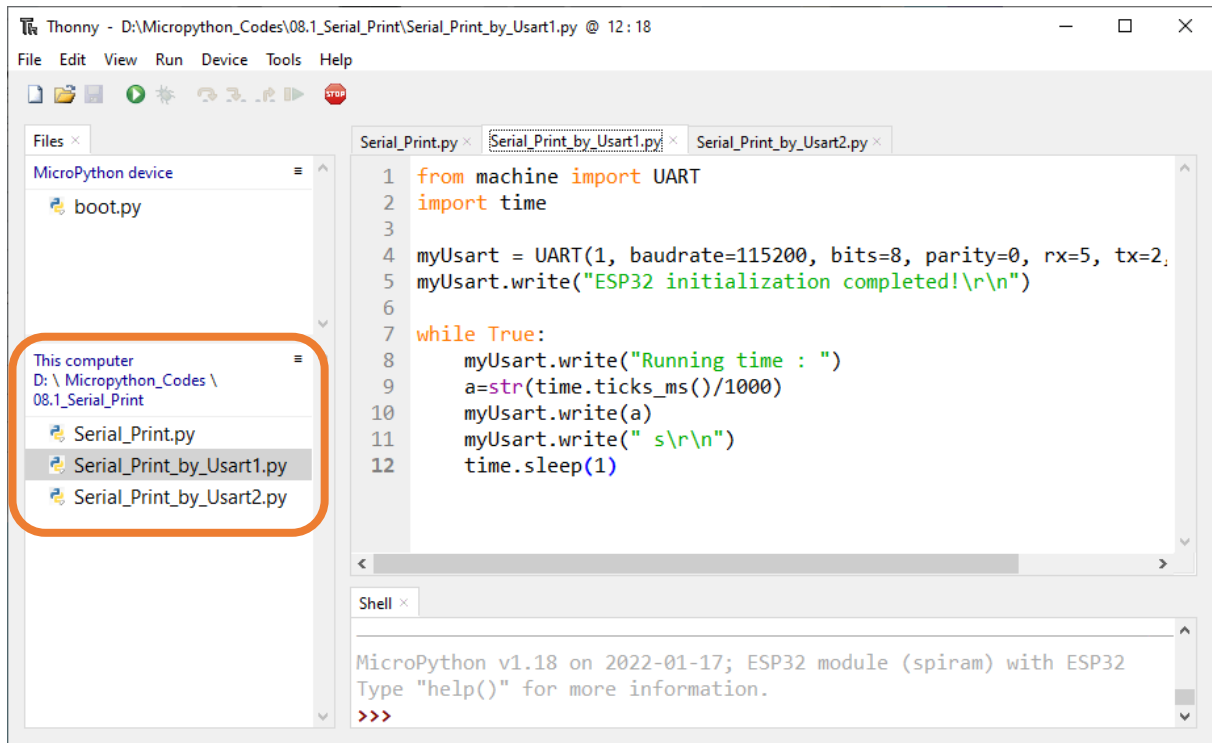
Click “Run current script” and observe the changes of “Shell”, which will display the time when ESP32 is powered on once per second.



The following is the program code:

```
1 import time
2
3 print("ESP32 initialization completed!")
4
5 while True:
6     print("Running time : ", time.ticks_ms()/1000, "s")
7     time.sleep(1)
```

ESP32-WROVER has 3 serial ports, one of which is used as REPL, that is, Pin(1) and Pin(3) are occupied, and generally it is not recommended to be used as tx, rx. The other two serial ports can be configured simply by calling the UART module.



Reference

Class UART

Before each use of **UART** module, please add the statement **"from machine import UART"** to the top of python file.

UART(id, baudrate, bits, parity, rx, tx, stop, timeout): Define serial ports and configure parameters for them.

id: Serial Number. The available serial port number is 1 or 2

baudrate: Baud rate

bits: The number of each character.

parity: Check even or odd, with 0 for even checking and 1 for odd checking.

rx, tx: UAPT's reading and writing pins

Pin(0)、Pin(2)、Pin(4)、Pin(5)、Pin(9)、Pin(10)、Pin(12~19)、Pin(21~23)、Pin(25)、Pin(26)、Pin(34~36)、Pin(39)

Note: Pin(1) and Pin(3) are occupied and not recommend to be used as tx,rx.

stop: The number of stop bits, and the stop bit is 1 or 2.

timeout: timeout period (Unit: millisecond)

$0 < \text{timeout} \leq 0x7FFF\ FFFF$ (decimal: $0 < \text{timeout} \leq 2147483647$)

UART.init(baudrate, bits, parity, stop, tx, rx, rts, cts): Initialize serial ports

tx: writing pins of uart

rx: reading pins of uart

rts: rts pins of uart

cts: cts pins of uart

UART.read(nbytes): Read nbytes bytes

UART.read(): Read data

UART.write(buf): Write byte buffer to UART bus

UART.readline(): Read a line of data, ending with a newline character.

UART.readinto(buf): Read and write data into buffer.

UART.readinto(buf, nbytes): Read and write data into buffer.

UART.any(): Determine whether there is data in serial ports. If yes, return the number of bytes; Otherwise, return 0.

Project 7.2 Serial Read and Write

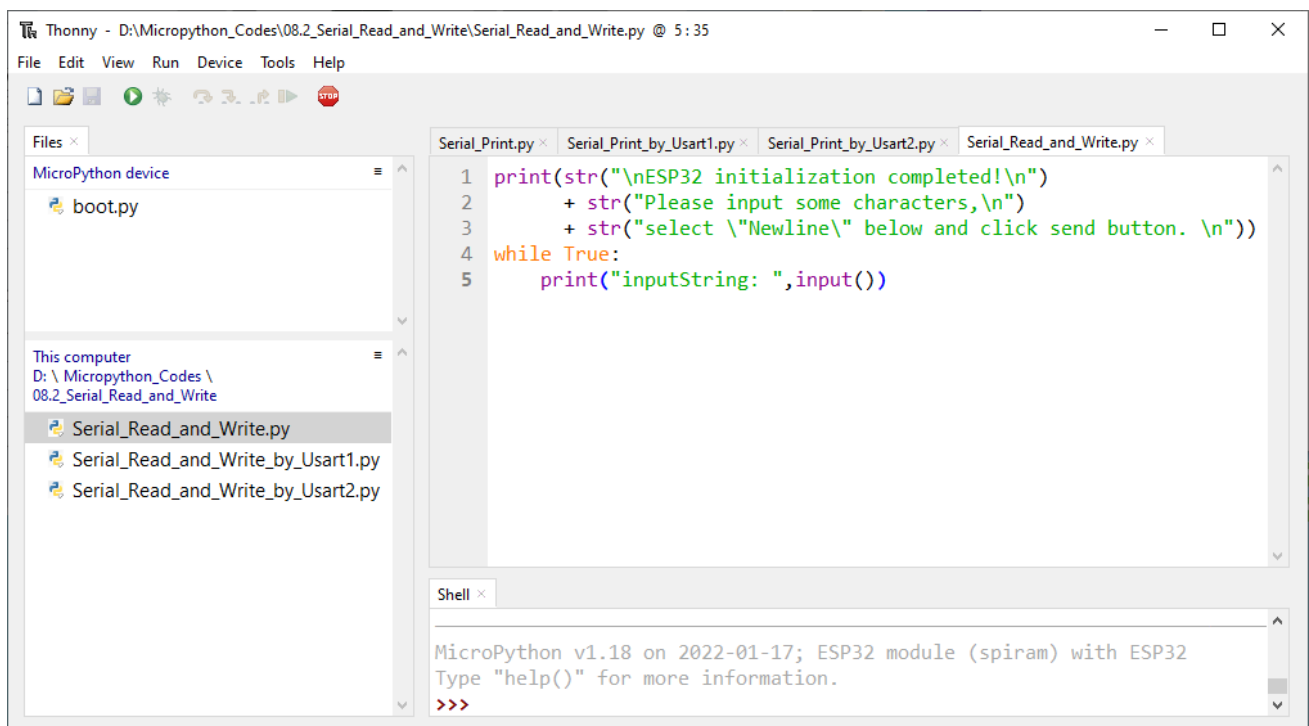
From last section, we use Serial port on Freenove ESP32 to send data to a computer, now we will use that to receive data from computer.

Component and Circuit are the same as in the previous project.

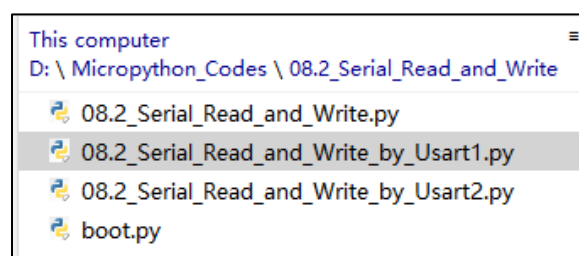
Code

Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "07.2_Serial_Read_and_Write" and double click "Serial_Read_and_Write.py".

07.2_Serial_Read_and_Write



Click "Run current script" and ESP32 will print out data at "Shell" and wait for users to enter any messages. Press Enter to end the input, and "Shell" will print out data that the user entered. If you want to use other serial ports, you can use other python files in the same directory.



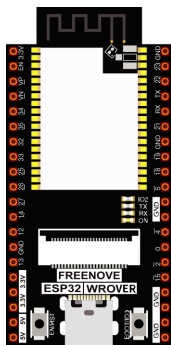
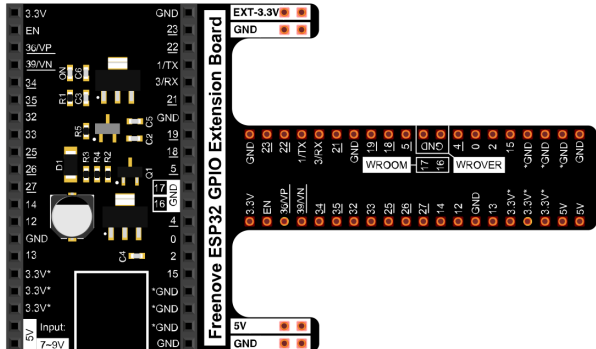
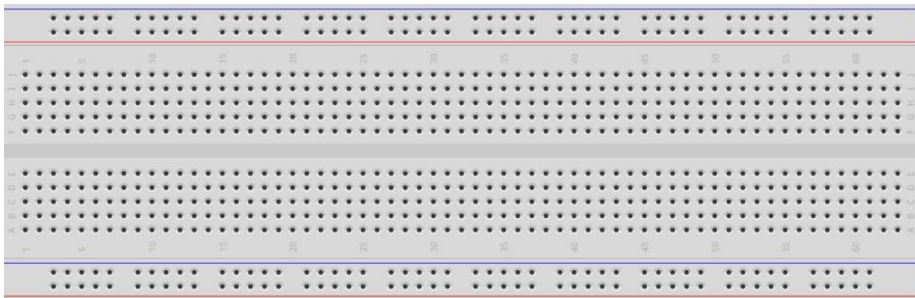
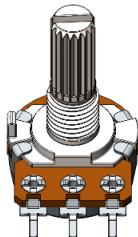
Chapter 8 AD/DA Converter

We have learned how to control the brightness of LED through PWM and understood that PWM is not the real analog before. In this chapter, we will learn how to read analog, convert it into digital and convert the digital into analog output. That is, ADC and DAC.

Project 8.1 Read the Voltage of Potentiometer

In this project, we will use the ADC function of ESP32 to read the voltage value of potentiometer. And then output the voltage value through the DAC to control the brightness of LED.

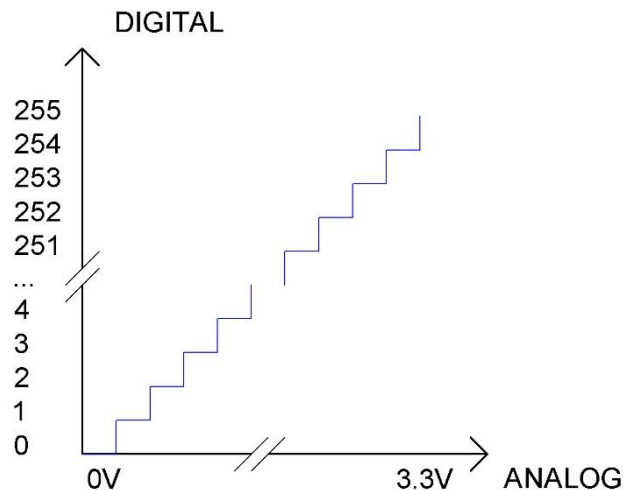
Component List

ESP32-WROVER x1 	GPIO Extension Board x1 		
Breadboard x1 			
Rotary potentiometer x1 	Resistor 220Ω x1 	LED x1 	Jumper M/M x5 

Related knowledge

ADC

An ADC is an electronic integrated circuit used to convert analog signals such as voltages to digital or binary form consisting of 1s and 0s. The range of our ADC on ESP32 is 12 bits, that means the resolution is $2^{12}=4096$, and it represents a range (at 3.3V) will be divided equally to 4096 parts. The range of analog values corresponds to ADC values. So the more bits the ADC has, the denser the partition of analog will be and the greater the precision of the resulting conversion.



Subsection 1: the analog in range of 0V---3.3/4095 V corresponds to digital 0;

Subsection 2: the analog in range of 3.3/4095 V---2*3.3 /4095V corresponds to digital 1;

...

The following analog will be divided accordingly.

The conversion formula is as follows:

$$ADCValue = \frac{\text{Analog Voltage}}{3.3} * 4095$$

DAC

The reversing of this process requires a DAC, Digital-to-Analog Converter. The digital I/O port can output high level and low level (0 or 1), but cannot output an intermediate voltage value. This is where a DAC is useful. ESP32 has two DAC output pins with 8-bit accuracy, GPIO25 and GPIO26, which can divide VDD (here is 3.3V) into $2^8=256$ parts. For example, when the digital quantity is 1, the output voltage value is $3.3/256 * 1$ V, and when the digital quantity is 128, the output voltage value is $3.3/256 * 128=1.65$ V, the higher the accuracy of DAC, the higher the accuracy of output voltage value will be.

The conversion formula is as follows:

$$\text{Analog Voltage} = \frac{\text{DAC Value}}{255} * 3.3 \text{ (V)}$$

ADC on ESP32

ESP32 has 6 pins can be used to measure analog signals. GPIO pin sequence number and analog pin definition are shown in the following table.

ADC number in ESP32	ESP32 GPIO number
ADC1	GPIO 36
ADC2	GPIO 39
ADC3	GPIO 34
ADC4	GPIO 35
ADC5	GPIO 32
ADC6	GPIO 33

DAC on ESP32

ESP32 has two 8-bit digital analog converters to be connected to GPIO25 and GPIO26 pins, respectively, and it is immutable. As shown in the following table,

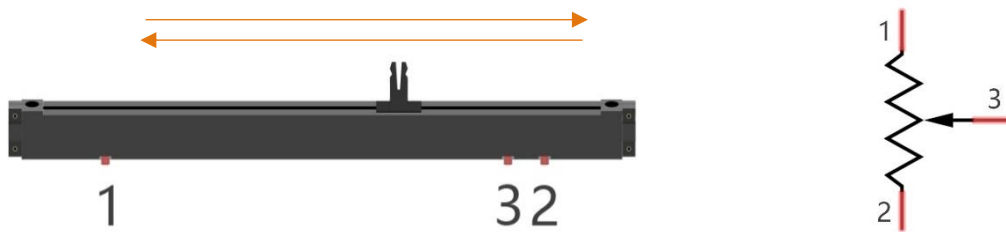
Simulate pin number	GPIO number
DAC1	25
DAC2	26

Note: In this ESP32, GPIO26 is connected to 3.3V through a resistor. Therefore, DAC2 cannot be used.

Component knowledge

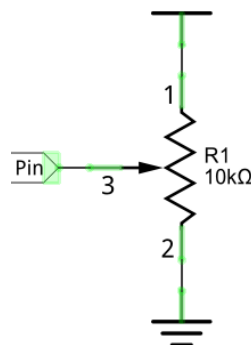
Potentiometer

Potentiometer is a resistive element with three Terminal parts. Unlike the resistors that we have used thus far in our project which have a fixed resistance value, the resistance value of a potentiometer can be adjusted. A potentiometer is often made up by a resistive substance (a wire or carbon element) and movable contact brush. When the brush moves along the resistor element, there will be a change in the resistance of the potentiometer's output side (3) (or change in the voltage of the circuit that is a part). The illustration below represents a linear sliding potentiometer and its electronic symbol on the right.



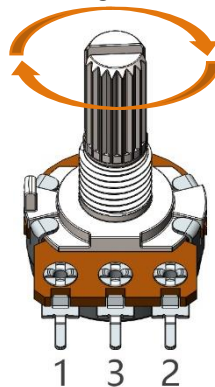
What between potentiometer pin 1 and pin 2 is the resistor body, and pins 3 is connected to brush. When brush moves from pin 1 to pin 2, the resistance between pin 1 and pin 3 will increase up to body resistance linearly, and the resistance between pin 2 and pin 3 will decrease down to 0 linearly.

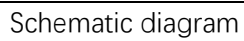
In the circuit. The both sides of resistance body are often connected to the positive and negative electrode of the power. When you slide the brush pin 3, you can get a certain voltage in the range of the power supply.



Rotary potentiometer

Rotary potentiometers and linear potentiometers have the same function; the only difference being the physical action being a rotational rather than a sliding movement.



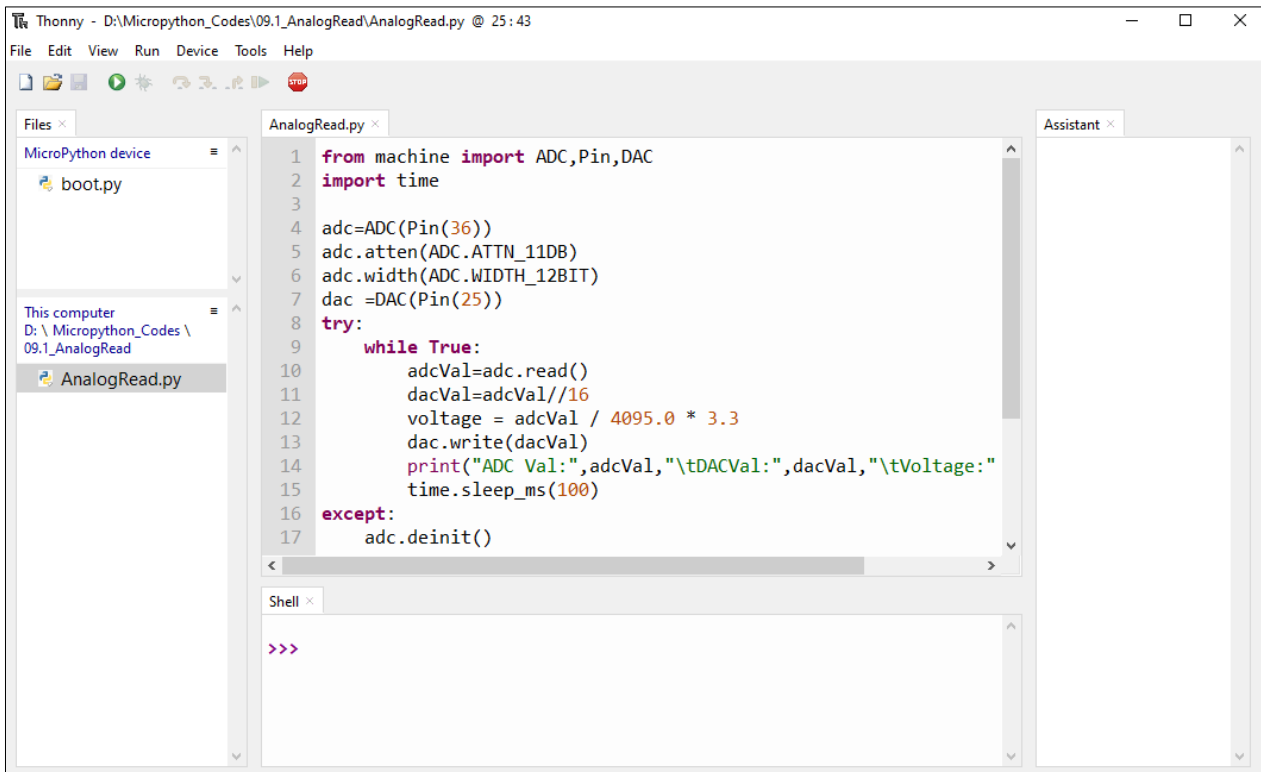


Code

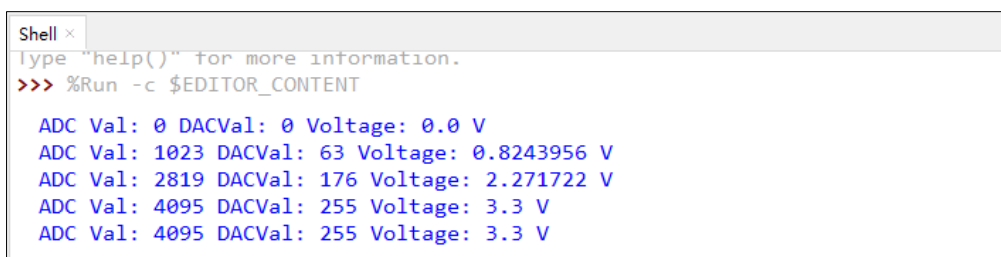
Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “08.1_AnalogRead” and then click “AnalogRead.py”.

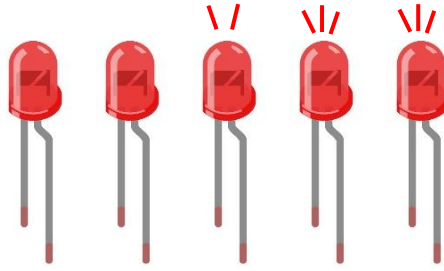
08.1_AnalogRead



Click “Run current script” and observe the message printed in “Shell”.



LEDs display as below:



"Shell" prints ADC value, DAC value, the output voltage of potentiometer and other information. In the code, we make the output voltage of the DAC pin equal to the input voltage of the ADC pin. Rotate the handle of the potentiometer, the printed information will change. When the voltage is greater than 1.6V (turn-on voltage of red LED), the LED starts to emit light. If you continue to increase the output voltage, the LED will gradually become brighter. And when the voltage is less than 1.6V, the LED will not light up, because this does not reach the turn-on voltage of the LED, which indirectly proves the difference between DAC and PWM. (If you have an oscilloscope, you can view the waveform output by the DAC through the oscilloscope)

The following is the code:

```

1  from machine import ADC, Pin, DAC
2  import time
3
4  adc=ADC(Pin(36))
5  adc.atten(ADC.ATTN_11DB)
6  adc.width(ADC.WIDTH_12BIT)
7  dac =DAC(Pin(25))
8
9  try:
10     while True:
11         adcVal=adc.read()
12         dacVal=adcVal//16
13         voltage = adcVal / 4095.0 * 3.3
14         dac.write(dacVal)
15         print("ADC Val:",adcVal," \tDACVal:",dacVal," \tVoltage:",voltage,"V")
16         time.sleep_ms(100)
17 except:
18     adc.deinit()

```

Import Pin, ADC and DAC modules.

```

1  from machine import ADC, Pin, DAC
2  import time

```

Turn on and configure the ADC with the range of 0-3.3V and the data width of 12-bit data width, and turn on the DAC pin.

```

4  adc=ADC(Pin(36))
5  adc.atten(ADC.ATTN_11DB)
6  adc.width(ADC.WIDTH_12BIT)
7  dac =DAC(Pin(25))

```


Read ADC value once every 100 milliseconds, convert ADC value to DAC value and output it, control the brightness of LED and print these data to "Shell".

```

10 while True:
11     adcVal=adc.read()
12     dacVal=adcVal//16
13     voltage = adcVal / 4095.0 * 3.3
14     dac.write(dacVal)
15     print("ADC Val:",adcVal, "\tDACVal:", dacVal, "\tVoltage:", voltage, "V")
16     time.sleep_ms(100)

```

Reference

Class ADC

Before each use of ACD module, please add the statement **"from machine import ADC"** to the top of the python file.

machine.ADC(pin): Create an ADC object associated with the given pin.

pin: Available pins are: Pin(36)、Pin(39)、Pin(34)、Pin(35)、Pin(32)、Pin(33)。

ADC.read(): Read ADC and return the value.

ADC.atten(db): Set attenuation ration (that is, the full range voltage, such as the voltage of 11db full range is 3.3V)

db: attenuation ratio

ADC.ATTIN_0DB —full range of 1.2V

ADC.ATTN_2_5_DB —full range of 1.5V

ADC.ATTN_6DB —full range of 2.0 V

ADC.ATTN_11DB —full range of 3.3V

ADC.width(bit): Set data width.

bit: data bit

ADC.WIDTH_9BIT —9 data width

ADC.WIDTH_10BIT — 10 data width

ADC.WIDTH_11BIT — 11 data width

ADC.WIDTH_12BIT — 12 data width

Class DAC

Before each use of **DAC** module, please add the statement **"from machine import DAC"** to the top of the python file.

machine.DAC(pin): Create a DAC object associated with the given pin.

pin: Available pins are: Pin(25)、Pin(26)

DAC.write(value): Output voltage

value: The range of data value: 0-255, corresponding output voltage of 0-3.3V

Chapter 9 TouchSensor

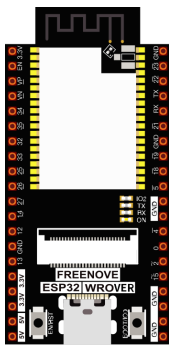
ESP32 offers up to 10 capacitive touch GPIO, and as you can see from the previous section, mechanical switches are prone to jitter that must be eliminated when used, which is not the case with ESP32's built-in touch sensor. In addition, on the service life, the touch switch also has advantages that mechanical switch is completely incomparable.

Project 9.1 Read Touch Sensor

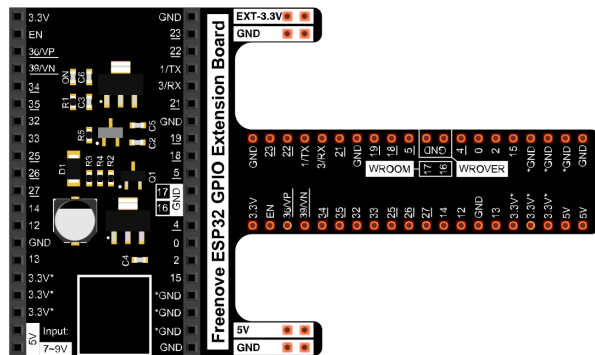
This project reads the value of the touch sensor and prints it out.

Component List

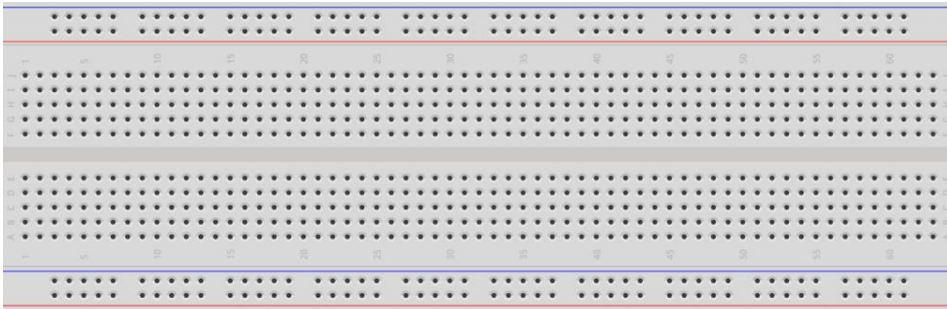
ESP32-WROVER x1



GPIO Extension Board x1



Breadboard x1



Jumper M/M x1



Related knowledge

Touch sensor

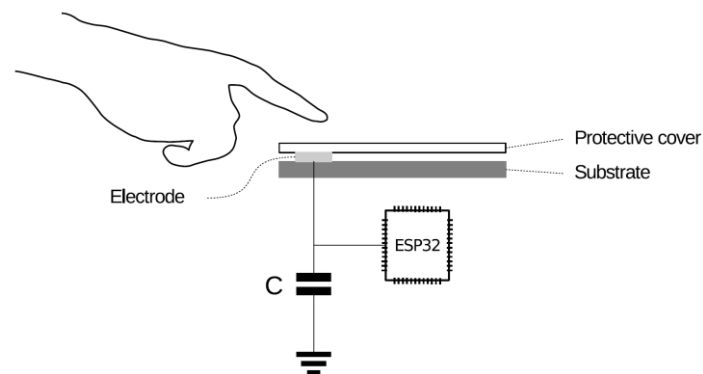
ESP32's touch sensor supports up to 7 GPIO channels as capacitive touch pins. Each pin can be used separately as an independent touch switch or be combined to produce multiple touch points. The following table is a list of available touch pins on ESP32.

Functions of pins	ESP32 GPIO number
GPIO4	GPIO4
MTDO	GPIO15
MTCK	GPIO13
MTDI	GPIO12
MTMS	GPIO14
32K_XN	GPIO33
32K_XP	GPIO32

The pin numbers are shown in the figure above. When you need to use the TouchPad, you only need to call the TouchPad function to initialize the corresponding pins.

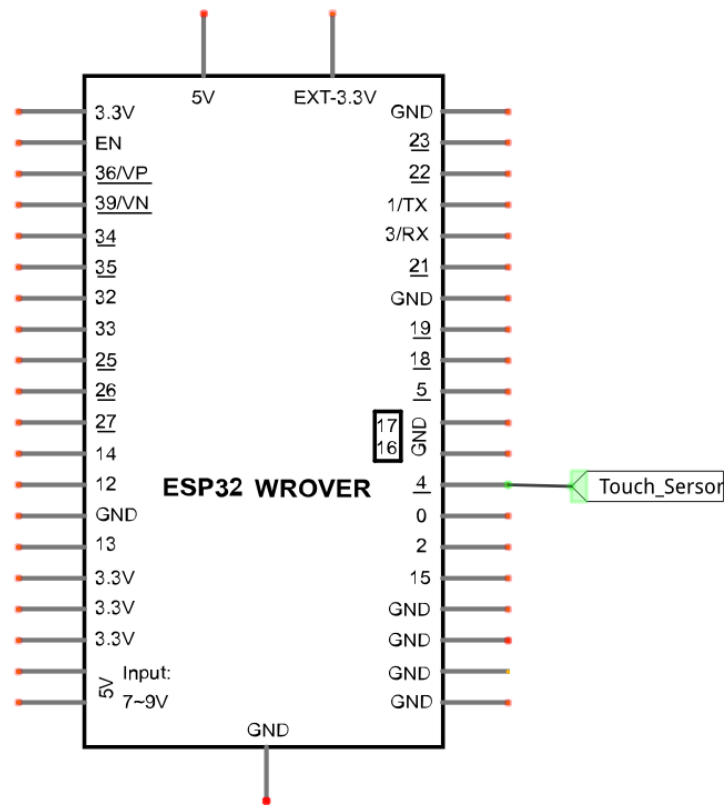
The electronic signals generated with the touch are analog, which are converted by the internal ADC. You may have found that all touch pins have featured with ADC.

The way to connect the hardware is as follows:

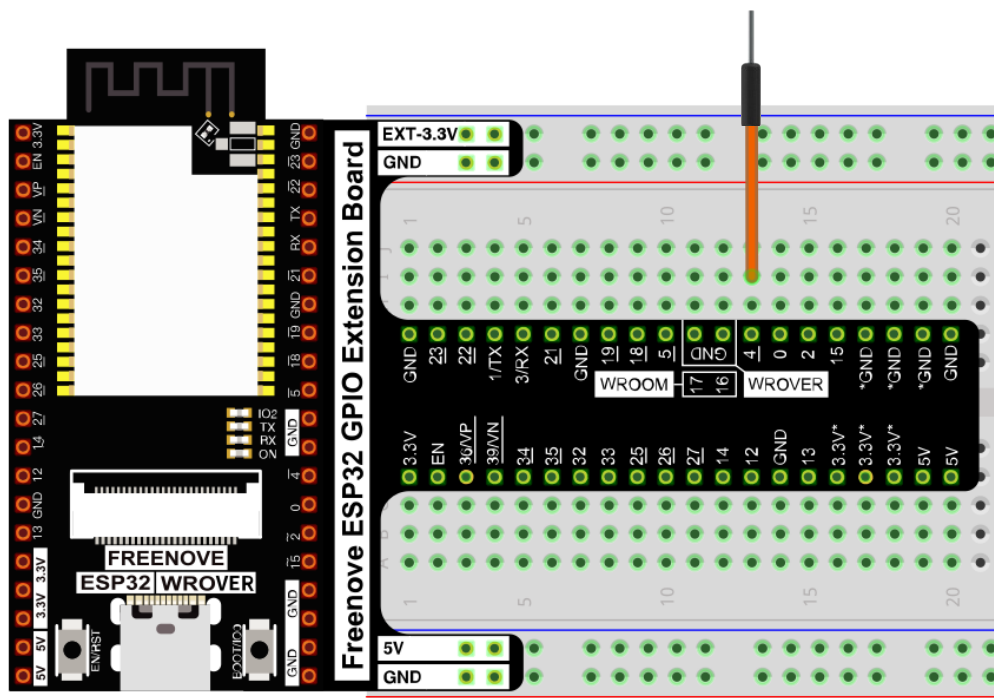


Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

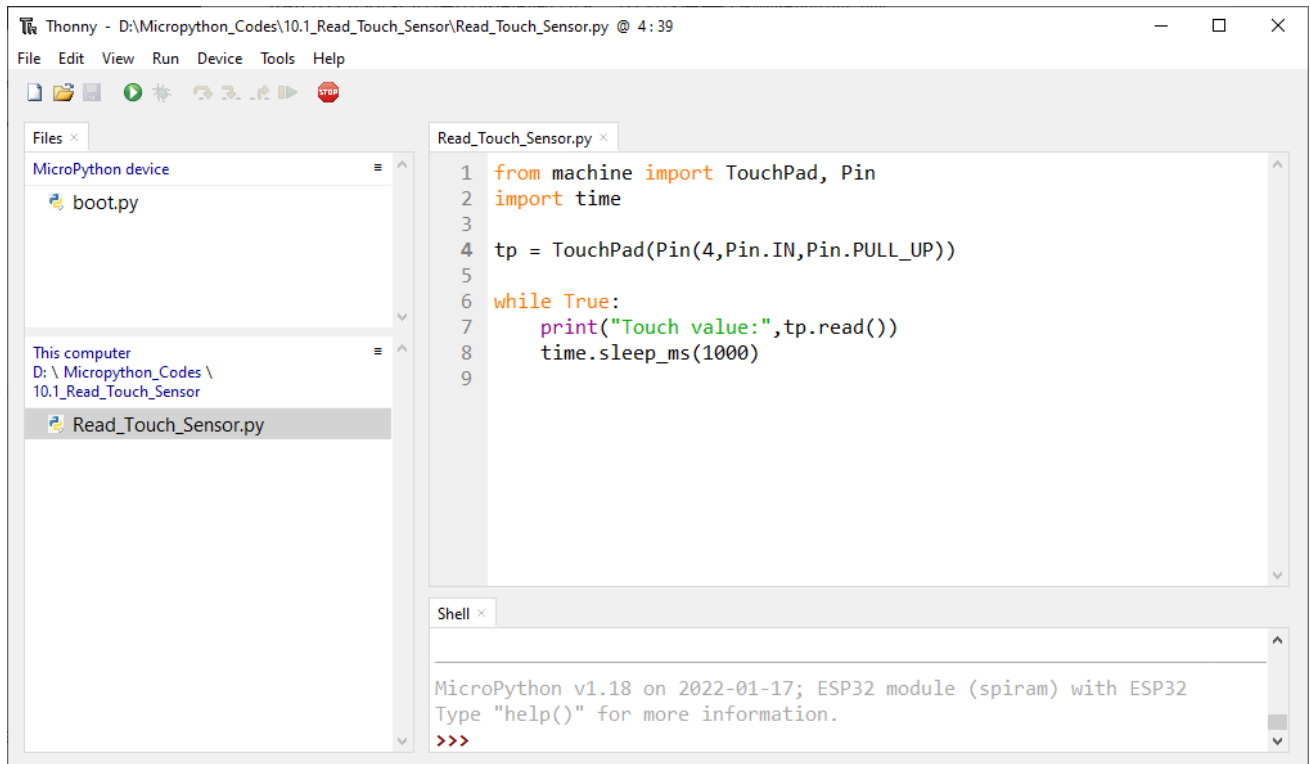


Code

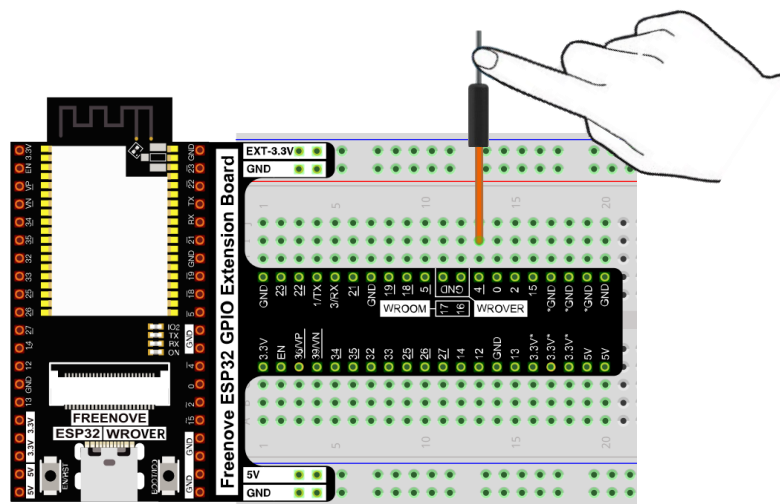
Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “09.1_Read_Touch_Sensor” and double click “Read_Touch_Sensor.py”.

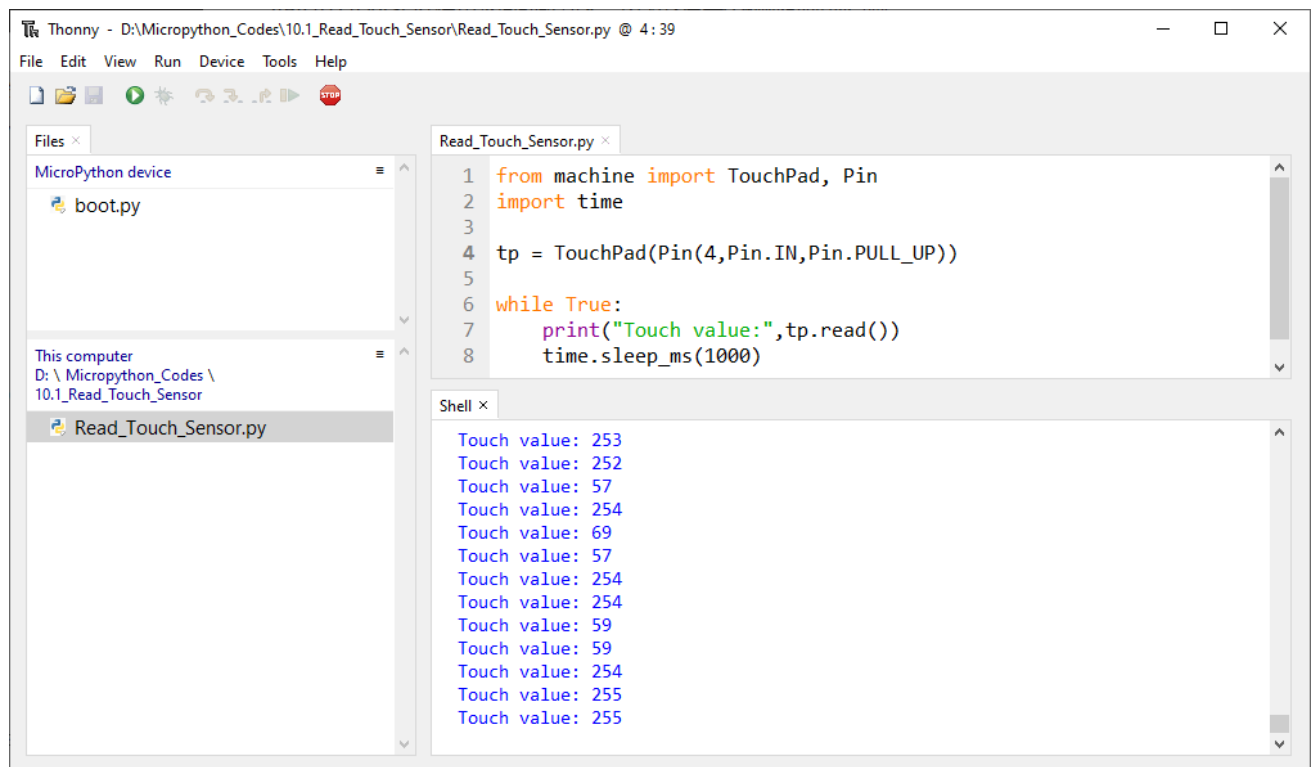
09.1_Read_Touch_Sensor



Click “Run current script”, touch the jumper wire with your finger and observe the messages printed in “Shell”.



Messages printed at "Shell":



The screenshot shows the Thonny IDE interface. The top menu bar includes File, Edit, View, Run, Device, Tools, and Help. The left sidebar has two panes: 'Files' and 'This computer'. The 'Files' pane shows a 'MicroPython device' with a 'boot.py' file. The 'This computer' pane shows the directory 'D:\Micropython_Codes\10.1_Read_Touch_Sensor' with a file 'Read_Touch_Sensor.py' selected. The main editor window displays the code for 'Read_Touch_Sensor.py':

```
1 from machine import TouchPad, Pin
2 import time
3
4 tp = TouchPad(Pin(4,Pin.IN,Pin.PULL_UP))
5
6 while True:
7     print("Touch value:",tp.read())
8     time.sleep_ms(1000)
```

Below the editor is a 'Shell' window showing the output of the script:

```
Touch value: 253
Touch value: 252
Touch value: 57
Touch value: 254
Touch value: 69
Touch value: 57
Touch value: 254
Touch value: 254
Touch value: 59
Touch value: 59
Touch value: 254
Touch value: 255
Touch value: 255
```

Reference

Class TouchPad

Before each use of **TouchPad** module, please add the statement "**from machine import TouchPad**" to the top of the python file.

TouchPad(pin): Initialize the TouchPad object and associate it with ESP32 pins.

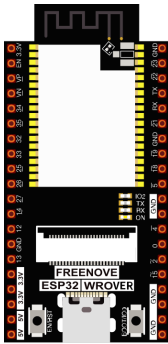
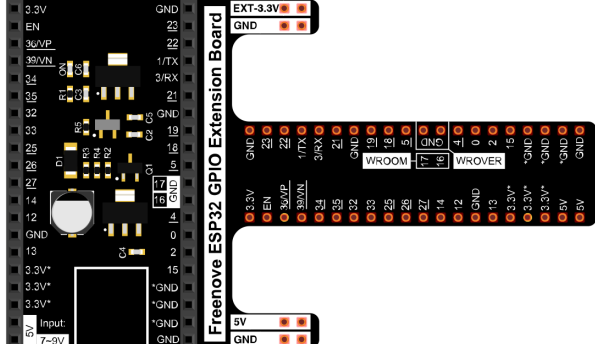
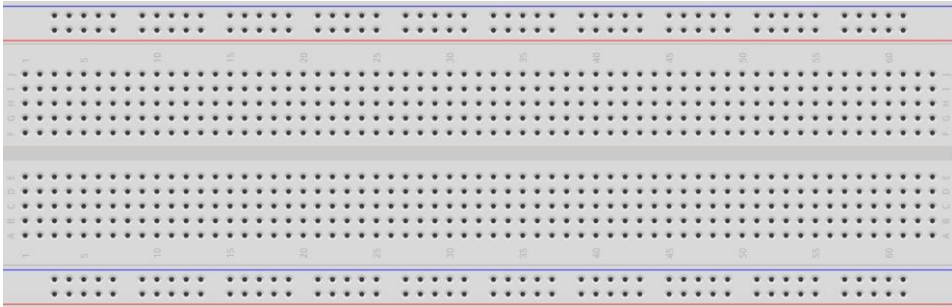
pin: Pin(4)、Pin(15)、Pin(13)、Pin(12)、Pin(14)、Pin(32)、Pin(33)

TouchPad.read(): Read the capacitance of touchpad. If your fingers touch TouchPad pins, the capacitance decreases; Otherwise, it will not change.

Project 9.2 TouchLamp

In this project, we will use ESP32's touch sensor to create a touch switch lamp.

Component List

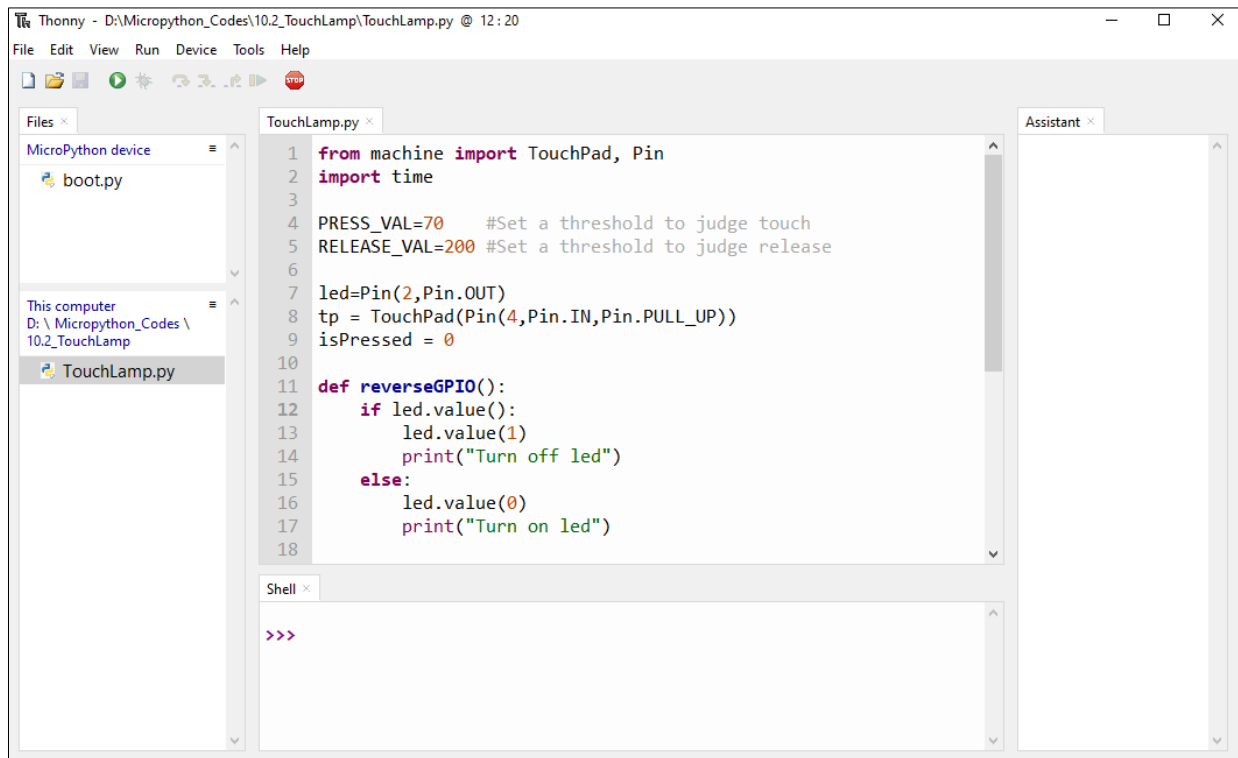
ESP32-WROVER x1 	GPIO Extension Board x1 	
Breadboard x1 		
Jumper M/M x3 	LED x1 	Resistor 220Ω x1 

Code

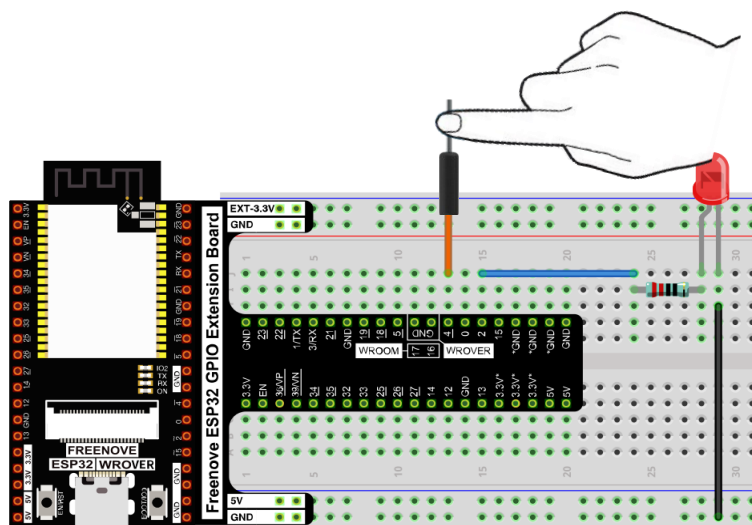
Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny, click “This computer” → “D:” → “Micropython_Codes” → “09.2_TouchLamp” and double click “TouchLamp.py”.

09.2_TouchLamp



Click “Run current script” and then touch the jumper wire with your finger. The state of LED will change with each touch and the detection state of the touch sensor will be printed in the “Shell”



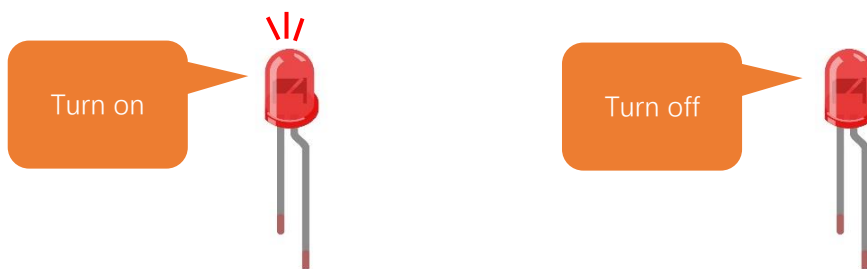
```

Shell x
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

Turn on led
Touch detected!
Touch released!
Turn off led
Touch detected!
Touch released!
Turn on led
Touch detected!
Touch released!
Turn off led
Touch detected!
Touch released!
Turn on led
Touch detected!
Touch released!
Turn off led
Touch detected!
Touch released!

```

LED displays as follows:



The following is the program code:

```

1  from machine import TouchPad, Pin
2  import time
3
4  PRESS_VAL=70    #Set a threshold to judge touch
5  RELEASE_VAL=200 #Set a threshold to judge release
6
7  led=Pin(2,Pin. OUT)
8  tp = TouchPad(Pin(4, Pin. IN, Pin. PULL_UP))
9  isPressed = 0
10
11 def reverseGPIO():
12     if led.value():
13         led.value(1)
14         print("Turn off led")
15     else:
16         led.value(0)
17         print("Turn on led")
18 while True:
19     if tp.read() < PRESS_VAL:

```

```

20         if not isPressed:
21             isPressed = 1
22             reverseGPIO()
23             print("Touch detected!")
24             time.sleep_ms(100)
25     if tp.read() > RELEASE_VAL:
26         if isPressed:
27             isPressed = 0
28             print("Touch released!")
29             time.sleep_ms(100)

```

Import Pin and TouchPad modules.

```

1  from machine import TouchPad, Pin
2  import time

```

The closer the return value of the function read() is to 0, the more obviously the touch action is detected. As this is not a fixed value, a threshold value needs to be defined. When the value of the sensor is less than the threshold, it is considered a valid touch action. Similarly, define a threshold value for the released state, and the value between the sensor value and the threshold is regarded as an invalid interference value.

```

4  PRESS_VAL=70    #Set a threshold to judge touch
5  RELEASE_VAL=200 #Set a threshold to judge release

```

First, decide whether the touch is detected. If yes, print some messages, reverse the state of LED and set the flag bit isProcessed to 1 to avoid repeatedly executing the program after a touch is detected.

```

19     if tp.read() < PRESS_VAL:
20         if not isPressed:
21             isPressed = 1
22             reverseGPIO()
23             print("Touch detected!")
24             time.sleep_ms(100)

```

And then decide whether the touch key is released. If yes, print some messages, and set isProcessed to 0 to avoid repeatedly executing the program after a touch is released and to prepare for the next touch detector.

```

25     if tp.read() > RELEASE_VAL:
26         if isPressed:
27             isPressed = 0
28             print("Touch released!")
29             time.sleep_ms(100)

```

Customize a function that reverses the output level of the LED each time it is called.

```

11  def reverseGPIO():
12      if led.value():
13          led.value(1)
14          print("Turn off led")
15      else:
16          led.value(0)
17          print("Turn on led")

```

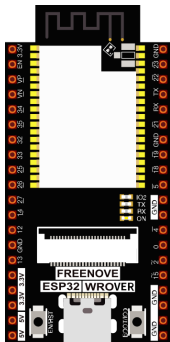
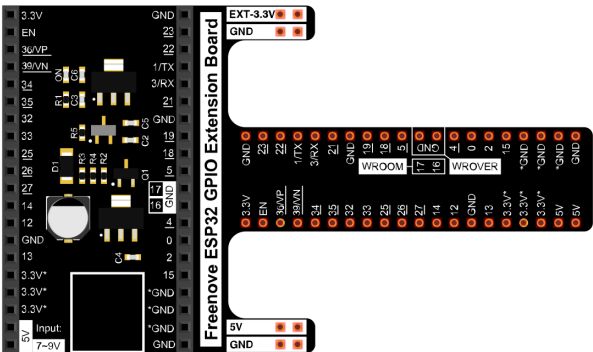
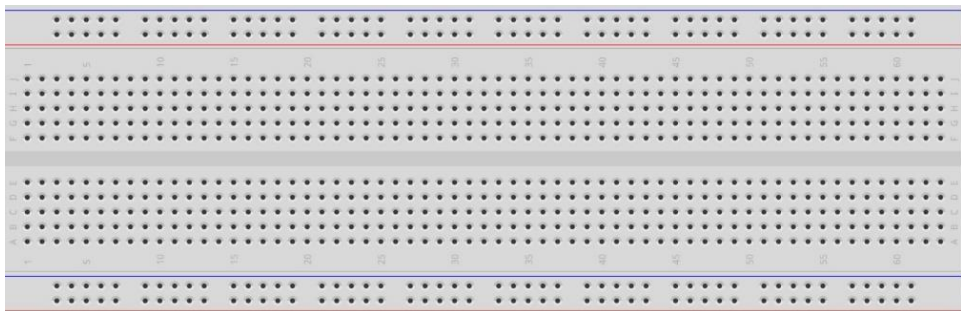
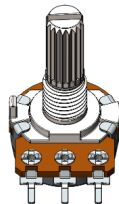
Chapter 10 Potentiometer & LED

We have learned how to use ADC and DAC before. When using DAC output analog to drive LED, we found that, when the output voltage is less than led turn-on voltage, the LED does not light; when the output analog voltage is greater than the LED voltage, the LED lights. This leads to a certain degree of waste of resources. Therefore, in the control of LED brightness, we should choose a more reasonable way of PWM control. In this chapter, we learn to control the brightness of LED through a potentiometer.

Project 10.1 Soft Light

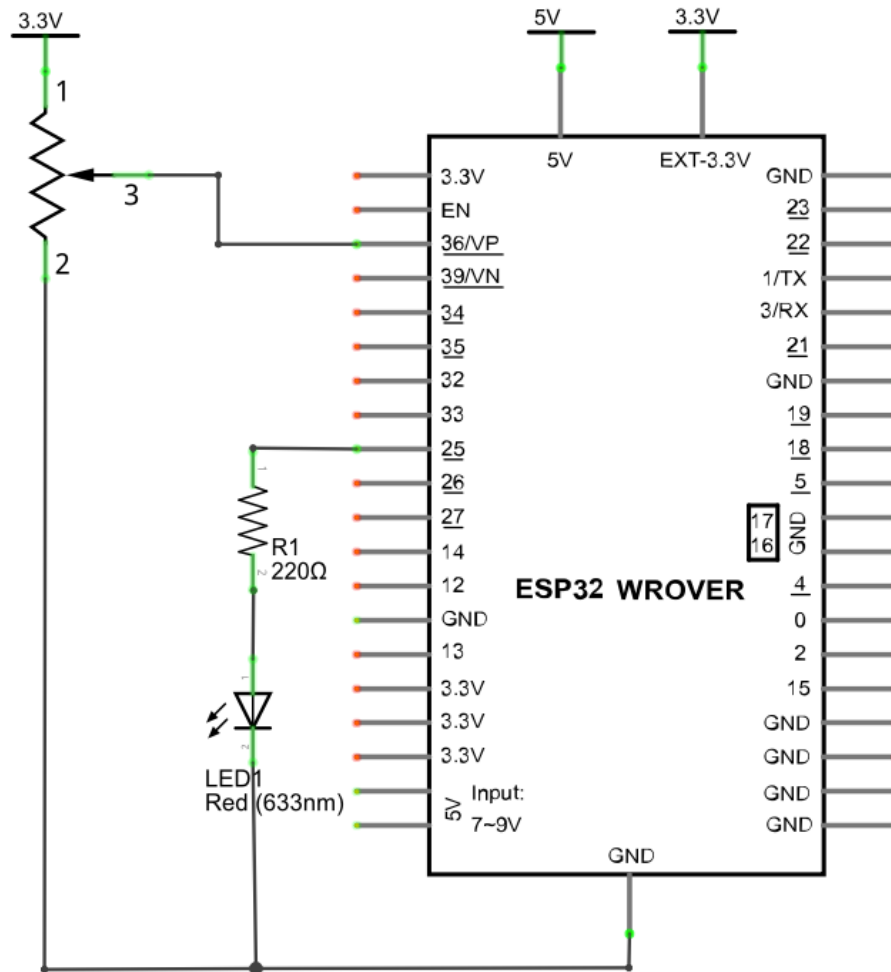
In this project, we will make a soft light. We will use an ADC Module to read ADC values of a potentiometer and map it to duty cycle of the PWM used to control the brightness of an LED. Then you can change the brightness of an LED by adjusting the potentiometer.

Component List

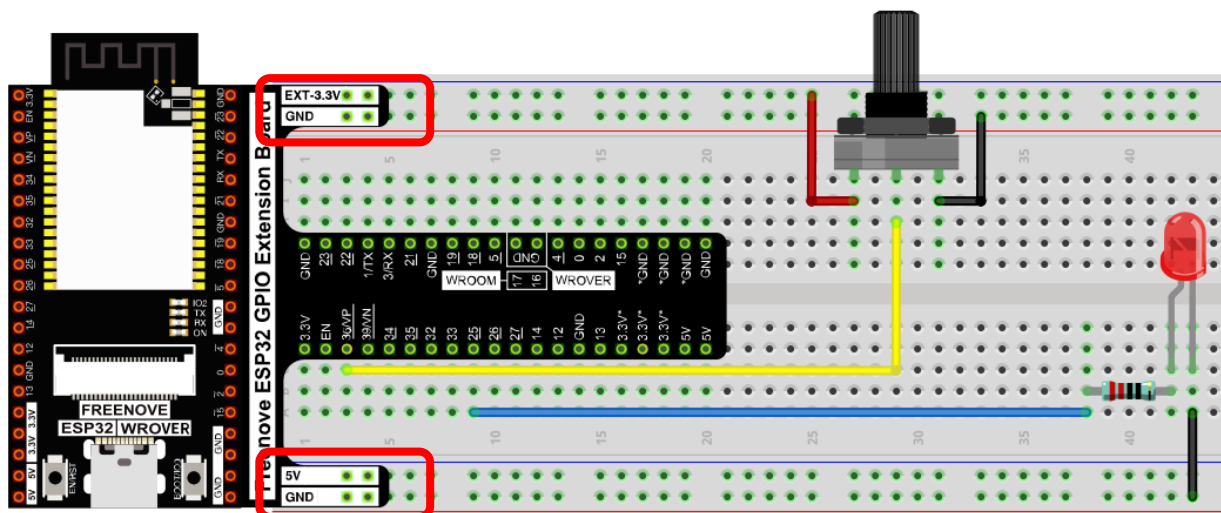
ESP32-WROVER x1 	GPIO Extension Board x1 		
Breadboard x1 			
Rotary potentiometer x1 	Resistor 220Ω x1 	LED x1 	Jumper M/M x5 

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

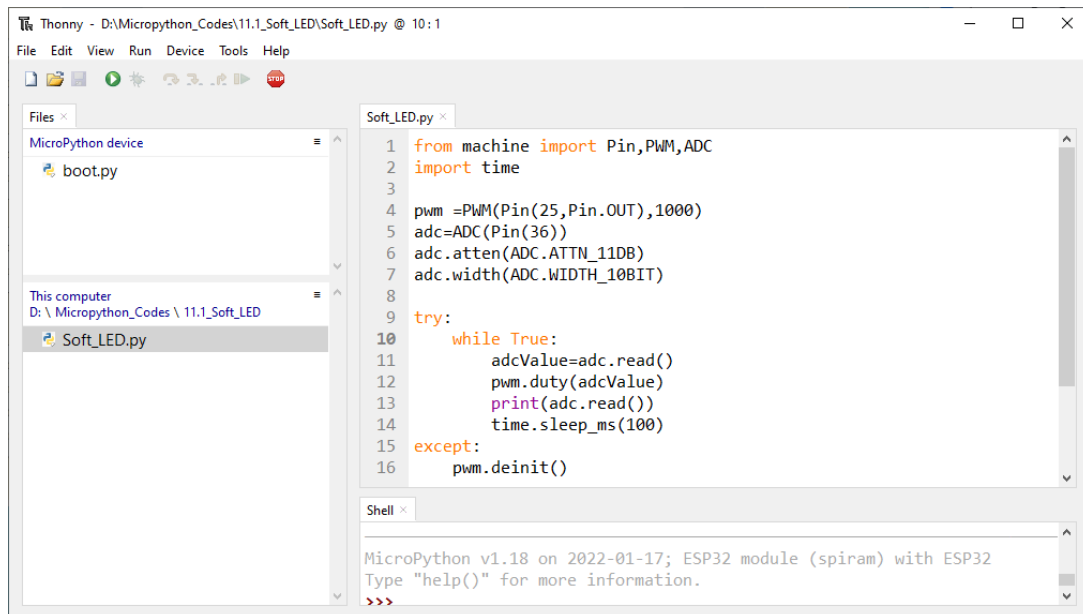


Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “10.1_Soft_LED” and double click “Soft_LED.py”.

10.1_Soft_LED



Click “Run current script”. Rotate the handle of potentiometer and the brightness of LED will change correspondingly.

The following is the code:

```

1  from machine import Pin, PWM, ADC
2  import time
3
4  pwm = PWM(Pin(25, Pin.OUT), 1000)
5  adc = ADC(Pin(36))
6  adc.atten(ADC.ATTN_11DB)
7  adc.width(ADC.WIDTH_10BIT)
8
9  try:
10     while True:
11         adcValue = adc.read()
12         pwm.duty(adcValue)
13         print(adc.read())
14         time.sleep_ms(100)
15 except:
16     pwm.deinit()

```

In the code, read the ADC value of potentiometer and map it to the duty cycle of PWM to control LED brightness.

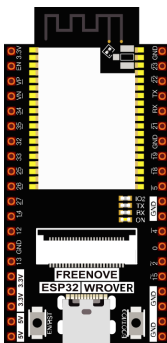
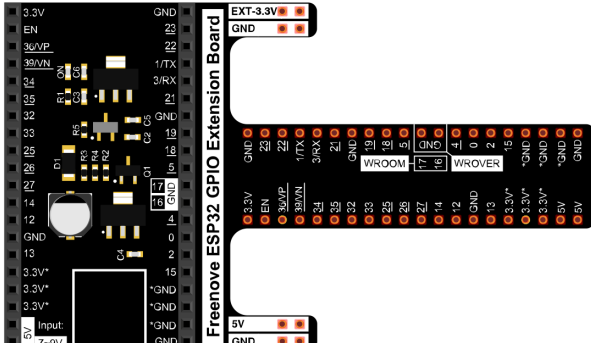
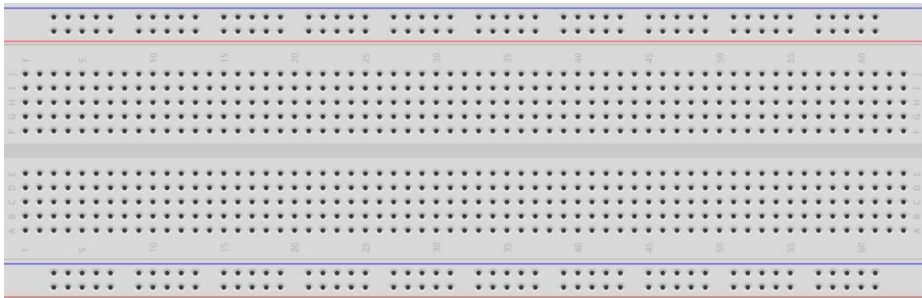
Chapter 11 Photoresistor & LED

In this chapter, we will learn how to use photoresistor.

Project 11.1 NightLamp

A Photoresistor is very sensitive to the amount of light present. We can take advantage of the characteristic to make a nightlight with the following function: when the ambient light is less (darker environment) the LED will automatically become brighter to compensate and when the ambient light is greater (brighter environment) the LED will automatically dim to compensate.

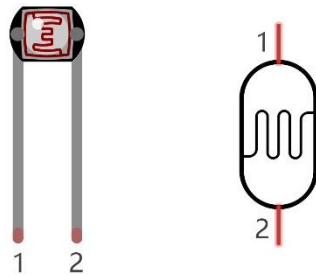
Component List

ESP32-WROVER x1		GPIO Extension Board x1		
				
Breadboard x1				
				
Photoresistor x1	Resistor		LED x1	Jumper M/M x4
	220Ω x1	10KΩ x1		
				

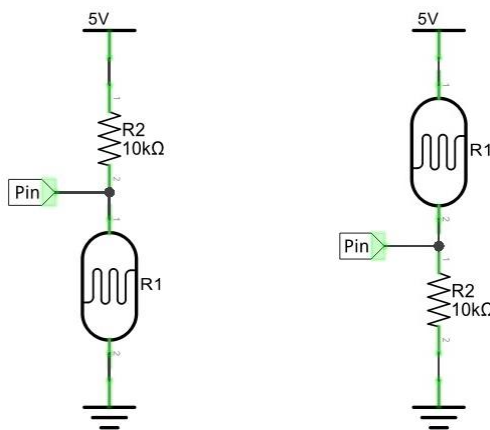
Component knowledge

Photoresistor

Photoresistor is simply a light sensitive resistor. It is an active component that decreases resistance with respect to receiving luminosity (light) on the component's light sensitive surface. Photoresistor's resistance value will change in proportion to the ambient light detected. With this characteristic, we can use a Photoresistor to detect light intensity. The Photoresistor and its electronic symbol are as follows.



The circuit below is used to detect the change of a Photoresistor's resistance value:



In the above circuit, when a Photoresistor's resistance value changes due to a change in light intensity, the voltage between the Photoresistor and Resistor R1 will also change. Therefore, the intensity of the light can be obtained by measuring this voltage.

The circuit of this project is similar to SoftLight. The only difference is that the input signal is changed from a potentiometer to a combination of a photoresistor and a resistor.

The diagram shows an ESP32 WROVER module with the following connections:

- Power:**
 - 5V pin connected to a 5V supply.
 - GND pin connected to ground.
 - EXT-3.3V pin connected to a 3.3V supply.
 - Other 3.3V pins are connected to a 3.3V supply.
- Buzzer:**
 - One terminal connected to pin 36 (VP).
 - Other terminal connected to pin 39 (VN).
- LED:**
 - Anode connected to pin 25.
 - Cathode connected to pin 26 through a 220Ω resistor (R1).
- Resistor R3:** A 10kΩ resistor connected between a 3.3V supply and pin 36.

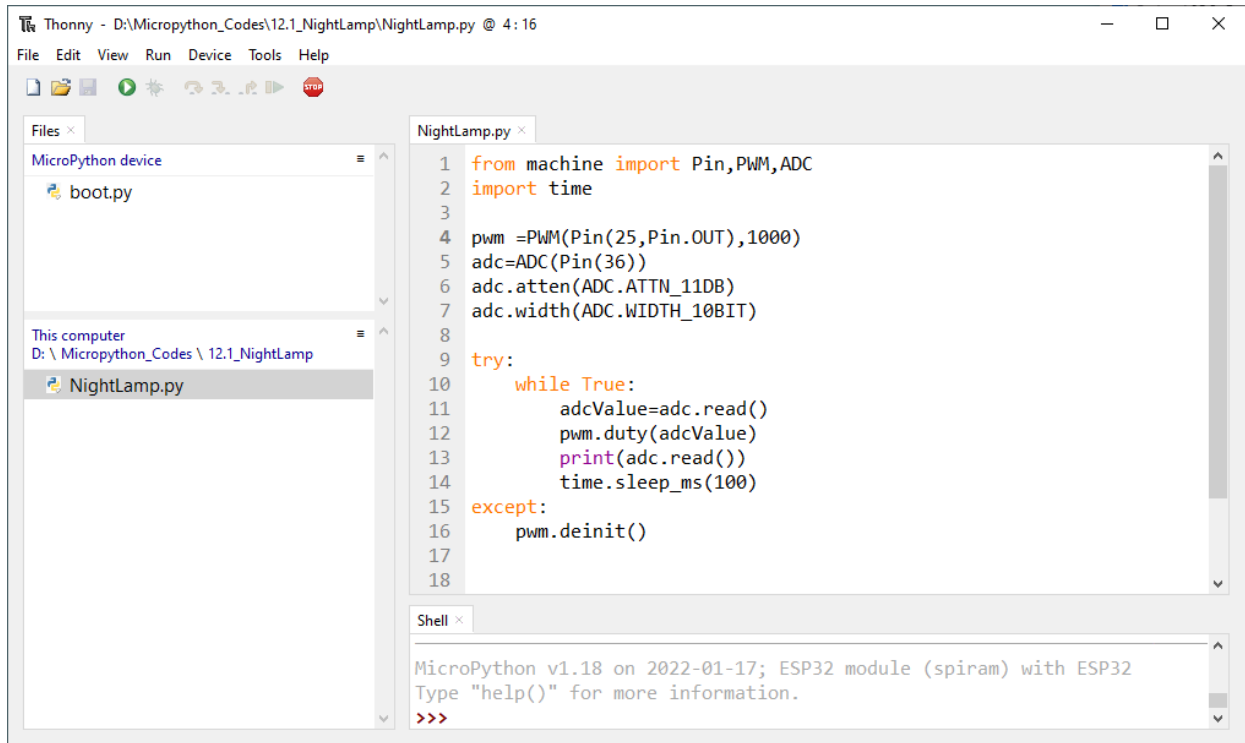
The diagram shows a breadboard circuit. On the left, a black PCB labeled "Freemove ESP32 GPIO Extension Board" is populated with components. It includes a USB Type-C port at the bottom, a white header labeled "FREEMOVE ESP32 WROOM", and several rows of pins. Pins are labeled with voltages (5V, 3.3V), ground (GND), and functions (EN, VP, V+, TX, RX). A small black component is soldered near the top right. In the center, a black PCB labeled "WROOM" (ESP8266) is placed. Its pins are labeled with voltages (5V, 3.3V, GND) and functions (EN, VP, V+, TX, RX). A red LED is connected to pin 17 (GPIO17) through a resistor. The LED's other end is connected to ground. The entire setup is mounted on a standard green breadboard.

Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Codes of this project is logically the same as the project [Soft Light](#).

11.1_Nightlamp



Click “Run current script”. Cover the photoresistor with your hands or illuminate it with lights, the brightness of LEDs will change.

The following is the program code:

```

1 from machine import Pin,PWM,ADC
2 import time
3
4 pwm =PWM(Pin(25,Pin.OUT),1000)
5 adc=ADC(Pin(36))
6 adc.atten(ADC.ATTN_11DB)
7 adc.width(ADC.WIDTH_10BIT)
8
9 try:
10     while True:
11         adcValue=adc.read()
12         pwm.duty(adcValue)
13         print(adc.read())
14         time.sleep_ms(100)
15 except:
16     pwm.deinit()

```

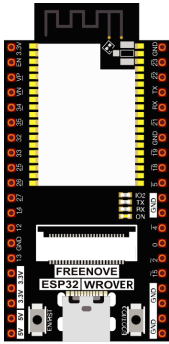
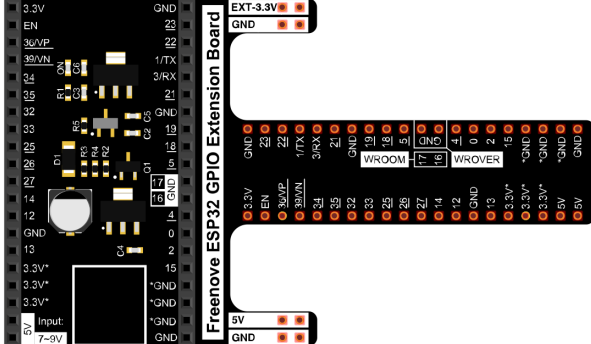
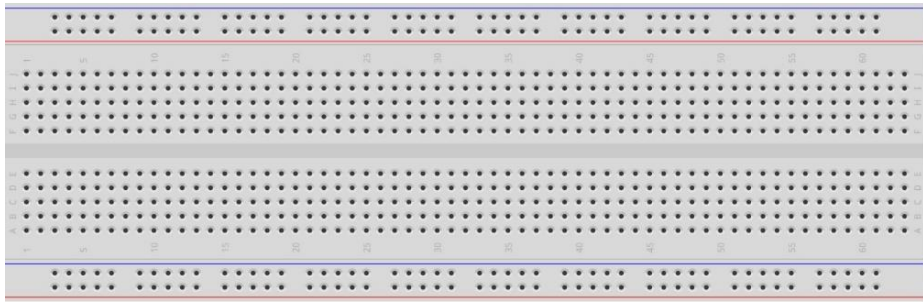
Chapter 12 Thermistor

In this chapter, we will learn about Thermistors which are another kind of Resistor

Project 12.1 Thermometer

A Thermistor is a type of Resistor whose resistance value is dependent on temperature and changes in temperature. Therefore, we can take advantage of this characteristic to make a Thermometer.

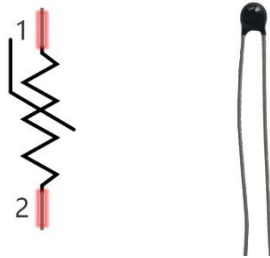
Component List

ESP32-WROVER x1 	GPIO Extension Board x1 	
Breadboard x1 		
Thermistor x1 	Resistor 1kΩ x1 	Jumper M/M x3 

Component knowledge

Thermistor

A Thermistor is a temperature sensitive resistor. When it senses a change in temperature, the resistance of the Thermistor will change. We can take advantage of this characteristic by using a Thermistor to detect temperature intensity. A Thermistor and its electronic symbol are shown below.



The relationship between resistance value and temperature of a thermistor is:

$$R_t = R * \text{EXP} \left[B * \left(\frac{1}{T_2} - \frac{1}{T_1} \right) \right]$$

Where:

R_t is the thermistor resistance under T_2 temperature;

R is the nominal resistance of thermistor under T_1 temperature;

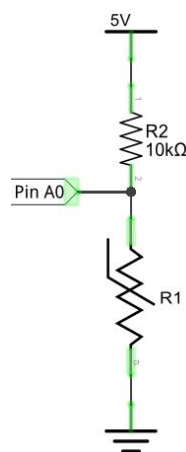
$\text{EXP}[n]$ is n th power of e ;

B is for thermal index;

T_1, T_2 is Kelvin temperature (absolute temperature). Kelvin temperature = 273.15 + Celsius temperature.

For the parameters of the Thermistor, we use: $B=3950$, $R=10k$, $T_1=25$.

The circuit connection method of the Thermistor is similar to photoresistor, as the following:



We can use the value measured by the ADC converter to obtain the resistance value of Thermistor, and then we can use the formula to obtain the temperature value.

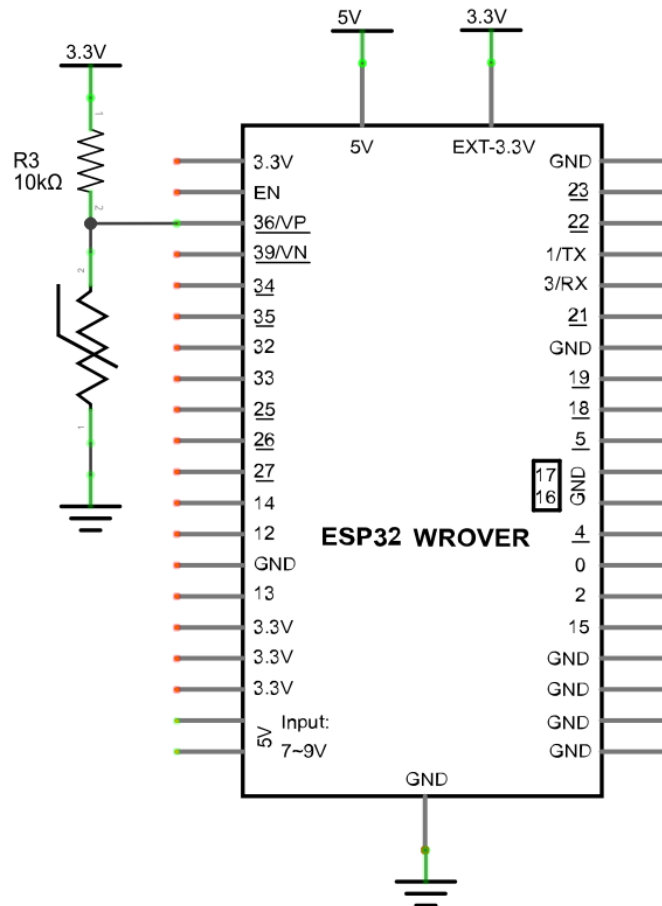
Therefore, the temperature formula can be derived as:

$$T_2 = 1 / \left(\frac{1}{T_1} + \ln \left(\frac{R_t}{R} \right) / B \right)$$

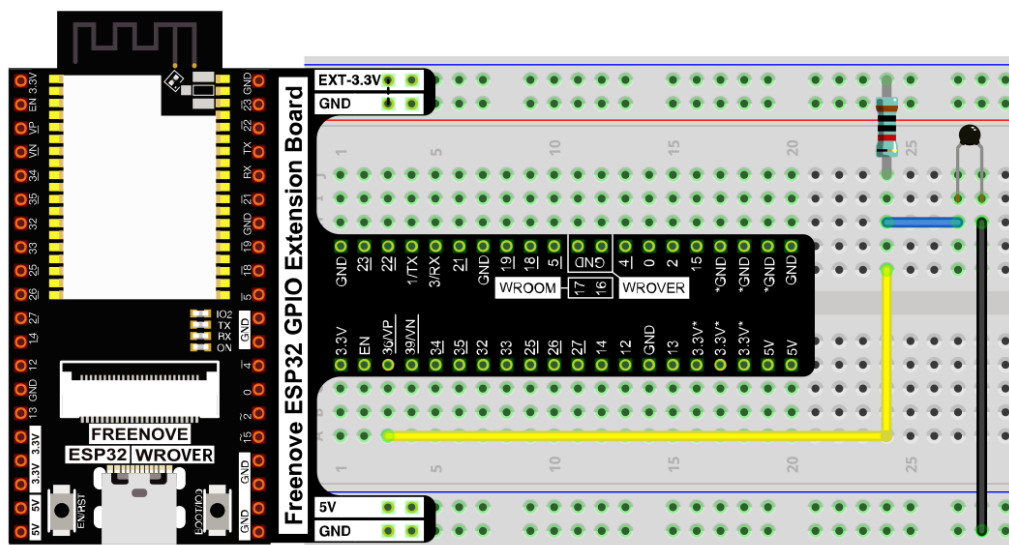
Circuit

The circuit of this project is similar to the one in the previous chapter. The only difference is that the Photoresistor is replaced by a Thermistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

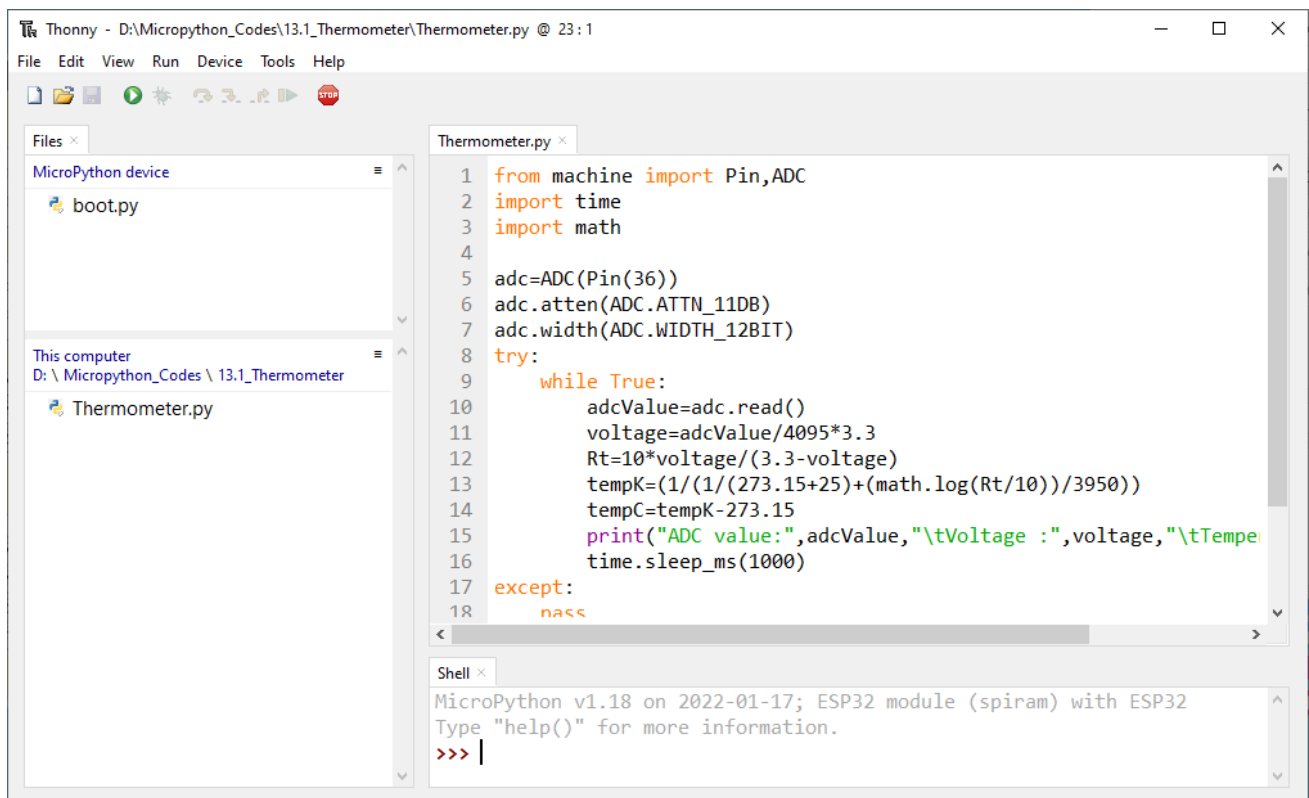


Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “12.1_Thermometer” and double click “Thermometer.py”.

12.1_Thermometer



The screenshot shows the Thonny IDE interface. The main editor displays the code for Thermometer.py, which uses an ADC to read a value from pin 36, converts it to voltage, then to resistance (Rt), and finally to temperature in Kelvin (tempK) and Celsius (tempC). The code prints the ADC value, voltage, and temperature, and sleeps for 1000ms between readings. The left sidebar shows the file explorer with 'Thermometer.py' selected under 'D:\Micropython_Codes\12.1_Thermometer'. The bottom shell window shows the MicroPython v1.18 environment on an ESP32 module, with the prompt '>>>' ready for input.

```

1 from machine import Pin,ADC
2 import time
3 import math
4
5 adc=ADC(Pin(36))
6 adc.atten(ADC.ATTN_11DB)
7 adc.width(ADC.WIDTH_12BIT)
8 try:
9     while True:
10         adcValue=adc.read()
11         voltage=adcValue/4095*3.3
12         Rt=10*voltage/(3.3-voltage)
13         tempK=(1/(1/(273.15+25)+(math.log(Rt/10))/3950))
14         tempC=tempK-273.15
15         print("ADC value:",adcValue,"\tVoltage :",voltage,"\tTempe
16             time.sleep_ms(1000)
17 except:
18     pass
  
```

Shell

MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> |

Click “Run current script” and “Shell” will constantly display the current ADC value, voltage value and temperature value. Try to “pinch” the thermistor (without touching the leads) with your index finger and thumb for a brief time, you should see that the temperature value increases.

If you have any concerns, please contact us via: support@freenove.com

Shell x

MicroPython V1.12.0 on 2019-12-20, ESP32 module (SPIRAM) with ESP32
Type "help()" for more information.

>>> %Run -c \$EDITOR_CONTENT

ADC value: 2175	Voltage : 1.752747	Temperature : 22.21976
ADC value: 2176	Voltage : 1.753553	Temperature : 22.19809
ADC value: 2141	Voltage : 1.725348	Temperature : 22.95731
ADC value: 1989	Voltage : 1.602857	Temperature : 26.2919
ADC value: 1943	Voltage : 1.565788	Temperature : 27.31705
ADC value: 1943	Voltage : 1.565788	Temperature : 27.31705
ADC value: 1943	Voltage : 1.565788	Temperature : 27.31705
ADC value: 1942	Voltage : 1.564982	Temperature : 27.33945
ADC value: 1942	Voltage : 1.564982	Temperature : 27.33945
ADC value: 1943	Voltage : 1.565788	Temperature : 27.31705
ADC value: 2097	Voltage : 1.68989	Temperature : 23.91562
ADC value: 2218	Voltage : 1.787399	Temperature : 21.29001

pinching the
thermistor

The following is the code:

```
1  from machine import Pin, ADC
2  import time
3  import math
4
5  adc=ADC(Pin(36))
6  adc.atten(ADC.ATTN_11DB)
7  adc.width(ADC.WIDTH_12BIT)
8
9  try:
10     while True:
11         adcValue=adc.read()
12         voltage=adcValue/4095*3.3
13         Rt=10*voltage/(3.3-voltage)
14         tempK=(1/(1/(273.15+25)+(math.log(Rt/10))/3950))
15         tempC=tempK-273.15
16         print("ADC value:", adcValue, "\tVoltage :", voltage, "\tTemperature :", tempC);
17         time.sleep_ms(1000)
18 except:
19     pass
```

In the code, the ADC value of ADC module A0 port is read, and then it calculates the voltage and the resistance of Thermistor according to Ohms Law. Finally, it calculates the temperature sensed by the Thermistor, according to the formula.

Chapter 13 Joystick

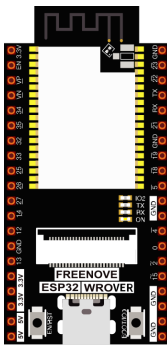
In the previous chapter, we have learned how to use rotary potentiometer. Now, let's learn a new electronic module Joystick which working on the same principle as rotary potentiometer.

Project 13.1 Joystick

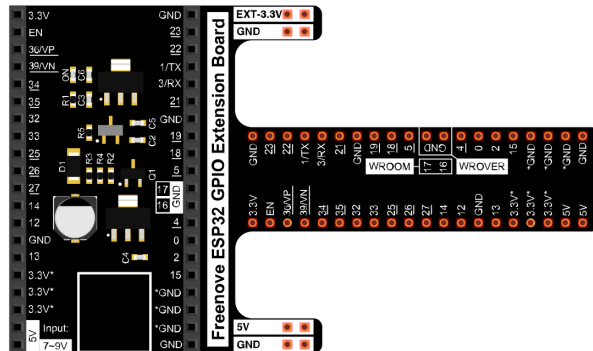
In this project, we will read the output data of a Joystick and display it to the Terminal screen.

Component List

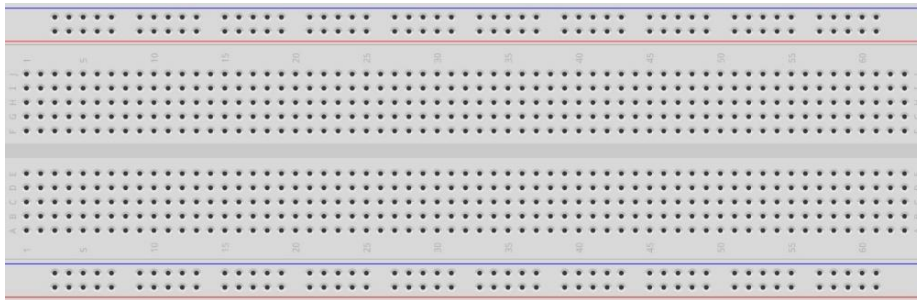
ESP32-WROVER x1



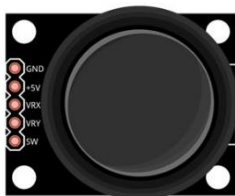
GPIO Extension Board x1



Breadboard x1



Joystick x1



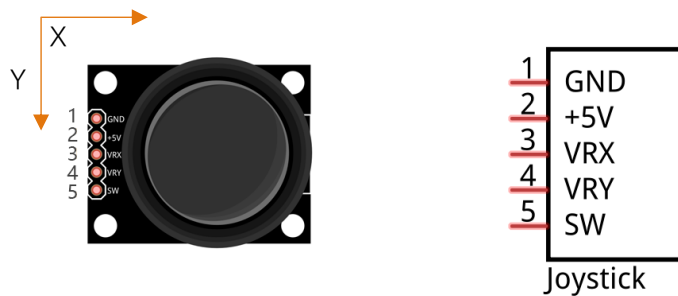
Jumper F/M x5



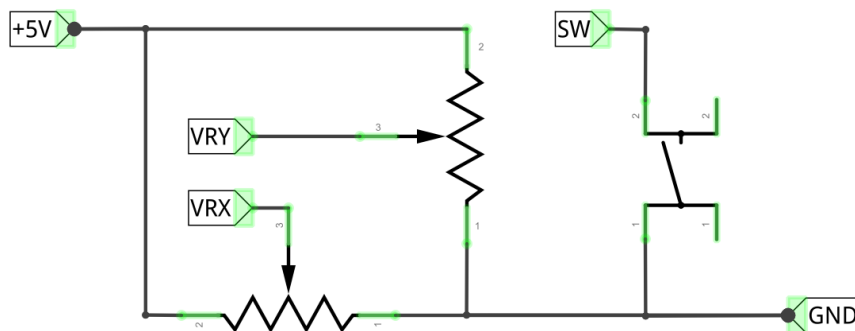
Component knowledge

Joystick

A Joystick is a kind of input sensor used with your fingers. You should be familiar with this concept already as they are widely used in gamepads and remote controls. It can receive input on two axes (Y and or X) at the same time (usually used to control direction on a two dimensional plane). And it also has a third direction capability by pressing down (Z axis/direction).



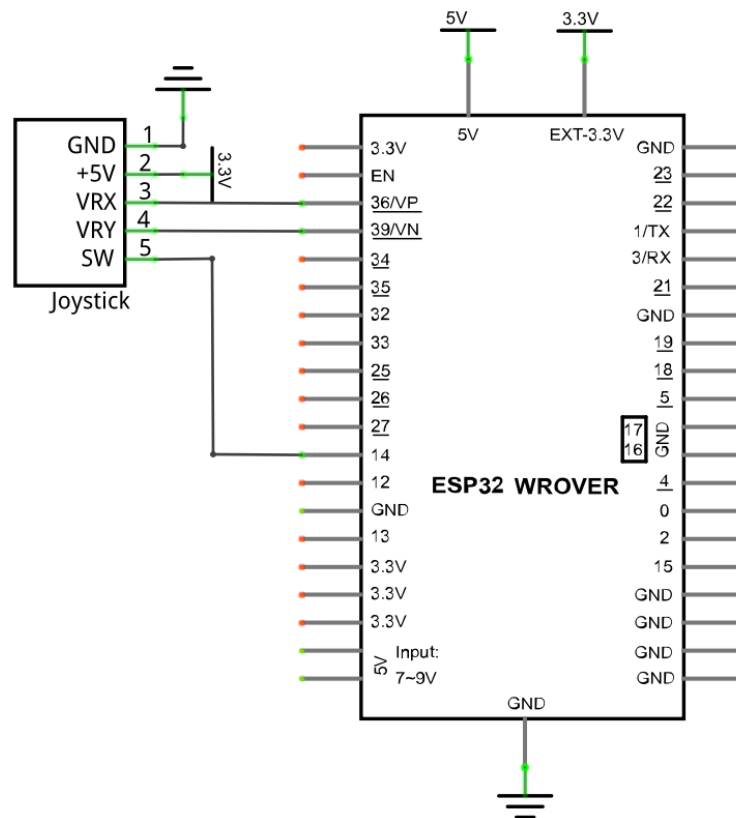
This is accomplished by incorporating two rotary potentiometers inside the Joystick Module at 90 degrees of each other, placed in such a manner as to detect shifts in direction in two directions simultaneously and with a Push Button Switch in the "vertical" axis, which can detect when a User presses on the Joystick.



When the Joystick data is read, there are some differences between the axes: data of X and Y axes is analog, which needs to use the ADC. The data of the Z axis is digital, so you can directly use the GPIO to read this data or you have the option to use the ADC to read this.

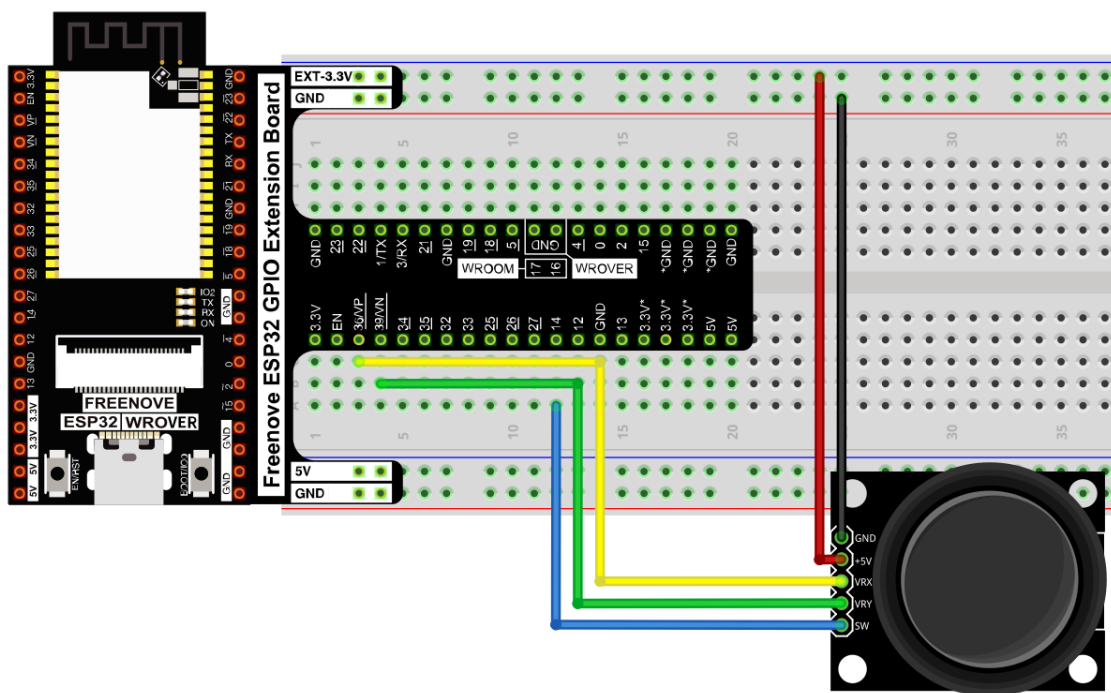
Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via:

support@freenove.com



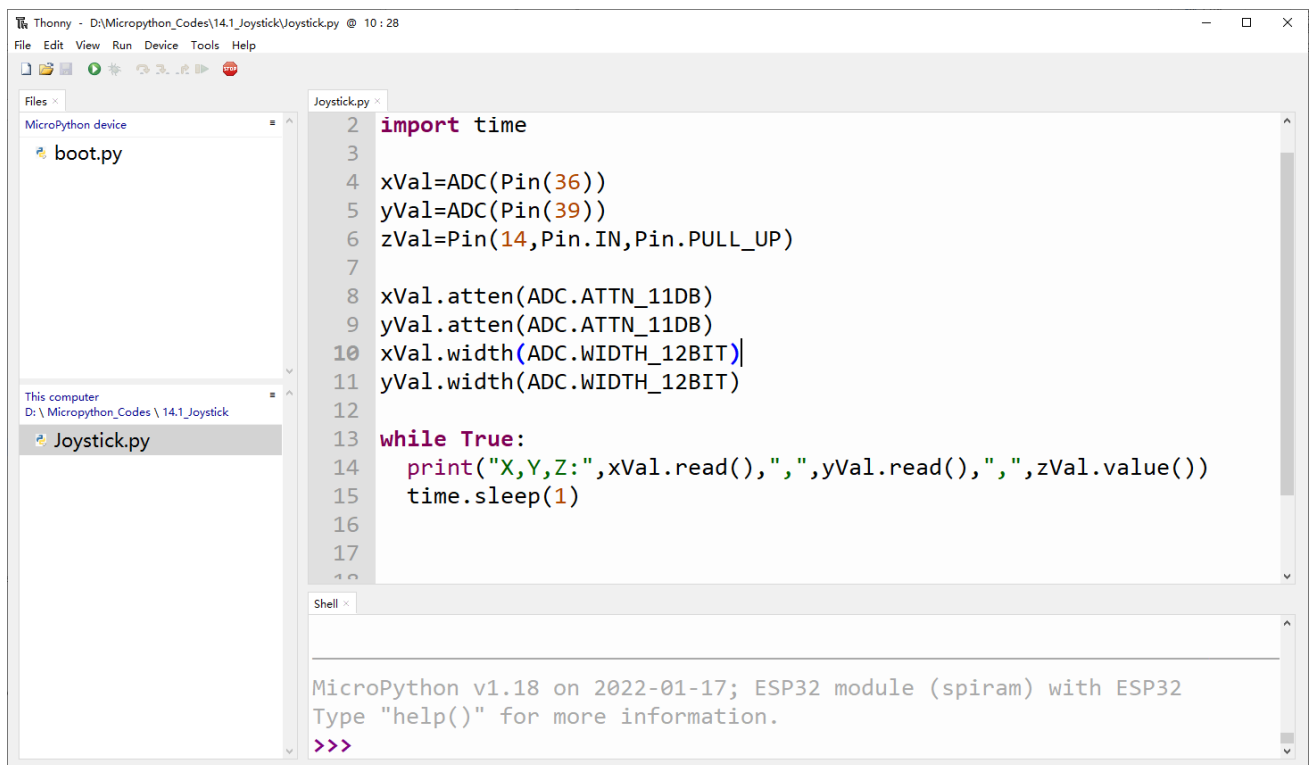
Code

In this project's code, we will read the ADC values of X and Y axes of the Joystick, and read digital quality of the Z axis, then display these out in Terminal.

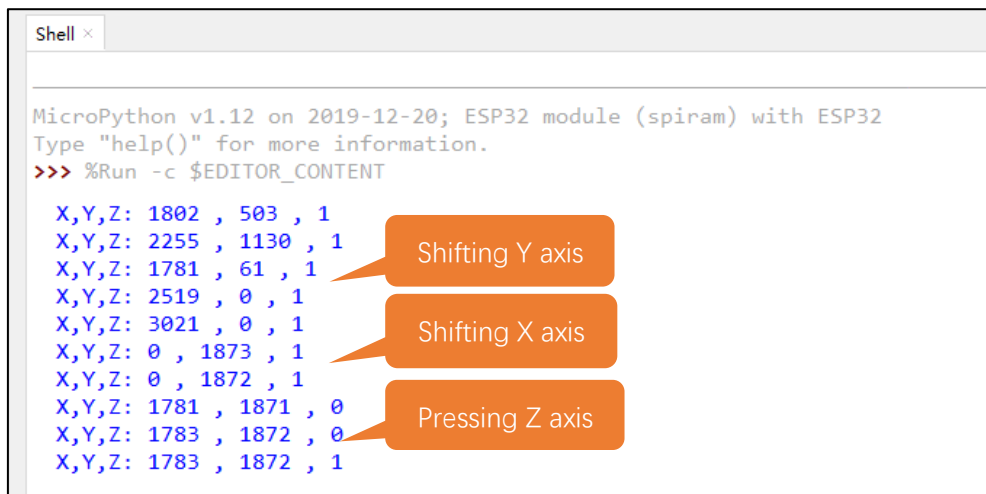
Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “13.1_Joystick” and double click “Joystick.py”.

13.1_Joystick



Click “Run current script”. Shifting the Joystick or pressing it down will make the printed data in “Shell” change.



The flowing is the code:

```
1  from machine import ADC, Pin
2  import time
3
4  xVal=ADC(Pin(36))
5  yVal=ADC(Pin(39))
6  zVal=Pin(14, Pin.IN, Pin.PULL_UP)
7
8  xVal atten(ADC.ATTN_11DB)
9  yVal atten(ADC.ATTN_11DB)
10 xVal.width(ADC.WIDTH_12BIT)
11 yVal.width(ADC.WIDTH_12BIT)
12
13 while True:
14     print("X,Y,Z:", xVal.read(), ",", yVal.read(), ",", zVal.value())
15     time.sleep(1)
```

Set the acquisition range of voltage of the two ADC channels to 0-3.3V, and the acquisition width of data to 0-4095.

```
8  xVal.atten(ADC.ATTN_11DB)
9  yVal.atten(ADC.ATTN_11DB)
10 xVal.width(ADC.WIDTH_12BIT)
11 yVal.width(ADC.WIDTH_12BIT)
```

In the code, configure Z_Pin to pull-up input mode. In loop(), use Read () to read the value of axes X and Y and use value() to read the value of axis Z, and then display them.

```
14 print("X,Y,Z:", xVal.read(), ",", yVal.read(), ",", zVal.value())
```

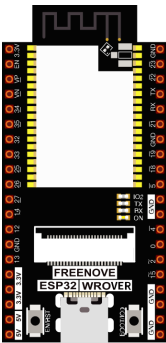
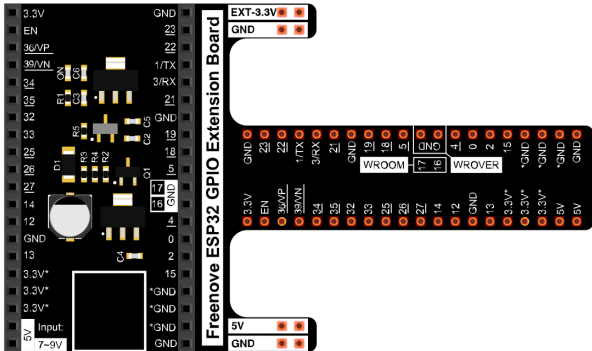
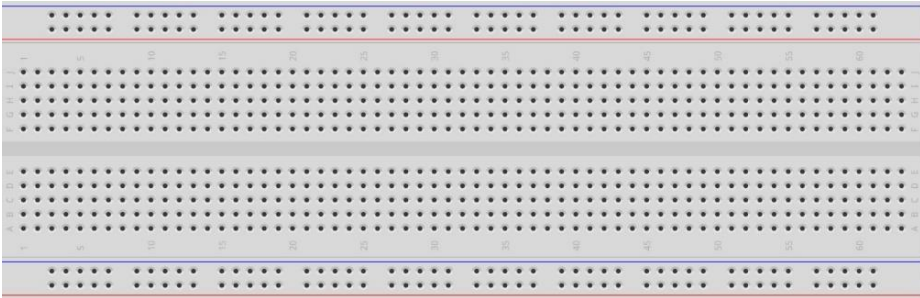
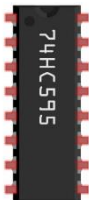
Chapter 14 74HC595 & LED Bar Graph

We have used LED Bar Graph to make a flowing water light, in which 10 GPIO ports of ESP32 is occupied. More GPIO ports mean that more peripherals can be connected to ESP32, so GPIO resource is very precious. Can we make flowing water light with less GPIO? In this chapter, we will learn a component, 74HC595, which can achieve the target.

Project 14.1 Flowing Water Light

Now let's learn how to use the 74HC595 IC Chip to make a flowing water light using less GPIO.

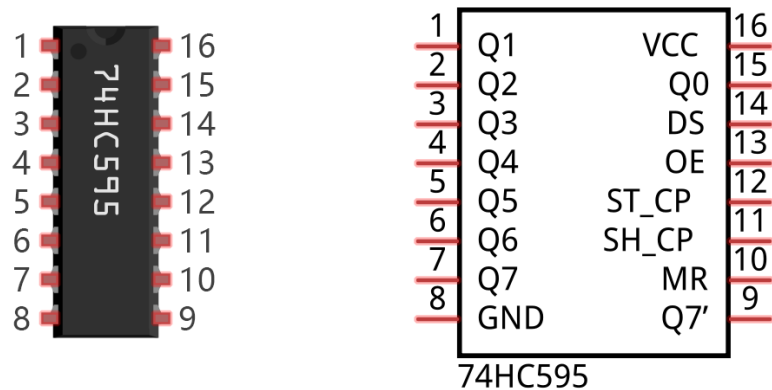
Component List

ESP32-WROVER x1 	GPIO Extension Board x1 		
Breadboard x1 			
74HC595 x1 	LED Bar Graph x1 	Resistor 220Ω x8 	Jumper M/M x15 

Related knowledge

74HC595

A 74HC595 chip is used to convert serial data into parallel data. A 74HC595 chip can convert the serial data of one byte into 8 bits, and send its corresponding level to each of the 8 ports correspondingly. With this characteristic, the 74HC595 chip can be used to expand the IO ports of an ESP32. At least 3 ports are required to control the 8 ports of the 74HC595 chip.



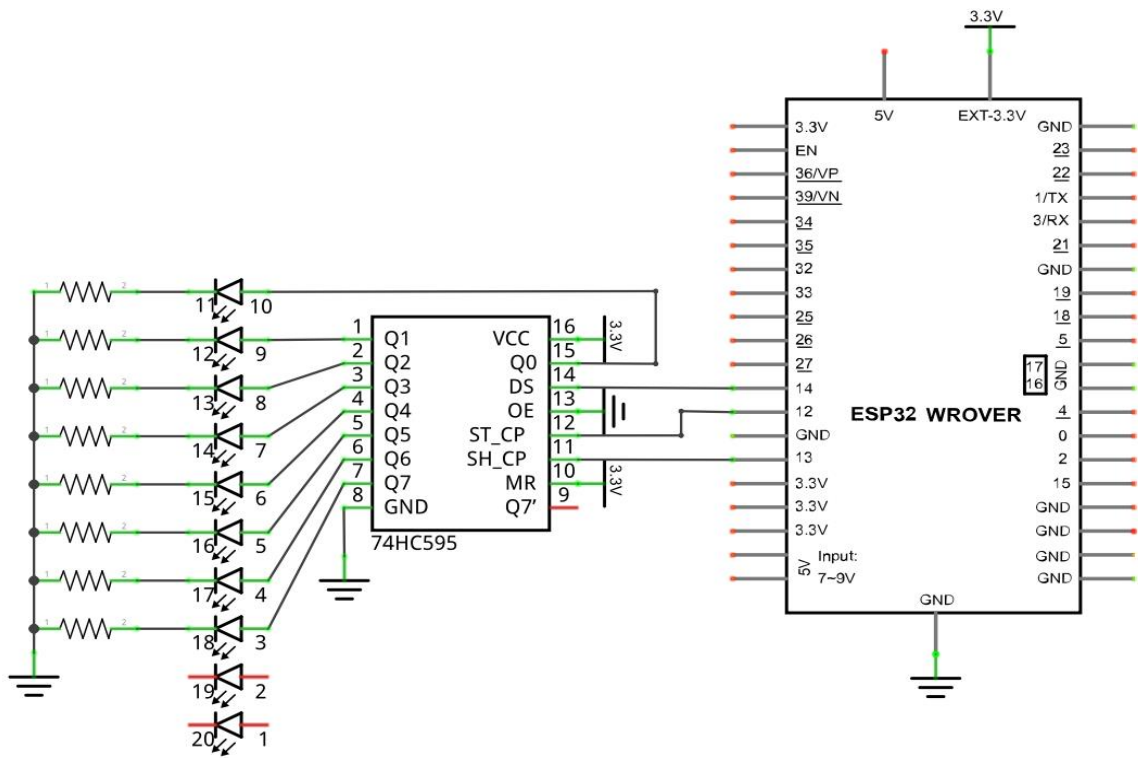
The ports of the 74HC595 chip are described as follows:

Pin name	GPIO number	Description
Q0-Q7	15, 1-7	Parallel data output
VCC	16	The positive electrode of power supply, the voltage is 2~6V
GND	8	The negative electrode of power supply
DS	14	Serial data Input
OE	13	Enable output, When this pin is in high level, Q0-Q7 is in high resistance state When this pin is in low level, Q0-Q7 is in output mode
ST_CP	12	Parallel Update Output: when its electrical level is rising, it will update the parallel data output.
SH_CP	11	Serial shift clock: when its electrical level is rising, serial data input register will do a shift.
MR	10	Remove shift register: When this pin is in low level, the content in shift register will be cleared.
Q7'	9	Serial data output: it can be connected to more 74HC595 in series.

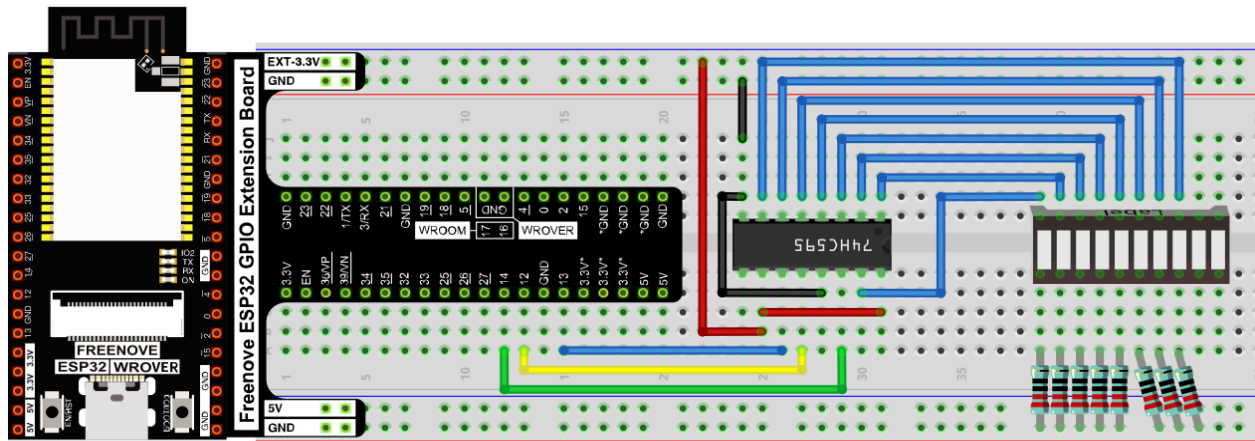
For more detail, please refer to the datasheet on the 74HC595 chip.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



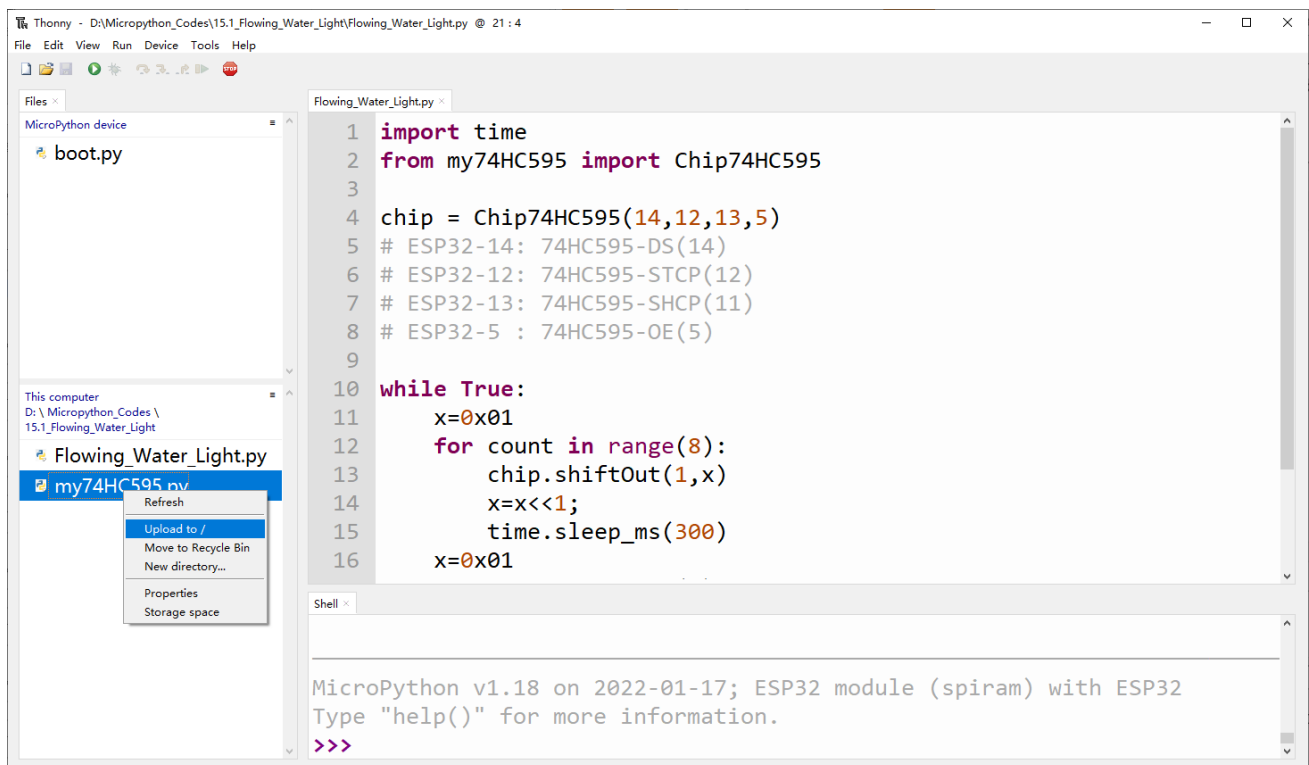
Code

In this project, we will make a flowing water light with a 74HC595 chip to learn about its functions.

Move the program folder “**Freenove Super Starter Kit for ESP32/Python/Python Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “14.1_Flowing_Water_Light”. Select “my74HC595.py”, right click your mouse to select “Upload to /”, wait for “my74HC595.py” to be uploaded to ESP32-WROVER and then double click “Flowing_Water_Light.py”.

14.1_Flowing_Water_Light



Click “Run current script” and you will see that Bar Graph LED starts with the flowing water pattern flashing from left to right and then back from right to left. If it displays nothing, maybe the LED Bar is connected upside down, please unplug it and then re-plug it reversely.

If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
1  import time
2  from my74HC595 import Chip74HC595
3
4  chip = Chip74HC595(14, 12, 13)
5  # ESP32-14: 74HC595-DS(14)
6  # ESP32-12: 74HC595-STCP(12)
7  # ESP32-13: 74HC595-SHCP(11)
8
9  while True:
10     x=0x01
11     for count in range(8):
12         chip.shiftOut(1, x) #High bit is sent first
13         x=x<<1
14         time.sleep_ms(300)
15     x=0x01
16     for count in range(8):
17         chip.shiftOut(0, x) #Low bit is sent first
18         x=x<<1
19         time.sleep_ms(300)
```

Import time and my74HC595 modules.

```
1  import time
2  from my74HC595 import Chip74HC595
```

Assign pins for ESP32-WROVER to connect to 74HC595.

```
4  chip = Chip74HC595(14, 12, 13)
```

The first for loop makes LED Bar display separately from left to right while the second for loop make it display separately from right to left.

```
10     x=0x01
11     for count in range(8):
12         chip.shiftOut(1, x) #High bit is sent first
13         x=x<<1
14         time.sleep_ms(300)
15     x=0x01
16     for count in range(8):
17         chip.shiftOut(0, x) #Low bit is sent first
18         x=x<<1
19         time.sleep_ms(300)
```

Reference

Class Chip74HC595

Before each use of the object **Chip74HC595**, make sure my74HC595.py has been uploaded to "/" of ESP32, and then add the statement "**from my74HC595 import Chip74HC595**" to the top of the python file.

Chip74HC595(): An object. By default, 74HC595's DS pin is connected to Pin(14) of ESP32, ST_CP pin is connected to ESP32's Pin(12) and OE pin is connected to ESP's Pin(5). If you need to modify the pins, just do the following operations.

chip=Chip74HC595() or **chip=Chip74HC595(14,12,13,5)**

shiftOut(direction, data): Write data to 74HC595.

direction: 1/0. "1" presents that high-order byte will be sent first while "0" presents that low-order byte will be sent first.

data: The content that is sent, which is one-byte data.

clear(): Clear the latch data of 74HC595.。

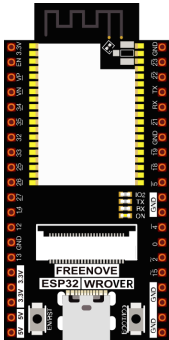
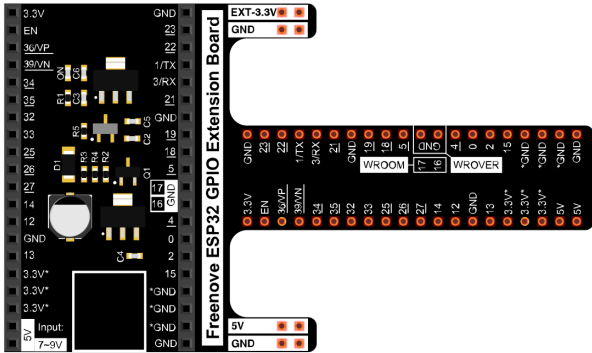
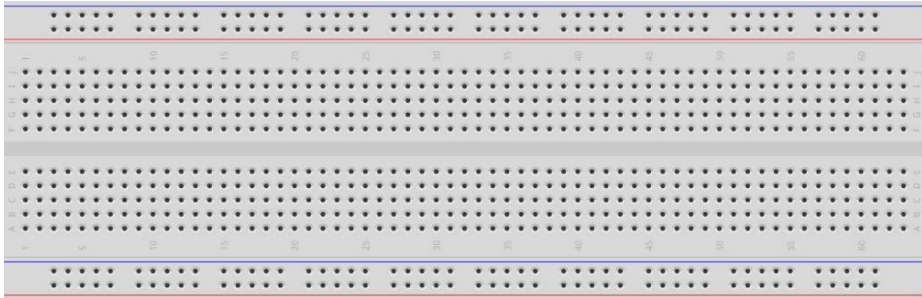
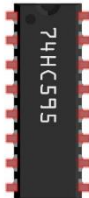
Chapter 15 74HC595 & 7-Segment Display.

In this chapter, we will introduce the 7-Segment Display.

Project 15.1 7-Segment Display.

We will use 74HC595 to control 7-segment display and make it display hexadecimal character "0-F".

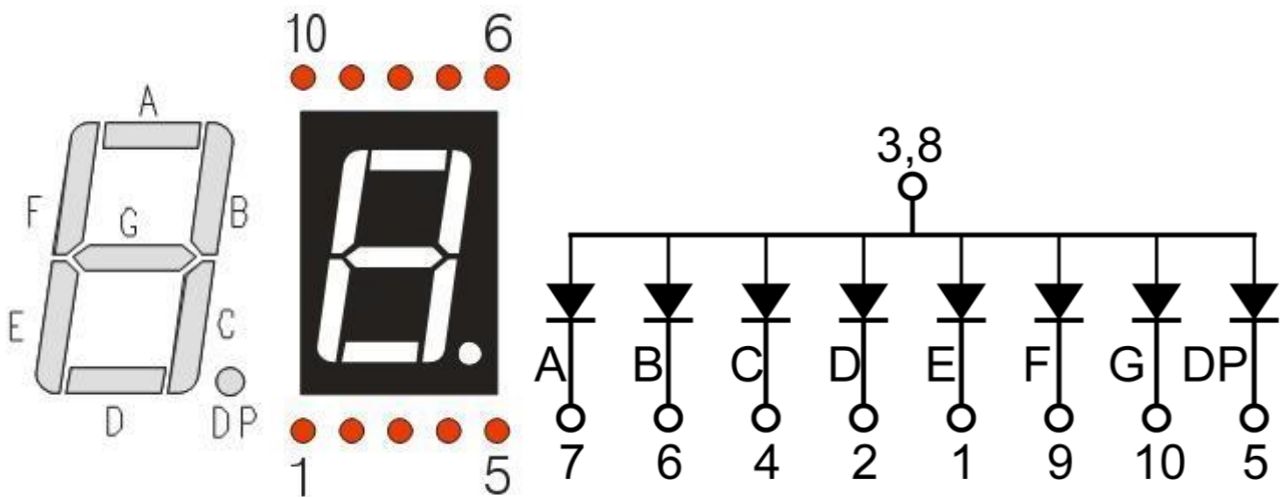
Component List

ESP32-WROVER x1 	GPIO Extension Board x1 		
Breadboard x1 			
74HC595 x1 	7-segment display x1 	Resistor 220Ω x8 	Jumper M/M 

Component knowledge

7-segment display

A 7-Segment Display is a digital electronic display device. There is a figure "8" and a decimal point represented, which consists of 8 LEDs. The LEDs have a Common Anode and individual Cathodes. Its internal structure and pin designation diagram is shown below:



As we can see in the above circuit diagram, we can control the state of each LED separately. Also, by combining LEDs with different states of ON and OFF, we can display different characters (Numbers and Letters). For example, to display a "0": we need to turn ON LED segments A, B, C, D, E and F, and turn OFF LED segments G and DP.



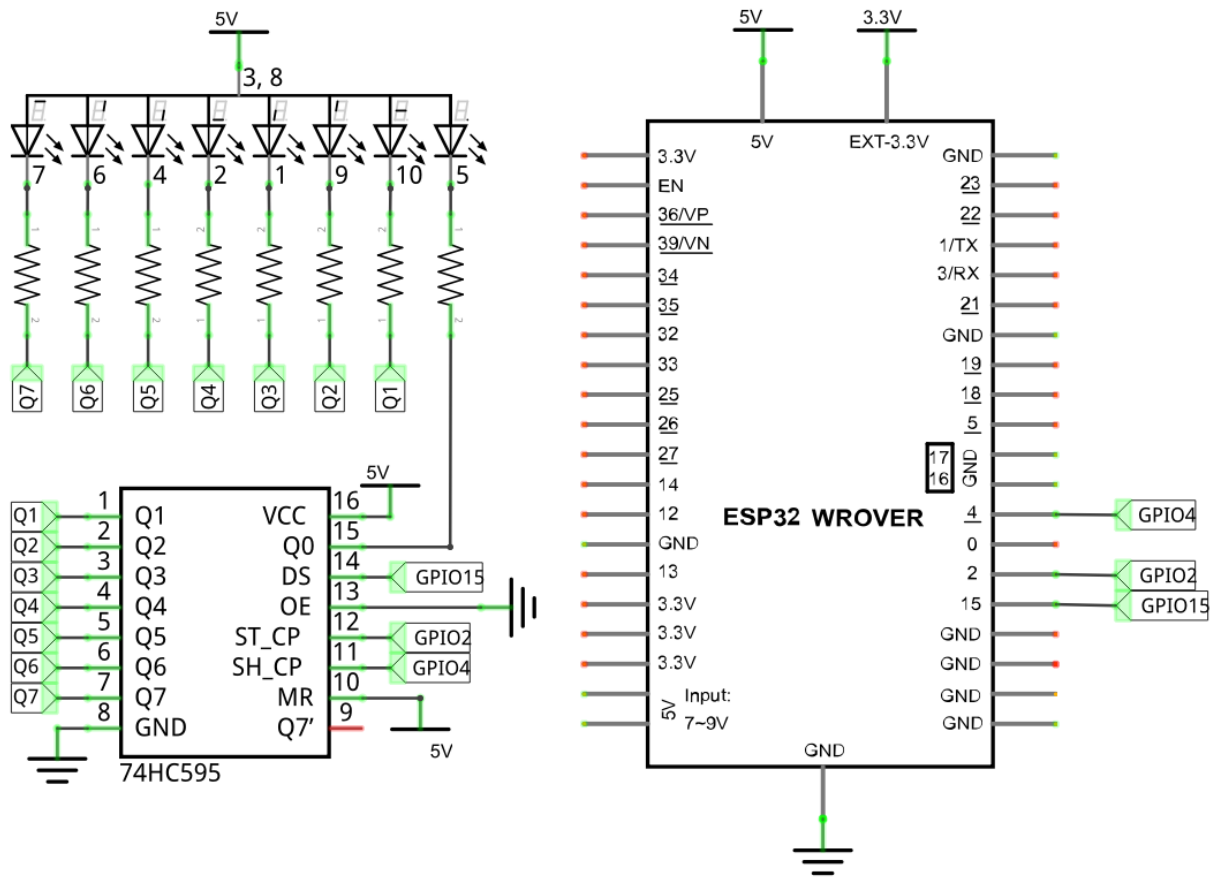
In this project, we will use a 7-Segment Display with a Common Anode. Therefore, when there is an input low level to an LED segment the LED will turn ON. Defining segment "A" as the lowest level and segment "DP" as the highest level, from high to low would look like this: "DP", "G", "F", "E", "D", "C", "B", "A". Character "0" corresponds to the code: `1100 0000b=0xc0`.

For detailed code values, please refer to the following table (common anode).

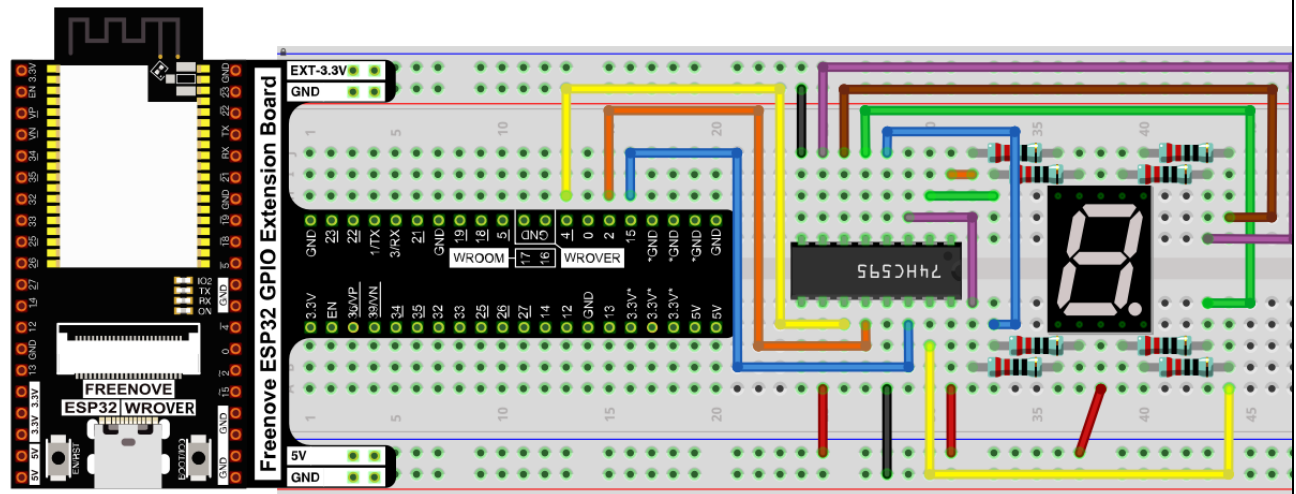
CHAR	DP	G	F	E	D	C	B	A	Hex	ASCII
0	1	1	0	0	0	0	0	0	0xc0	1100 0000
1	1	1	1	1	1	0	0	1	0xf9	1111 1001
2	1	0	1	0	0	1	0	0	0xa4	1010 0100
3	1	0	1	1	0	0	0	0	0xb0	1011 0000
4	1	0	0	1	1	0	0	1	0x99	1001 1001
5	1	0	0	1	0	0	1	0	0x92	1001 0010
6	1	0	0	0	0	0	1	0	0x82	1000 0010
7	1	1	1	1	1	0	0	0	0xf8	1111 1000
8	1	0	0	0	0	0	0	0	0x80	1000 0000
9	1	0	0	1	0	0	0	0	0x90	1001 0000
A	1	0	0	0	1	0	0	0	0x88	1000 1000
B	1	0	0	0	0	0	1	1	0x83	1000 0011
C	1	1	0	0	0	1	1	0	0xc6	1100 0110
D	1	0	1	0	0	0	0	1	0xa1	1010 0001
E	1	0	0	0	0	1	1	0	0x86	1000 0110
F	1	0	0	0	1	1	1	0	0x8e	1000 1110

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

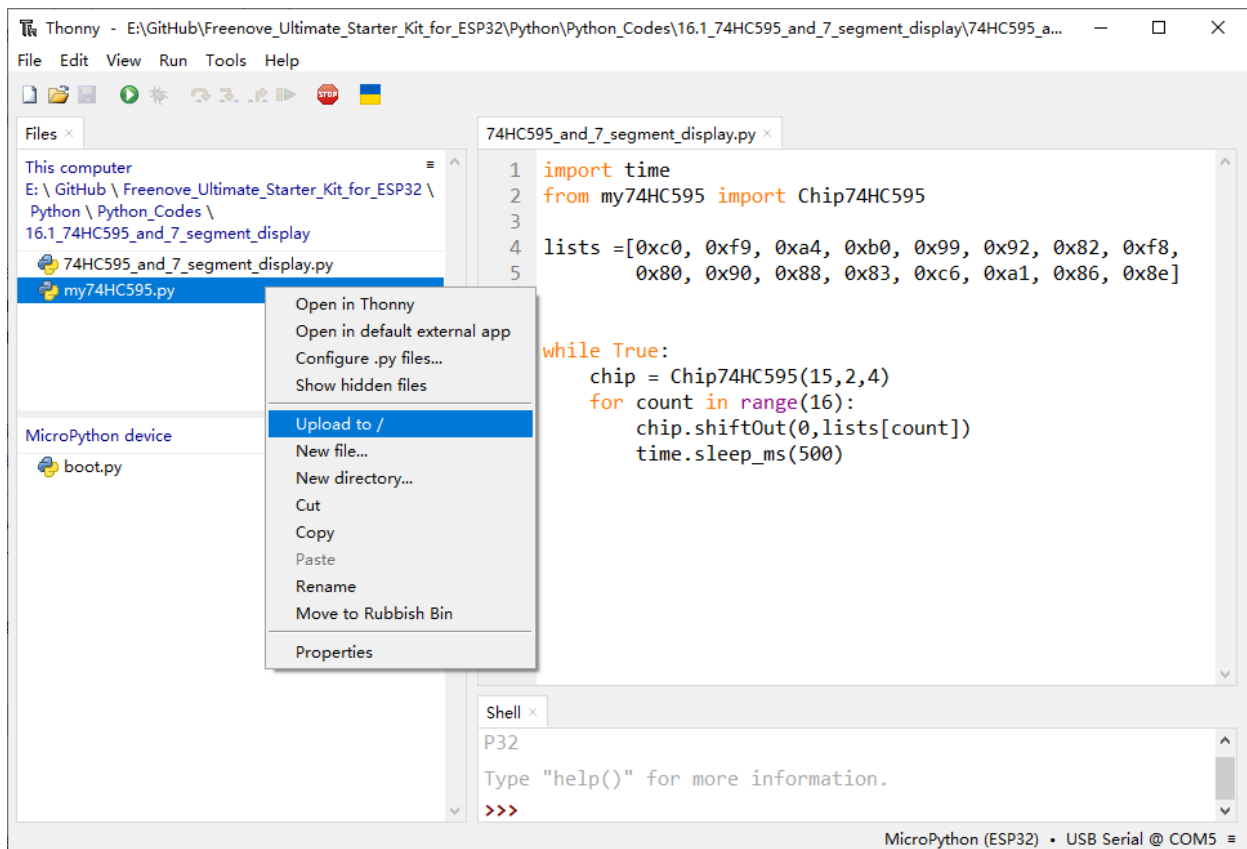
In this section, the 74HC595 is used in the same way as in the previous section, but with different values transferred. We can learn how to master the digital display by sending the code value of "0" - "F".

Move the program folder "**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

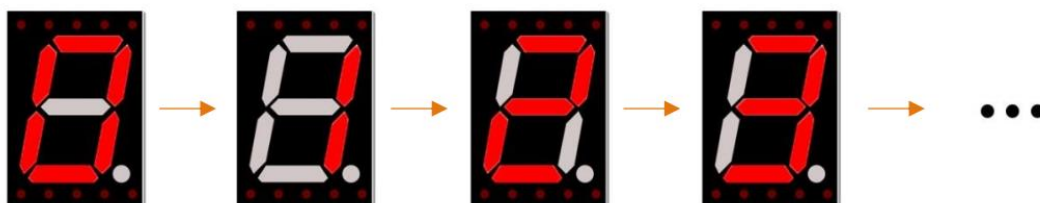
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "15.1_74HC595_and_7_segment_display".

Select "my74HC595.py", right click your mouse to select "Upload to /", wait for "my74HC595.py" to be uploaded to ESP32-WROVER and then double click "74HC595_and_7_segment_display.py".

15.1_74HC595_and_7_segment_display



Click "Run current script", and you'll see a 1-bit, 7-segment display displaying 0-f in a loop.



The following is the program code:

```
1  import time
2  from my74HC595 import Chip74HC595
3
4  lists =[0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8,
5          0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
6
7  chip = Chip74HC595(15,2,4)
8  try:
9      while True:
10         for count in range(16):
11             chip.shiftOut(0, lists[count])
12             time.sleep_ms(500)
13 except:
14     pass
```

Import time and my74HC595 modules.

```
1  import time
2  from my74HC595 import Chip74HC595
```

Put the encoding "0" - "F" into the list.

```
4  lists =[0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8,
5          0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
```

Define an object, whose pins applies default configuration, to drive 74HC595.

```
7  chip = Chip74HC595(15,2,4)
```

Send data of digital tube to 74HC595 chip.

```
11 chip.shiftOut(0, lists[count])
```

Chapter 16 Relay & Motor

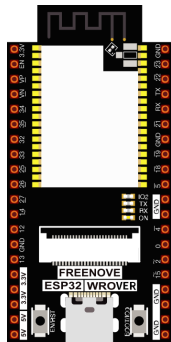
In this chapter, we will learn a kind of special switch module, Relay Module.

Project 16.1 Control Motor with Potentiometer

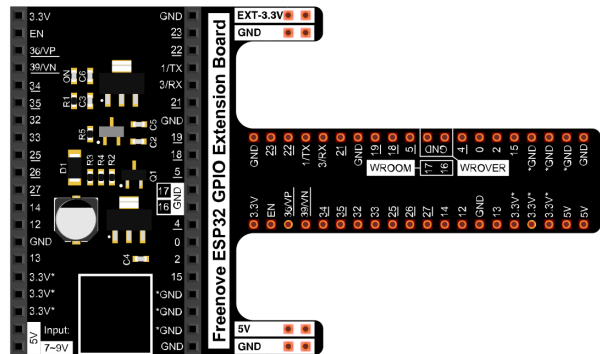
Control the direction and speed of the motor with a potentiometer.

Component List

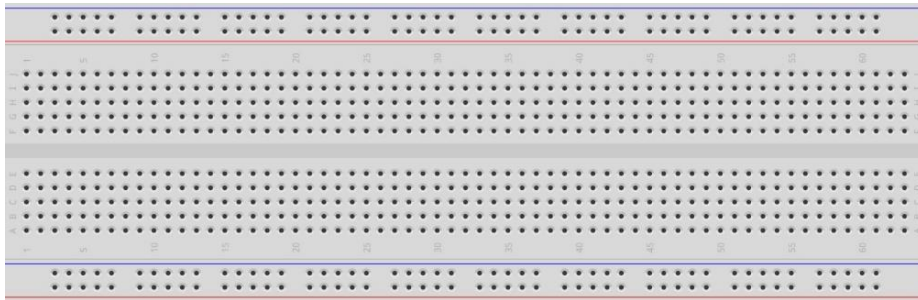
ESP32-WROVER x1



GPIO Extension Board x1



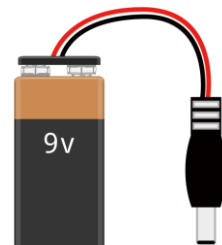
Breadboard x1

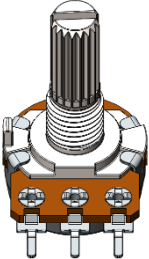
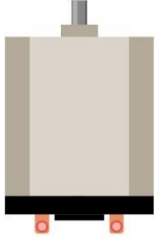
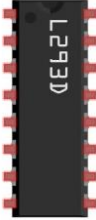


Jumper M/M



9V battery (prepared by yourself) & battery line

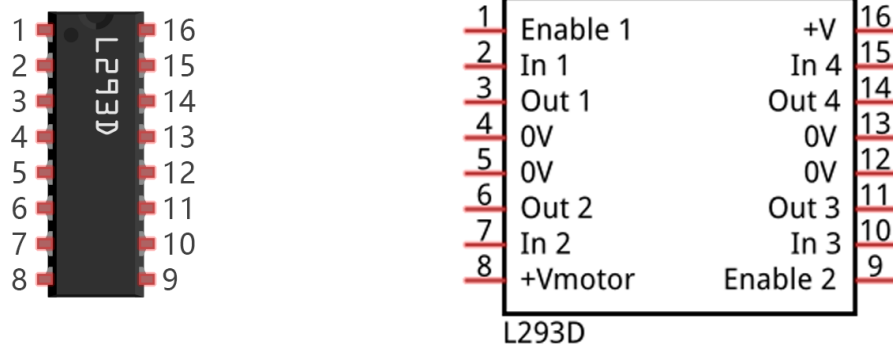


Rotary potentiometer x1 	Motor x1 	L293D 
--	---	--

Component knowledge

L293D

L293D is an IC Chip (Integrated Circuit Chip) with a 4-channel motor drive. You can drive a Unidirectional DC Motor with 4 ports or a Bi-Directional DC Motor with 2 ports or a Stepper Motor (Stepper Motors are covered later in this Tutorial).



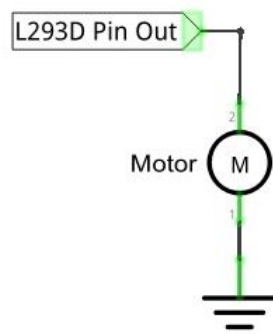
Port description of L293D module is as follows:

Pin name	Pin number	Description
In x	2, 7, 10, 15	Channel x digital signal input pin
Out x	3, 6, 11, 14	Channel x output pin, input high or low level according to In x pin, get connected to +Vmotor or 0V
Enable1	1	Channel 1 and channel 2 enable pin, high level enable
Enable2	9	Channel 3 and channel 4 enable pin, high level enable
0V	4, 5, 12, 13	Power cathode (GND)
+V	16	Positive electrode (VCC) of power supply, supply voltage 3.0~36V
+Vmotor	8	Positive electrode of load power supply, provide power supply for the Out pin x, the supply voltage is +V~36V

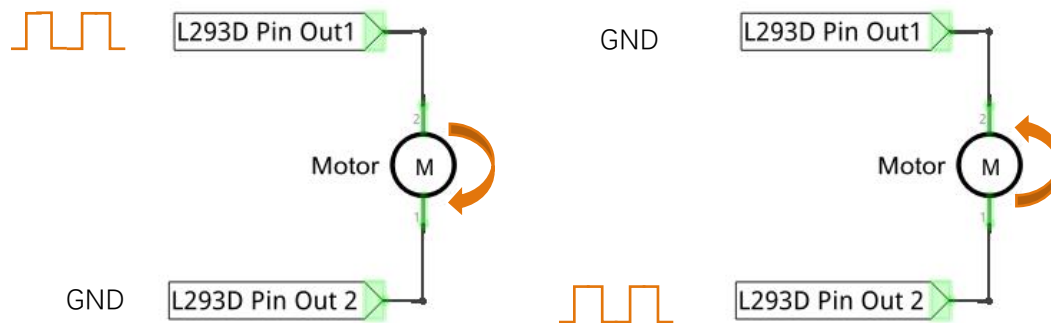
For more detail, please refer to the datasheet for this IC Chip.

When using L293D to drive DC motor, there are usually two connection options.

The following connection option uses one channel of the L293D, which can control motor speed through the PWM, However the motor then can only rotate in one direction.



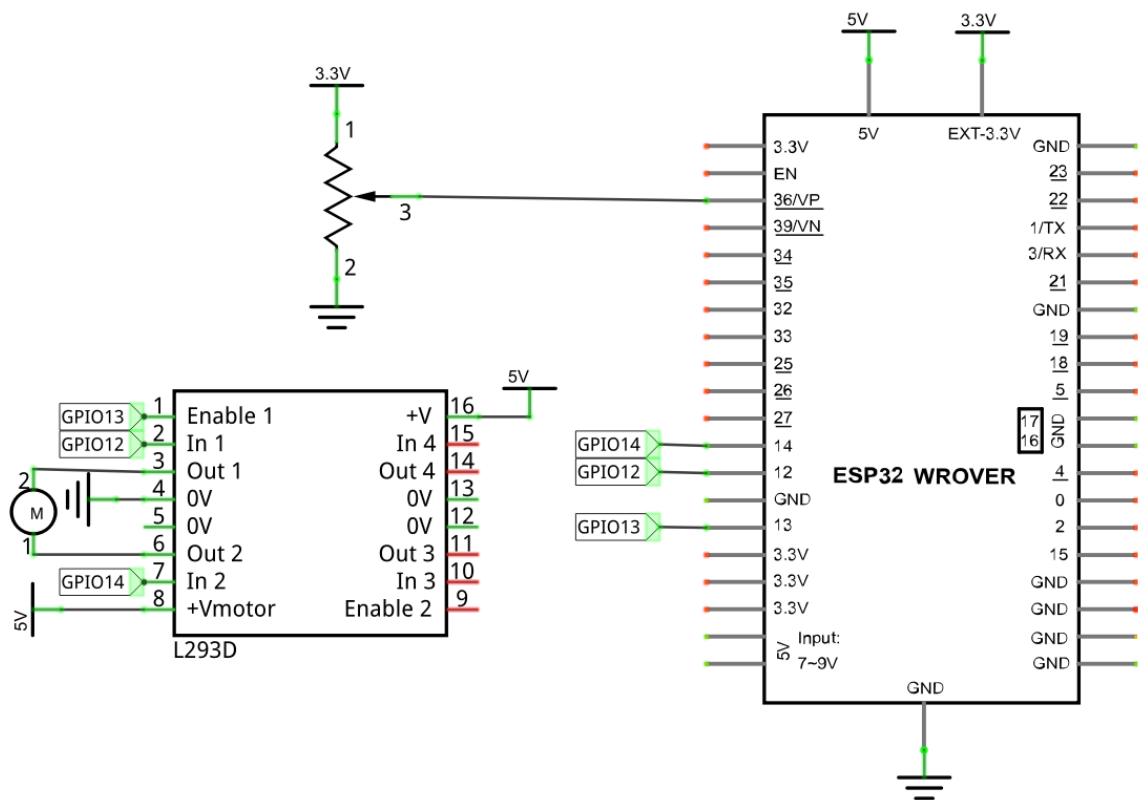
The following connection uses two channels of the L293D: one channel outputs the PWM wave, and the other channel connects to GND, therefore you can control the speed of the motor. When these two channel signals are exchanged, not only can they control the speed of motor, but also control the direction of the motor.



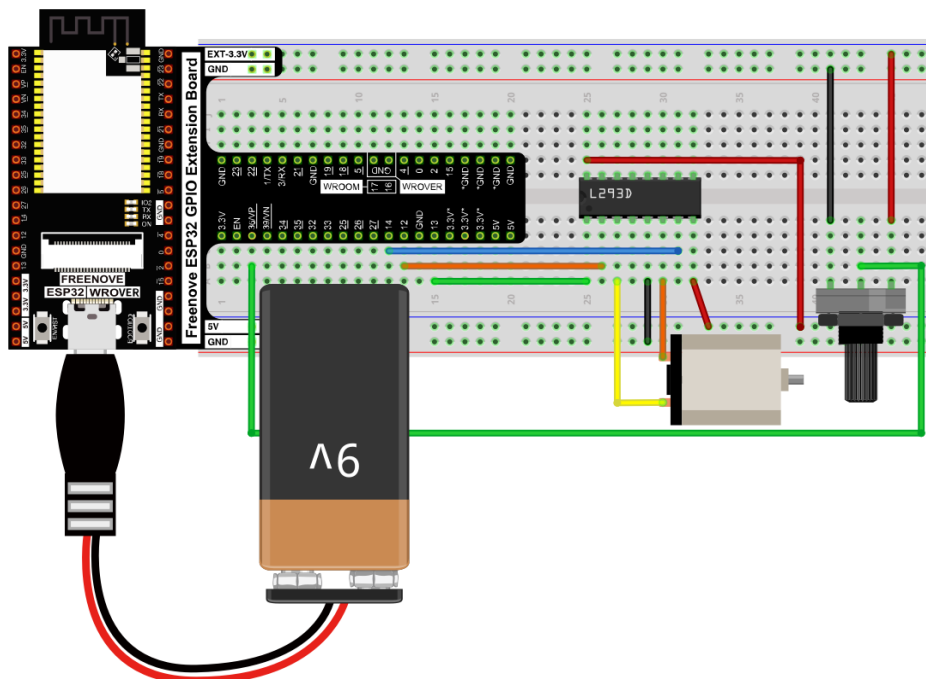
In practical use the motor is usually connected to channels 1 and 2 by outputting different levels to in1 and in2 to control the rotational direction of the motor, and output to the PWM wave to Enable1 port to control the motor's rotational speed. If the motor is connected to channel 3 and 4 by outputting different levels to in3 and in4 to control the motor's rotation direction, and output to the PWM wave to Enable2 pin to control the motor's rotational speed.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



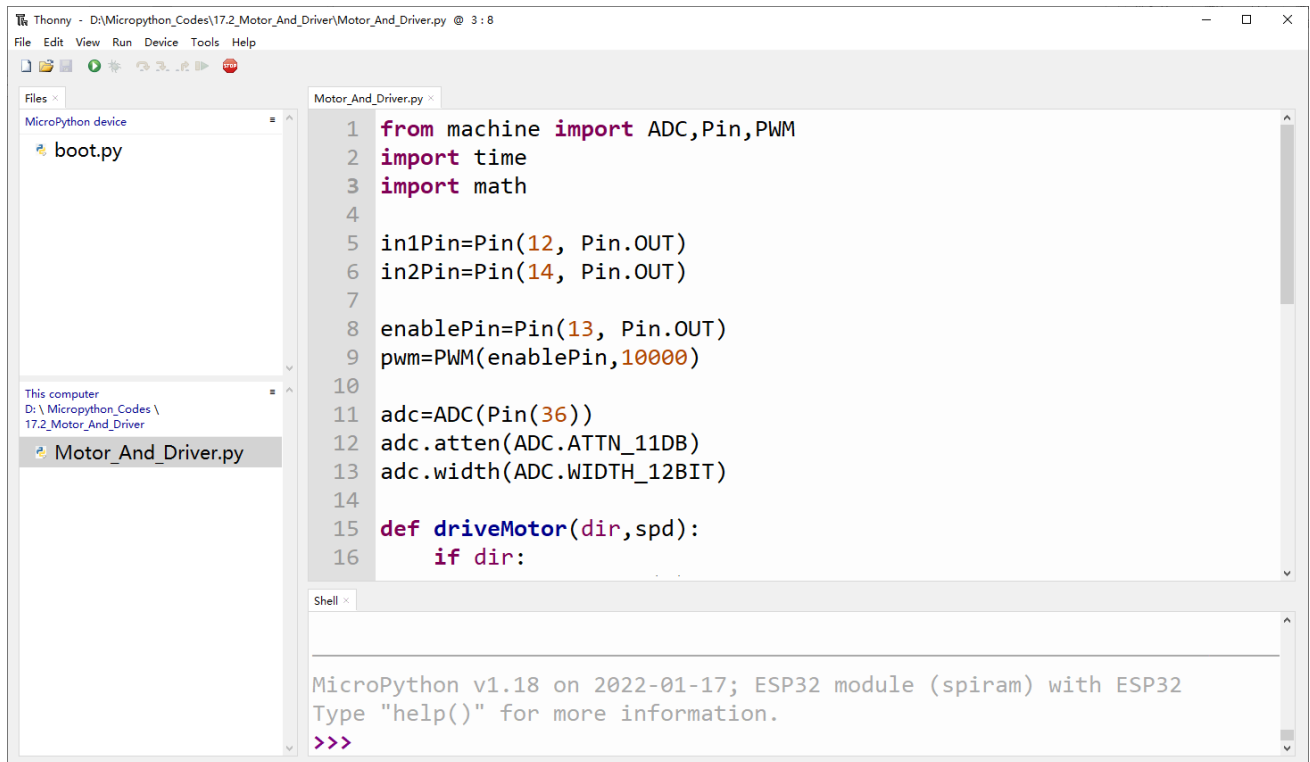
Note: the motor circuit uses A large current, about 0.2-0.3A without load. We recommend that you use a 9V battery to power the extension board.

Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “16.1_Motor_And_Driver” and double click “Motor_And_Driver.py”.

16.1_Motor_And_Driver



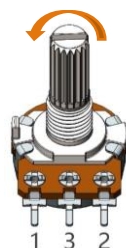
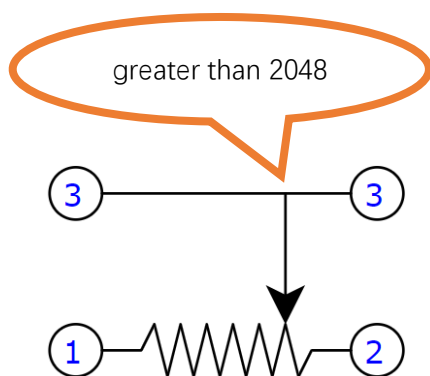
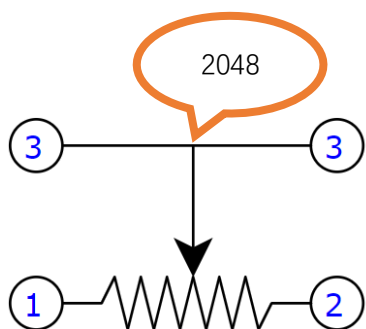
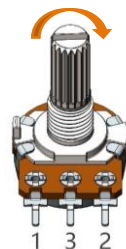
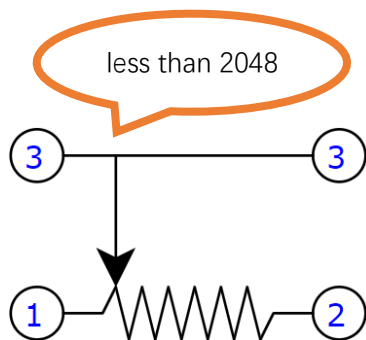
```
Thonny - D:\Micropython_Codes\17.2_Motor_And_Driver\Motor_And_Driver.py @ 3 : 8
File Edit View Run Device Tools Help

Files
  MicroPython device
    boot.py
  This computer
    D:\Micropython_Codes\
      17.2_Motor_And_Driver
        Motor_And_Driver.py

Motor_And_Driver.py
1 from machine import ADC,Pin,PWM
2 import time
3 import math
4
5 in1Pin=Pin(12, Pin.OUT)
6 in2Pin=Pin(14, Pin.OUT)
7
8 enablePin=Pin(13, Pin.OUT)
9 pwm=PWM(enablePin,10000)
10
11 adc=ADC(Pin(36))
12 adc.atten(ADC.ATTN_11DB)
13 adc.width(ADC.WIDTH_12BIT)
14
15 def driveMotor(dir,spd):
16     if dir:
```

```
Shell
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>>
```

Click “Run current script”, rotate the potentiometer in one direction and the motor speeds up slowly in one direction. Rotate the potentiometer in the other direction and the motor will slow down to stop. And then rotate it in the original direction to accelerate the motor.



The following is the Code:

```
1  from machine import ADC, Pin, PWM
2  import time
3  import math
4
5  in1Pin=Pin(12, Pin.OUT)
6  in2Pin=Pin(14, Pin.OUT)
7
8  enablePin=Pin(13, Pin.OUT)
9  pwm=PWM(enablePin, 10000)
10
11  adc=ADC(Pin(36))
12  adc.atten(ADC.ATTN_11DB)
13  adc.width(ADC.WIDTH_12BIT)
14
15  def driveMotor(dir, spd):
16      if dir :
17          in1Pin.value(1)
18          in2Pin.value(0)
19      else :
20          in1Pin.value(0)
21          in2Pin.value(1)
22      pwm.duty(spd)
23
24  try:
25      while True:
26          potenVal = adc.read()
27          rotationSpeed = potenVal - 2048
28          if (potenVal > 2048):
29              rotationDir = 1;
30          else:
31              rotationDir = 0;
32          rotationSpeed=int(math.fabs((potenVal-2047)//2)-1)
33          driveMotor(rotationDir, rotationSpeed)
34          time.sleep_ms(10)
35  except:
36      adc.deinit()
37      pwm.deinit()
```

The ADC of ESP32 has a 12-bit accuracy, corresponding to a range from 0 to 4095. In this program, set the number 2048 as the midpoint. If the value of ADC is less than 2048, make the motor rotate in one direction. If the value of ADC is greater than 2048, make the motor rotate in the other direction. Subtract 2048 from the ADC value and take the absolute value, and then divide this result by 2 to be the speed of the motor.

```

26     potenVal = adc.read()
27     rotationSpeed = potenVal - 2048
28     if (potenVal > 2048):
29         rotationDir = 1;
30     else:
31         rotationDir = 0;
32     rotationSpeed=int(math.fabs((potenVal-2047)//2)-1)
33     driveMotor(rotationDir, rotationSpeed)
34     time.sleep_ms(10)

```

Initialize pins of L293D chip.

```

5     in1Pin=Pin(12, Pin.OUT)
6     in2Pin=Pin(14, Pin.OUT)
7
8     enablePin=Pin(13, Pin.OUT)
9     pwm=PWM(enablePin, 10000, 512)

```

Initialize ADC pins, set the range of voltage to 0-3.3V and the acquisition width of data to 0-4095.

```

11    adc=ADC(Pin(36))
12    adc.atten(ADC.ATTN_11DB)
13    adc.width(ADC.WIDTH_12BIT)

```

Function driveMotor is used to control the rotation direction and speed of the motor. The **dir** represents direction while **spd** refers to speed.

```

15    def driveMotor(dir, spd):
16        if dir :
17            in1Pin.value(1)
18            in2Pin.value(0)
19        else :
20            in1Pin.value(0)
21            in2Pin.value(1)
22        pwm.duty(spd)

```

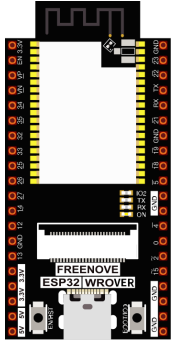
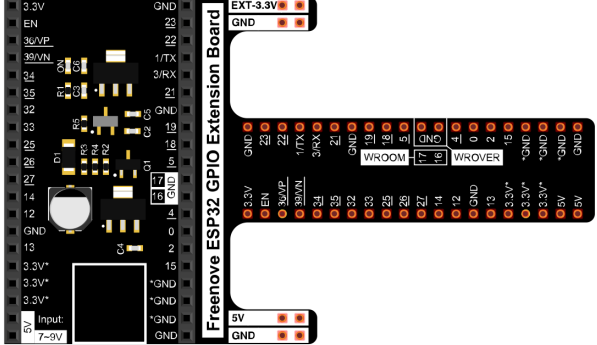
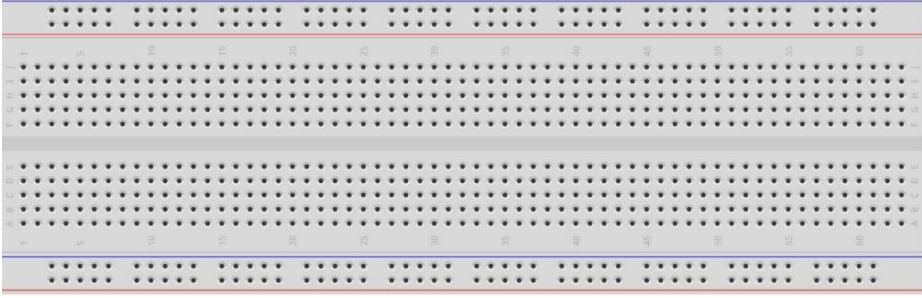
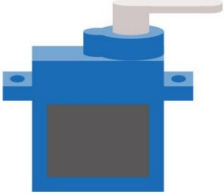
Chapter 17 Servo

Previously, we learned how to control the speed and rotational direction of a Motor. In this chapter, we will learn about Servos which are a rotary actuator type motor that can be controlled to rotate to specific angles.

Project 17.1 Servo Sweep

First, we need to learn how to make a Servo rotate.

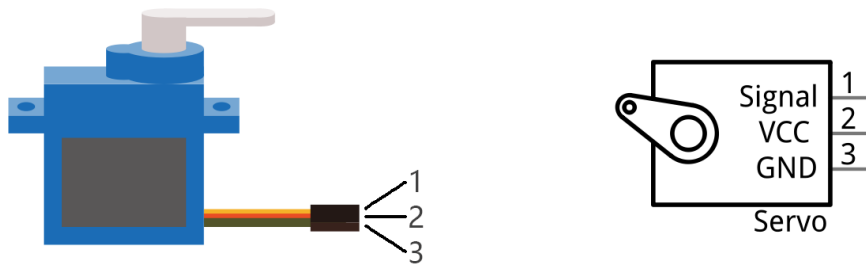
Component List

ESP32-WROVER x1 	GPIO Extension Board x1 
Breadboard x1 	
Servo x1 	Jumper M/M x3 

Component knowledge

Servo

Servo is a compact package which consists of a DC Motor, a set of reduction gears to provide torque, a sensor and control circuit board. Most Servos only have a 180-degree range of motion via their “horn”. Servos can output higher torque than a simple DC Motor alone and they are widely used to control motion in model cars, model airplanes, robots, etc. Servos have three wire leads which usually terminate to a male or female 3-pin plug. Two leads are for electric power: Positive (2-VCC, Red wire), Negative (3-GND, Brown wire), and the signal line (1-Signal, Orange wire) as represented in the Servo provided in your Kit.



We will use a 50Hz PWM signal with a duty cycle in a certain range to drive the Servo. The time interval of 0.5ms-2.5ms of PWM single cycle high level corresponds to the Servo angle 0 degrees - 180 degree linearly. Part of the corresponding values are as follows:

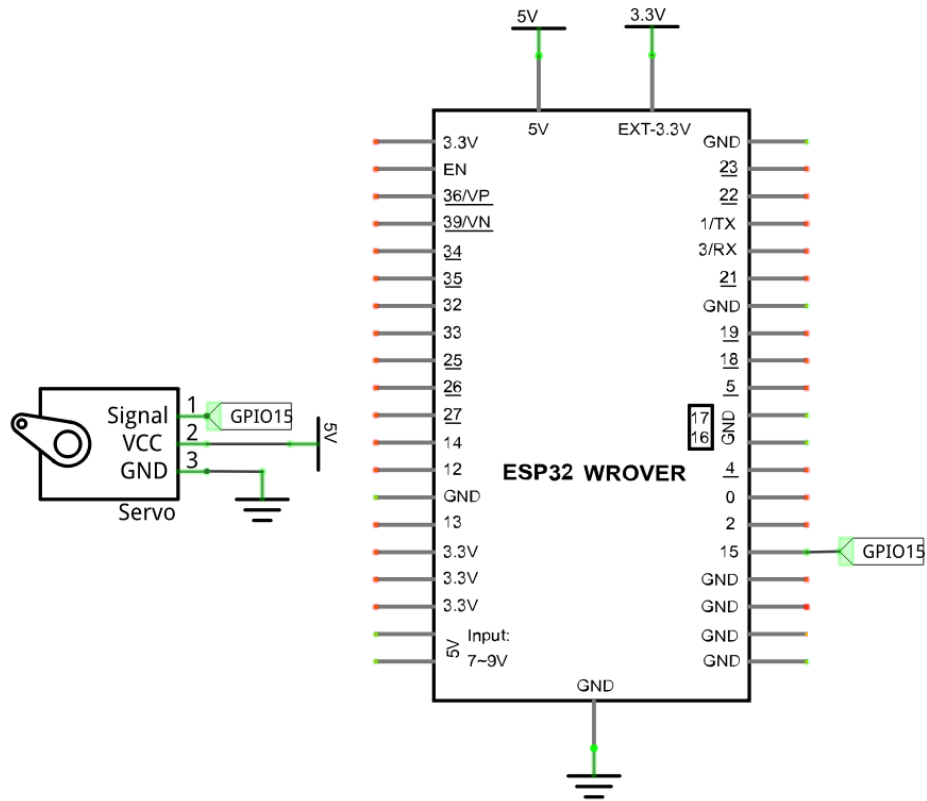
High level time	Servo angle
0.5ms	0 degree
1ms	45 degree
1.5ms	0 degree
2ms	45 degree
2.5ms	180 degree

When you change the Servo signal value, the Servo will rotate to the designated angle.

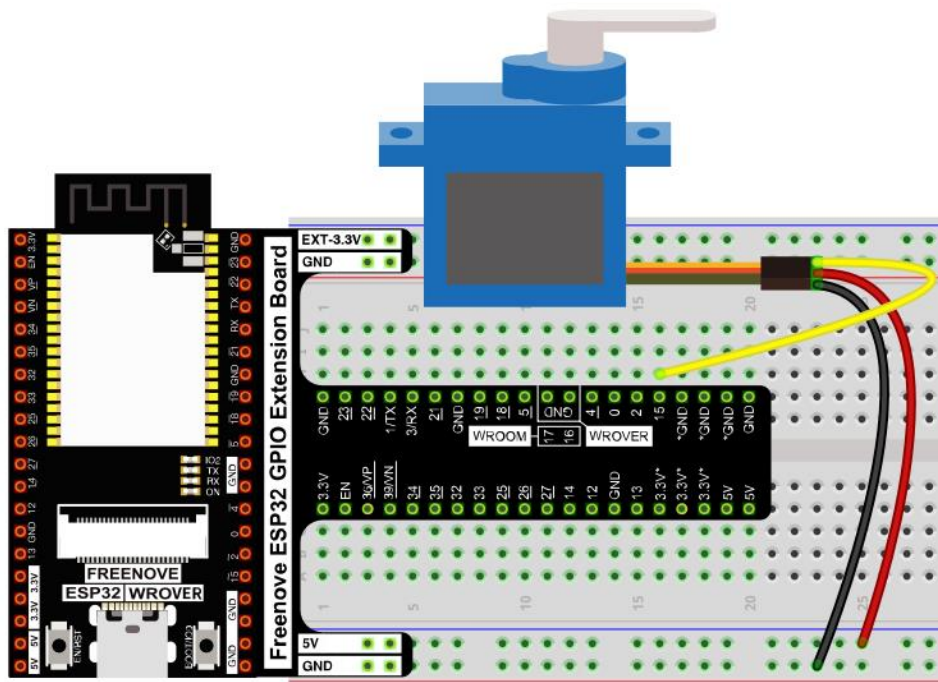
Circuit

Use caution when supplying power to the Servo, it should be 5V. Make sure you do not make any errors when connecting the Servo to the power supply.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

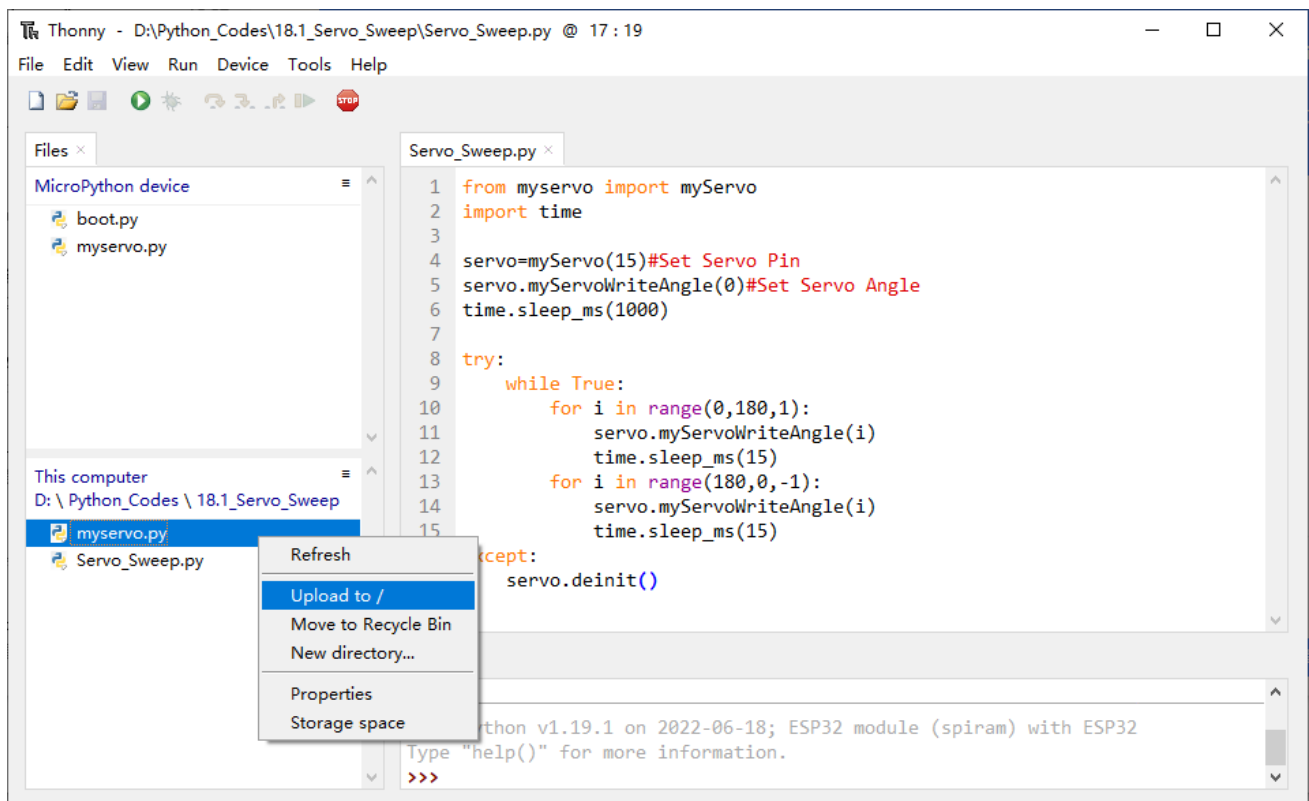


Code

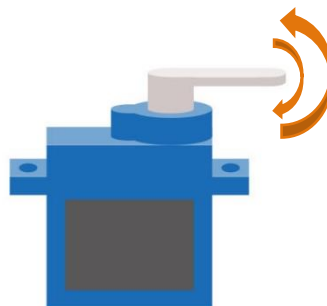
Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “17.1_Servo_Sweep”. Select “myservo.py”, right click your mouse to select “Upload to /”, wait for “myservo.py” to be uploaded to ESP32-WROVER and then double click “Servo_Sweep.py”.

17.1_Servo_Sweep



Click “Run current script”, the Servo will rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.



The following is the program code:

```
1  from myservo import myServo
2  import time
3
4  servo=myServo(15) #Set Servo Pin
5  servo.myServoWriteAngle(0) #Set Servo Angle
6  time.sleep_ms(1000)
7
8  try:
9      while True:
10         for i in range(0, 180, 1):
11             servo.myServoWriteAngle(i)
12             time.sleep_ms(15)
13         for i in range(180, 0, -1):
14             servo.myServoWriteAngle(i)
15             time.sleep_ms(15)
16 except:
17     servo.deinit()
```

Import myservo module.

```
1  from myservo import myServo
```

Initialize pins of the servo and set the starting point of the servo to 0 degree.

```
4  servo=myServo(15)
5  servo.myServoWriteAngle(0)
6  time.sleep_ms(1000)
```

Control the servo to rotate to a specified angle within the range of 0-180 degrees.

```
8  servo.myServoWriteAngle(i)
```

Use two for loops. The first one controls the servo to rotate from 0 degree to 180 degrees while the other controls it to rotate back from 180 degrees to 0 degree.

```
10         for i in range(0, 180, 1):
11             servo.myServoWriteAngle(i)
12             time.sleep_ms(15)
13         for i in range(180, 0, -1):
14             servo.myServoWriteAngle(i)
15             time.sleep_ms(15)
```

Reference

class myServo

Before each use of **myServo**, please make sure myservo.py has been uploaded to "/" of ESP32, and then add the statement "**from myservo import myServo**" to the top of the python file.

myServo (): The object that controls the servo, with the default pin Pin(15), default frequency 50Hz and default duty cycle 512.

myServoWriteDuty(duty): The function that writes duty cycle to control the servo.

duty: Range from 26 to 128, with 26 corresponding to the servo's 0 degree and 128 corresponding to 180 degrees.

myServoWriteAngle(pos): Function that writes angle value to control the servo.

pos: Ranging from 0-180, corresponding the 0-180 degrees of the servo.

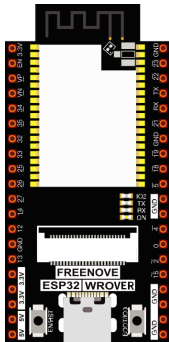
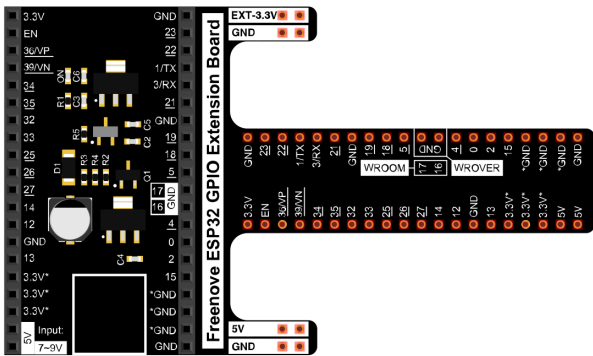
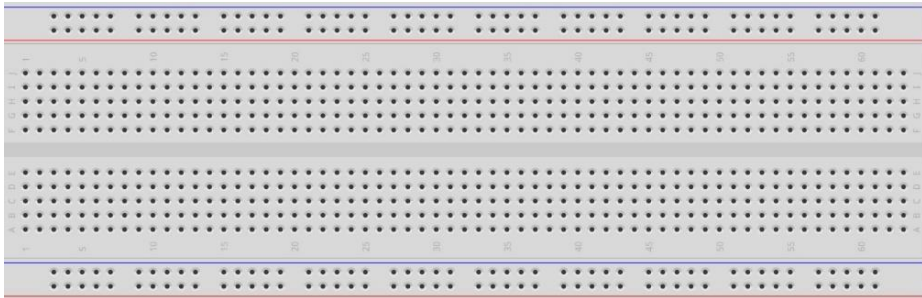
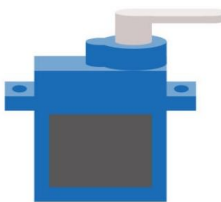
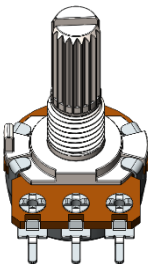
myServoWriteTime(us): Writes time to control the servo.

us: Range from 500-2500, with 500 corresponding to the servo's 0 degree and 2500 corresponding to 180 degrees.

Project 17.2 Servo Knop

Use a potentiometer to control the servo motor to rotate at any angle.

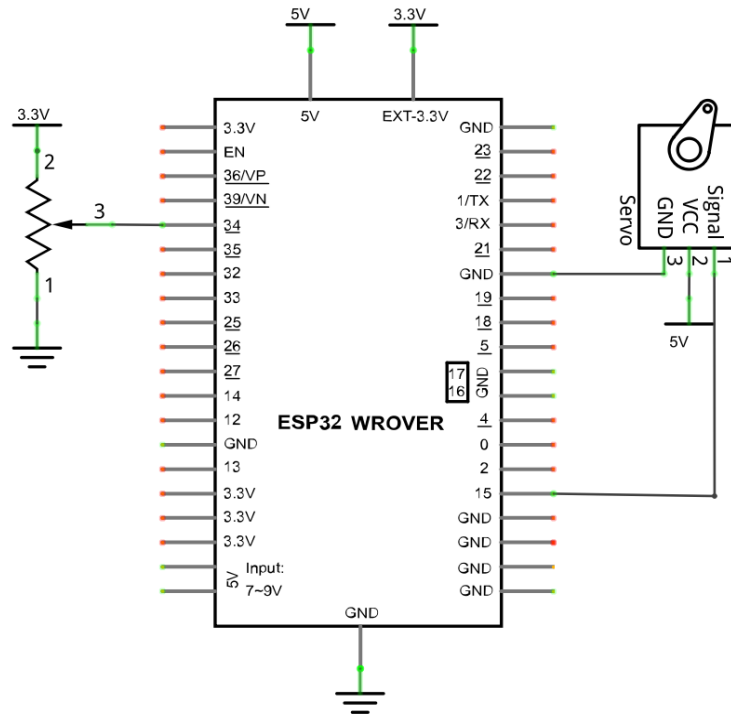
Component List

ESP32-WROVER x1 	GPIO Extension Board x1 	
Breadboard x1 		
Servo x1 	Jumper M/M x6 	Rotary potentiometer x1 

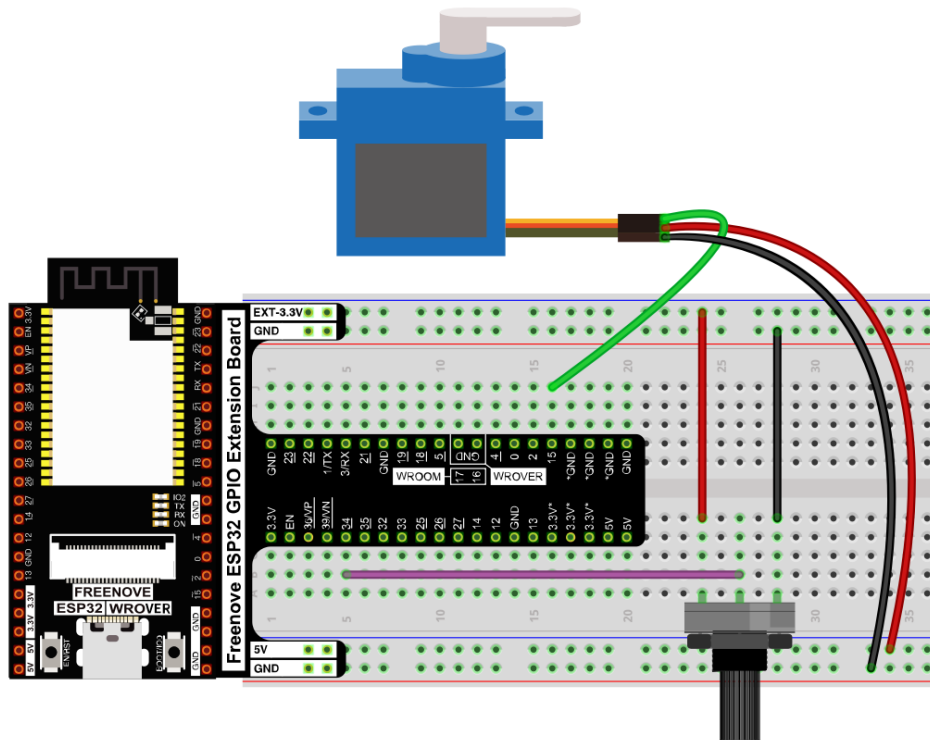
Circuit

Use caution when supplying power to the Servo, it should be 5V. Make sure you do not make any errors when connecting the Servo to the power supply.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

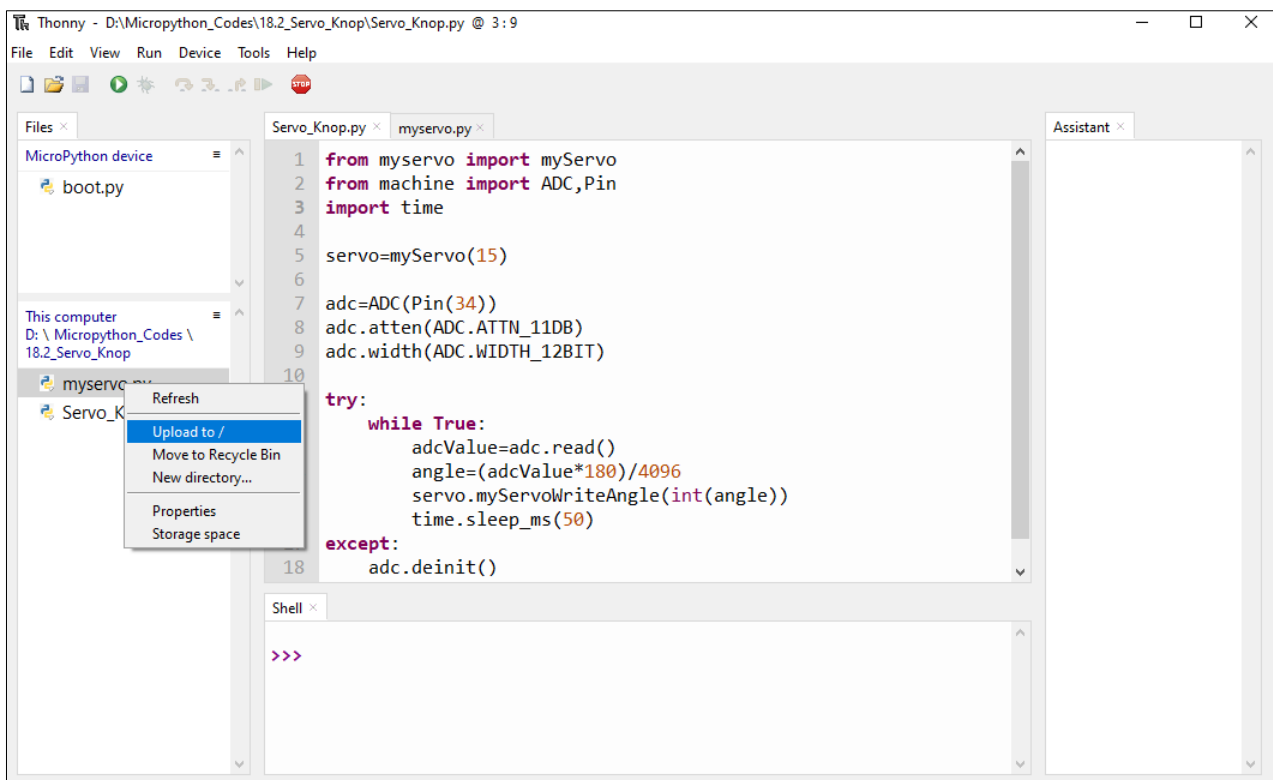


Code

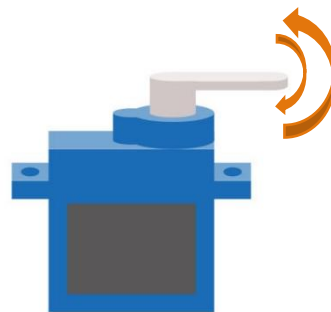
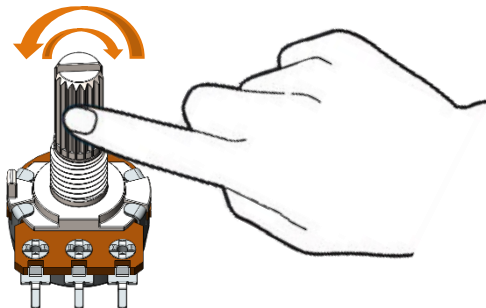
Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “17.2_Servo_Knop”. Select “myservo.py”, right click your mouse to select “Upload to /”, wait for “myservo.py” to be uploaded to ESP32-WROVER and then double click “Servo_Knop.py”.

17.2_Servo_Knop



Click “Run current script”, twist the potentiometer back and forth, and the servo motor rotates accordingly.



The following is the program code:

```
1  from myservo import myServo
2  from machine import ADC, Pin
3  import time
4
5  servo=myServo(15)
6
7  adc=ADC(Pin(34))
8  adc.atten(ADC. ATTN_11DB)
9  adc.width(ADC. WIDTH_12BIT)
10
11  try:
12      while True:
13          adcValue=adc.read()
14          angle=(adcValue*180)/4096
15          servo.myServoWriteAngle(int(angle))
16          time.sleep_ms(50)
17  except:
18      adc.deinit()
19      servo.deinit()
```

In this project, we will use Pin(34) of ESP32 to read the ADC value of the rotary potentiometer and then convert it to the angle value required by the servo and control the servo to rotate to the corresponding angle.

Chapter 18 LCD1602

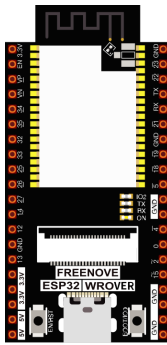
In this chapter, we will learn about the LCD1602 Display Screen

Project 18.1 LCD1602

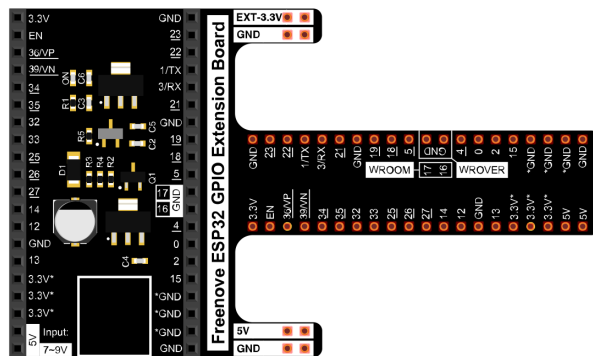
In this section we learn how to use lcd1602 to display something.

Component List

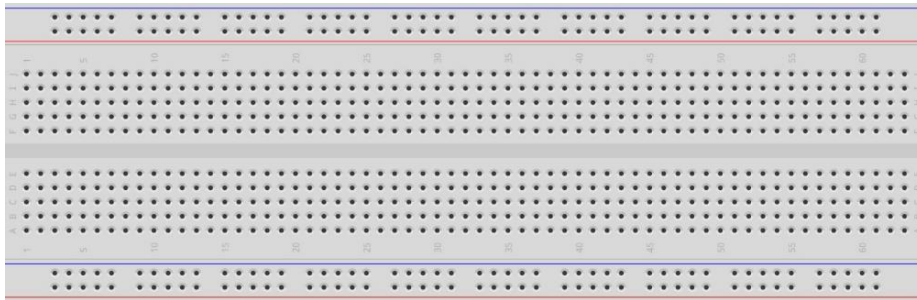
ESP32-WROVER x1



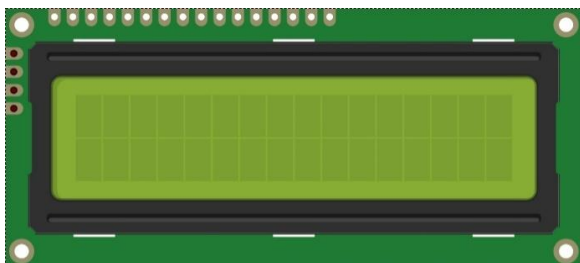
GPIO Extension Board x1



Breadboard x1



LCD1602 Module x1



Jumper F/M x4



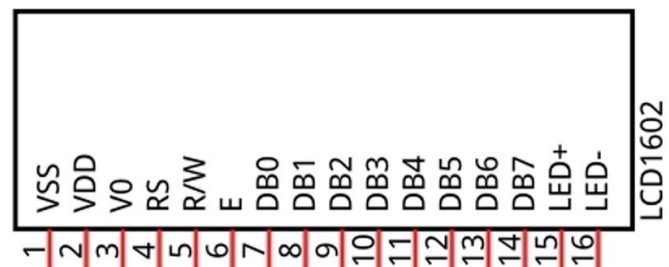
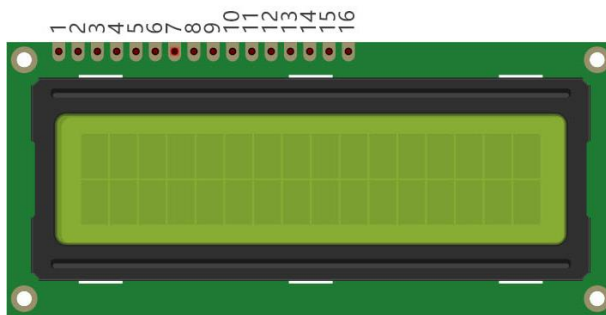
Component knowledge

I2C communication

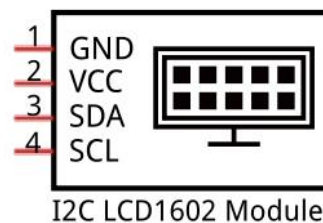
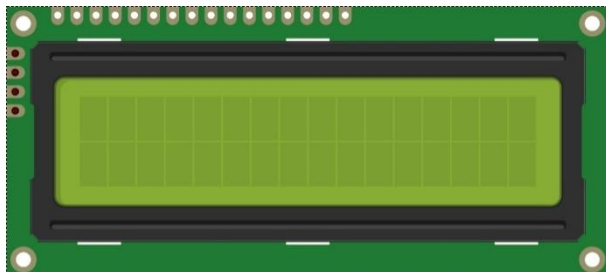
I2C (Inter-Integrated Circuit) is a two-wire serial communication mode, which can be used for the connection of micro controllers and their peripheral equipment. Devices using I2C communication must be connected to the serial data (SDA) line, and serial clock (SCL) line (called I2C bus). Each device has a unique address and can be used as a transmitter or receiver to communicate with devices connected to the bus.

LCD1602 communication'

The LCD1602 Display Screen can display 2 lines of characters in 16 columns. It is capable of displaying numbers, letters, symbols, ASCII code and so on. As shown below is a monochrome LCD1602 Display Screen along with its circuit pin diagram

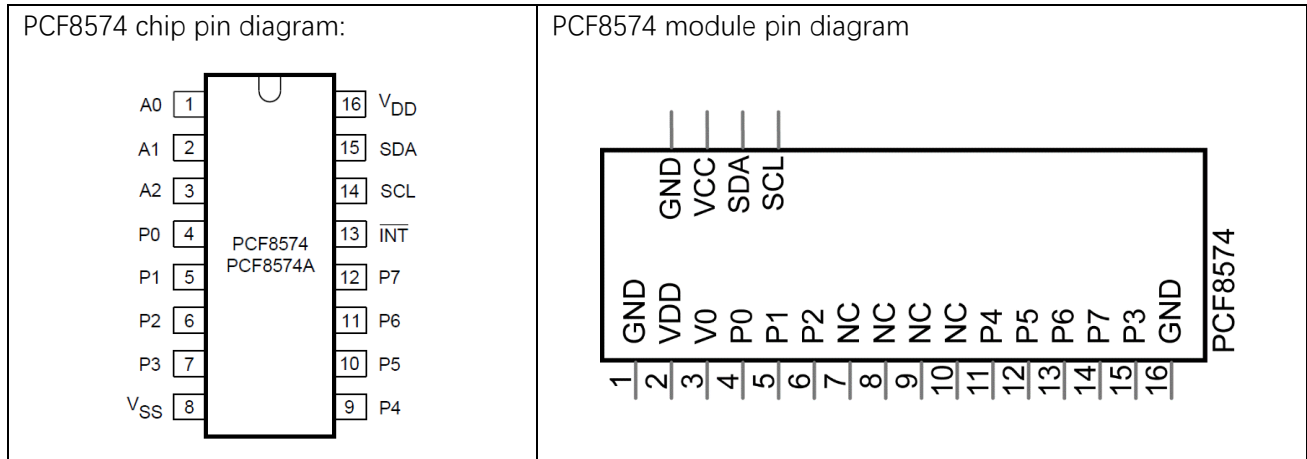


I2C LCD1602 Display Screen integrates an I2C interface, which connects the serial-input & parallel-output module to the LCD1602 Display Screen. This allows us to use only 4 lines to operate the LCD1602.

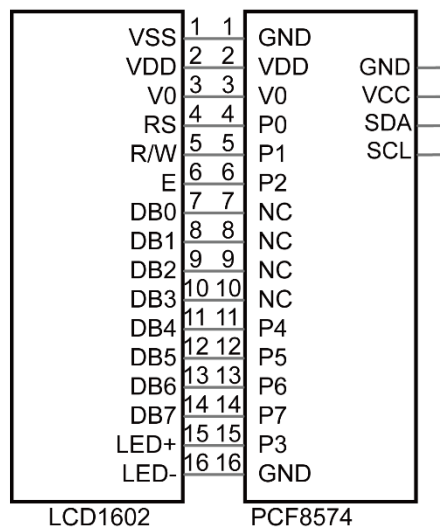


The serial-to-parallel IC chip used in this module is PCF8574T (PCF8574AT), and its default I2C address is 0x27(0x3F). You can also view the ESP32 bus on your I2C device address through command "i2cdetect -y 1".

Below is the PCF8574 pin schematic diagram and the block pin diagram:



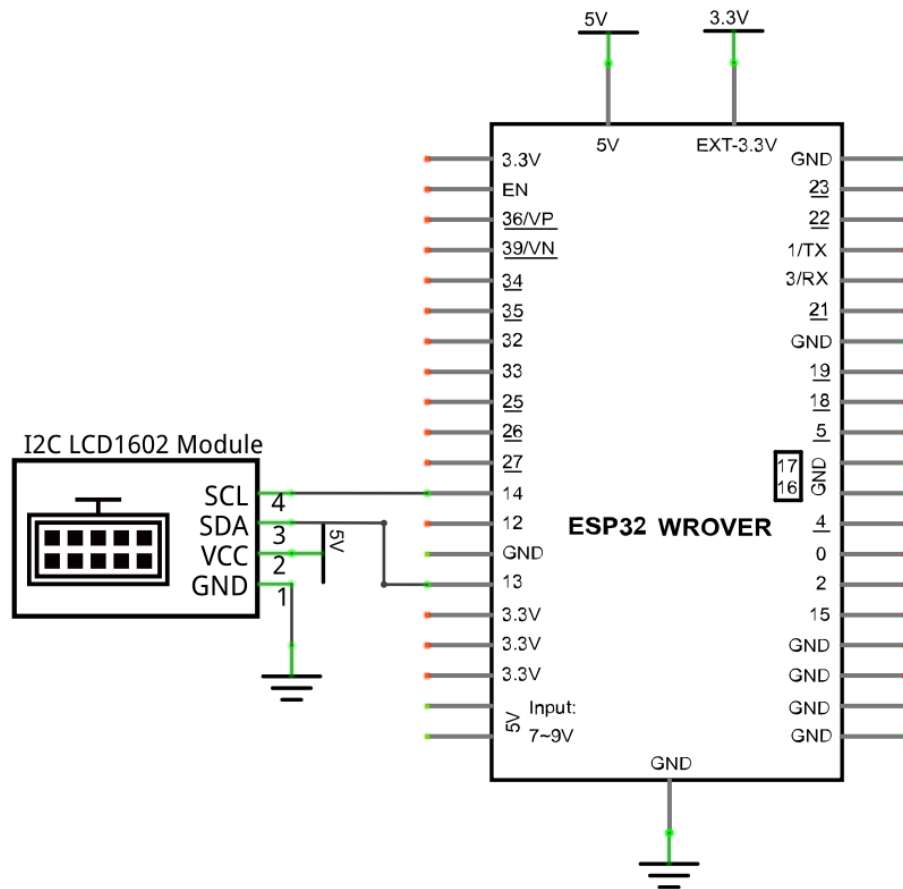
PCF8574 module pin and LCD1602 pin are corresponding to each other and connected with each other:



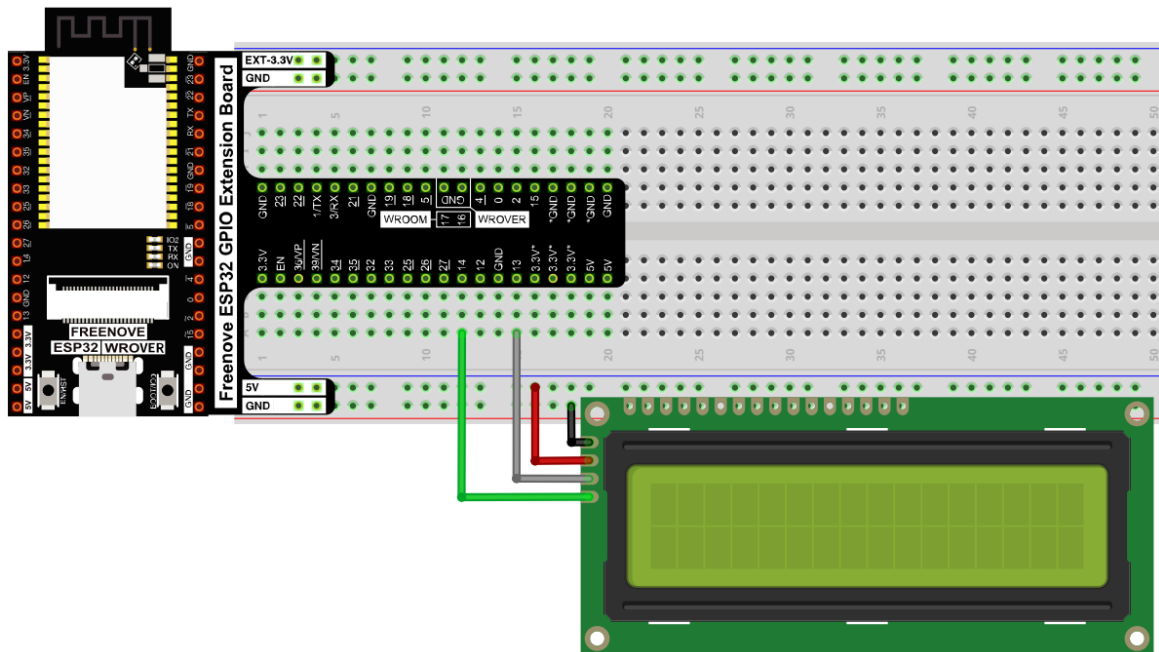
So we only need 4 pins to control the 16 pins of the LCD1602 Display Screen through the I2C interface. In this project, we will use the I2C LCD1602 to display some static characters and dynamic variables.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

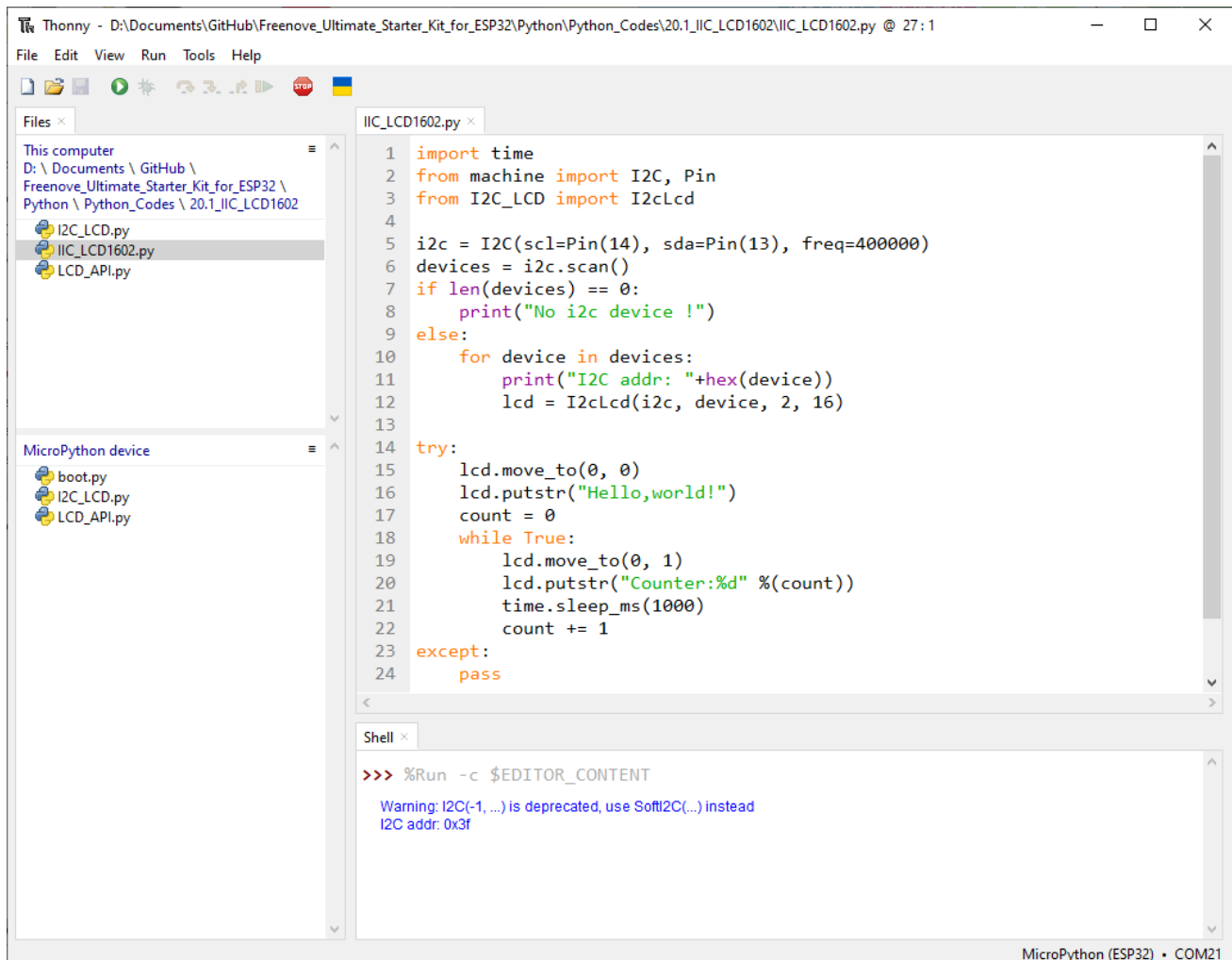


Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “18.1_I2C_LCD1602”. Select “I2C_LCD.py” and “LCD_API.py”, right click your mouse to select “Upload to /”, wait for “I2C_LCD.py” and “LCD_API.py” to be uploaded to ESP32-WROVER and then double click “I2C_LCD1602.py”.

18.1_I2C_LCD1602



The screenshot shows the Thonny IDE interface. The left sidebar displays the file explorer with the project structure: D:\Documents\GitHub\Freenove_Ultimate_Starter_Kit_for_ESP32\Python\Python_Codes\20.1_I2C_LCD1602. The main editor window shows the code for I2C_LCD1602.py. The code imports time, I2C, and Pin from machine, and I2CLcd from I2C_LCD. It initializes an I2C object with scl=Pin(14) and sda=Pin(13) at 400000 Hz. It scans for I2C devices and prints their addresses. A try block contains the logic to move the cursor to (0,0), print "Hello, world!", and enter a while loop that prints a counter and sleeps for 1000ms. An except block passes. The bottom shell window shows the command %Run -c \$EDITOR_CONTENT and the output: Warning: I2C(-1, ...) is deprecated, use SoftI2C(...) instead, followed by I2C addr: 0x3f.

```

1 import time
2 from machine import I2C, Pin
3 from I2C_LCD import I2CLcd
4
5 i2c = I2C(scl=Pin(14), sda=Pin(13), freq=400000)
6 devices = i2c.scan()
7 if len(devices) == 0:
8     print("No i2c device !")
9 else:
10     for device in devices:
11         print("I2C addr: "+hex(device))
12         lcd = I2CLcd(i2c, device, 2, 16)
13
14 try:
15     lcd.move_to(0, 0)
16     lcd.putstr("Hello,world!")
17     count = 0
18     while True:
19         lcd.move_to(0, 1)
20         lcd.putstr("Counter:%d" %(count))
21         time.sleep_ms(1000)
22         count += 1
23 except:
24     pass
  
```

Shell

```

>>> %Run -c $EDITOR_CONTENT
Warning: I2C(-1, ...) is deprecated, use SoftI2C(...) instead
I2C addr: 0x3f
  
```

Click “Run current script” and LCD1602 displays some characters.



So far, at this writing, we have two types of LCD1602 on sale. One needs to adjust the backlight, and the other does not.

The LCD1602 that does not need to adjust the backlight is shown in the figure below.



The following is the program code:

```

1  import time
2  from machine import I2C, Pin
3  from I2C_LCD import I2cLcd
4
5  i2c = I2C(scl=Pin(14), sda=Pin(13), freq=400000)
6  devices = i2c.scan()
7  if len(devices) == 0:
8      print("No i2c device !")
9  else:
10     for device in devices:
11         print("I2C addr: "+hex(device))
12         lcd = I2cLcd(i2c, device, 2, 16)
13
14  try:
15     lcd.move_to(0, 0)
16     lcd.putstr("Hello,world!")
17     count = 0
18     while True:
19         lcd.move_to(0, 1)
20         lcd.putstr("Counter:%d" %(count))
21         time.sleep_ms(1000)
22         count += 1
23  except:
24     pass

```

Import time, I2C and I2C_LCD modules.

```

1  import time
2  from machine import I2C, Pin
3  from I2C_LCD import I2cLcd

```

Initialize I2C pins and associate them with I2CLCD module, and then set the number of rows and columns for LCD1602.

Instantiate the I2C LCD1602 screen. It should be noted here that if your LCD driver chip uses PCF8574T, set the I2C address to 0x27, and if uses PCF8574AT, set the I2C address to 0x3F.

```

5  i2c = I2C(scl=Pin(14), sda=Pin(13), freq=400000)
6  devices = i2c.scan()
7  if len(devices) == 0:
8      print("No i2c device !")
9  else:
10     for device in devices:
11         print("I2C addr: "+hex(device))
12         lcd = I2cLcd(i2c, device, 2, 16)

```

Move the cursor of LCD1602 to the first row, first column, and print out "Hello, world!"

```

15     lcd.move_to(0, 0)
16     lcd.putstr("Hello,world!")

```

The second line of LCD1602 continuously prints the number of seconds after the ESP32 program runs.

```

18     while True:
19         lcd.move_to(0, 1)
20         lcd.putstr("Counter:%d" %(count))
21         time.sleep_ms(1000)
22         count += 1

```

Reference

Class I2cLcd

Before each use of the object **I2cLcd**, please make sure that **I2C_LCD.py** and **LCD_API.py** have been uploaded to "/" of ESP32, and then add the statement **"from I2C_LCD import I2cLcd"** to the top of the python file.

clear(): Clear the LCD1602 screen display.

show_cursor(): Show the cursor of LCD1602.

hide_cursor(): Hide the cursor of LCD1602.

blink_cursor_on(): Turn on cursor blinking.

blink_cursor_off(): Turn off cursor blinking.

display_on(): Turn on the display function of LCD1602.

display_off(): Turn on the display function of LCD1602.

backlight_on(): Turn on the backlight of LCD1602.

backlight_off(): Turn on the backlight of LCD1602.

move_to(cursor_x, cursor_y): Move the cursor to a specified position.

cursor_x: Column cursor_x

cursor_y: Row cursor_y

putchar(char): Print the character in the bracket on LCD1602

putstr(string): Print the string in the bracket on LCD1602.

Chapter 19 Ultrasonic Ranging

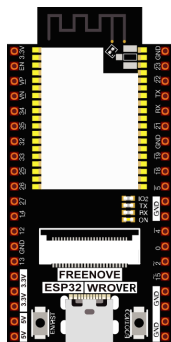
In this chapter, we learn a module which use ultrasonic to measure distance, HC SR04.

Project 19.1 Ultrasonic Ranging

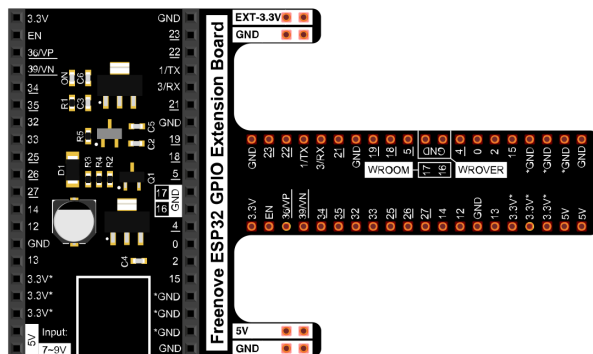
In this project, we use ultrasonic ranging module to measure distance, and print out the data in the terminal.

Component List

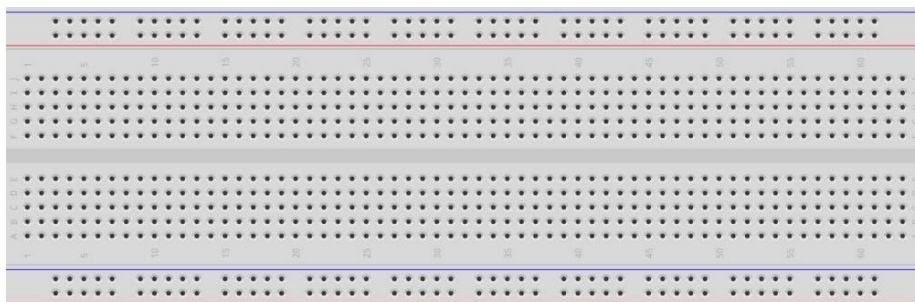
ESP32-WROVER x1



GPIO Extension Board x1



Breadboard x1



Jumper F/M x4

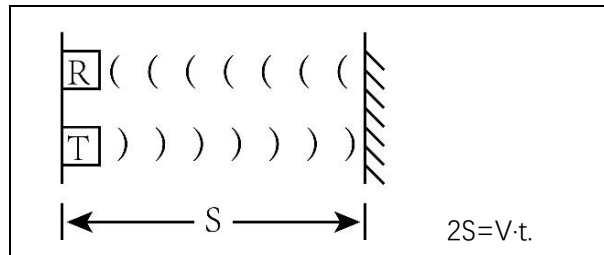


HC SR04 x1

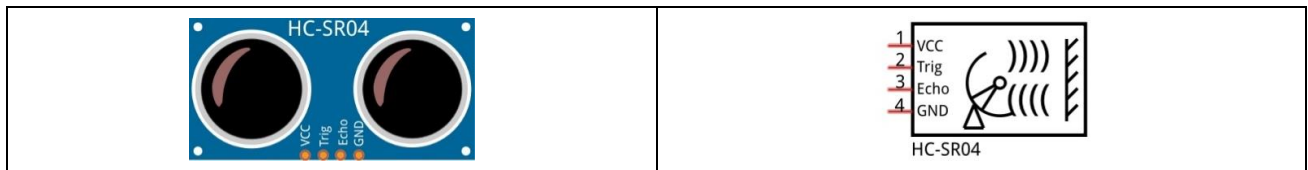


Component Knowledge

The Ultrasonic Ranging Module uses the principle that ultrasonic waves will reflect when they encounter any obstacles. This is possible by counting the time interval between when the ultrasonic wave is transmitted to when the ultrasonic wave reflects back after encountering an obstacle. Time interval counting will end after an ultrasonic wave is received, and the time difference (Δt) is the total time of the ultrasonic wave's journey from being transmitted to being received. Because the speed of sound in air is a constant, and is about $v=340\text{m/s}$, we can calculate the distance between the Ultrasonic Ranging Module and the obstacle: $s=vt/2$.



The HC-SR04 Ultrasonic Ranging Module integrates both an ultrasonic transmitter and a receiver. The transmitter is used to convert electrical signals (electrical energy) into high frequency (beyond human hearing) sound waves (mechanical energy) and the function of the receiver is opposite of this. The picture and the diagram of the HC SR04 Ultrasonic Ranging Module are shown below:



Pin description:

Pin	Description
VCC	power supply pin
Trig	trigger pin
Echo	Echo pin
GND	GND

Technical specs:

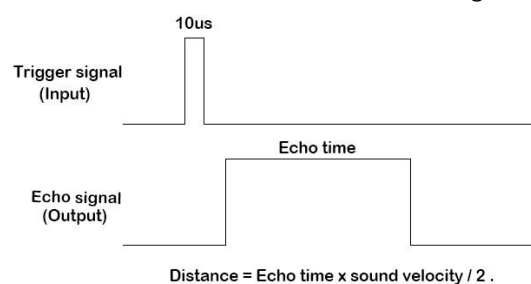
Working voltage: 5V

Working current: 12mA

Minimum measured distance: 2cm

Maximum measured distance: 200cm

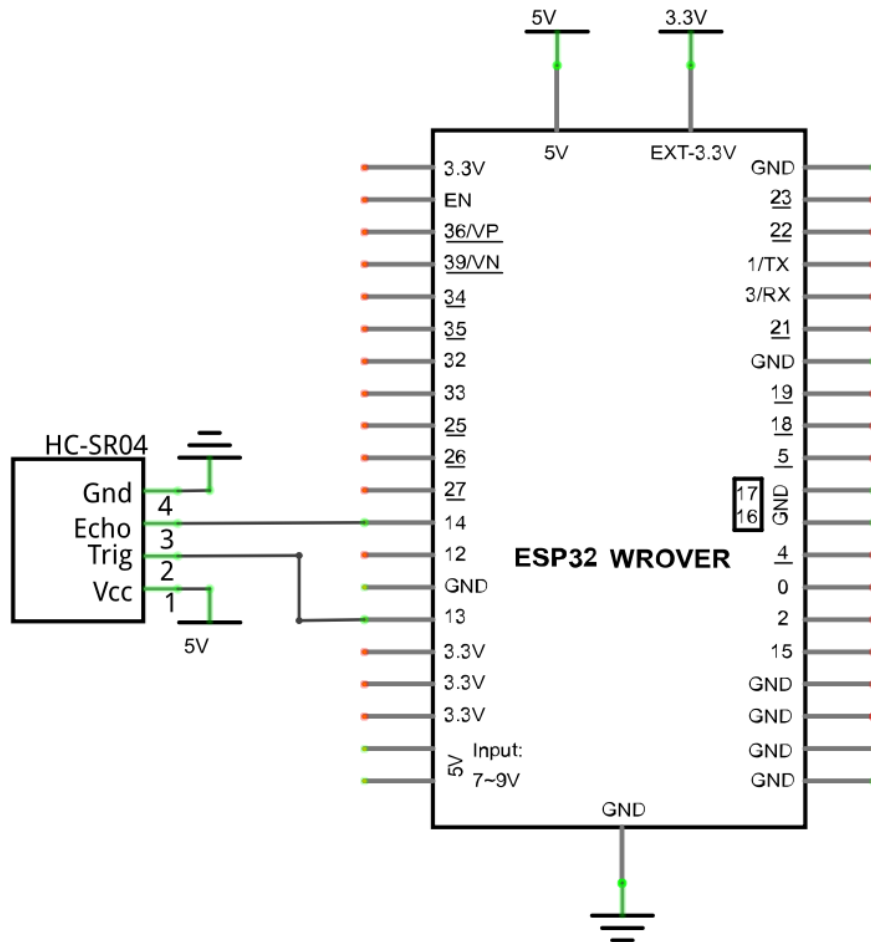
Instructions for Use: output a high-level pulse in Trig pin lasting for least 10 μs , the module begins to transmit ultrasonic waves. At the same time, the Echo pin is pulled up. When the module receives the returned ultrasonic waves from encountering an obstacle, the Echo pin will be pulled down. The duration of high level in the Echo pin is the total time of the ultrasonic wave from transmitting to receiving, $s=vt/2$.



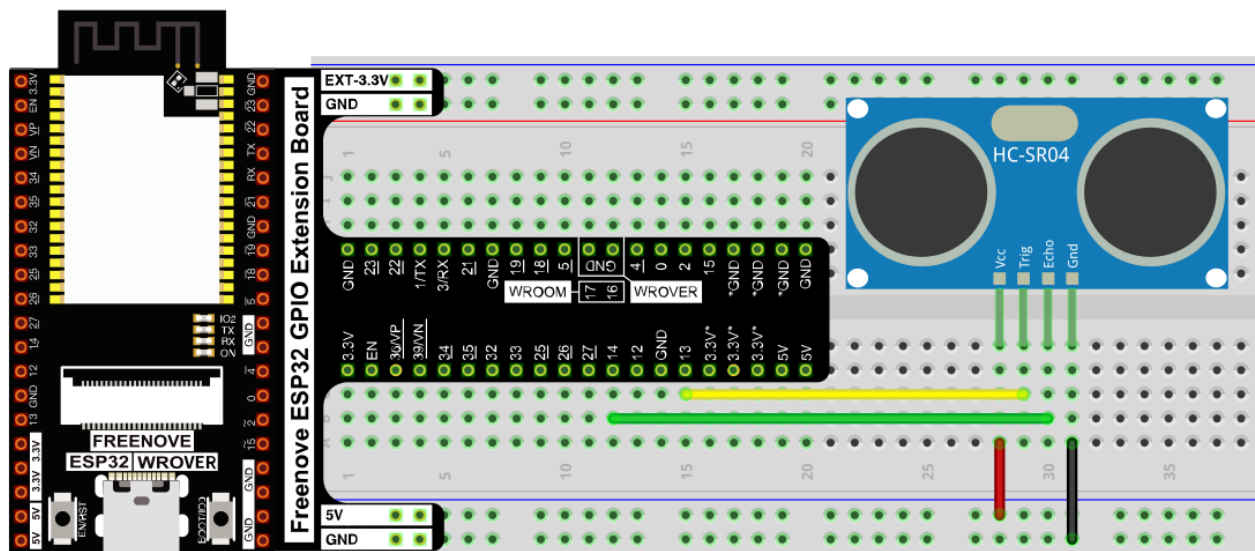
Circuit

Note that the voltage of ultrasonic module is 5V in the circuit.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “19.1_Ultrasonic_Ranging” and double click “Ultrasonic_Ranging.py”.

19.1_Ultrasonic_Ranging

The screenshot shows the Thonny IDE interface. The main editor displays the code for `Ultrasonic_Ranging.py`. The code defines a `getSonar()` function that uses the `machine` module to control pins and the `time` module for delays. It calculates the distance by measuring the time it takes for an ultrasonic wave to travel to an object and back.

```

1  from machine import Pin
2  import time
3
4  trigPin=Pin(13,Pin.OUT,0)
5  echoPin=Pin(14,Pin.IN,0)
6
7  soundVelocity=340
8  distance=0
9
10 def getSonar():
11     trigPin.value(1)
12     time.sleep_us(10)
13     trigPin.value(0)
14     while not echoPin.value():
15         pass
16     pingStart=time.ticks_us()
17     while echoPin.value():
18         pass
19     pingStop=time.ticks_us()
20     pingTime=time.ticks_diff(pingStop,pingStart)
21     distance=pingTime*soundVelocity//2//10000
22     return int(distance)

```

The Shell window at the bottom shows the output of the script: `X,Y,Z: 4095 , 368 , 1`. Below the shell output, the MicroPython version and hardware information are displayed: `MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32`.

Click “Run current script”, you can use it to measure the distance between the ultrasonic module and the object. As shown in the following figure:

The screenshot shows the Shell window output. It displays the MicroPython version and hardware information: `MicroPython v1.12 on 2019-12-20; ESP32 module (spiram) with ESP32`. Below this, the command `>>> %Run -c $EDITOR_CONTENT` is executed, resulting in a series of distance measurements in centimeters:

```

Distance: 14 cm
Distance: 15 cm
Distance: 18 cm
Distance: 21 cm
Distance: 22 cm
Distance: 14 cm
Distance: 9 cm
Distance: 7 cm
Distance: 7 cm
Distance: 12 cm

```

The following is the program code:

```

1  from machine import Pin
2  import time
3
4  trigPin=Pin(13,Pin.OUT,0)
5  echoPin=Pin(14,Pin.IN,0)
6
7  soundVelocity=340
8  distance=0
9
10 def getSonar():
11     trigPin.value(1)
12     time.sleep_us(10)
13     trigPin.value(0)
14     while not echoPin.value():
15         pass
16     pingStart=time.ticks_us()
17     while echoPin.value():
18         pass
19     pingStop=time.ticks_us()
20     pingTime=time.ticks_diff(pingStop,pingStart)
21     distance=pingTime*soundVelocity//2//10000
22     return int(distance)
23
24 time.sleep_ms(2000)
25 while True:
26     time.sleep_ms(500)
27     print('Distance: ',getSonar(),'cm')

```

Define the control pins of the ultrasonic ranging module.

```

4  trigPin=Pin(13,Pin.OUT,0)
5  echoPin=Pin(14,Pin.IN,0)

```

Set the speed of sound.

```

7  soundVelocity=340
8  distance=0

```

Subfunction getSonar() is used to start the Ultrasonic Module to begin measurements, and return the measured distance in centimeters. In this function, first let trigPin send 10us high level to start the Ultrasonic Module. Then use pulseIn() to read the Ultrasonic Module and return the duration time of high level. Finally, the measured distance according to the time is calculated.

```

10 def getSonar():
11     trigPin.value(1)
12     time.sleep_us(10)
13     trigPin.value(0)
14     while not echoPin.value():
15         pass

```



```
16 pingStart=time.ticks_us()  
17 while echoPin.value():  
18     pass  
19 pingStop=time.ticks_us()  
20 pingTime=time.ticks_diff(pingStop,pingStart)  
21 distance=pingTime*soundVelocity//2//10000  
22 return int(distance)
```

Delay for 2 seconds and wait for the ultrasonic module to stabilize. Print data obtained from ultrasonic module every 500 milliseconds

```
24 time.sleep_ms(2000)  
25 while True:  
26     time.sleep_ms(500)  
27     print('Distance: ',getSonar(),'cm')
```

Project 19.2 Ultrasonic Ranging

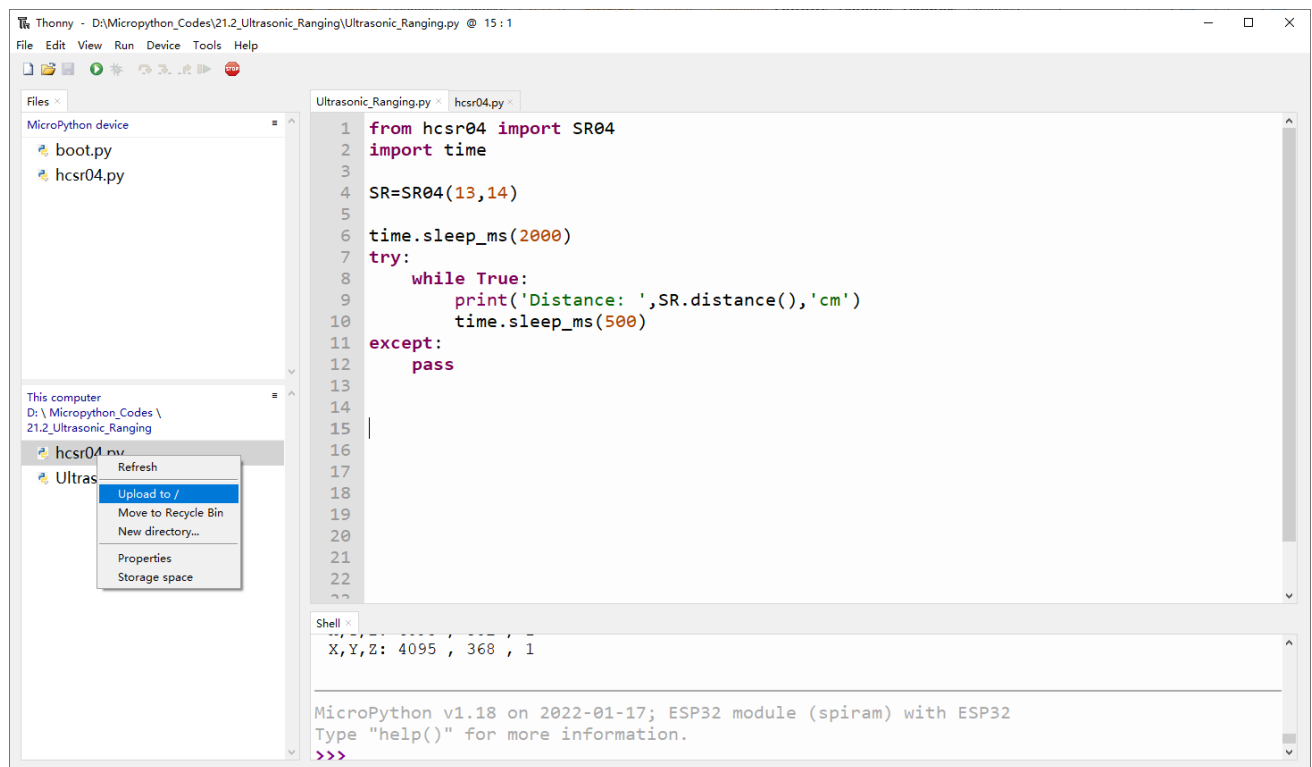
Component List and Circuit

Component List and Circuit are the same as the previous section.

Code

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “19.2_Ultrasonic_Ranging”. Select “hcsr04.py”, right click your mouse to select “Upload to /”, wait for “hcsr04.py” to be uploaded to ESP32-WROVER and then double click “Ultrasonic_Ranging.py”.

19.2_Ultrasonic_Ranging



Click “Run current script”. Use the ultrasonic module to measure distance. As shown in the following figure:

```
Shell ×
Distance: 6.64/ cm
Distance: 5.151 cm
Distance: 5.151 cm
Distance: 6.052 cm
Distance: 7.225 cm
Distance: 7.531 cm
Distance: 8.721 cm
Distance: 12.121 cm
Distance: 11.798 cm
Distance: 13.6 cm
Distance: 14.467 cm
Distance: 16.116 cm
Distance: 17.442 cm
Distance: 19.652 cm
Distance: 22.015 cm
```

The following is the program code:

```
1 from hcsr04 import SR04
2 import time
3
4 SR=SR04(13,14)
5
6 time.sleep_ms(2000)
7 try:
8     while True:
9         print('Distance: ',SR.distance(), 'cm')
10        time.sleep_ms(500)
11 except:
12     pass
```

Import hcsr04 module.

```
1 from hcsr04 import SR04
```

Define an ultrasonic object and associate with the pins.

```
3 SR=SR04(13,14)
```

Obtain the distance data returned from the ultrasonic ranging module.

```
9 SR.distance()
```

Obtain the ultrasonic data every 500 milliseconds and print them out in “Shell”.

```
8 while True:
9     print('Distance: ',SR.distance(), 'cm')
10    time.sleep_ms(500)
```

Reference

Class hcsr04

Before each use of object **SR04**, please add the statement “**from hcsr04 import SR04**” to the top of python file.

SR04(): Object of ultrasonic module. By default, trig pin is Pin(13) and echo pin is Pin(14).

distanceCM(): Obtain the distance from the ultrasonic to the measured object with the data type being int type, and the unit being cm.

distanceMM(): Obtain the distance from the ultrasonic to the measured object with the data type being int type, and the unit being mm.

distance(): Obtain the distance from the ultrasonic to the measured object with the data type being float type, and the unit being cm.

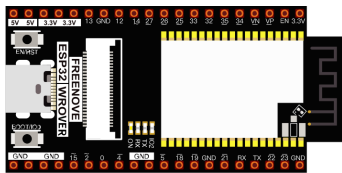
Chapter 20 Bluetooth

This chapter mainly introduces how to make simple data transmission through Bluetooth of ESP32-WROVER and mobile phones.

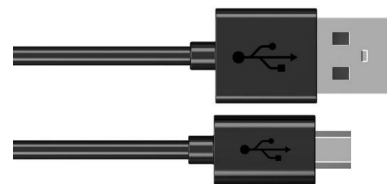
Project 20.1 Bluetooth Low Energy Data Passthrough

Component List

ESP32-WROVER x1



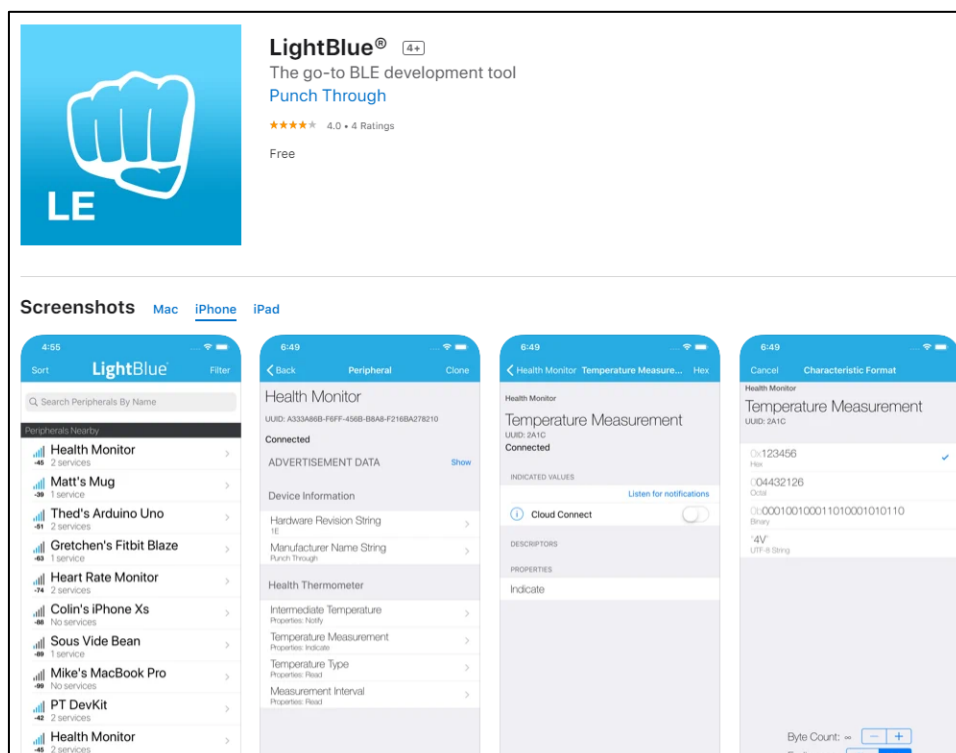
Micro USB Wire x1



Lightblue

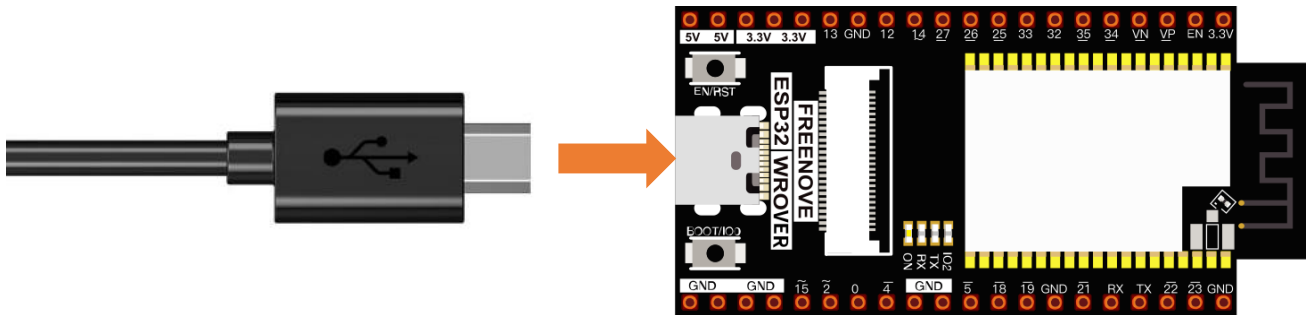
If you can't install Serial Bluetooth on your phone, try LightBlue. If you do not have this software installed on your phone, you can refer to this link:

<https://apps.apple.com/us/app/lightblue/id557428110?platform=iphone>



Circuit

Connect Freenove ESP32 to the computer using the USB cable.

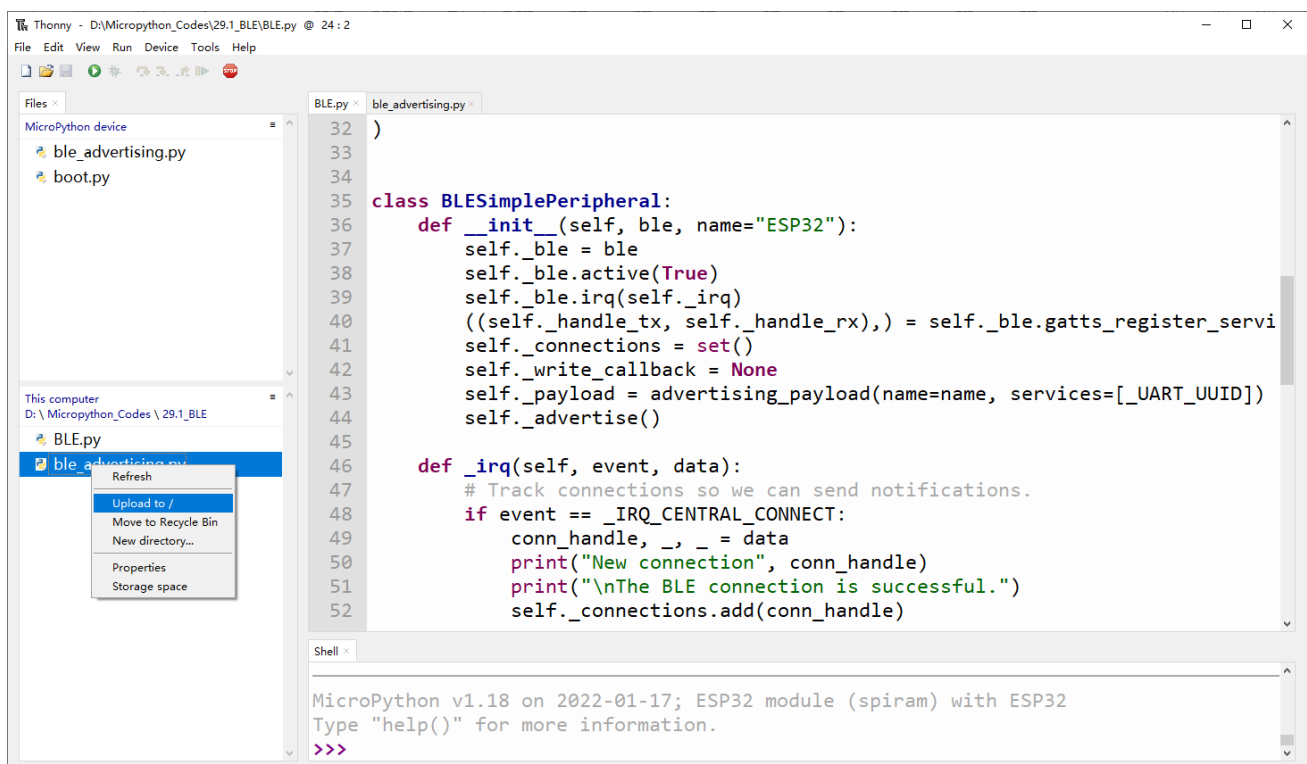


Code

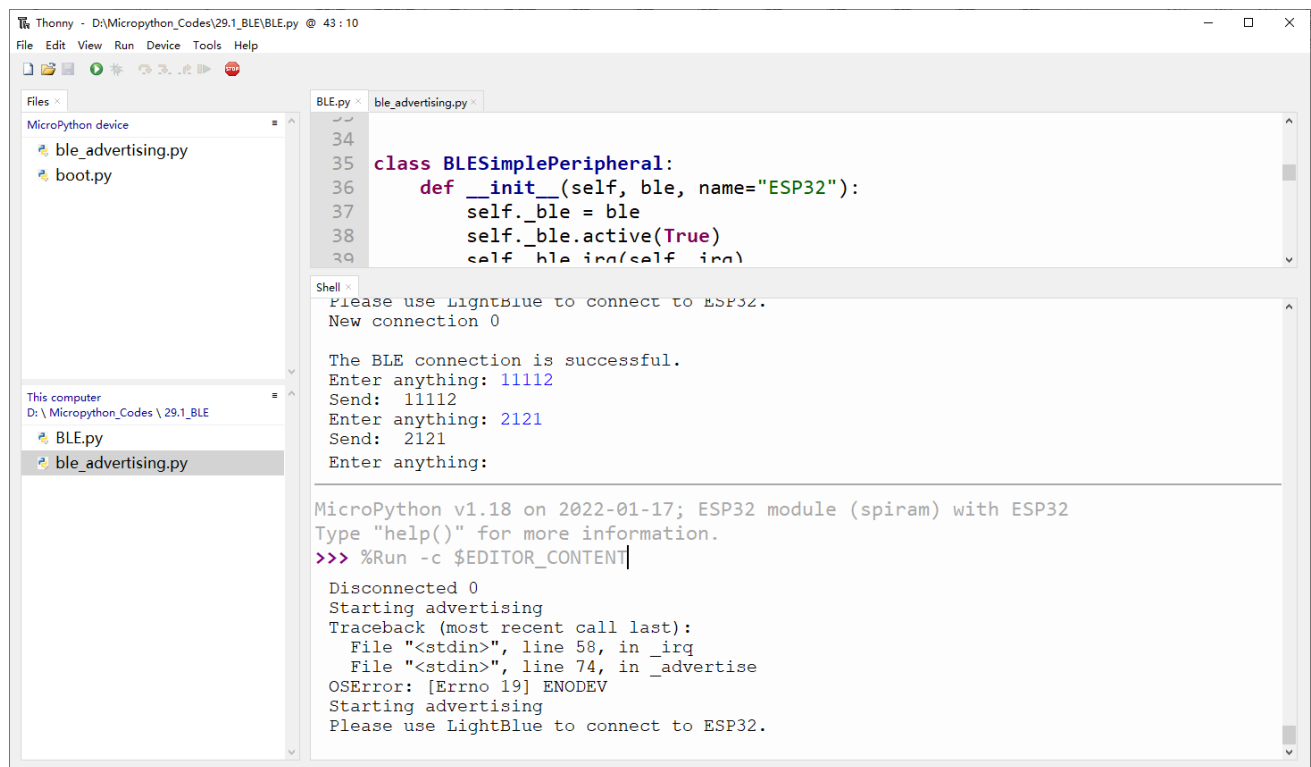
Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “20.1_BLE”. Select “ble_advertising.py”, right click your mouse to select “Upload to /”, wait for “ble_advertising.py” to be uploaded to ESP32-WROVER and then double click “BLE.py”.

20.1_BLE



Click run for BLE.py.



```
Thonny - D:\MicroPython_Codes\29.1_BLE\BLE.py @ 43 : 10
File Edit View Run Device Tools Help

Files
MicroPython device
  ble_advertising.py
  boot.py

This computer
D:\MicroPython_Codes\29.1_BLE
  BLE.py
  ble_advertising.py

BLE.py
34
35 class BLESimplePeripheral:
36     def __init__(self, ble, name="ESP32"):
37         self._ble = ble
38         self._ble.active(True)
39         self._ble.irq(self._irq)

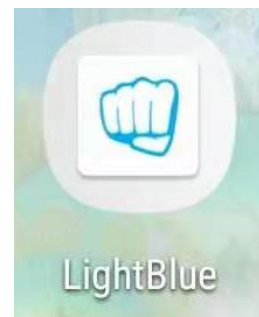
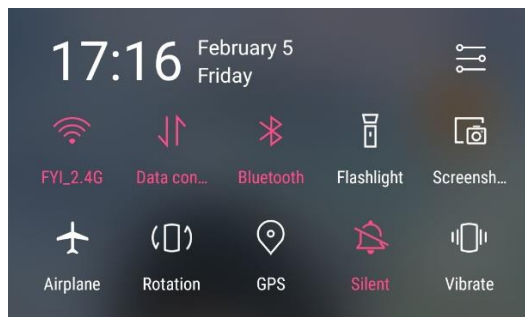
Shell
Please use LightBlue to connect to ESP32.
New connection 0

The BLE connection is successful.
Enter anything: 11112
Send: 11112
Enter anything: 2121
Send: 2121
Enter anything:

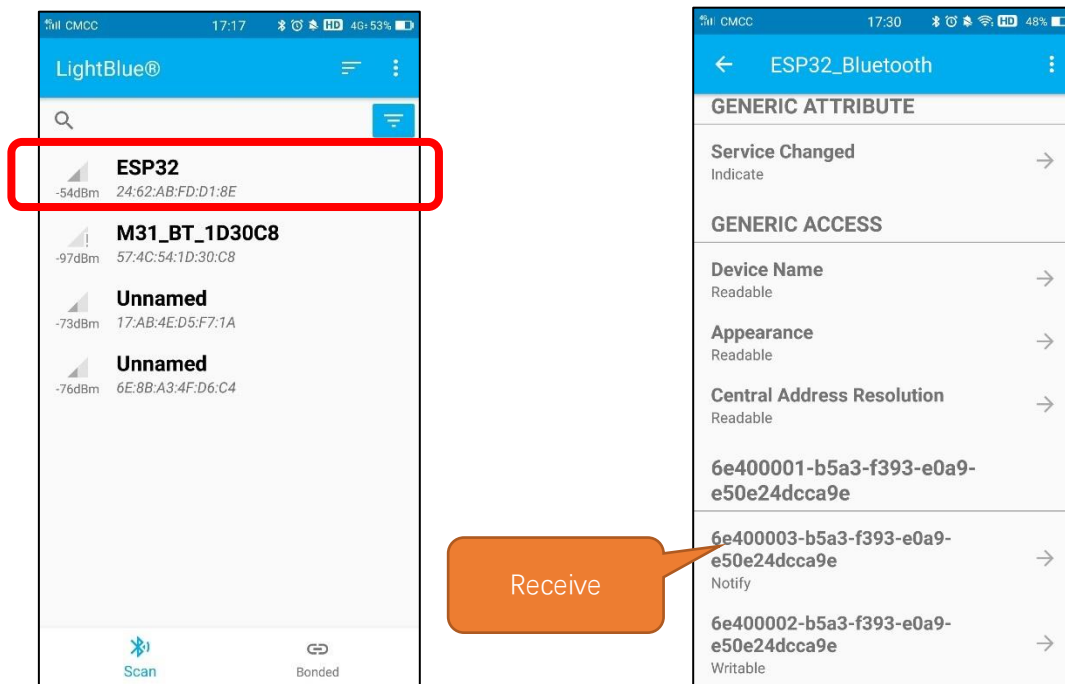
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

Disconnected 0
Starting advertising
Traceback (most recent call last):
  File "<stdin>", line 58, in _irq
  File "<stdin>", line 74, in _advertise
OSError: [Errno 19] ENODEV
Starting advertising
Please use LightBlue to connect to ESP32.
```

Turn ON Bluetooth on your phone, and open the Lightblue APP.



In the Scan page, swipe down to refresh the name of Bluetooth that the phone searches for. Click ESP32.



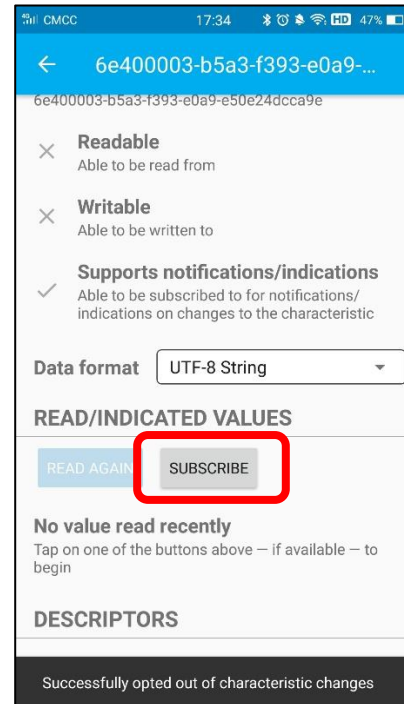
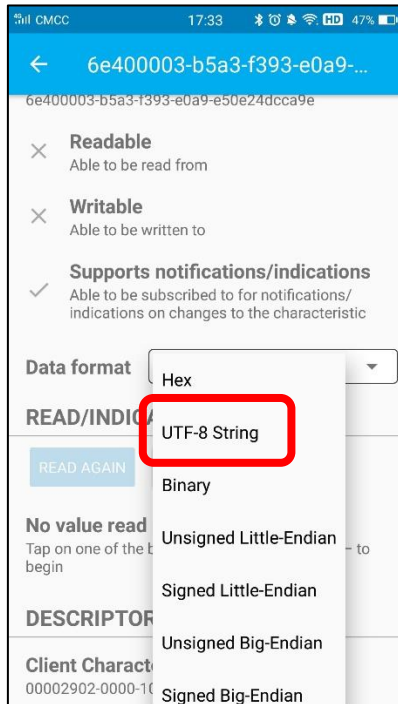
After Bluetooth is connect successfully, Shell will printer the information.

```
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

Disconnected 0
Starting advertising
Traceback (most recent call last):
  File "<stdin>", line 58, in _irq
  File "<stdin>", line 74, in _advertise
OSError: [Errno 19] ENODEV
Starting advertising
Please use LightBlue to connect to ESP32.
New connection 0

The BLE connection is successful.
Enter anything:
```

Click "Receive". Select the appropriate Data format in the box to the right of Data Format. For example, HEX for hexadecimal, utf-string for character, Binary for Binary, etc. Then click SUBSCRIBE.



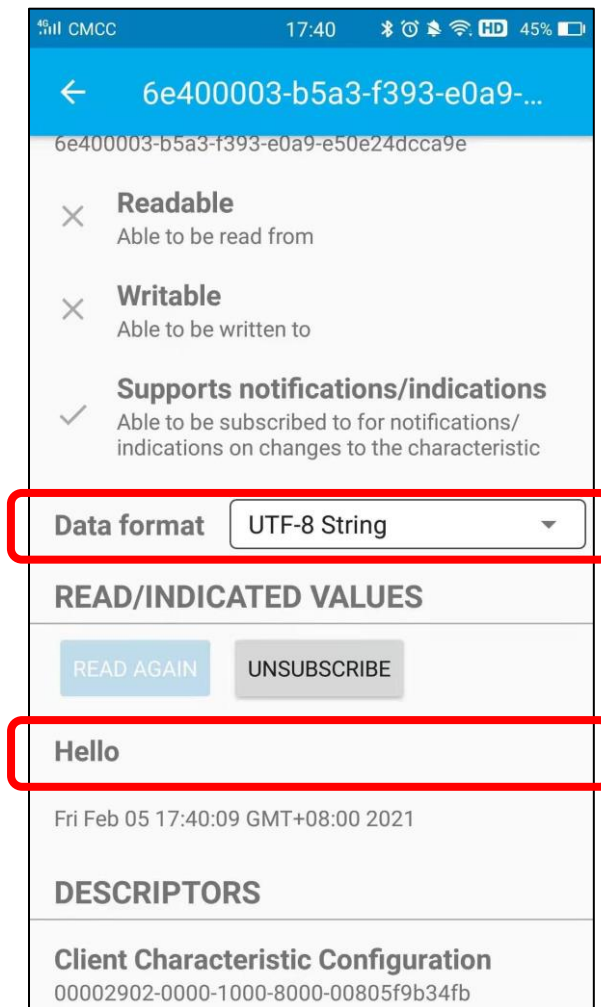
You can type "Hello" in Shell and press "Enter" to send.

```
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

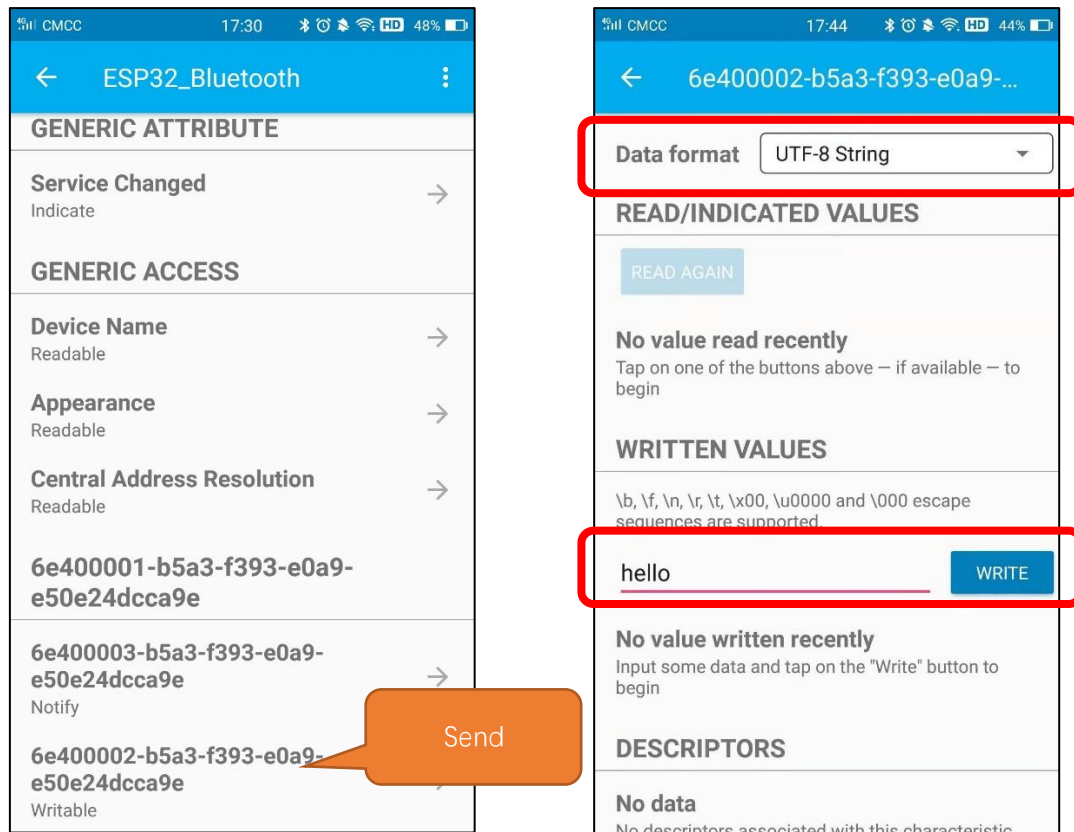
Disconnected 0
Starting advertising
Traceback (most recent call last):
  File "<stdin>", line 58, in _irq
  File "<stdin>", line 74, in _advertise
OSError: [Errno 19] ENODEV
Starting advertising
Please use LightBlue to connect to ESP32.
New connection 0

The BLE connection is successful.
Enter anything: Hello
Send: Hello
Enter anything:
```

And then you can see the mobile Bluetooth has received the message.



Similarly, you can select “Send” on your phone. Set Data format, and then enter anything in the sending box and click Write to send.



You can check the message from Bluetooth in “Shell”.

```
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

Disconnected 0
Starting advertising
Traceback (most recent call last):
  File "<stdin>", line 58, in _irq
  File "<stdin>", line 74, in _advertise
OSError: [Errno 19] ENODEV
Starting advertising
Please use LightBlue to connect to ESP32.
New connection 0

The BLE connection is successful.
Enter anything: Hello
Send: Hello
Enter anything: RX b'hello'
```

And now data can be transferred between your mobile phone and computer via ESP32-WROVER.

The following is the program code:

```

1  import bluetooth
2  import random
3  import struct
4  import time
5  from ble_advertising import advertising_payload
6  from micropython import const
7
8  _IRQ_CENTRAL_CONNECT = const(1)
9  _IRQ_CENTRAL_DISCONNECT = const(2)
10 _IRQ_GATTS_WRITE = const(3)
11 _FLAG_READ = const(0x0002)
12 _FLAG_WRITE_NO_RESPONSE = const(0x0004)
13 _FLAG_WRITE = const(0x0008)
14 _FLAG_NOTIFY = const(0x0010)
15
16 _UART_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
17 _UART_TX = (
18     bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"),
19     _FLAG_READ | _FLAG_NOTIFY,
20 )
21 _UART_RX = (
22     bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"),
23     _FLAG_WRITE | _FLAG_WRITE_NO_RESPONSE,
24 )
25 _UART_SERVICE = (
26     _UART_UUID,
27     (_UART_TX, _UART_RX),
28 )
29 class BLESimplePeripheral:
30     def __init__(self, ble, name="ESP32"):
31         self._ble = ble
32         self._ble.active(True)
33         self._ble.irq(self._irq)
34         ((self._handle_tx, self._handle_rx),) =
35 self._ble.gatts_register_services((_UART_SERVICE,))
36         self._connections = set()
37         self._write_callback = None
38         self._payload = advertising_payload(name=name, services=[_UART_UUID])
39         self._advertise()
40     def _irq(self, event, data):
41         # Track connections so we can send notifications.
42         if event == _IRQ_CENTRAL_CONNECT:

```

```

43         conn_handle, _, _ = data
44         print("New connection", conn_handle)
45         print("\nThe BLE connection is successful.")
46         self._connections.add(conn_handle)
47     elif event == _IRQ_CENTRAL_DISCONNECT:
48         conn_handle, _, _ = data
49         print("Disconnected", conn_handle)
50         self._connections.remove(conn_handle)
51         # Start advertising again to allow a new connection.
52         self._advertise()
53     elif event == _IRQ_GATTS_WRITE:
54         conn_handle, value_handle = data
55         value = self._ble.gatts_read(value_handle)
56         if value_handle == self._handle_rx and self._write_callback:
57             self._write_callback(value)
58     def send(self, data):
59         for conn_handle in self._connections:
60             self._ble.gatts_notify(conn_handle, self._handle_tx, data)
61     def is_connected(self):
62         return len(self._connections) > 0
63     def _advertise(self, interval_us=500000):
64         print("Starting advertising")
65         self._ble.gap_advertise(interval_us, adv_data=self._payload)
66     def on_write(self, callback):
67         self._write_callback = callback
68     def demo():
69         ble = bluetooth.BLE()
70         p = BLESimplePeripheral(ble)
71         def on_rx(rx_data):
72             print("RX", rx_data)
73         p.on_write(on_rx)
74         print("Please use LightBlue to connect to ESP32.")
75         while True:
76             if p.is_connected():
77                 # Short burst of queued notifications.
78                 tx_data = input("Enter anything: ")
79                 print("Send: ", tx_data)
80                 p.send(tx_data)
81     if __name__ == "__main__":
82         demo()

```

Define the specified UUID number for BLE vendor.

```

18 _UART_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
19 _UART_TX = (
20     bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"),

```

```

21     _FLAG_READ | _FLAG_NOTIFY,
22 )
23 _UART_RX = (
24     bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"),
25     _FLAG_WRITE | _FLAG_WRITE_NO_RESPONSE,
26 )

```

Write an `_irq` function to manage BLE interrupt events.

```

42     def _irq(self, event, data):
43         # Track connections so we can send notifications.
44         if event == _IRQ_CENTRAL_CONNECT:
45             conn_handle, _, _ = data
46             print("New connection", conn_handle)
47             print("\nThe BLE connection is successful.")
48             self._connections.add(conn_handle)
49         elif event == _IRQ_CENTRAL_DISCONNECT:
50             conn_handle, _, _ = data
51             print("Disconnected", conn_handle)
52             self._connections.remove(conn_handle)
53             # Start advertising again to allow a new connection.
54             self._advertise()
55         elif event == _IRQ_GATTS_WRITE:
56             conn_handle, value_handle = data
57             value = self._ble.gatts_read(value_handle)
58             if value_handle == self._handle_rx and self._write_callback:
59                 self._write_callback(value)

```

Initialize the BLE function and name it.

```

33     def __init__(self, ble, name="ESP32"):

```

When the mobile phone send data to ESP32 via BLE Bluetooth, it will print them out with serial port; When the serial port of ESP32 receive data, it will send them to mobile via BLE Bluetooth.

```

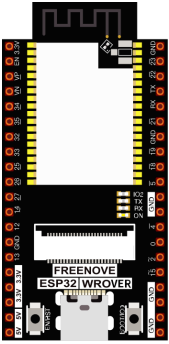
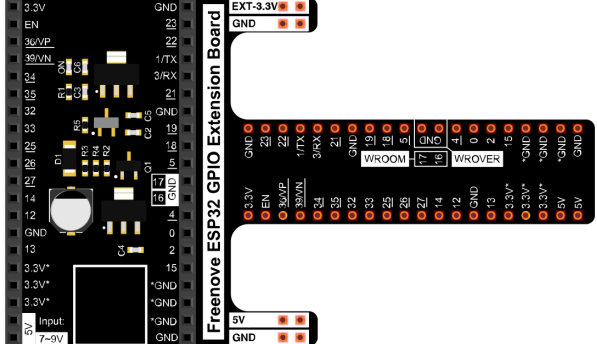
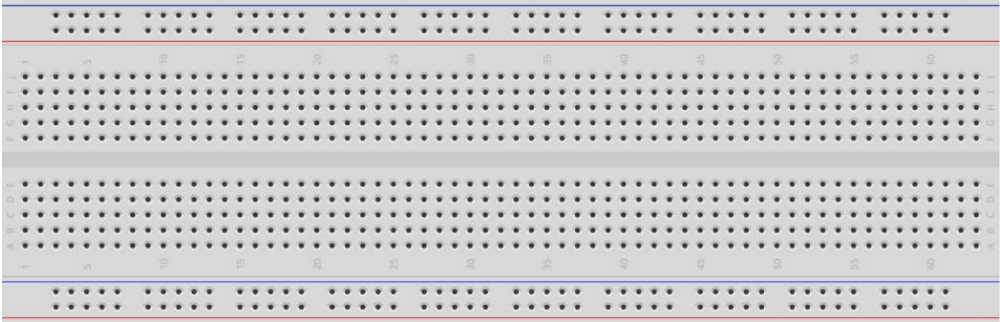
70     def demo():
71         ble = bluetooth.BLE()
72         p = BLESimplePeripheral(ble)
73         def on_rx(rx_data):
74             print("RX", rx_data)
75         p.on_write(on_rx)
76         print("Please use LightBlue to connect to ESP32.")
77         while True:
78             if p.is_connected():
79                 # Short burst of queued notifications.
80                 tx_data = input("Enter anything: ")
81                 print("Send: ", tx_data)
82                 p.send(tx_data)
83         lastMsg = now;
84     }

```

Project 20.2 Bluetooth Control LED

In this section, we will control the LED with Bluetooth.

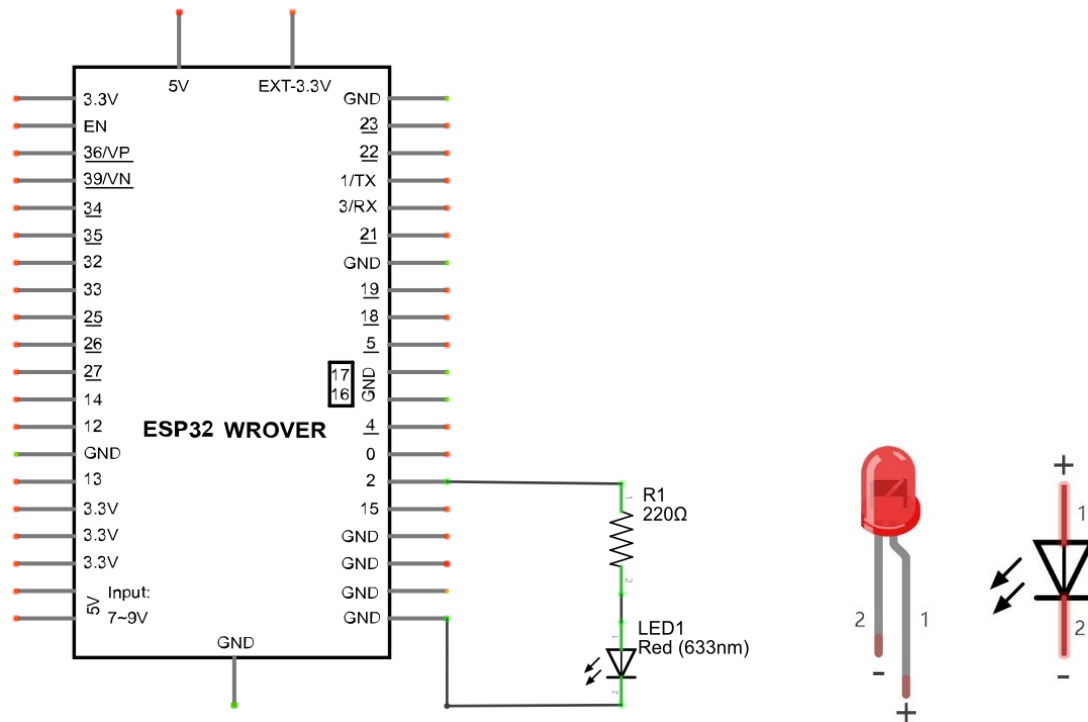
Component List

ESP32-WROVER x1 	GPIO Extension Board x1 		
Micro USB Wire x1 	LED x1 	Resistor 220Ω x1 	Jumper M/M x2 
Breadboard x1 			

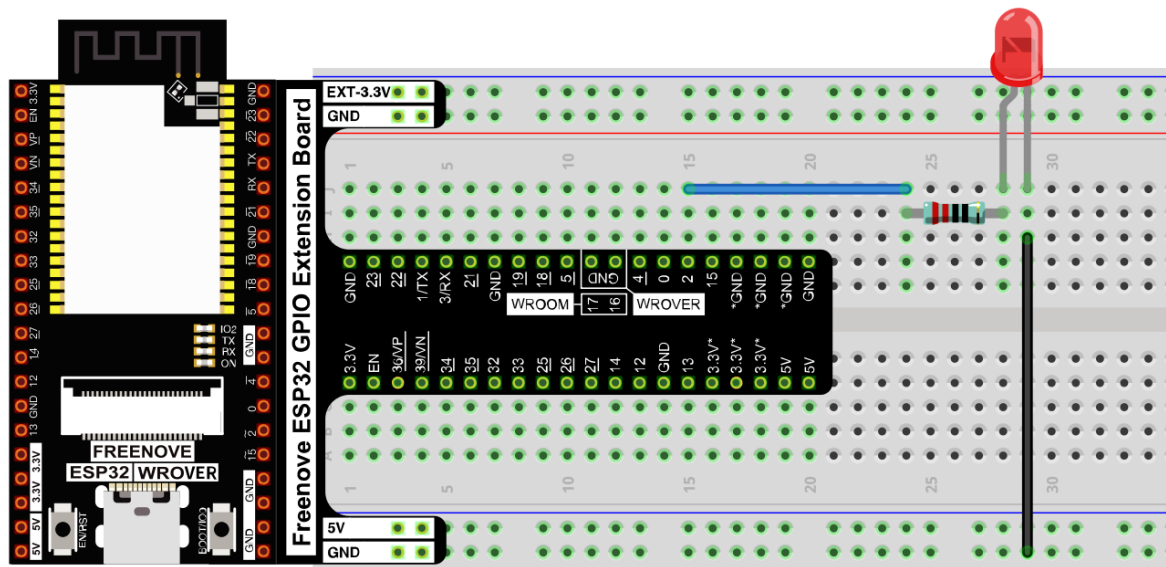
Circuit

Connect Freenove ESP32 to the computer using a USB cable.

Schematic diagram



Hardware connection. **If you need any support, please contact us via: support@freenove.com**

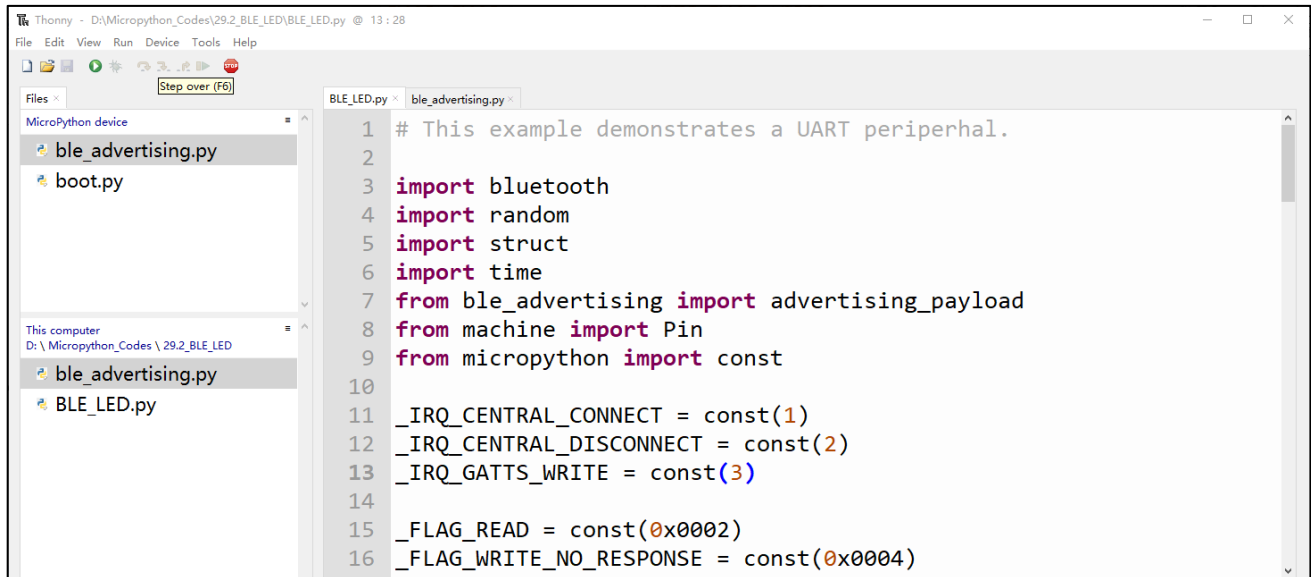


Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “20.2_BLE_LED”. Select “ble_advertising.py”, right click your mouse to select “Upload to /”, wait for “ble_advertising.py” to be uploaded to ESP32-WROVER and then double click “BLE_LED.py”.

20.2_BLE_LED



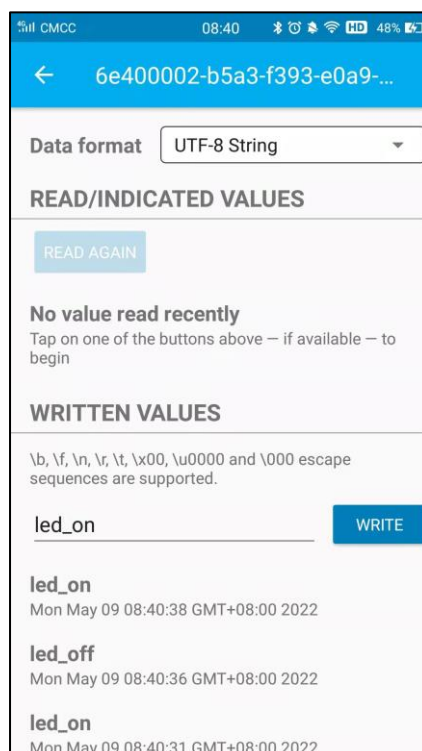
```

1 # This example demonstrates a UART peripheral.
2
3 import bluetooth
4 import random
5 import struct
6 import time
7 from ble_advertising import advertising_payload
8 from machine import Pin
9 from micropython import const
10
11 _IRQ_CENTRAL_CONNECT = const(1)
12 _IRQ_CENTRAL_DISCONNECT = const(2)
13 _IRQ_GATTS_WRITE = const(3)
14
15 _FLAG_READ = const(0x0002)
16 _FLAG_WRITE_NO_RESPONSE = const(0x0004)

```

Compile and upload code to ESP32. The operation of the APP is the same as 20.1, you only need to change the sending content to "led_on" and "led_off" to operate LEDs on the ESP32-WROVER.

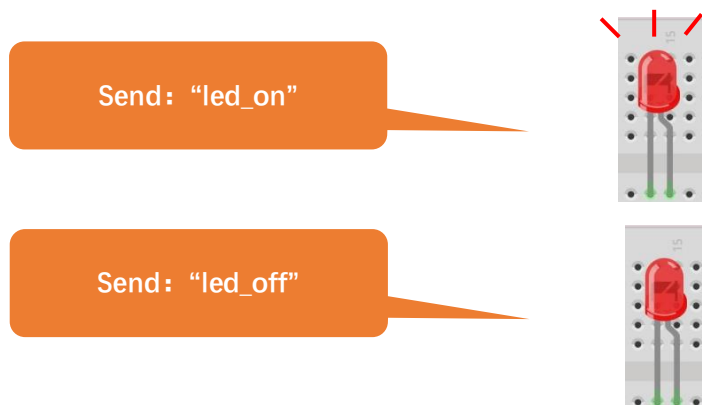
Data sent from mobile APP:



You can check the message sent by Bluetooth in "Shell".

```
Shell x
The BLE connection is successful.
Received: b'led_on'
Received: b'led_off'
Received: b'led_on'
```

The phenomenon of LED



Attention: If the sending content isn't "led_on" or "led_off", then the state of LED will not change. If the LED is on, when receiving irrelevant content, it keeps on; Correspondingly, if the LED is off, when receiving irrelevant content, it keeps off.

The following is the program code:

```
1  import bluetooth
2  import random
3  import struct
4  import time
5  from ble_advertising import advertising_payload
6  from machine import Pin
7  from micropython import const
8
9  _IRQ_CENTRAL_CONNECT = const(1)
10 _IRQ_CENTRAL_DISCONNECT = const(2)
11 _IRQ_GATTS_WRITE = const(3)
12
13 _FLAG_READ = const(0x0002)
14 _FLAG_WRITE_NO_RESPONSE = const(0x0004)
15 _FLAG_WRITE = const(0x0008)
16 _FLAG_NOTIFY = const(0x0010)
17
18 _UART_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
19 _UART_TX = (
20     bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"),
21     _FLAG_READ | _FLAG_NOTIFY,
```

```

22 )
23 _UART_RX = (
24     bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"),
25     _FLAG_WRITE | _FLAG_WRITE_NO_RESPONSE,
26 )
27 _UART_SERVICE = (
28     _UART_UUID,
29     (_UART_TX, _UART_RX),
30 )
31 class BLESimplePeripheral:
32     def __init__(self, ble, name="ESP32"):
33         self._ble = ble
34         self._ble.active(True)
35         self._ble.irq(self._irq)
36         ((self._handle_tx, self._handle_rx),) =
37 self._ble.gatts_register_services((_UART_SERVICE,))
38         self._connections = set()
39         self._write_callback = None
40         self._payload = advertising_payload(name=name, services=[_UART_UUID])
41         self._advertise()
42     def _irq(self, event, data):
43         # Track connections so we can send notifications.
44         if event == _IRQ_CENTRAL_CONNECT:
45             conn_handle, _, _ = data
46             print("New connection", conn_handle)
47             print("\nThe BLE connection is successful.")
48             self._connections.add(conn_handle)
49         elif event == _IRQ_CENTRAL_DISCONNECT:
50             conn_handle, _, _ = data
51             print("Disconnected", conn_handle)
52             self._connections.remove(conn_handle)
53             # Start advertising again to allow a new connection.
54             self._advertise()
55         elif event == _IRQ_GATTS_WRITE:
56             conn_handle, value_handle = data
57             value = self._ble.gatts_read(value_handle)
58             if value_handle == self._handle_rx and self._write_callback:
59                 self._write_callback(value)
60     def send(self, data):
61         for conn_handle in self._connections:
62             self._ble.gatts_notify(conn_handle, self._handle_tx, data)
63     def is_connected(self):
64         return len(self._connections) > 0
65     def _advertise(self, interval_us=500000):

```

```
66         print("Starting advertising")
67         self._ble.gap_advertise(interval_us, adv_data=self._payload)
68     def on_write(self, callback):
69         self._write_callback = callback
70 def demo():
71     ble = bluetooth.BLE()
72     p = BLESimplePeripheral(ble)
73     led=Pin(2,Pin.OUT)
74     def on_rx(rx_data):
75         print("Received: ", rx_data)
76         if rx_data == b'led_on':
77             led.value(0)
78         elif rx_data == b'led_off':
79             led.value(1)
80         else:
81             pass
82     p.on_write(on_rx)
83     print("Please use LightBlue to connect to ESP32.")
84 if __name__ == "__main__":
85     demo()
```

Compare received message with "led_on" and "led_off" and take action accordingly.

```
76     if rx_data == b'led_on':
77         led.value(0)
78     elif rx_data == b'led_off':
79         led.value(1)
```

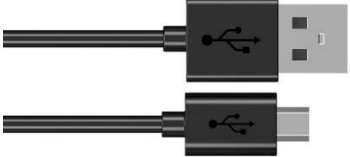
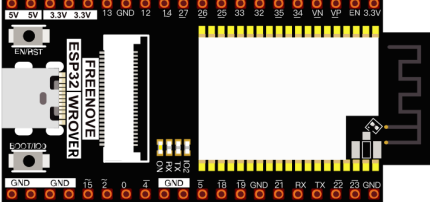
Chapter 21 WiFi Working Modes

In this chapter, we'll focus on the WiFi infrastructure for ESP32-WROVER.

ESP32-WROVER has 3 different WiFi operating modes: Station mode, AP mode and AP+Station mode. All WiFi programming projects must be configured with WiFi operating mode before using WiFi, otherwise WiFi cannot be used.

Project 21.1 Station mode

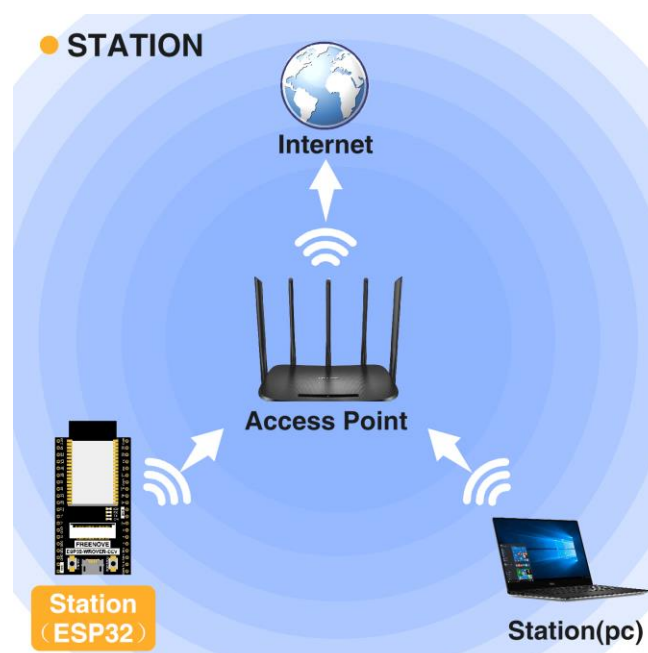
Component List

Micro USB Wire x1	ESP32-WROVER x1
	

Component knowledge

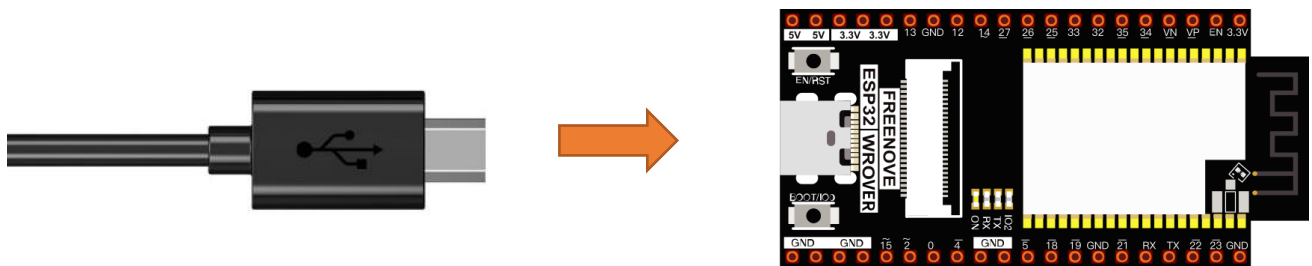
Station mode

When ESP32 selects Station mode, it acts as a WiFi client. It can connect to the router network and communicate with other devices on the router via WiFi connection. As shown below, the PC is connected to the router, and if ESP32 wants to communicate with the PC, it needs to be connected to the router.



Circuit

Connect Freenove ESP32 to the computer using the USB cable.



Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “21.1_Station_mode” and double click “Station_mode.py”.

21.1_Station_mode



Because the names and passwords of routers in various places are different, before the Code runs, users need to enter the correct router's name and password in the box as shown in the illustration above.

After making sure the router name and password are entered correctly, compile and upload codes to ESP32-WROVER, wait for ESP32 to connect to your router and print the IP address assigned by the router to ESP32 in "Shell".

```
MicroPython v1.12 on 2019-12-20; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

Setup start
connecting to FYI_2.4G
I (4155528) phy: phy_version: 4102, 2fa7a43, Jul 15 2019, 13:06:06,
0, 2
Connected, IP address: ('192.168.1.102', '255.255.255.0', '192.168.
1.1', '192.168.1.1')
Setup End

>>>
```

The following is the program code:

```
1  import time
2  import network
3
4  ssidRouter    = '*****' #Enter the router name
5  passwordRouter = '*****' #Enter the router password
6
7  def STA_Setup(ssidRouter, passwordRouter):
8      print("Setup start")
9      sta_if = network.WLAN(network.STA_IF)
10     if not sta_if.isconnected():
11         print('connecting to', ssidRouter)
12         sta_if.active(True)
13         sta_if.connect(ssidRouter, passwordRouter)
14         while not sta_if.isconnected():
15             pass
16         print('Connected, IP address:', sta_if.ifconfig())
17         print("Setup End")
18
19     try:
20         STA_Setup(ssidRouter, passwordRouter)
21     except:
22         sta_if.disconnect()
```

Import network module.

```
2  import network
```

Enter correct router name and password.

```
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
```

Set ESP32 in Station mode.

```
9      sta_if = network.WLAN(network.STA_IF)
```

Activate ESP32's Station mode, initiate a connection request to the router and enter the password to connect.

```
12     sta_if.active(True)
13     sta_if.connect(ssidRouter, passwordRouter)
```

Wait for ESP32 to connect to router until they connect to each other successfully.

```
14     while not sta_if.isconnected():
15         pass
```

Print the IP address assigned to ESP32-WROVER in "Shell".

```
16     print('Connected, IP address:', sta_if.ifconfig())
```

Reference

Class network

Before each use of **network**, please add the statement "**import network**" to the top of the python file.

WLAN(interface_id): Set to WiFi mode.

network.STA_IF: Client, connecting to other WiFi access points.

network.AP_IF: Access points, allowing other WiFi clients to connect.

active(is_active): With parameters, it is to check whether to activate the network interface; Without parameters, it is to query the current state of the network interface.

scan(ssid, bssid, channel, RSSI, authmode, hidden): Scan for wireless networks available nearby (only scan on STA interface), return a tuple list of information about the WiFi access point.

bssid: The hardware address of the access point, returned in binary form as a byte object. You can use `ubinascii.hexlify()` to convert it to ASCII format.

authmode: Access type

AUTH_OPEN = 0

AUTH_WEP = 1

AUTH_WPA_PSK = 2

AUTH_WPA2_PSK = 3

AUTH_WPA_WPA2_PSK = 4

AUTH_MAX = 6

Hidden: Whether to scan for hidden access points

False: Only scanning for visible access points

True: Scanning for all access points including the hidden ones.

isconnected(): Check whether ESP32 is connected to AP in Station mode. In STA mode, it returns True if it is connected to a WiFi access point and has a valid IP address; Otherwise it returns False.

connect(ssid, password): Connecting to wireless network.

ssid: WiFi name

password: WiFi password

disconnect(): Disconnect from the currently connected wireless network.

Project 21.2 AP mode

Component List & Circuit

Component List & Circuit are the same as in Section 21.1.

Component knowledge

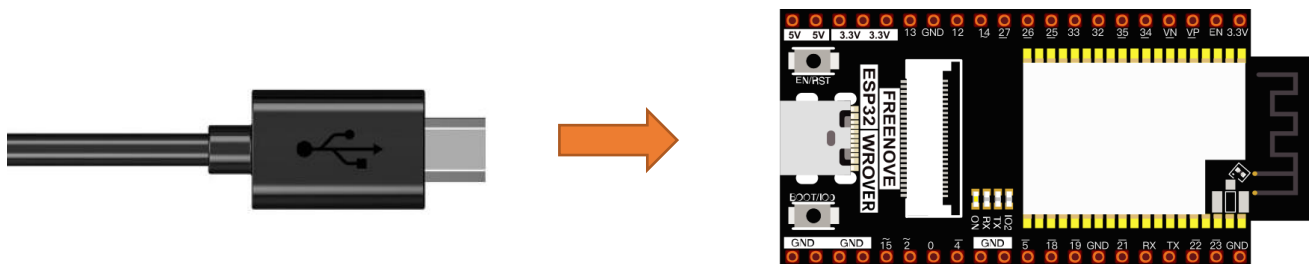
AP mode

When ESP32 selects AP mode, it creates a hotspot network that is separated from the Internet and waits for other WiFi devices to connect. As shown in the figure below, ESP32 is used as a hotspot. If a mobile phone or PC wants to communicate with ESP32, it must be connected to the hotspot of ESP32. Only after a connection is established with ESP32 can they communicate.



Circuit

Connect Freenove ESP32 to the computer using the USB cable.

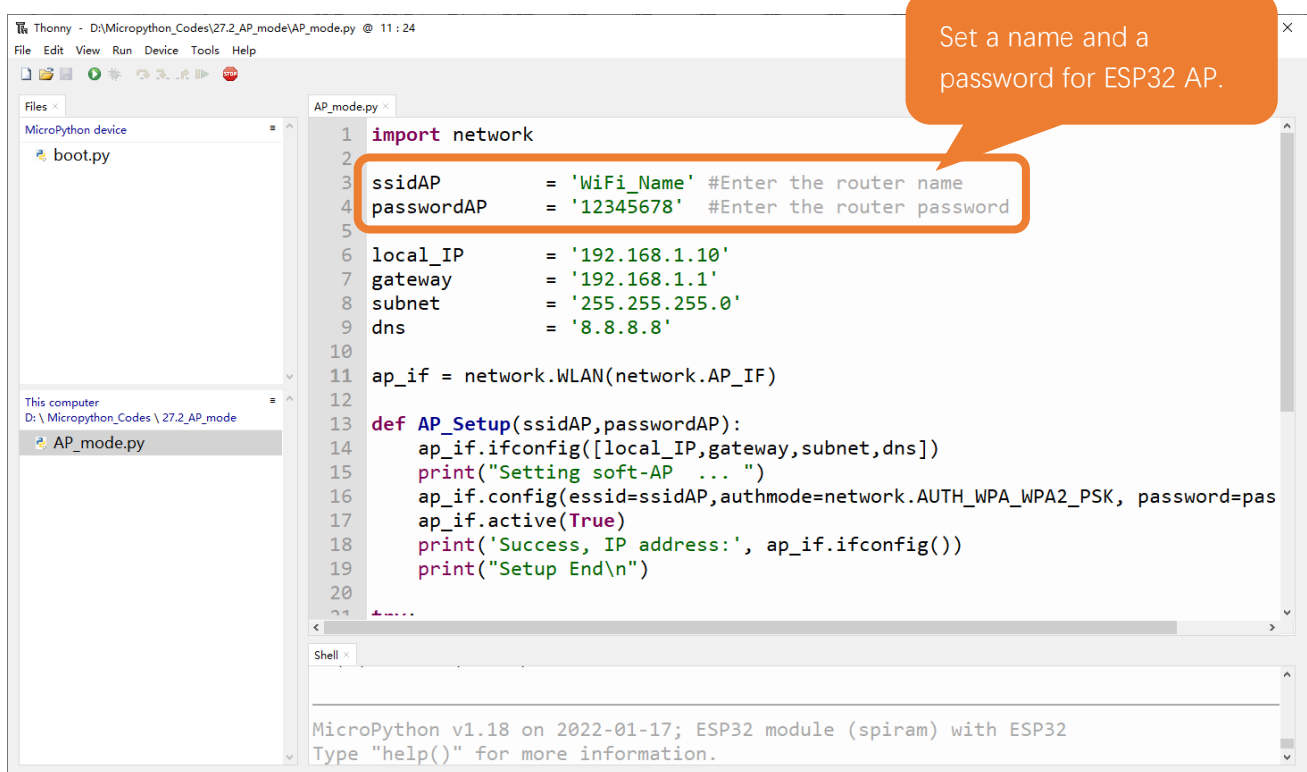


Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

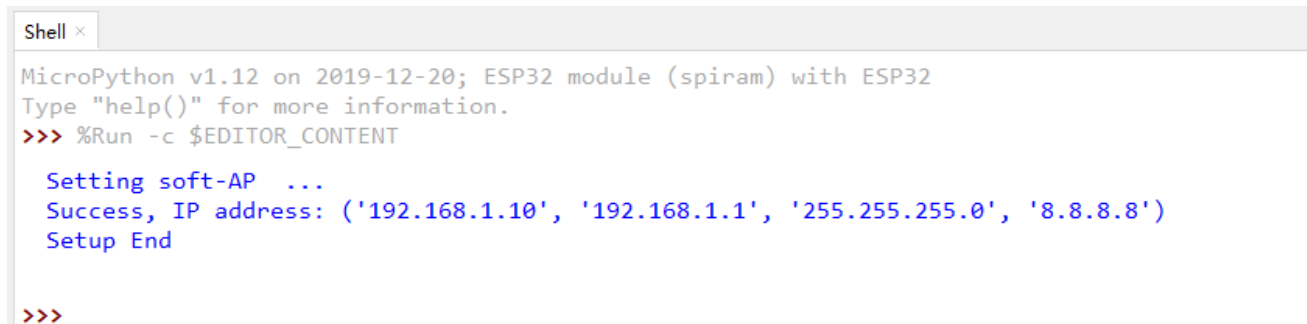
Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “21.2_AP_mode”. and double click “AP_mode.py”.

21.2_AP_mode

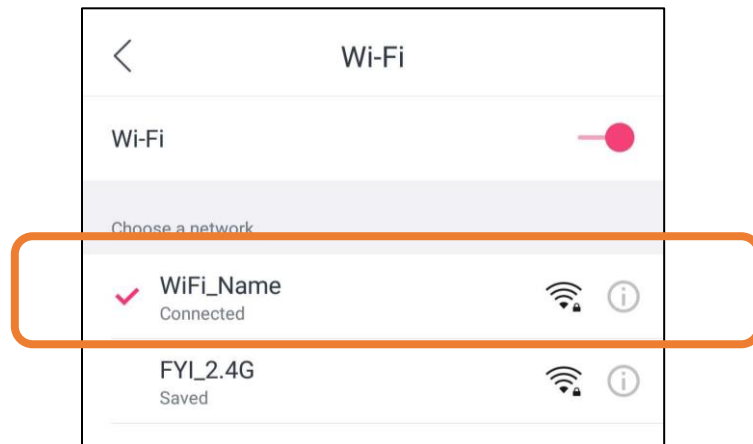


Before the Code runs, you can make any changes to the AP name and password for ESP32 in the box as shown in the illustration above. Of course, you can leave it alone by default.

Click “Run current script”, open the AP function of ESP32 and print the access point information.



Turn on the WiFi scanning function of your phone, and you can see the ssid_AP on ESP32, which is called "WiFi_Name" in this Code. You can enter the password "12345678" to connect it or change its AP name and password by modifying Code.



The following is the program code:

```

1  import network
2
3  ssidAP      = 'WiFi_Name' #Enter the router name
4  passwordAP  = '12345678' #Enter the router password
5
6  local_IP    = '192.168.1.10'
7  gateway     = '192.168.1.1'
8  subnet      = '255.255.255.0'
9  dns         = '8.8.8.8'
10
11 ap_if = network.WLAN(network.AP_IF)
12
13 def AP_Setup(ssidAP, passwordAP):
14     ap_if.ifconfig([local_IP, gateway, subnet, dns])
15     print("Setting soft-AP ... ")
16     ap_if.config(essid=ssidAP, authmode=network.AUTH_WPA_WPA2_PSK, password=passwordAP)
17     ap_if.active(True)
18     print('Success, IP address:', ap_if.ifconfig())
19     print("Setup End\n")
20
21 try:
22     AP_Setup(ssidAP, passwordAP)
23 except:
24     ap_if.disconnect()

```

Import network module.

```

1  import network

```

Enter correct AP name and password.

```
3 ssidAP = 'WiFi_Name' #Enter the router name
4 passwordAP = '12345678' #Enter the router password
```

Set ESP32 in AP mode.

```
11 ap_if = network.WLAN(network.AP_IF)
```

Configure IP address, gateway and subnet mask for ESP32.

```
14 ap_if.ifconfig([local_IP, gateway, subnet, dns])
```

Turn on an AP in ESP32, whose name is set by ssid_AP and password is set by password_AP.

```
16 ap_if.config(essid=ssidAP, authmode=network.AUTH_WPA_WPA2_PSK, password=passwordAP)
17 ap_if.active(True)
```

If the program is running abnormally, the AP disconnection function will be called.

```
14 ap_if.disconnect()
```

Reference

Class network

Before each use of **network**, please add the statement “**import network**” to the top of the python file.

WLAN(interface_id): Set to WiFi mode.

network.STA_IF: Client, connecting to other WiFi access points

network.AP_IF: Access points, allowing other WiFi clients to connect

active(is_active): With parameters, it is to check whether to activate the network interface; Without parameters, it is to query the current state of the network interface

isconnected(): In AP mode, it returns True if it is connected to the station; otherwise it returns False.

connect(ssid, password): Connecting to wireless network

ssid: WiFiname

password: WiFipassword

config(essid, channel): To obtain the MAC address of the access point or to set the WiFi channel and the name of the WiFi access point.

ssid: WiFi account name

channel: WiFichannel

ifconfig([(ip, subnet, gateway, dns)]): Without parameters, it returns a 4-tuple (ip, subnet_mask, gateway, DNS_server); With parameters, it configures static IP.

ip: IPaddress

subnet_mask: subnet mask

gateway: gateway

DNS_server: DNSserver

disconnect(): Disconnect from the currently connected wireless network

status(): Return the current status of the wireless connection

Project 21.3 AP+Station mode

Component List

Micro USB Wire x1	ESP32-WROVER x1
-------------------	-----------------

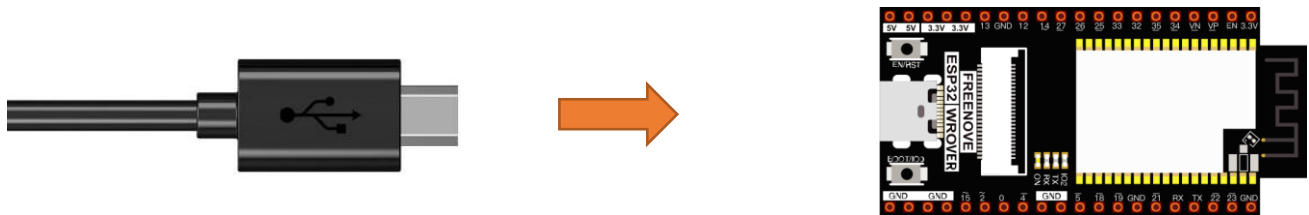
Component knowledge

AP+Station mode

In addition to AP mode and Station mode, ESP32 can also use AP mode and Station mode at the same time. This mode contains the functions of the previous two modes. Turn on ESP32's Station mode, connect it to the router network, and it can communicate with the Internet via the router. At the same time, turn on its AP mode to create a hotspot network. Other WiFi devices can choose to connect to the router network or the hotspot network to communicate with ESP32.

Circuit

Connect Freenove ESP32 to the computer using the USB cable.

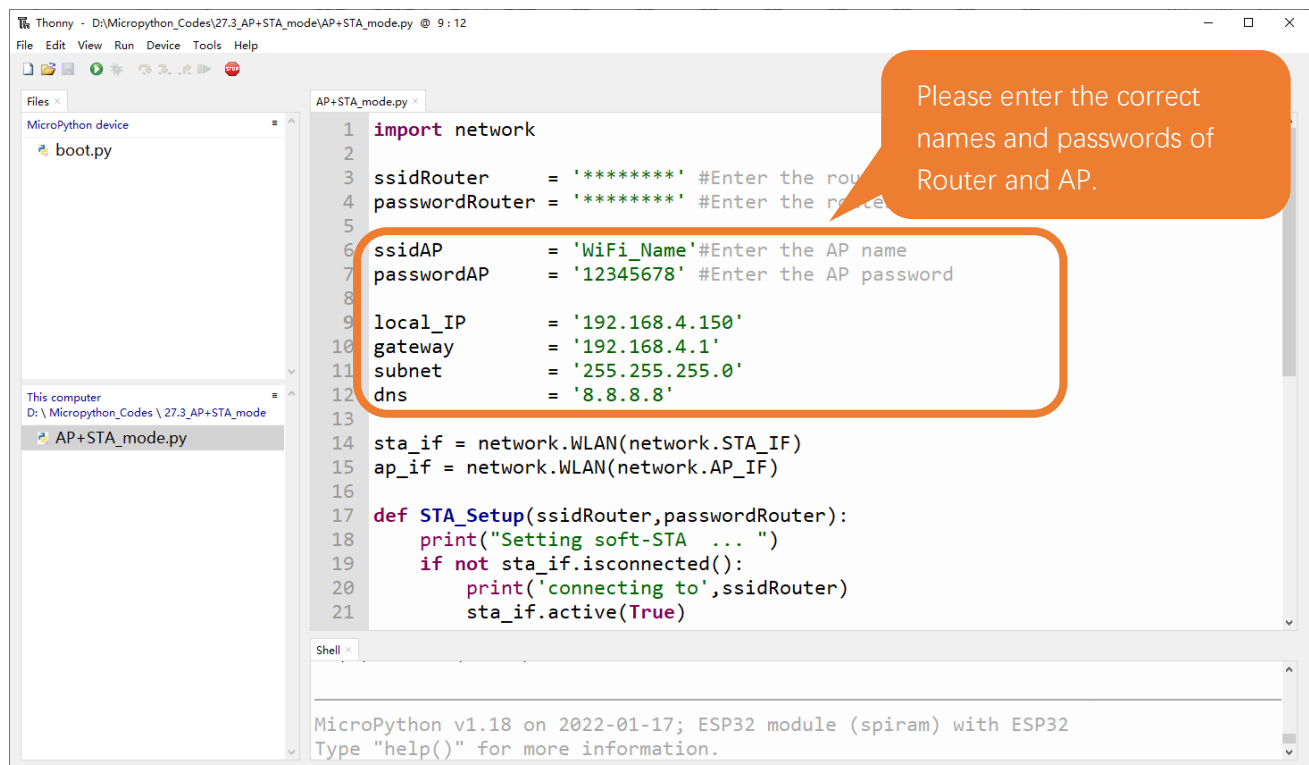


Code

Move the program folder "**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

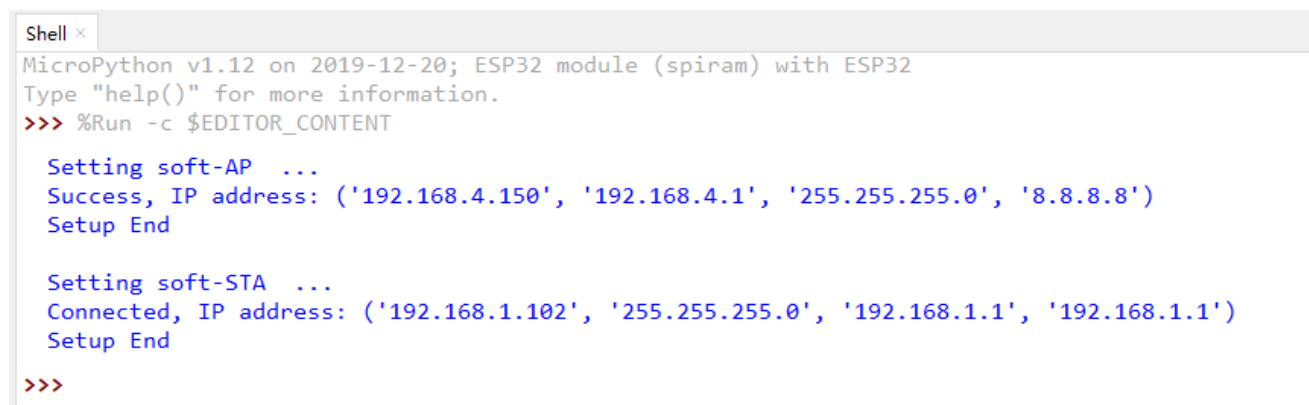
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "21.3_AP+STA_mode" and double click "AP+STA_mode.py".

21.3_AP+STA_mode

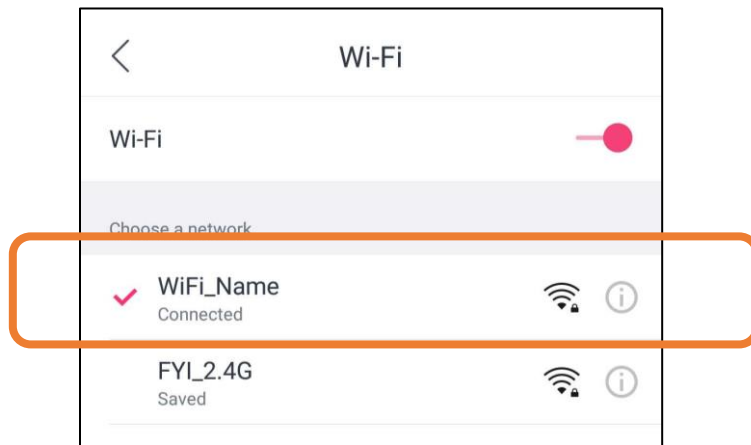


It is analogous to Project 21.1 and Project 21.2. Before running the Code, you need to modify ssidRouter, passwordRouter, ssidAP and passwordAP shown in the box of the illustration above.

After making sure that the code is modified correctly, click "Run current script" and the "Shell" will display as follows:



Turn on the WiFi scanning function of your phone, and you can see the ssidAP on ESP32.



The following is the program code:

```

1  import network
2
3  ssidRouter    = '*****' #Enter the router name
4  passwordRouter = '*****' #Enter the router password
5
6  ssidAP        = 'WiFi_Name' #Enter the AP name
7  passwordAP    = '12345678' #Enter the AP password
8
9  local_IP      = '192.168.4.150'
10 gateway       = '192.168.4.1'
11 subnet        = '255.255.255.0'
12 dns           = '8.8.8.8'
13
14 sta_if = network.WLAN(network.STA_IF)
15 ap_if = network.WLAN(network.AP_IF)
16
17 def STA_Setup(ssidRouter, passwordRouter):
18     print("Setting soft-STA ... ")
19     if not sta_if.isconnected():
20         print('connecting to', ssidRouter)
21         sta_if.active(True)
22         sta_if.connect(ssidRouter, passwordRouter)
23         while not sta_if.isconnected():
24             pass
25     print('Connected, IP address:', sta_if.ifconfig())
26     print("Setup End")
27
28 def AP_Setup(ssidAP, passwordAP):
29     ap_if.ifconfig([local_IP, gateway, subnet, dns])
30     print("Setting soft-AP ... ")

```

```
31     ap_if.config(essid=ssidAP,authmode=network.AUTH_WPA_WPA2_PSK, password=passwordAP)
32     ap_if.active(True)
33     print('Success, IP address:', ap_if.ifconfig())
34     print("Setup End\n")
35
36 try:
37     AP_Setup(ssidAP, passwordAP)
38     STA_Setup(ssidRouter, passwordRouter)
39 except:
40     sta_if.disconnect()
41     ap_if.iddisconnect()
```

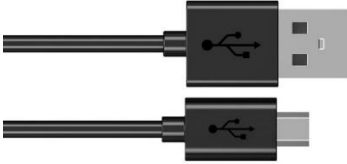
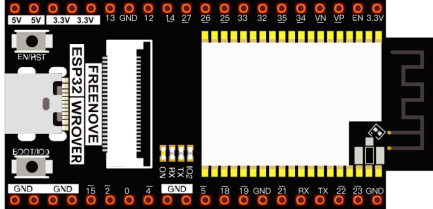

Chapter 22 TCP/IP

In this chapter, we will introduce how ESP32 implements network communications based on TCP/IP protocol. There are two roles in TCP/IP communication, namely Server and Client, which will be implemented respectively with two projects in this chapter.

Project 22.1 As Client

In this section, ESP32 is used as Client to connect Server on the same LAN and communicate with it.

Component List

Micro USB Wire x1 	ESP32-WROVER x1 
---	--

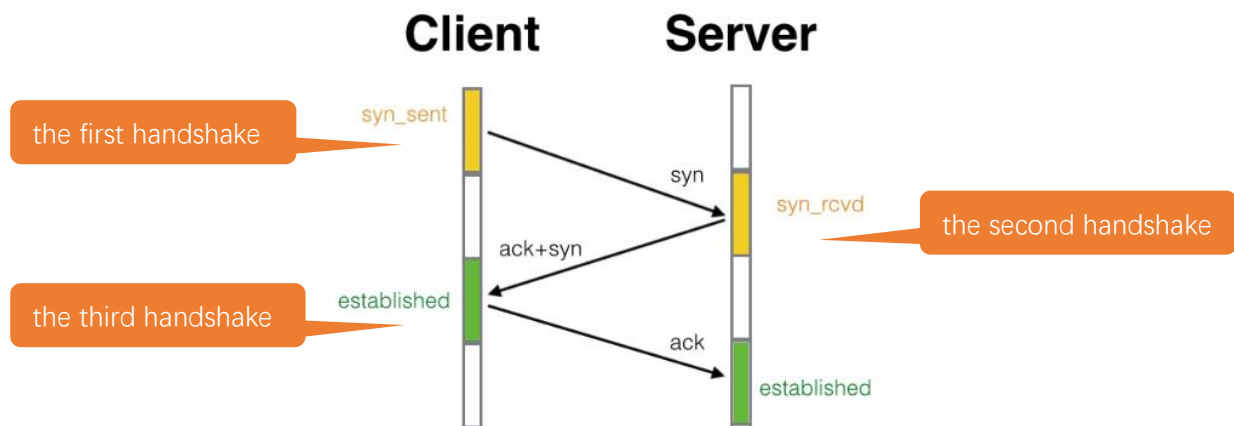
Component knowledge

TCP connection

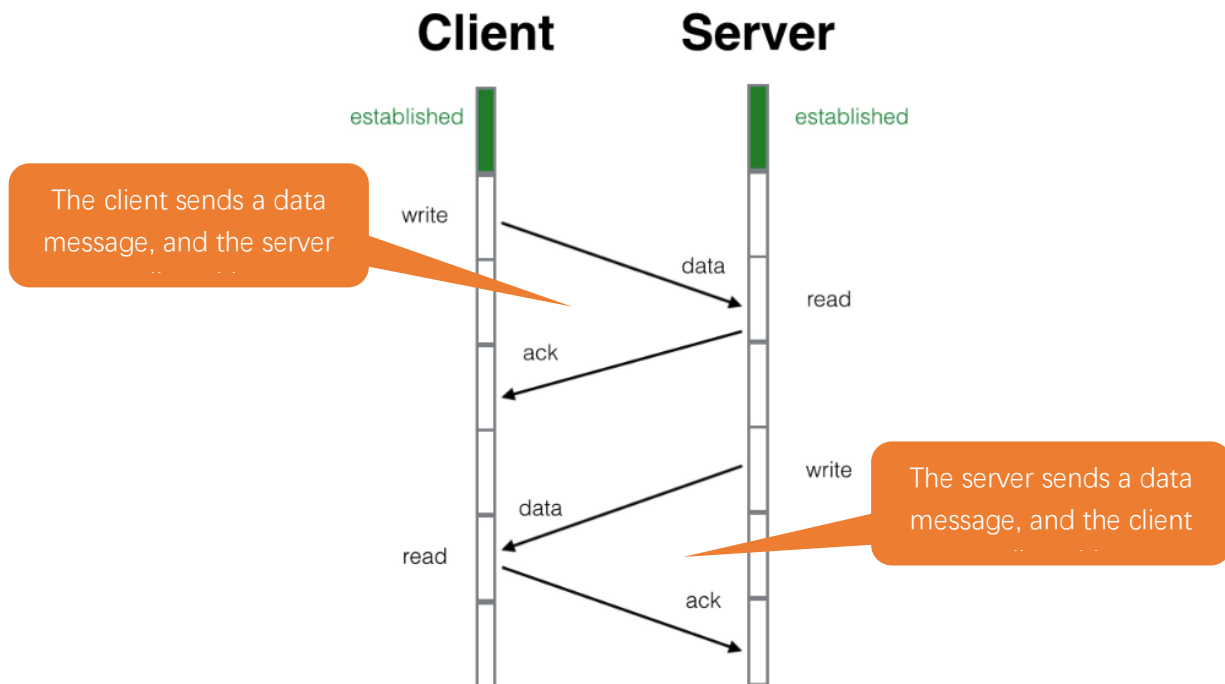
Before transmitting data, TCP needs to establish a logical connection between the sending end and the receiving end. It provides reliable and error-free data transmission between the two computers. In the TCP connection, the client and the server must be clarified. The client sends a connection request to the server, and each time such a request is proposed, a "three-times handshake" is required.

Three-times handshake: In the TCP protocol, during the preparation phase of sending data, the client and the server interact three times to ensure the reliability of the connection, which is called "three-times handshake". The first handshake, the client sends a connection request to the server and waits for the server to confirm. The second handshake, the server sends a response back to the client informing that it has received the connection request.

The third handshake, the client sends a confirmation message to the server again to confirm the connection.



TCP is a connection-oriented, low-level transmission control protocol. After TCP establishes a connection, the client and server can send and receive messages to each other, and the connection will always exist as long as the client or server does not initiate disconnection. Each time one party sends a message, the other party will reply with an ack signal.



Install Processing

In this tutorial, we use Processing to build a simple TCP/IP communication platform.

If you've not installed Processing, you can download it by clicking <https://processing.org/download/>. You can choose an appropriate version to download according to your PC system.

Processing p5.js Processing.py Processing for Android Processing for Pi Processing Foundation

Processing

Download Processing. Processing is available for Linux, Mac OS X, and Windows. Select your choice to download the software below.

3.5.4 (17 January 2020)

[Windows 64-bit](#) [Linux 64-bit](#) [Mac OS X](#)

[Windows 32-bit](#)

» [Github](#) [Report Bugs](#) [Wiki](#) [Supported Platforms](#)

Read about the [changes in 3.0](#). The [list of revisions](#) covers the differences between releases in detail.

Unzip the downloaded file to your computer. Click "processing.exe" as the figure below to run this software.

	core	2020/1/17 12:16
	java	2020/1/17 12:17
	lib	2020/1/17 12:16
	modes	2020/1/17 12:16
	tools	2020/1/17 12:16
	processing.exe	2020/1/17 12:16
	processing-java.exe	2020/1/17 12:16
	revisions.txt	2020/1/17 12:16

Use Server mode for communication

Open the “Freenove_Super_Starter_Kit_for_ESP32/Codes/Micropython_Codes/22.1_TCP_as_Client/sketchWiFi/sketchWiFi.pde”. Click “Run”.

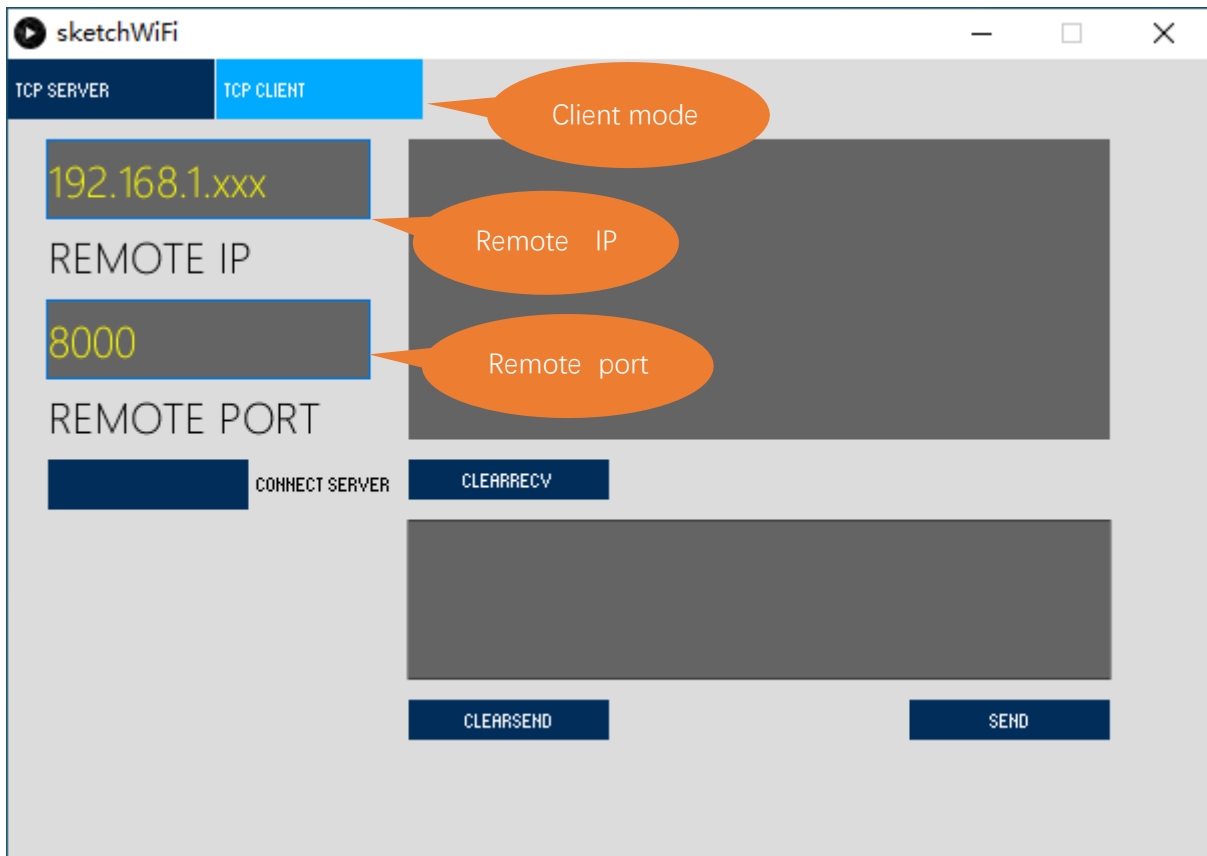


The new pop-up interface is as follows. If ESP32 is used as Client, select TCP SERVER mode for sketchWiFi.



When sketchWiFi selects TCP SERVER mode, ESP32 Code needs to be changed according to sketchWiFi's displaying of LOCAL IP or LOCAL PORT.

If ESP32 serves as Server, select TCP CLIENT mode for sketchWiFi.



When sketchWiFi selects TCP CLIENT mode, the LOCAL IP and LOCAL PORT of sketchWiFi need to be changed according to the IP address and port number printed by the serial monitor.

Mode selection: select **Server mode/Client mode**.

IP address: In Server mode, this option does not need to be filled in, and the computer will automatically obtain the IP address.

In Client mode, fill in the remote IP address to be connected.

Port number: In Server mode, fill in a port number for client devices to make an access connection.

In client mode, fill in port number given by the Server devices to make an access connection.

Start button: In server mode, push the button, and then the computer will serve as Server and open a port number for Client to make access connection. During this period, the computer will keep monitoring.

In client mode, before pushing the button, please make sure the server is on, remote IP address and remote port number is correct; push the button, and the computer will make access connection to the remote port number of the remote IP as a Client.

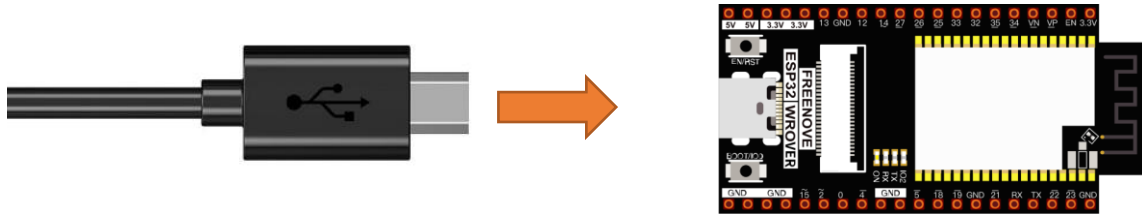
clear receive: clear out the content in the receiving text box

clear send: clear out the content in the sending text box

Sending button: push the sending button, the computer will send the content in the text box to others.

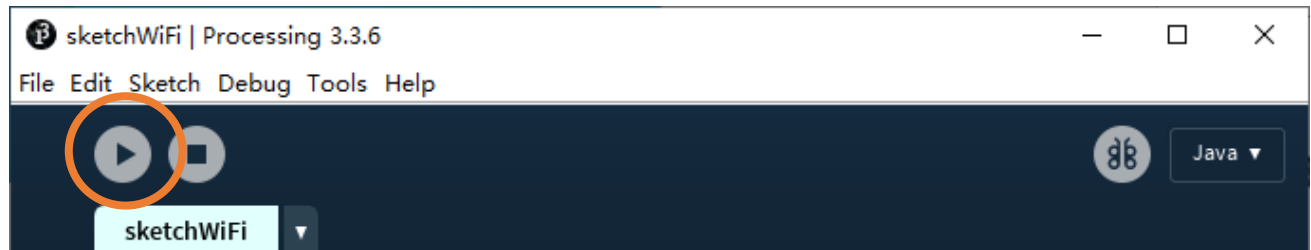
Circuit

Connect Freenove ESP32 to the computer using USB cable.

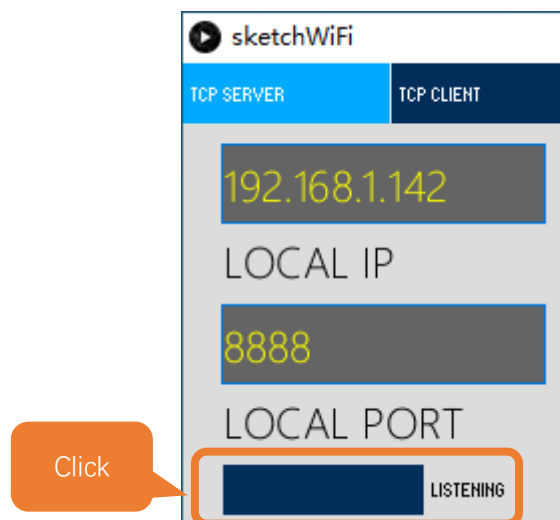


Code

Before running the Code, please open "sketchWiFi.pde." first, and click "Run".



The newly pop up window will use the computer's IP address by default and open a data monitor port. Click "Listening".

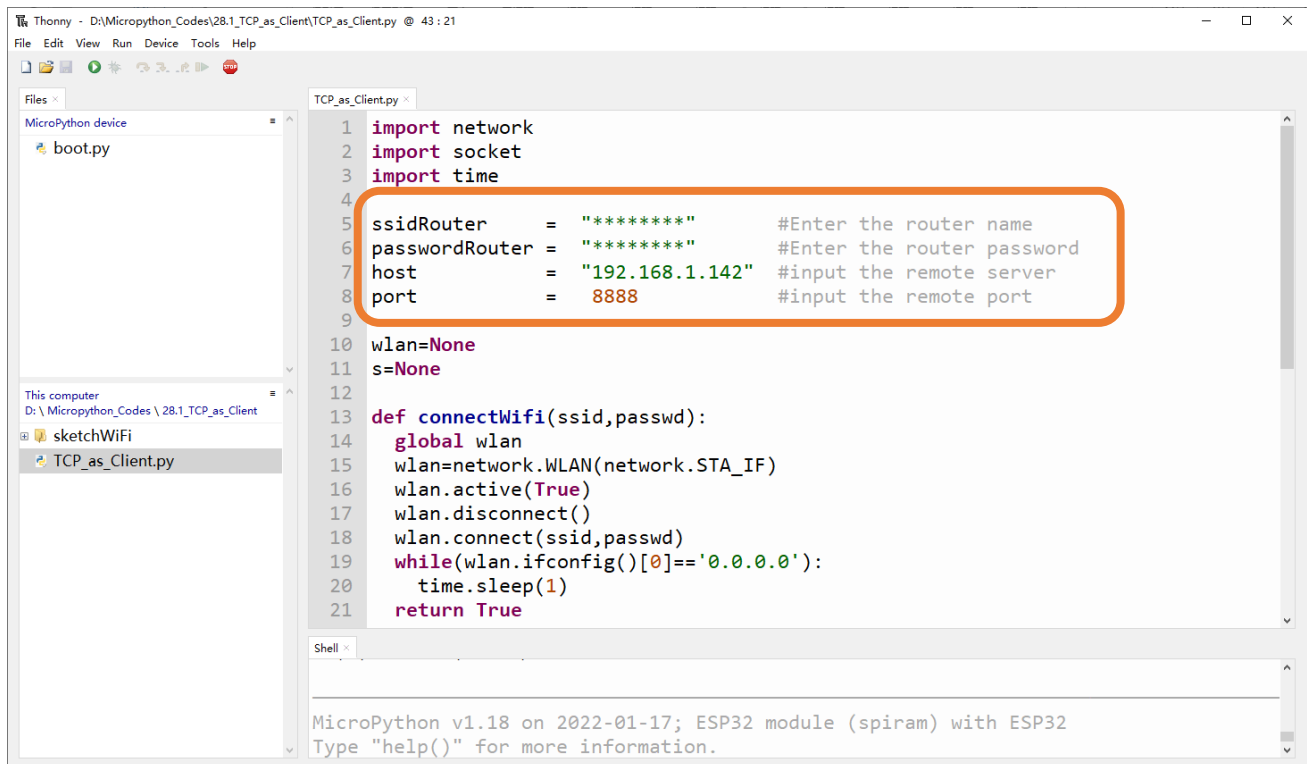


Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “22.1_TCP_as_Client” and double click “TCP_as_Client.py”.

Before clicking “Run current script”, please modify the name and password of your router and fill in the “host” and “port” according to the IP information shown in the box below:

22.1_TCP_as_Client



Click “Run current script” and in “Shell”, you can see ESP32-WROVER automatically connects to sketchWiFi.



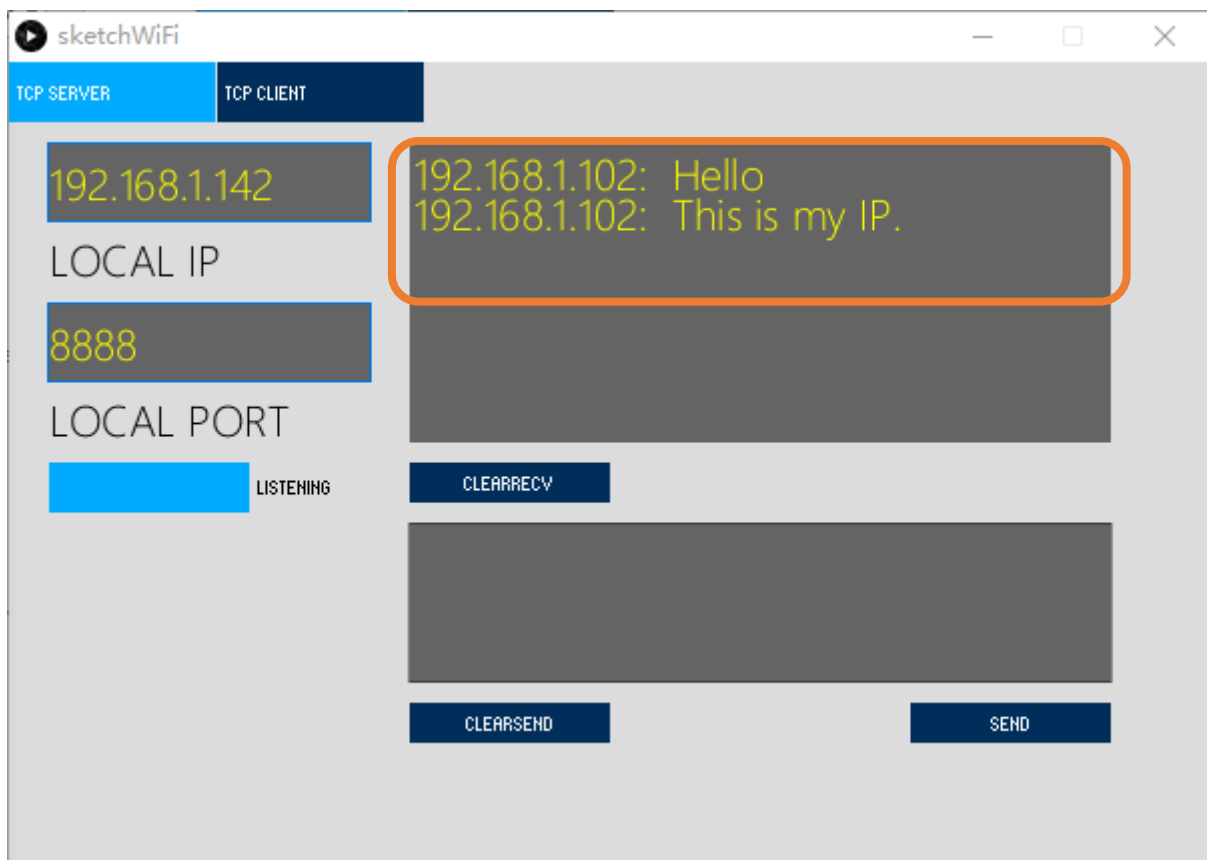
If you don't click "Listening" for sketchWiFi, ESP32-WROVER will fail to connect and will print information as follows:

```

Shell x
TCP connected to: 192.168.1.142 : 8888
Close socket
>>> %Run -c $EDITOR_CONTENT
TCP close, please reset!
>>>

```

ESP32 connects with TCP SERVER, and TCP SERVER receives messages from ESP32, as shown in the figure below.



The following is the program code:

```

1  import network
2  import socket
3  import time
4
5  ssidRouter    = "*****"    #Enter the router name
6  passwordRouter = "*****"    #Enter the router password
7  host          = "*****"    #input the remote server
8  port          = 8888        #input the remote port
9
10 wlan=None
11 s=None

```



```

12
13 def connectWifi(ssid,passwd):
14     global wlan
15     wlan= network.WLAN(network.STA_IF)
16     wlan.active(True)
17     wlan.disconnect()
18     wlan.connect(ssid,passwd)
19     while(wlan.ifconfig()[0]=='0.0.0.0'):
20         time.sleep(1)
21     return True
22 try:
23     connectWifi(ssidRouter,passwordRouter)
24     s = socket.socket()
25     s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
26     s.connect((host,port))
27     print("TCP Connected to:", host, ":", port)
28     s.send('Hello')
29     s.send('This is my IP.')
30     while True:
31         data = s.recv(1024)
32         if(len(data) == 0):
33             print("Close socket")
34             s.close()
35             break
36         print(data)
37         ret=s.send(data)
38 except:
39     print("TCP close, please reset!")
40     if (s):
41         s.close()
42     wlan.disconnect()
43     wlan.active(False)

```

Import network、socket、time modules.

```

1 import network
2 import socket
3 import time

```

Enter the actual router name, password, remote server IP address, and port number.

```

5 ssidRouter    = "*****"    #Enter the router name
6 passwordRouter = "*****"    #Enter the router password
7 host          = "*****"    #input the remote server
8 port          = 8888        #input the remote port

```

Connect specified Router until it is successful.

```

13 def connectWifi(ssid,passwd):
14     global wlan
15     wlan= network.WLAN(network.STA_IF)
16     wlan.active(True)
17     wlan.disconnect()
18     wlan.connect(ssid,passwd)
19     while(wlan.ifconfig()[0]!='0.0.0.0'):
20         time.sleep(1)
21     return True

```

Connect router and then connect it to remote server.

```

23 connectWifi(ssidRouter,passwordRouter)
24 s = socket.socket()
25 s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
26 s.connect((host,port))
27 print("TCP Connected to:", host, ":", port)

```

Send messages to the remote server, receive the messages from it and print them out, and then send the messages back to the server.

```

28 s.send('Hello')
29 s.send('This is my IP.')
30 while True:
31     data = s.recv(1024)
32     if(len(data) == 0):
33         print("Close socket")
34         s.close()
35         break
36     print(data)
37     ret=s.send(data)

```

If an exception occurs in the program, for example, the remote server is shut down, execute the following program, turn off the socket function, and disconnect the WiFi.

```

39 print("TCP close, please reset!")
40 if (s):
41     s.close()
42 wlan.disconnect()
43 wlan.active(False)

```

Reference

Class socket

Before each use of **socket**, please add the statement “**import socket**” to the top of the python file.

socket([af, type, proto]): Create a socket.

af: address

socket.AF_INET: IPv4

socket.AF_INET6: IPv6

type: type

socket.SOCK_STREAM : TCP stream

socket.SOCK_DGRAM : UDP datagram

socket.SOCK_RAW : Original socket

socket.SO_REUSEADDR : socket reusable

proto: protocol number

socket.IPPROTO_TCP: TCPmode

socket.IPPROTO_UDP: UDPmode

socket.setsockopt(level, optname, value): Set the socket according to the options.

Level: Level of socket option

socket.SOL_SOCKET: Level of socket option. By default, it is 4095.

optname: Options of socket

socket.SO_REUSEADDR: Allowing a socket interface to be tied to an address that is already in use.

value: The value can be an integer or a bytes-like object representing a buffer.

socket.connect(address): To connect to server.

Address: Tuple or list of the server's address and port number

send(bytes): Send data and return the bytes sent.

recv(bufsize): Receive data and return a bytes object representing the data received.

close(): Close socket.

To learn more please visit: <http://docs.micropython.org/en/latest/>

Project 22.2 As Server

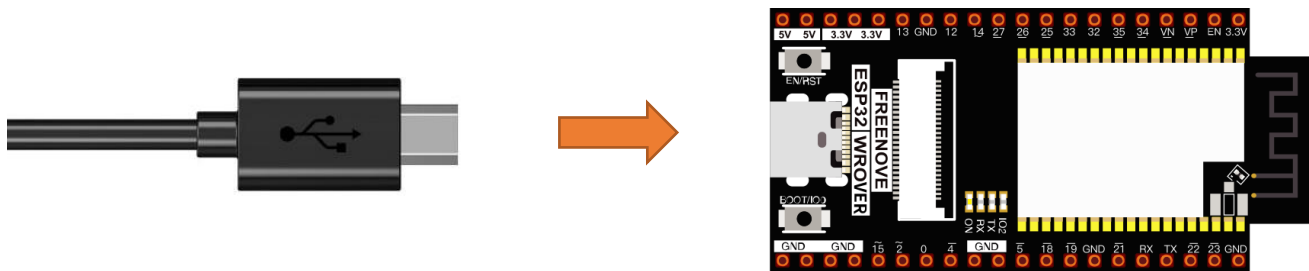
In this section, ESP32 is used as a Server to wait for the connection and communication with Client on the same LAN.

Component List

Micro USB Wire x1	ESP32-WROVER x1
-------------------	-----------------

Circuit

Connect Freenove ESP32 to the computer using the USB cable.



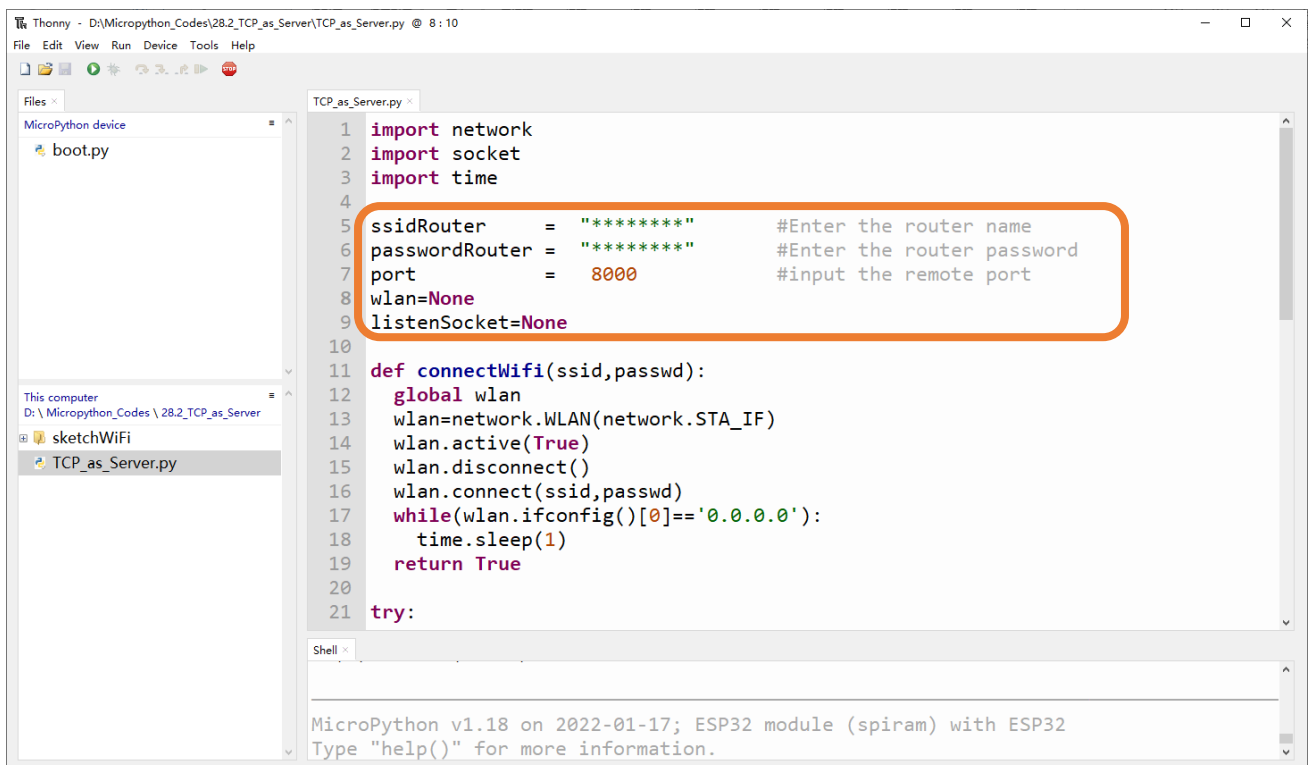
Code

Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

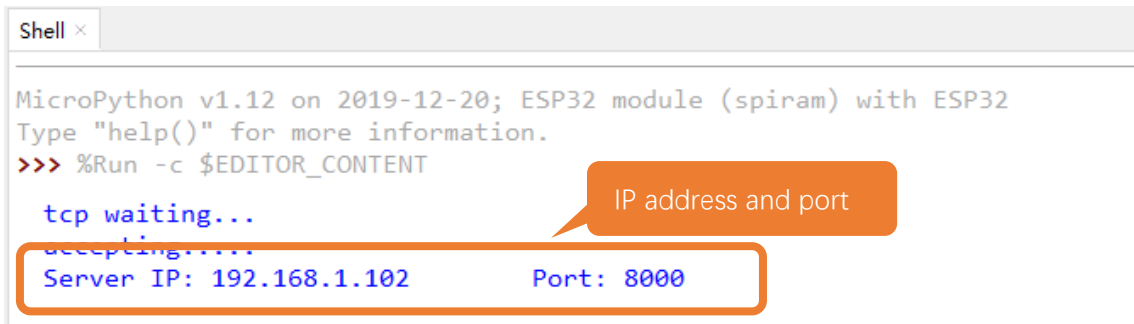
Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “22.2_TCP_as_Server” and double click “TCP_as_Server.py”.

Before clicking “Run current script”, please modify the name and password of your router shown in the box below.

22.2_TCP_as_Server



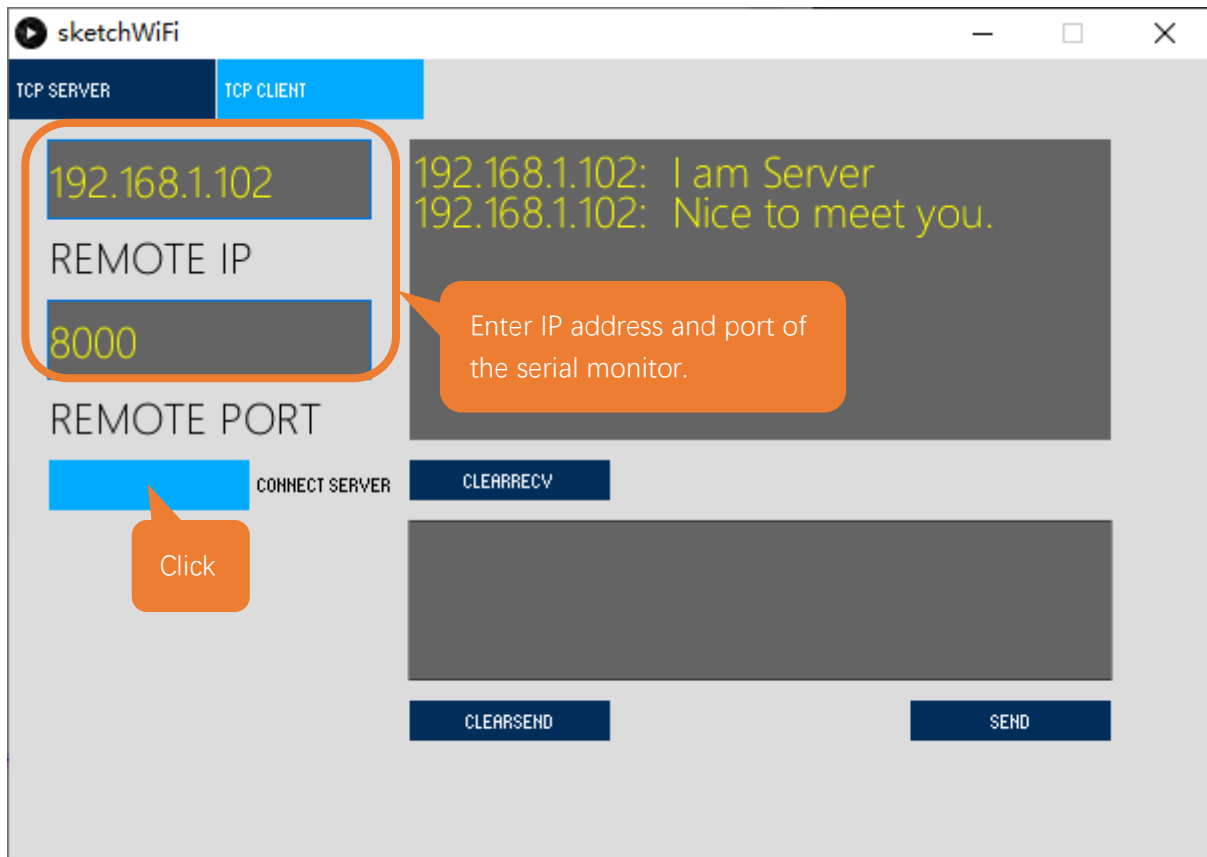
After making sure that the router's name and password are correct, click “Run current script” and in “Shell”, you can see a server opened by the ESP32- WROVER waiting to connecting to other network devices.



Processing:

Open the "Freenove_Super_Starter_Kit_for_ESP32/Codes/MicroPython_Codes/22.2_TCP_as_Server/sketchWiFi/sketchWiFi.pde".

Based on the message printed in "Shell", enter the correct IP address and port when processing, and click to establish a connection with ESP32 to communicate.



You can enter any information in the "Send Box" of sketchWiFi. Click "Send" and ESP32 will print the received messages to "Shell" and send them back to sketchWiFi.

```

Shell x
MicroPython v1.12 on 2019-12-20; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT|

tcp waiting...
accepting.....
Server IP: 192.168.1.102      Port: 8000
('192.168.1.142', 51326) connected
b'Nice to meet you.'
```

The following is the program code:

```
1  import network
2  import socket
3  import time
4
5  ssidRouter    = "*****"      #Enter the router name
6  passwordRouter = "*****"      #Enter the router password
7  port          = 8000           #input the remote port
8  wlan          = None
9  listenSocket  = None
10
11 def connectWifi(ssid,passwd):
12     global wlan
13     wlan=network.WLAN(network.STA_IF)
14     wlan.active(True)
15     wlan.disconnect()
16     wlan.connect(ssid,passwd)
17     while(wlan.ifconfig()[0]!='0.0.0.0'):
18         time.sleep(1)
19     return True
20
21 try:
22     connectWifi(ssidRouter,passwordRouter)
23     ip=wlan.ifconfig()[0]
24     listenSocket = socket.socket()
25     listenSocket.bind((ip,port))
26     listenSocket.listen(1)
27     listenSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
28     print('tcp waiting...')
29     while True:
30         print("Server IP:", ip, "\tPort:", port)
31         print("accepting....")
32         conn,addr = listenSocket.accept()
33         print(addr,"connected")
34         break
35     conn.send(' I am Server')
36     while True:
37         data = conn.recv(1024)
38         if(len(data) == 0):
39             print("close socket")
40             listenSocket.close()
41             wlan.disconnect()
42             wlan.active(False)
43             break
```

```

44     else:
45         print(data)
46         ret = conn.send(data)
47 except:
48     print("Close TCP-Server, please reset.")
49     if(listenSocket):
50         listenSocket.close()
51     wlan.disconnect()
52     wlan.active(False)

```

Call function connectWifi() to connect to router and obtain the dynamic IP that it assigns to ESP32.

```

22     connectWifi(ssidRouter, passwordRouter)
23     ip=wlan.ifconfig()[0]

```

Open the socket server, bind the server to the dynamic IP, and open a data monitoring port.

```

24     listenSocket = socket.socket()
25     listenSocket.bind((ip, port))
26     listenSocket.listen(1)
27     listenSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

```

Print the server's IP address and port, monitor the port and wait for the connection of other network devices.

```

29     while True:
30         print("Server IP:", ip, "\tPort:", port)
31         print("accepting....")
32         conn, addr = listenSocket.accept()
33         print(addr, "connected")
34         break

```

Each time receiving data, print them in "Shell" and send them back to the client.

```

36     while True:
37         data = conn.recv(1024)
38         if(len(data) == 0):
39             print("close socket")
40             listenSocket.close()
41             wlan.disconnect()
42             wlan.active(False)
43             break
44         else:
45             print(data)
46             ret = conn.send(data)

```

If the client is disconnected, close the server and disconnect WiFi.

```

47 except:
48     print("Close TCP-Server, please reset.")
49     if(listenSocket):
50         listenSocket.close()
51     wlan.disconnect()
52     wlan.active(False)

```

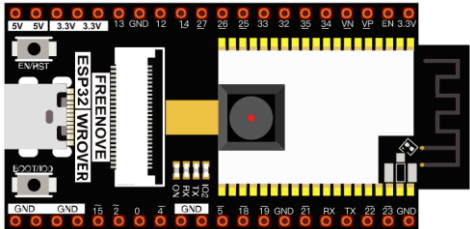

Chapter 23 Camera Web Server

In this section, we'll use ESP32's video function as an example to study.

Project 23.1 Camera Web Server

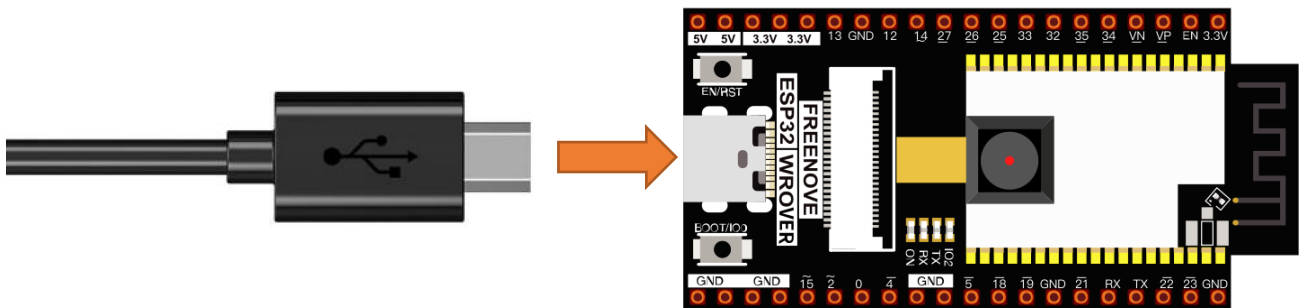
Connect ESP32 using USB and check its IP address through serial monitor. Use web page to access IP address to obtain video and image data.

Component List

Micro USB Wire x1 	ESP32-WROVER x1 
--	--

Circuit

Connect Freenove ESP32 to the computer using USB cable.



Code

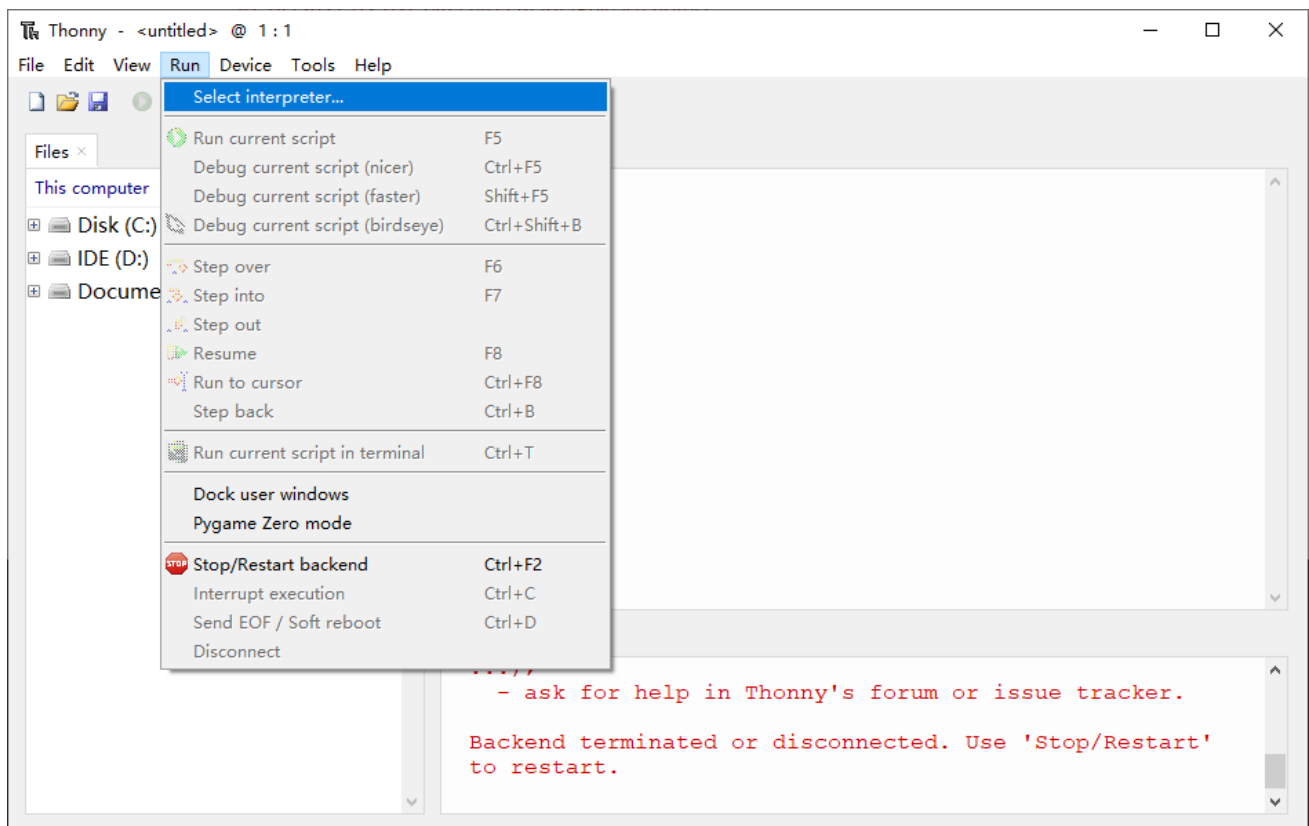
Move the program folder “**Freenove_Super_Starter_Kit_for_ESP32/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Since Micropython does not provide firmware including camera module, in this chapter, we will use the camera based on the firmware in lemariva's Github project, micropython-camera-driver.

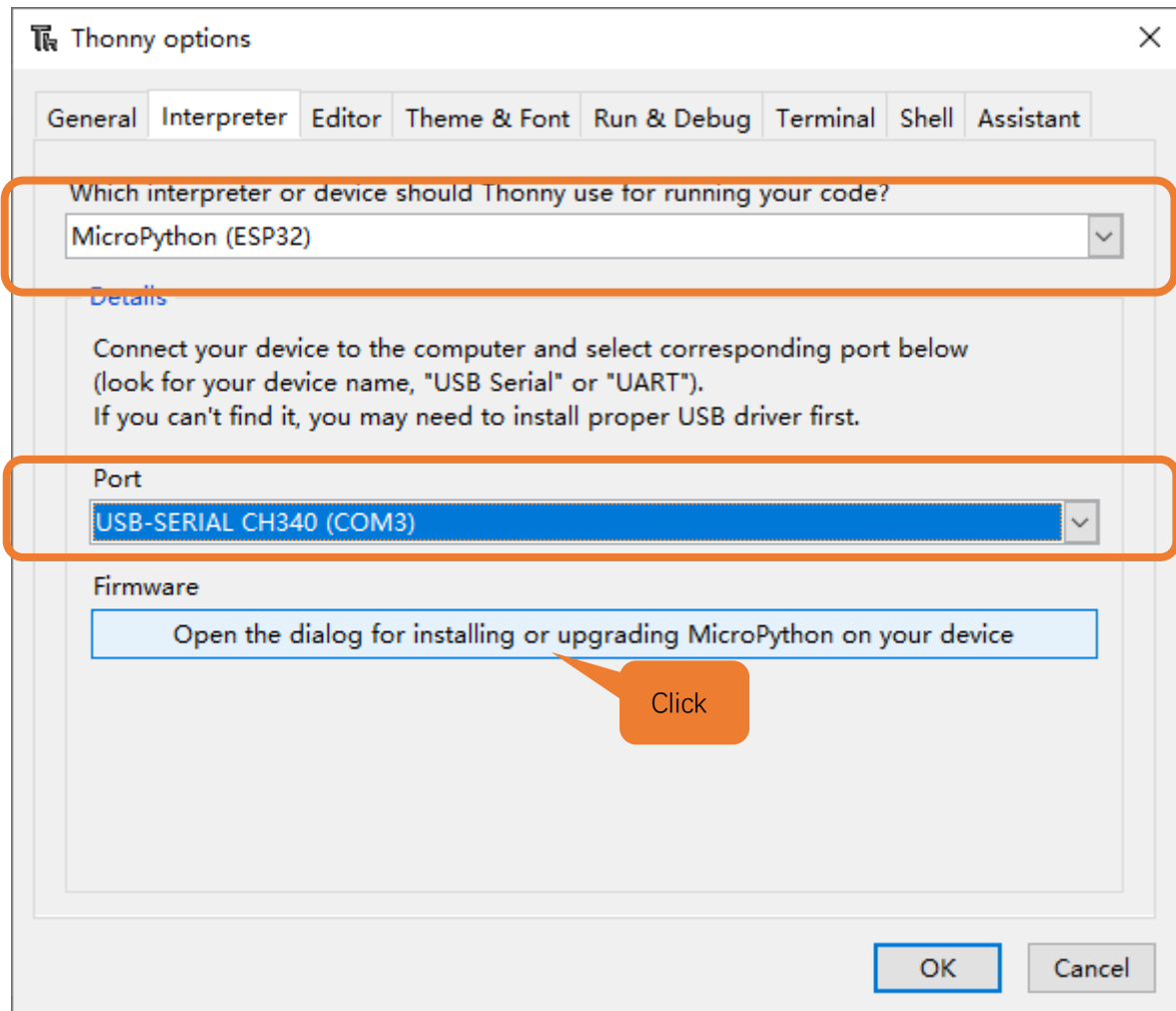
Project link: <https://github.com/lemariva/micropython-camera-driver>

Before starting the project, we need to re-upload the firmware with the camera module via steps below.

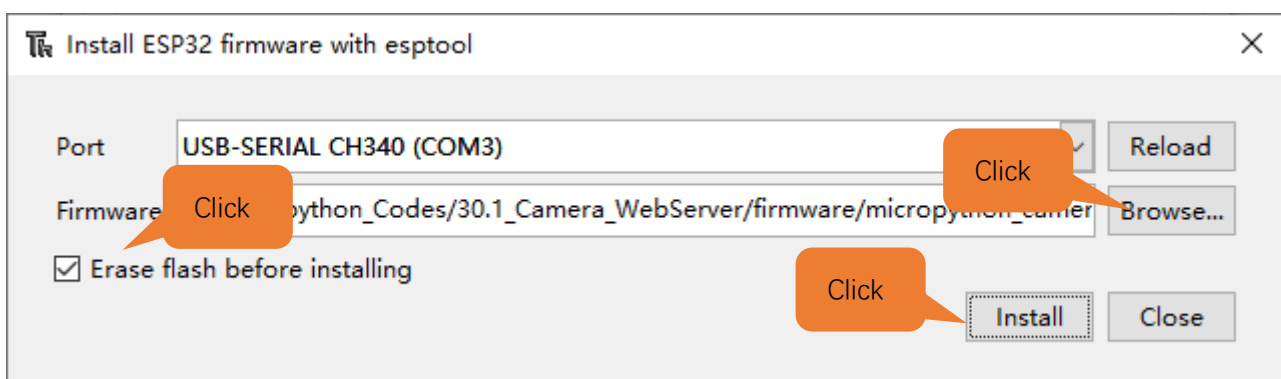
Open Thonny, click “run” and select “Select interpreter...”



Select “Micropython (ESP32)”, select “USB-SERIAL CH340 (COM4)”, and then click the long button under “Firmware”.



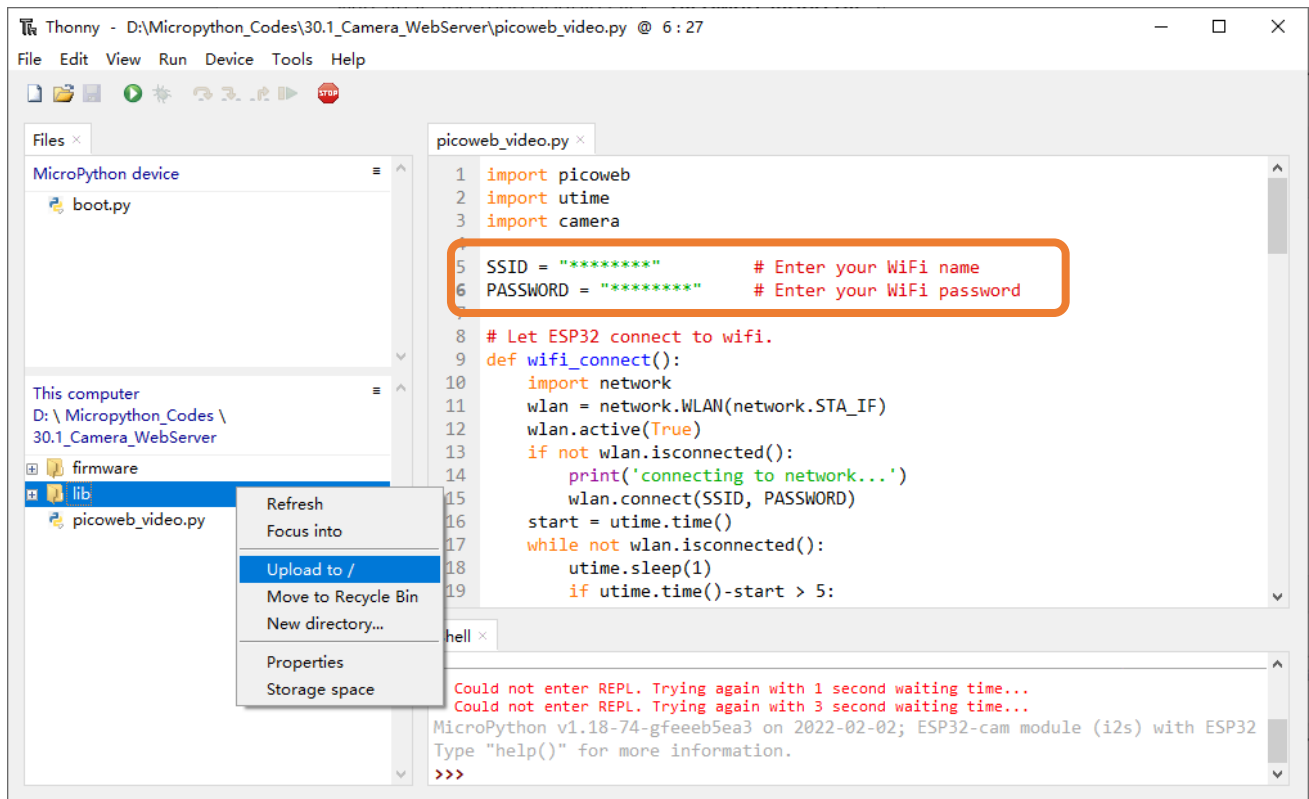
In the new popup window, select corresponding “USB-SERIAL CH340 (COM3)” for port. Click “Browse”, select “23.1_Camera_WebServer\firmware\micropython_camera_feeeb5ea3_esp32_idf4_4.bin”. Select “Erase...” and click “Install”.



Wait for completion.

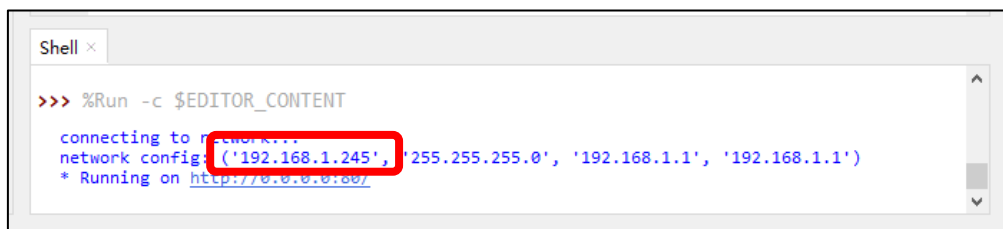
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "23.1_Camera_WebServer". Select folder "lib", right click your mouse to select "Upload to /", wait for "lib" to be uploaded to ESP32-WROVER and then double click "picoweb_video.py".

23.1_Camera_WebServer



Before running the program, please modify your router's name and password in the box shown in the illustration above to make sure that your code can compile and work successfully.

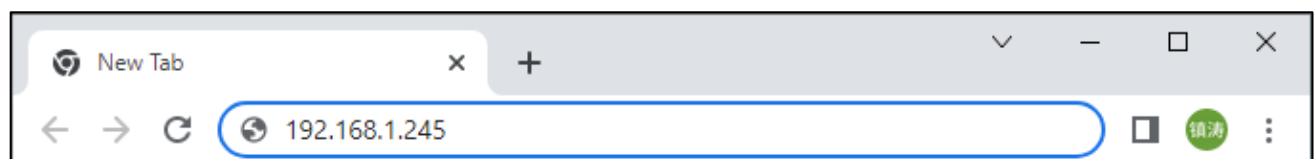
Click "run" to run the code "picoweb_video.py", then you can see the following content in the shell area.



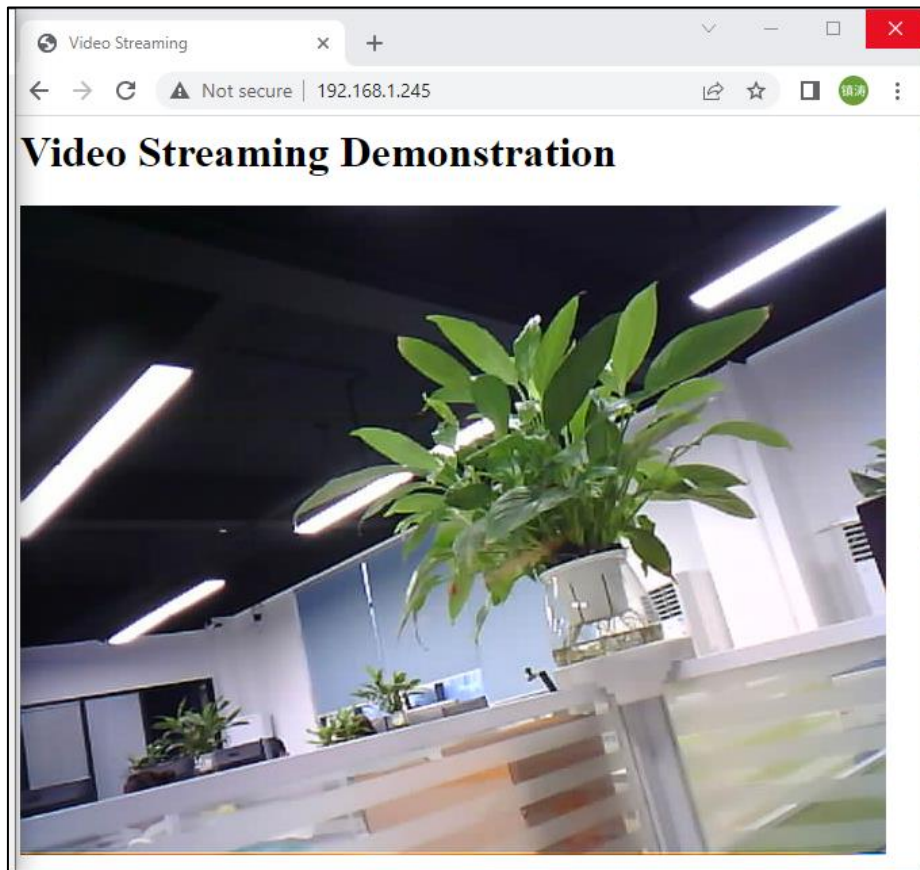
If your ESP32 has been in the process of connecting to router, but the information above has not been printed out, please re-check whether the router name and password have been entered correctly and press the reset key on ESP32-WROVER to wait for a successful connection prompt.

Open a web browser, enter the IP address printed by the serial monitor in the address bar, and access it.

Taking the Google browser as an example, here's what the browser prints out after successful access to ESP32's IP.



The effect is shown in the image below.



Please note:

If the shell area prompts an error when you click to run the code, please press the rst button on the esp32, wait for the system reset to complete, and then re-run the code.

The following is the program code.

```
1  import picoweb
2  import utime
3  import camera
4  import gc
5
6  SSID = "*****"          # Enter your WiFi name
7  PASSWORD = "*****"      # Enter your WiFi password
8
9  # Let ESP32 connect to wifi.
10 def wifi_connect():
11     import network
12     wlan = network.WLAN(network.STA_IF)
13     wlan.active(True)
14     if not wlan.isconnected():
15         print('connecting to network...')
16         wlan.connect(SSID, PASSWORD)
17     start = utime.time()
18     while not wlan.isconnected():
```

```

19         utime.sleep(1)
20         if utime.time()-start > 5:
21             print("connect timeout!")
22             break
23     if wlan.isconnected():
24         print('network config:', wlan.ifconfig())
25
26 # Initializing the Camera
27 def camera_init():
28     # Disable camera initialization
29     camera.deinit()
30     # Enable camera initialization
31     camera.init(0, d0=4, d1=5, d2=18, d3=19, d4=36, d5=39, d6=34, d7=35,
32                 format=camera.JPEG, framesize=camera.FRAME_VGA,
33                 xclk_freq=camera.XCLK_20MHz,
34                 href=23, vsync=25, reset=-1, pwn=-1,
35                 sioc=27, siod=26, xclk=21, pclk=22, fb_location=camera.PSRAM)
36
37     camera.framesize(camera.FRAME_VGA) # Set the camera resolution
38     # The options are the following:
39     # FRAME_96X96 FRAME_QQVGA FRAME_QCIF FRAME_HQVGA FRAME_240X240
40     # FRAME_QVGA FRAME_CIF FRAME_HVGA FRAME_VGA FRAME_SVGA
41     # FRAME_XGA FRAME_HD FRAME_SXGA FRAME_UXGA
42     # Note: The higher the resolution, the more memory is used.
43     # Note: And too much memory may cause the program to fail.
44
45     camera.flip(1) # Flip up and down window: 0-1
46     camera.mirror(1) # Flip window left and right: 0-1
47     camera.saturation(0) # saturation: -2,2 (default 0). -2 grayscale
48     camera.brightness(0) # brightness: -2,2 (default 0). 2 brightness
49     camera.contrast(0) # contrast: -2,2 (default 0). 2 highcontrast
50     camera.quality(10) # quality: # 10-63 lower number means higher quality
51     # Note: The smaller the number, the sharper the image. The larger the number, the more
52     blurry the image
53
54     camera.speffect(camera.EFFECT_NONE) # special effects:
55     # EFFECT_NONE (default) EFFECT_NEG EFFECT_BW EFFECT_RED EFFECT_GREEN EFFECT_BLUE
56     EFFECT_RETRO
57     camera.whitebalance(camera.WB_NONE) # white balance
58     # WB_NONE (default) WB_SUNNY WB_CLOUDY WB_OFFICE WB_HOME
59
60 # HTTP Response Content
61 index_web="""
62 HTTP/1.0 200 OK\r\n

```

```
61 <html>
62   <head>
63     <title>Video Streaming</title>
64   </head>
65   <body>
66     <h1>Video Streaming Demonstration</h1>
67     
68   </body>
69 </html>
70 """
71
72 # HTTP Response
73 def index(req, resp):
74     # You can construct an HTTP response completely yourself, having
75     yield from resp.awrite(index_web)
76
77 # Send camera pictures
78 def send_frame():
79     buf = camera.capture()
80     yield (b'--frame\r\n'
81           b'Content-Type: image/jpeg\r\n\r\n'
82           + buf + b'\r\n')
83     del buf
84     gc.collect()
85
86 # Video transmission
87 def video(req, resp):
88     yield from picoweb.start_response(resp, content_type="multipart/x-mixed-replace;
89 boundary=frame")
90     while True:
91         yield from resp.awrite(next(send_frame()))
92         gc.collect()
93
94 ROUTES = [
95     # You can specify exact URI string matches...
96     ("/", index),
97     ("/video", video),
98 ]
99
100 if __name__ == '__main__':
101
102     import ulogging as logging
103     logging.basicConfig(level=logging.INFO)
104     camera_init()
```

```

105     wifi_connect()
106
107     #Create an app object that contains two decorators
108     app = picoweb.WebApp(__name__, ROUTES)
109
110     app.run(debug=1, port=80, host="0.0.0.0")
111     # debug values:
112     # -1 disable all logging
113     # 0 (False) normal logging: requests and errors
114     # 1 (True) debug logging
115     # 2 extra debug logging

```

Import picoweb、utime、camera、gc modules.

```

1     import picoweb
2     import utime
3     import camera
4     import gc

```

Before running the code, please modify the WiFi name and password in the code to ensure that the ESP32 can connect to the network.

```

6     SSID = "*****"           # Enter your WiFi name
7     PASSWORD = "*****"       # Enter your WiFi password

```

Define the WiFi connection function, set the ESP32 to STA mode, and let the ESP32 connect to the nearby WiFi. If the connection is successful, the WiFi configuration information of the ESP32 will be printed; if the connection fails, the connection timeout will be printed.

```

10     def wifi_connect():
11         import network
12         wlan = network.WLAN(network.STA_IF)
13         wlan.active(True)
14         if not wlan.isconnected():
15             print('connecting to network...')
16             wlan.connect(SSID, PASSWORD)
17         start = utime.time()
18         while not wlan.isconnected():
19             utime.sleep(1)
20             if utime.time()-start > 5:
21                 print("connect timeout!")
22                 break
23         if wlan.isconnected():
24             print('network config:', wlan.ifconfig())

```

The deinit() is used to disable the configuration of the camera to prevent the previous configuration from interfering with the following configuration.

The init() is used to configure the camera's pin driver, image data format, resolution and other information. By default, please do not modify this function, otherwise the camera initialization fails and the image cannot be obtained.


```

29 camera.deinit()
30 # Enable camera initialization
31 camera.init(0, d0=4, d1=5, d2=18, d3=19, d4=36, d5=39, d6=34, d7=35,
32             format=camera.JPEG, framesize=camera.FRAME_VGA,
33             xclk_freq=camera.XCLK_20MHz,
34             href=23, vsync=25, reset=-1, pwn=-1,
35             sioc=27, siod=26, xclk=21, pclk=22, fb_location=camera.PSRAM)

```

This function can set the resolution of the camera individually, you can refer to the notes below to select the appropriate resolution size.

```

37 camera.framesize(camera.FRAME_VGA) # Set the camera resolution
38 # The options are the following:
39 # FRAME_96X96 FRAME_QQVGA FRAME_QCIF FRAME_HQVGA FRAME_240X240
40 # FRAME_QVGA FRAME_CIF FRAME_HVGA FRAME_VGA FRAME_SVGA
41 # FRAME_XGA FRAME_HD FRAME_SXGA FRAME_UXGA
42 # Note: The higher the resolution, the more memory is used.
43 # Note: And too much memory may cause the program to fail.

```

The following functions can modify the image information obtained by the camera.

```

45 camera.flip(1) # Flip up and down window: 0-1
46 camera.mirror(1) # Flip window left and right: 0-1
47 camera.saturation(0) # saturation: -2,2 (default 0). -2 grayscale
48 camera.brightness(0) # brightness: -2,2 (default 0). 2 brightness
49 camera.contrast(0) # contrast: -2,2 (default 0). 2 highcontrast
50 camera.quality(10) # quality: # 10-63 lower number means higher quality
51 # Note: The smaller the number, the sharper the image. The larger the number, the more
    blurry the image
52
53 camera.speffect(camera.EFFECT_NONE) # special effects:
54 # EFFECT_NONE (default) EFFECT_NEG EFFECT_BW EFFECT_RED EFFECT_GREEN EFFECT_BLUE
    EFFECT_RETRO
55 camera.whitebalance(camera.WB_NONE) # white balance
56 # WB_NONE (default) WB_SUNNY WB_CLOUDY WB_OFFICE WB_HOME

```

This is the code for a simple web interface, used here as an example.

```

59 index_web="""
60 HTTP/1.0 200 OK\r\n
61 <html>
62   <head>
63     <title>Video Streaming</title>
64   </head>
65   <body>
66     <h1>Video Streaming Demonstration</h1>
67     
68   </body>
69 </html>
70 """

```

Web page response function. When a user visits the webpage "/" built by ESP32, ESP32 calls this function, allowing the user to observe a display interface in the browser.

```
73 def index(req, resp):
74     # You can construct an HTTP response completely yourself, having
75     yield from resp.awrite(index_web)
```

send_frame() can send the image obtained by ESP32 in web page format. When someone visits the webpage "/video" built by the ESP32, the video(req, resp) function is used to continuously fetch images and send them to the browser.

```
77 # Send camera pictures
78 def send_frame():
79     buf = camera.capture()
80     yield (b'--frame\r\n'
81           b'Content-Type: image/jpeg\r\n\r\n'
82           + buf + b'\r\n')
83     del buf
84     gc.collect()
85
86 # Video transmission
87 def video(req, resp):
88     yield from picoweb.start_response(resp, content_type="multipart/x-mixed-replace;
89     boundary=frame")
90     while True:
91         yield from resp.awrite(next(send_frame()))
92         gc.collect()
```

Create two route decorators and declare their listening strings and corresponding response handlers respectively.

```
94 ROUTES = [
95     # You can specify exact URI string matches...
96     ("/", index),
97     ("/video", video),
98 ]
```

This is the main part of the program. First initialize the ESP32 camera, and then configure WiFi to connect the ESP32 to the network. Call the picoweb library, build a webserver, and run it.

```
103 import ulogging as logging
104 logging.basicConfig(level=logging.INFO)
105 camera_init()
106 wifi_connect()
107
108 #Create an app object that contains two decorators
109 app = picoweb.WebApp(__name__, ROUTES)
110
111 app.run(debug=1, port=80, host="0.0.0.0")
```

Reference

Image resolution	Sharpness	Image resolution	Sharpness
FRAMESIZE_96x96	96x96	FRAMESIZE_HVGA	480x320
FRAMESIZE_QQVGA	160x120	FRAMESIZE_VGA	640x480
FRAMESIZE_QCIF	176x144	FRAMESIZE_SVGA	800x600
FRAMESIZE_HQVGA	240x176	FRAMESIZE_XGA	1024x768
FRAMESIZE_240x240	240x240	FRAMESIZE_HD	1280x720
FRAMESIZE_QVGA	320x240	FRAMESIZE_SXGA	1280x1024
FRAMESIZE_CIF	400x296	FRAMESIZE_UXGA	1600x1200

We recommend that the resolution not exceed VGA(640x480).

What's next?

Thanks for your reading. This tutorial is all over here. If you find any mistakes, omissions or you have other ideas and questions about contents of this tutorial or the kit and etc., please feel free to contact us:

support@freenove.com

We will check and correct it as soon as possible.

If you want learn more about ESP32, you view our ultimate tutorial:

https://github.com/Freenove/Freenove_Ultimate_Starter_Kit_for_ESP32/archive/master.zip

If you want to learn more about Arduino, Raspberry Pi, smart cars, robots and other interesting products in science and technology, please continue to focus on our website. We will continue to launch cost-effective, innovative and exciting products.

<http://www.freenove.com/>

End of the Tutorial

Thank you again for choosing Freenove products.